

Embedded Linux and Kernel Drivers

Embedded Linux and Kernel Drivers

*From the boot chain to a driver
measured on a real-time kernel*

Erik Pihl

Embedded Linux and Kernel Drivers

Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

The code in this book may also be used under the MIT License, which is the license of the code in the companion repository, apart from the few files that name another in an SPDX line of their own. Excerpts quoted from the Linux kernel's own source, such as a Kconfig entry or a comment, remain under the kernel's license, GPL-2.0. The repository holds the lecture material, the specification, build files and tests of every lab, and the QEMU device model the whole course runs on:

<https://github.com/qrtech-academy/embedded-linux>

The kernel in this book is Linux 6.12, cross-built for arm64 and run under QEMU with a BusyBox userland. Linux® is the registered trademark of Linus Torvalds in the U.S. and other countries. Arm is a trademark of Arm Limited.

Set in TeX Gyre Pagella and DejaVu Sans Mono with Lua^ATeX. The figures are drawn from code by the course's own figure pipeline.

Typeset from the twelve lectures of the course, their exercises, and the two self-assessment papers. This edition: 12 September 2026.

Contents

Preface	ix
 I THE SYSTEM	
1 Embedded Linux, and the Licence You Ship With It	3
1.1 The Four Pieces, and How They Assemble Themselves	4
1.2 Open Source Licences, and What You Have to Publish	10
1.3 Review	15
1.4 Exercises	16
2 The Command Line, and the System Underneath It	21
2.1 The Shell, the Process Tree, and Where Things Live	22
2.2 /proc, /sys, and /dev.	27
2.3 Review	34
2.4 Exercises	36
3 The Kernel: Architecture, Source Tree, and Configuration	43
3.1 The Kernel as a Program, and Its Source Tree	44
3.2 Kconfig and Kbuild.	50
3.3 Review	56
3.4 Exercises	58
 II WRITING A DRIVER	
4 Kernel Modules	67
4.1 Kernel Modules, and the C You Are Allowed to Write	68
4.2 Review	76
4.3 Exercises	77
5 Character Device Drivers	83
5.1 Character Device Drivers	84
5.2 The Driver, Specified	90

5.3	Review	95
5.4	Exercises	96
6	Memory, MMIO, and Resource Management	101
6.1	Kernel Memory	102
6.2	MMIO and Resource Management.	108
6.3	Review	115
6.4	Exercises	116
III	CONCURRENCY, INTERRUPTS AND TIME	
7	Concurrency and Locking	125
7.1	Where Concurrency Comes From, and What to Do About It	126
7.2	Deadlock, Lock Ordering, and Lockdep.	133
7.3	Review	137
7.4	Exercises	139
8	Interrupts and Deferred Work	145
8.1	Interrupts.	146
8.2	Deferred Work.	153
8.3	Review	157
8.4	Exercises	158
9	Sleeping, Waiting, and Time	165
9.1	Sleeping and Waiting.	166
9.2	Time, Delays and Timers.	172
9.3	Review	176
9.4	Exercises	177
IV	FITTING INTO THE KERNEL	
10	The Device Model and the Device Tree	185
10.1	The Device Model.	186
10.2	The Device Tree	191
10.3	Review	195
10.4	Exercises	196
11	Kernel Frameworks	203
11.1	Kernel Frameworks.	204
11.2	Review	211
11.3	Exercises	212

12	Real-Time Linux	219
12.1	Real Time	220
12.2	Measuring Latency Honestly	225
12.3	Review	230
12.4	Exercises	231
A	Solutions to the Exercises	237
A.1	Chapter 1: Embedded Linux, and the Licence You Ship With It	237
A.2	Chapter 2: The Command Line, and the System Underneath It. . . .	243
A.3	Chapter 3: The Kernel: Architecture, Source Tree, and Configuration .	249
A.4	Chapter 4: Kernel Modules	256
A.5	Chapter 5: Character Device Drivers.	263
A.6	Chapter 6: Memory, MMIO, and Resource Management.	269
A.7	Chapter 7: Concurrency and Locking	277
A.8	Chapter 8: Interrupts and Deferred Work.	284
A.9	Chapter 9: Sleeping, Waiting, and Time.	292
A.10	Chapter 10: The Device Model and the Device Tree.	299
A.11	Chapter 11: Kernel Frameworks	307
A.12	Chapter 12: Real-Time Linux	314
B	The Self-Assessment Papers	325
C	Self-Assessment Paper A	329
D	Self-Assessment Paper B	335
E	Model Answers to Paper A	343
	Question 1: The image, and the licence on it.	343
	Question 2: Asking a running kernel a question	345
	Question 3: Configuring a kernel, and what it costs	345
	Question 4: A module, written out	346
	Question 5: The character device contract	348
	Question 6: An address, translated and mapped	349
	Question 7: A race, traced.	350
	Question 8: An interrupt, acknowledged.	351
	Question 9: A reader put to sleep	352
	Question 10: Frameworks, and the trade	353
F	Model Answers to Paper B	355
	Question 1: What leaves the building, and when it does not	355
	Question 2: A number that is not what it says.	356
	Question 3: An option that did not survive	358

Question 4: A module that will not load	360
Question 5: A read that returns zero	361
Question 6: A mapping that is wrong	362
Question 7: A deadlock that has not happened	364
Question 8: An interrupt that never stops	365
Question 9: A driver that finds its own hardware	366
Question 10: Worse average, better maximum	367

Preface

This book is for embedded engineers who write C and have never written it for a kernel. It goes from what a Linux system on a board is made of, through the commands that interrogate a running one, to a device driver you write yourself: memory-mapped registers, interrupts, locking, a device tree node, a kernel framework, and finally what all of that does to latency on a real-time kernel.

It covers Linux as embedded engineers meet it: as a kernel you configure and build, a userland you assemble, and a driver you write for hardware nobody has written one for yet. Unlike an introductory Linux text, it assumes you can already write C and read a datasheet, and spends its time on the parts that are specific to kernel space:

- What an embedded Linux system is made of, and the boot chain that assembles it.
- Open source licences, and what shipping a product built on this kernel obliges you to publish.
- The command line, taught as the set of questions you can ask a running kernel.
- Kernel architecture, the source tree, and Kconfig.
- Kernel modules: building them, loading them, and the C environment they run in.
- Character device drivers, and the userspace interface a driver actually presents.
- Kernel memory allocation, memory-mapped I/O, DMA, and managed resources.
- Concurrency, locking, and the contexts in which you are not allowed to sleep.
- Interrupts, deferred work, wait queues, and time.
- The driver model, the device tree, and how a driver finds its hardware.
- Kernel frameworks, and why a driver that is a character device is usually a driver in the wrong subsystem.
- Real-time Linux: where latency comes from, what `PREEMPT_RT` changes, and what makes a driver hostile to it.

Everything here runs in an emulator, and that is the design

There is no board. Every chapter, every lab and every measurement runs on `qemu-system-aarch64`, and the device your driver talks to is one the companion repository ships the source of.

That is a real cost and it is worth naming before you start. You will not learn here what a floating input feels like, what a badly seated ribbon cable does to an I2C bus, or how long a board takes to come back after you brick its bootloader. Those are things only hardware teaches.

What it buys is everything else. The device has a register that no real device has: it records the exact instant it raised its interrupt line, and lets your driver read that instant back. That one register turns interrupt latency from something you infer into something you subtract, and Chapter 12 is built on it. Beyond that, every reader gets the same machine, a wrong `writel` costs you a reboot that takes two seconds rather than a trip to a bench, and a clone of the repository plus three make targets is the whole of the setup.

What you should be able to do at the end

Having worked through the book, you should be able to:

- Name the four pieces of an embedded Linux system and say what each one does at boot.
- Read an open source licence well enough to say what a product built on it must publish.
- Configure and cross-build a kernel, and say what a given config option costs.
- Write, build and load an out-of-tree kernel module.
- Write a character device driver, and say what its `read()` must do that a userspace `read()` need not.
- Choose an allocator and a GFP flag from the context the code runs in.
- Map a device's registers and access them without the compiler or the CPU reordering the accesses out from under you.
- Say what address a device doing DMA actually sees, and why a buffer you may dereference is not automatically a buffer you may hand to hardware.
- Pick a lock from what it protects and from where the contention comes from, and say why sleeping in an interrupt handler is not a style question.
- Write an interrupt handler, defer the work that does not belong in it, and block a reader until the data exists.
- Write a device tree node for a device, and a driver that probes from it with no address in it.
- Say which kernel framework a device belongs to, and what registering with it gives you free.
- Explain where latency comes from on a Linux system, what `PREEMPT_RT` changes about it, and what a measurement taken under an emulator is not evidence of.

What you are assumed to know

You are expected to arrive already comfortable with:

- Programming in C at the level of writing and debugging a few hundred lines: pointers, structs, arrays, function pointers, and manual memory management.
- Bit manipulation, and reading a register map out of a datasheet.
- Working on a command line: files, paths, redirection, pipes, and running make.
- What an interrupt is, at the level of the hardware: a line, a handler, and a return.

None of that is taught from scratch. It is refreshed where a lab needs it, briefly.

What the book does teach from nothing is everything above the C: what a kernel is as a program, what a module is as a thing you load into a running one, what a driver owes to the userspace on one side and the hardware on the other, and why almost every rule in kernel programming turns out to be a rule about which context the code is running in.

The code is C, and this is not a C book. Everything you write is ordinary C99 with a kernel API in front of it. There are no templates, no exceptions and no standard library; that last one is a bigger change than it sounds, and Chapter 4 spends a section on it.

Where this book sits. It is written to be read on its own. It assumes, and does not derive, the process, scheduler and address-translation theory that Chapters 3 and 7 lean on, and the memory ordering that Chapter 6 states without proving; where a chapter needs one of them, it says so. The QAcademy course *Embedded C*¹ covers the C itself.

How the book is organised

The book is typeset from a twelve-lecture course, and each chapter is one lecture. Its sections are the lecture's appendices, in the same order and with the same subsections, so a chapter here and a lecture directory in the repository can be read side by side. The chapters fall into four parts:

¹<https://github.com/qrtech-academy/embedded-c>

Chapter	Lecture	Topic
<i>Part I: The system</i>		
1	L01	Embedded Linux, and the licence you ship with it
2	L02	The command line, and the system underneath it
3	L03	The kernel: architecture, source tree, and configuration
<i>Part II: Writing a driver</i>		
4	L04	Kernel modules
5	L05	Character device drivers
6	L06	Memory, MMIO, and resource management
<i>Part III: Concurrency, interrupts and time</i>		
7	L07	Concurrency and locking
8	L08	Interrupts and deferred work
9	L09	Sleeping, waiting, and time
<i>Part IV: Fitting into the kernel</i>		
10	L10	The device model and the device tree
11	L11	Kernel frameworks
12	L12	Real-time Linux

Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises. The appendices hold the solutions to the exercises that are not code, and the two written papers described below.

Three topics are not covered here. Yocto and Buildroot get a section in Chapter 1 and no more, because teaching either honestly costs two chapters that would come out of the driver material, and how to build a distribution is a different subject with different tools. Networking drivers are absent for the same reason: the network stack is the largest subsystem in the kernel, and a driver for it is not a first driver. CPU isolation and IRQ affinity are absent from Chapter 12 for a different reason: the target is a two-CPU QEMU guest, and isolating one of two CPUs from a scheduler that is itself emulated would produce numbers that say nothing about a real machine. A claim about pinning that the target cannot demonstrate is worse than no claim, and Chapter 12 is the chapter that argues that.

One driver, grown

Every chapter has a lab, and from Chapter 4 onwards they are one driver, grown. You start with a module that prints a line, and finish with a driver that probes off a device tree node, services interrupts, exposes its device through a kernel framework, and has had its latency measured under two different preemption models.

The code you write is yours. The repository ships the specification in prose, which is in this book as the section of each chapter that specifies its lab, together with the Kbuild files, the KUnit suites and the on-target test scripts. It never ships the driver itself.

The harness is the machine. The code you write is a kernel module, and a kernel module runs in a kernel: there is no main, no libc, and nothing on the host that can call your `read()`. So `make test` builds your module, drops it into an `initramfs` with the lecture's `run.sh`, boots `qemu-system-aarch64`, and reads the serial console. Two things are tested that way:

- **KUnit**, for the parts that are pure logic. A ring buffer's wrap-around and a register field's decoding are functions with no hardware in them, and KUnit runs them inside the kernel with a real test framework around them. It is also what mainline uses, which makes it worth meeting.
- **The on-target script**, for everything else. Loading the module, reading `/dev`, checking `dmesg`, counting interrupts in `/proc/interrupts`; all the things that are only true of a driver in a running kernel.

Both report through the same three markers on the console, and `make test` distinguishes a test that failed from a kernel that panicked before the test began. That distinction matters more here than it does in userspace, because in kernel space the second one is a normal Tuesday.

What the exercises look like

Every chapter ends with eight to ten exercises, numbered within the chapter, and each labelled with its kind:

Kind	What it asks of you
RECALL	Say it back, in your own words, with nothing in front of you.
HAND CALCULATION	Work out a number on paper.
DESIGN	Choose between two mechanisms and defend the choice.
CODE	Write something and run it.
CROSS-CHECK	Exactly one per chapter, always last.

The Cross-check is the signature exercise and the reason the other four exist. You compute a number by hand, you run your own code on the same problem, and then you reconcile the two. They will rarely agree exactly, and reconciling them is the point: “my two answers differ by eight percent” is not a failure of the exercise, it is the exercise. Each one names the artefacts to measure and the questions to ask about the gap, so that you can tell a discrepancy you have explained from one you have merely noticed.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

The solutions to the exercises that are not code are in Appendix A: every Recall, Hand calculation and Design exercise, and the parts of each Cross-check that are worked on paper. **The Code exercises have no solution**, here or in the repository. Each is checked by running it on the target, and its Check yourself line says how; an answer to read would replace the work. Try each exercise before you turn to its solution.

The figures are drawn from code

The nineteen figures in this book are the course's own, generated by the figure pipeline in the repository's `diagrams/` directory and printed here exactly as the lectures show them. None of them is decoration: half are block diagrams of something the prose can only describe one step at a time, and the rest are charts of numbers the course measured on its own target. Every number on every chart comes from one file, `diagrams/measured.py`, which cites the section that publishes it, so a chart and the table beside it cannot disagree.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the lab after every chapter, checked on the target by `make test`, and the exercises that end every chapter.

Appendices [B](#) to [F](#) hold two four-hour papers with model answers, and they check something else: **what you can reconstruct on paper, with nothing in front of you**. Ten questions each, mixing prose with kernel C you either write or repair, and no machine in the room to tell you which it is.

They exist purely so that you can test your own knowledge after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one after the last chapter. Both papers draw on all twelve chapters, so sitting one partway through examines material you have not read yet. Their model answers mark the code questions against checklists rather than printing listings, so that they do not become back-door solutions to the labs.

Setting up

Everything builds inside a container, so the only thing you install is Docker. On Windows, install it under WSL2 and work from the WSL side of the filesystem. Then clone the companion repository, <https://github.com/qrtech-academy/embedded-linux>, and from its root:

```
make env      # Build the container image. Once, and slow.
make kernel  # Cross-build the arm64 kernel. Once, and slower.
make qemu    # Build QEMU with the course's device in it. Once.
make rootfs  # Build the BusyBox initramfs.
make boot    # Boot the target and get a shell. Ctrl-A X quits.
```

`make help` lists every target. If you would rather install the toolchain natively, set `QA_NO_CONTAINER=1` and install the packages listed in `docker/Dockerfile`; nothing in the course requires the container except the reproducibility of it.

Once a lab is written, `make build` compiles it and `make test` boots it and runs its tests. Both narrow to one lecture with `L=`:

```
make build L=L06
make test L=L06
```

Every target skips loudly rather than failing when what it needs does not exist yet, and a skip names the target that would fix it.

A note on the code style. The C in the repository is formatted with `clang-format` against the `.clang-format` shared by every QAcademy course, so the kernel code in it does *not* follow the Linux kernel's own coding style, and would be rejected upstream on that basis alone. That is a deliberate trade for consistency with the rest of QAcademy, and §4.1.9 shows the same function in both styles, so that opening a file in `drivers/` is not a surprise.

PART I

The System

Embedded Linux, and the Licence You Ship With It

What an embedded Linux system is made of, how it assembles itself at power-on, and what the licences on the pieces oblige you to hand over.

IN THIS CHAPTER

- What is actually on the flash of a product that runs Linux, in four pieces, and what each one does at power-on.
- The boot chain from the on-chip ROM to the first userspace process, and the three steps QEMU's -kernel lets you skip.
- Cross-compilation: why gcc on your laptop is the wrong compiler, and what a sysroot is for.
- Buildroot and Yocto in one section, which is all they get in this course, and why.
- Copyleft against permissive, and the one question a licence actually answers.
- The kernel's syscall exception, MODULE_LICENSE, and a licence boundary the linker enforces.
- `make kernel`, `make qemu`, `make rootfs`, `make boot`, and a shell on a machine you just built.

This chapter is about what a product that runs Linux is made of, and what shipping it obliges you to hand over. It takes the system apart into four pieces, the bootloader, the kernel, the root filesystem and the toolchain that builds the other three, and follows the chain of programs that assembles them between power-on and the first userspace process, including the three steps QEMU's -kernel skips and what skipping them costs. It explains why the compiler on your laptop is the wrong one and what a sysroot is for. Then it asks the one question a licence answers, what you must give somebody along with a binary, works it through for the licences an embedded product carries, and follows the two boundaries that matter for the rest of the book: the syscall exception, which is why your application is yours, and `EXPORT_SYMBOL_GPL`, where the argument about modules becomes a build error.

Read §1.1 first; it is the anatomy and the boot chain, and it ends with the machine this course builds, piece by piece and with its sizes, in §1.1.7. That machine is what the lab checks: what `make kernel`, `make qemu`, `make rootfs` and `make boot` produce, rather than code you write. §1.2 is the licensing material, the one section in the book with no code in it and the one

most likely to come up in a design review. The exercises in §1.4 end with the Cross-check: classify the licences of ten kernel source files by hand, predict the whole directory from them, then run your own audit script over it and reconcile the two, which will not agree. The lab is done when `make test L=L01` reports **PASSED**, with all nine of the target's checks on the machine you built, from the kernel version to the qa-dev node in its device tree.

1.1 The Four Pieces, and How They Assemble Themselves

This section is the anatomy of an embedded Linux system: what is actually stored on a product's flash, what each piece does, and the order in which they hand control to one another between power being applied and a prompt appearing. It ends with the machine this course runs on, and an honest account of which parts of the chain that machine skips.

1.1.1 What is on the flash

An embedded Linux product contains four things. Three of them ship; the fourth never leaves your build machine and determines everything about the other three.

Piece	Ships on the device	What it does
Bootloader	Yes	Brings up DRAM, finds the kernel, hands it a device tree
Kernel	Yes	Drives the hardware and provides the system call interface
Root filesystem	Yes	Everything that runs as a process, starting with PID 1
Toolchain	No	Compiles the other three; fixes the ABI they share

Almost every question in the rest of this course is a question about which of the four a thing belongs to. “Why does my driver not load” is a kernel question. “Why is my binary 40 MB” is a toolchain question. “Why does nothing happen after `Run /init` as `init` process” is a root filesystem question, and the answer is nearly always that `/init` is not there, is not executable, or is dynamically linked against a library that is not there either.

The fourth piece is the one people forget. A toolchain is not just a compiler; it is a compiler, a C library, a linker, and a set of headers, and the choice of C library in particular is baked into every binary you ship. You cannot mix a glibc binary and a musl root filesystem, and the failure when you try is a loader error at run time and not a link error at build time.

1.1.2 The boot chain

Power is applied. What happens next is a chain of five programs, each one loading the next, and each one existing because the one before it did not have enough memory to do the next one's job.

```
power on
|
```

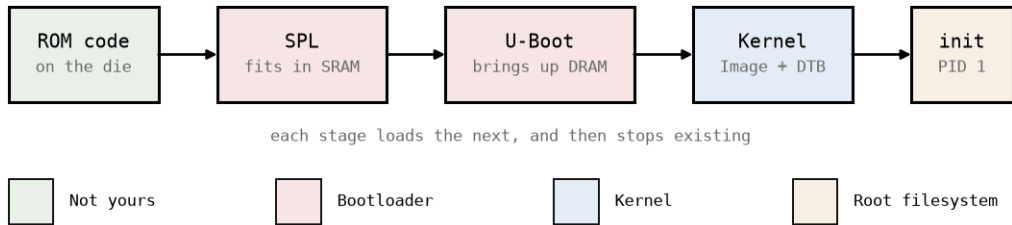
v	
[1] Boot ROM	on-chip, mask-programmed, a few kilobytes
	reads a strap pin, loads the next stage into on-chip SRAM
v	
[2] SPL / first stage	~64 KB, runs from SRAM
	its whole purpose is to initialise the DRAM controller
v	
[3] Bootloader	~1 MB, runs from DRAM (U-Boot, or an EFI loader)
	loads the kernel and the DTB into DRAM, then jumps
v	
[4] Kernel	decompresses itself, drives the hardware
	mounts a root filesystem, then hands over
v	
[5] /init	PID 1, the first userspace process
	starts everything else and never exits

Two things about this chain are worth stating explicitly because they are the parts people get wrong.

The SPL exists because of a chicken-and-egg problem. U-Boot is around a megabyte and needs DRAM to run in. DRAM does not work until its controller has been configured, with a long sequence of board-specific timing values. That configuration code has to run from somewhere, and the only memory available at that point is the few tens of kilobytes of SRAM on the SoC itself. So the build produces a second, tiny bootloader whose entire job is to set up DRAM and then load the real one. If you have ever wondered why a board has both `ML0` and `u-boot.img`, or both `SPL` and `u-boot`, that is why.

The boot ROM is the only piece you cannot replace. It is mask-programmed into the silicon. This is what makes bricking recoverable on some boards and terminal on others: if the ROM will fall back to loading over USB or UART when the flash is empty, you can always recover; if it will not, a bad write to the bootloader partition turns the board into a coaster.

The boot chain, and the four pieces



The toolchain is the piece that never ships, and the one that decides whether the other three fit together: it fixes the C library and the ABI of every binary it builds.

Figure 1.1 The boot chain drawn as five boxes, each loading the next: ROM code on the die, SPL in SRAM, U-Boot bringing up DRAM, the kernel Image and DTB, and init as PID 1. The boxes are coloured by which of the four pieces supplies them, with the toolchain named as the piece that never ships.

1.1.3 What the kernel is handed, and by whom

Step 3 hands step 4 two things: the kernel image, and a **device tree blob**.

The device tree matters enough to have a chapter of its own (Chapter 10), but the reason it exists belongs here. An x86 PC can discover its own hardware: PCI is enumerable, ACPI describes the rest, and a single kernel image boots on any machine. An embedded SoC can do none of that. There is no bus to walk that will tell you a UART lives at 0x09000000 and raises interrupt 33. Somebody has to say so, and the device tree is a data file that says so, kept separate from the kernel so that one kernel image can boot many boards.

So the kernel receives, in a register, a pointer to a blob that describes the machine. It parses it, and every driver that later probes gets its addresses from it. That is the whole idea, and Chapter 10 is about the mechanism.

1.1.4 What this course skips, and what that costs

make boot runs this:

```
qemu-system-aarch64 -machine virt -kernel Image -initrd rootfs.cpio.gz -append "
  ..."
```

-kernel skips steps 1, 2 and 3 entirely. QEMU writes the kernel image into the guest's memory, generates a device tree describing the machine it is emulating, puts a pointer to it in the right register, and jumps to the kernel's entry point. It is doing U-Boot's job, without being U-Boot.

What that costs you is real and worth naming. You will not learn here how to configure a DRAM controller, how to write a U-Boot environment, how to recover a board whose bootloader you overwrote, or what the serial output of a failing SPL looks like. Those are important skills and this course does not teach them.

What it buys is that everything after step 3 is identical. The kernel does not know or care whether U-Boot or QEMU put it in memory; it reads the same device tree and probes the same drivers either way. Every chapter from Chapter 3 onwards would be word-for-word the same on a real board.

1.1.5 The toolchain, and why your compiler is the wrong one

The target runs 64-bit ARM. Your build machine almost certainly does not. So the compiler that produces target binaries is not gcc but aarch64-linux-gnu-gcc, and that prefix is a **triplet**, or in this case something closer to a quadruplet:

```
aarch64 - linux - gnu
|         |         |
|         |         +-- C library and ABI: gnu (glibc). Others: musl, uclibc.
|         +-----+ operating system: linux. Others: none, elf, darwin.
+-----+ architecture: aarch64. Others: arm, x86_64, riscv64.
```

The single most useful demonstration of what that means is one command:

```
$ gcc -o hello-host hello.c && file hello-host
hello-host: ELF 64-bit LSB pie executable, x86-64, ...

$ aarch64-linux-gnu-gcc -o hello-target hello.c && file hello-target
hello-target: ELF 64-bit LSB pie executable, ARM aarch64, ...
```

Same source, same command line, two files that share nothing. The second will not run on your laptop and the first will not run on the target.

The sysroot is the other half. A cross-compiler must not use your machine's libraries, because those describe your machine. It uses a *sysroot*: a directory that mirrors the target's filesystem, holding the target's headers and the target's libraries. A toolchain built by Buildroot or Yocto names that directory when asked. Debian's, the one in the course's container, answers /:

```
$ aarch64-linux-gnu-gcc -print-sysroot
/
```

That is because Debian keeps the target's files beside your machine's rather than in a tree of their own: the target's C library and headers in /usr/aarch64-linux-gnu, other arm64 libraries in /usr/lib/aarch64-linux-gnu, and headers that are the same on every

architecture in `/usr/include`, which this compiler searches too. What it never searches is `/usr/lib/x86_64-linux-gnu`, where your machine's own libraries are.

Two consequences follow, and both of them bite people. A library your program needs must be installed *for the target*, not for your machine; installing `libfoo-dev` with `apt` installs your machine's library, which the cross linker cannot use. And the target's `glibc` version sets the oldest target root filesystem your binary will run on, because `glibc` guarantees that a binary runs on a newer `glibc` than the one it was built against, and not on an older one.

Kernel code has no sysroot and no C library at all. It is compiled `-nostdinc`, against the kernel's own headers. There is no `printf`, no `malloc`, and no `double`. Chapter 4 covers what that environment is actually like; the point here is that the kernel does not use the C library the rest of the toolchain is built around, which is why “the toolchain” and “the kernel” are two of the four pieces and not one.

1.1.6 Buildroot and Yocto, in one section

The three shipping pieces have to be built, configured and packaged together, reproducibly, by somebody who is not you and possibly on a build server. That is a real problem and there are two mainstream answers to it.

	Buildroot	Yocto / OpenEmbedded
Model	One make, one config, one image	Layers of recipes, one per package
Output	A root filesystem image	A root filesystem image, and a package feed
Rebuild cost	Often a full rebuild	Incremental, per recipe
Learning cost	An afternoon	Weeks
Fits	Small, fixed products	Products with variants, updates, or a team

The honest summary is that Buildroot is a build system and Yocto is a distribution builder, and teams pick the wrong one about half the time. A product with one variant and no field updates does not need Yocto's machinery. A product with four hardware variants, two customers and an OTA update channel will outgrow Buildroot's single-config model.

This course uses neither. Its root filesystem is one statically linked BusyBox and a shell script, assembled by `ci/rootfs.sh` in about a hundred lines. That is not what a product does, and it is not held up as an example of what to do. It is the smallest thing that boots, which makes it the right thing to learn on, and it means every file on the target is one you can account for.

1.1.7 The machine this course builds

Running `make kernel, make qemu, make rootfs` and `make boot` produces this:

Piece	What it is here	Size
Bootloader	None. QEMU's -kernel does the job	0
Kernel	Image, Linux 6.12.30, cross-built for arm64	~24 MB
Root filesystem	rootfs.cpio.gz: BusyBox, static, plus three modules	~1.3 MB
Toolchain	aarch64-linux-gnu-gcc 12, from the container image	n/a

The two sizes are worth staring at for a moment, because the ratio is the wrong way round from what most people expect. **The kernel is roughly twenty times the size of the entire userland.** That is not because the kernel is bloated in some abstract sense; it is because the configuration it was built from still enables a great deal that this machine cannot use. Finding out how much, and taking it out, is Chapter 3's lab.

The exercises in §1.4 ask you to produce both of those numbers yourself rather than take them from this table, and the Cross-check asks you to predict one of them before you measure it.

1.1.8 Where to look on a running system

Once `make boot` gives you a prompt, four files tell you which of the four pieces you are looking at. Chapter 2 is about this in earnest; these are the ones worth knowing from the start.

```
/proc/version      the kernel: version, compiler, and when it was built
/proc/cmdline      what the bootloader (here, QEMU) told the kernel at handover
/proc/iomem        the physical address map, as the kernel understands it
/sys/firmware/devicetree/base
                   the device tree, unpacked into a directory per node
```

That last one is worth a minute now even though Chapter 10 is where it is explained. The device tree the kernel was handed is exposed as a directory tree, one directory per node and one file per property. The device this course is built around appears there:

```
/sys/firmware/devicetree/base/platform-bus@c0000000/qa-dev@0/
  compatible      "qacademy,qa-dev-1.0"
  reg              00 00 00 00 00 00 10 00
  interrupts       00 00 00 00 00 00 00 70 00 00 00 04
```

Nobody typed those numbers. QEMU chose where to put the device, wrote the node, and handed the result to the kernel, which is exactly what a bootloader does on a real board. Reading them back and working out what physical address they describe is Chapter 6's Cross-check.

1.2 Open Source Licences, and What You Have to Publish

This section is about one question. It is the only question a licence actually answers, and almost every argument about open source licensing is an argument that has lost sight of it:

If I give somebody this binary, what am I obliged to give them along with it?

Not “is this free software”. Not “can I use this commercially”; you almost always can. The question is what leaves your building when the product does.

This is not legal advice, and the author is not a lawyer. What it is is the engineering model you need to have in your head so that you know which decisions need a lawyer and which do not. Getting this wrong is expensive, and the expensive cases are almost always cases where nobody asked the question at all until a customer did.

1.2.1 Two families

Every licence you will meet on an embedded product falls into one of two families, distinguished by what happens when you combine the code with your own.

Permissive licences let you do essentially anything, including shipping binaries with no source, provided you carry the copyright notice along. MIT, BSD-2-Clause, BSD-3-Clause and Apache-2.0 are the ones you will actually see.

Copyleft licences let you do essentially anything, provided that whoever receives the binary can also receive the source, under the same licence. GPL-2.0, GPL-3.0, LGPL-2.1 and MPL-2.0 are the ones you will see.

Licence	Family	If you ship a binary containing it, you must publish
MIT	Permissive	Nothing. Keep the notice in your documentation
BSD-3-Clause	Permissive	Nothing, plus you may not use the authors' names to endorse
Apache-2.0	Permissive	Nothing, plus a NOTICE file, plus a patent grant you give
MPL-2.0	Weak copyleft (file)	The source of the MPL files you changed
LGPL-2.1	Weak copyleft (library)	The library's source, and the means to relink your binary against a modified copy
GPL-2.0	Strong copyleft	The complete corresponding source of the whole work
GPL-3.0	Strong copyleft	The above, plus installation information for consumer devices

The two rows worth reading twice are LGPL and GPL-3.0, because both carry an obligation that is not about source code at all, and both are where embedded products get caught.

1.2.2 The word that does the work: derivative

Copyleft only reaches code that is part of the same work. So everything depends on where one work ends and the next begins, and that boundary is not defined by the licence in terms an engineer can apply mechanically. What exists instead is a set of positions, some of which are near-universally accepted and some of which are contested.

Broadly accepted:

- **Static linking creates one work.** If your proprietary code is linked into the same binary as GPL code, the result is a derivative work of both.
- **Separate processes communicating over a pipe or a socket do not.** Your closed daemon talking to a GPL program over a socket is two works.
- **Mere aggregation does not.** Two unrelated programs on the same filesystem image are not one work because they share a partition.

Contested, and where you need a lawyer rather than an opinion:

- **Dynamic linking.** The Free Software Foundation holds that linking against a GPL library creates a derivative work even at run time. Others disagree. This is why the LGPL exists: it settles the question for libraries that want to be linkable by proprietary code.
- **Kernel modules.** See §1.2.4, which is the case this course actually cares about.

1.2.3 The syscall exception, and why your application is yours

The Linux kernel is GPL-2.0-only. Every program on a Linux system makes system calls into it. If that made every program a derivative work of the kernel, no proprietary software could run on Linux at all, and Linux would have no commercial users.

Linus Torvalds added a note to the top of the kernel's `COPYING` file that says so explicitly. The note now lives in `LICENSES/exceptions/Linux-syscall-note`, which `COPYING` names as the kernel's explicit syscall exception:

NOTE! This copyright does *not* cover user programs that use kernel services by normal system calls — this is merely considered normal use of the kernel, and does *not* fall under the heading of “derived work”.

That paragraph is the legal foundation of embedded Linux as a commercial platform, and it is worth knowing that it is a clarification granted by the copyright holder rather than something that falls out of the GPL itself. **The boundary it draws is the system call boundary.** Your application talks to the kernel through `read`, `write`, `ioctl` and the rest, and that is normal use.

The practical consequence is the one your colleague gets wrong. “We ship Linux, so all our code must be open source” is false: your application, running as its own process and making system calls, is yours, and you may ship it as a binary with no source at all. What *is* true is the narrower claim, that the kernel you ship, and any modifications you made to it, must be published.

1.2.4 Kernel modules, and the licence the linker enforces

A kernel module is not a separate process. It is compiled against the kernel's own headers, linked into the kernel's address space, and calls kernel functions directly. Under every version of the derivative-work argument, that is a much harder position to defend than an

application making system calls, and the widely held view among kernel developers is that a module is a derivative work of the kernel.

The kernel does not leave this as an argument. It enforces a version of it mechanically, and this is the part that matters to you as a driver author.

Every module declares its licence in its own binary:

```
MODULE_LICENSE("GPL");
```

The string is not documentation. It is read by the module loader, and the set of accepted values is fixed: "GPL", "GPL v2", "GPL and additional rights", "Dual BSD/GPL", "Dual MIT/GPL", "Dual MPL/GPL", and "Proprietary". Anything else counts as proprietary. A module with no `MODULE_LICENSE` at all is not built: `modpost` fails with missing `MODULE_LICENSE()`.

Two things follow when a module is not GPL-compatible.

First, the kernel is **tainted**. `/proc/sys/kernel/tainted` gains a bit, every subsequent oops carries a taint flag, and kernel developers will decline to debug it. That is a social mechanism rather than a legal one, but it is a real cost: a bug report from a tainted kernel is a bug report nobody upstream will look at.

Second, and concretely, **a large part of the kernel's internal API becomes unavailable**. Symbols are exported in one of two ways:

```
EXPORT_SYMBOL(foo);      /* any module may use this */
EXPORT_SYMBOL_GPL(bar);  /* only GPL-compatible modules may use this */
```

A module declaring a proprietary licence and calling `bar()` is refused, and it is refused in **two different places depending on what the build could see**.

Usually the build fails, at the `modpost` step that runs after linking each module:

```
ERROR: modpost: GPL-incompatible module mine.ko uses GPL-only symbol 'bar'
```

That is the good case: the message names the licence, the module and the symbol, and no `.ko` is produced. It happens whenever `modpost` can see that `bar` is GPL-only, which is nearly always, because the built-in kernel's exports are listed in its `Module.symvers`.

When `modpost` could not have known, the module loader refuses instead, and its message is much worse. It does not report a licence problem; it **hides GPL-only symbols from a proprietary module's symbol lookup entirely**, so what you see is:

```
mine: Unknown symbol bar (err -2)
```

which is indistinguishable from a typo or a missing dependency. That path is what you hit after a kernel upgrade in which a symbol you used changed from `EXPORT_SYMBOL` to

EXPORT_SYMBOL_GPL; your module was built when it was legal and is loaded when it is not. Chapter 4 causes both on purpose.

Which symbols are _GPL is a live decision, not a historical accident. Maintainers choose, and over the years the boundary has moved steadily toward _GPL for anything that looks like a core internal interface. If your business model depends on a proprietary module, it depends on a set of exported symbols that other people control and may narrow in the next release.

1.2.5 What “complete corresponding source” means in practice

GPL-2.0 section 3 requires you to give the recipient “the complete corresponding machine-readable source code”, defined as “all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable”.

Read the last clause again, because it is the one people fail. **The build scripts are part of the obligation.** A tarball of kernel sources with no .config in it does not satisfy this: the recipient cannot produce your binary from it. Neither does a source tree that only builds with an internal toolchain nobody outside your company has.

The practical checklist for a product that ships a GPL kernel is:

- The kernel source, at the exact version you shipped, including every patch you applied.
- The .config you built it with.
- The device tree sources.
- Enough instructions to build it: which toolchain, which commands.
- The same for U-Boot, BusyBox, and every other GPL component in the image.

You may either ship this alongside the product, or include a **written offer** valid for three years to supply it on request. The written offer is what most products use, and it is the thing that has to survive in your build system for three years after the last unit ships. That is a release-engineering requirement that comes directly out of a licence, and it is the single most common thing to discover too late.

GPL-3.0 adds installation information. For a consumer device, you must also provide whatever is needed to actually install a modified version and have it run: keys, signing tools, flashing instructions. This is why a great many embedded products deliberately stay on GPL-2.0-only components, and why the kernel’s own GPL-2.0-only status without the “or later” clause is a decision rather than an oversight.

1.2.6 SPDX, and how you find out what you have

You cannot answer the question in the opening of this section without an inventory. Reading every file is not possible; almost every project now says so in a machine-readable way instead.

An **SPDX identifier** is a single comment line at the top of a file:

```
// SPDX-License-Identifier: GPL-2.0-only
```

The kernel adopted these in 2017 and nearly every file now carries one. They are exact, they are greppable, and a file that has one needs no further interpretation:

```
grep -rh 'SPDX-License-Identifier:' drivers/ | sort | uniq -c | sort -rn
```

Three cautions, and they are the whole of what makes this a skill rather than a grep.

- **A file with no SPDX tag is not unlicensed.** It is a file whose licence has to be found some other way: a LICENSE file in its directory, a header comment, or the project's top-level terms. These are the interesting files in any audit, and there are always some.
- **GPL-2.0-only and GPL-2.0-or-later are different licences.** The second lets a downstream recipient move to GPL-3.0 and its installation-information obligation. The kernel is GPL-2.0-only. Some of the components around it are not.
- **A build system is not an inventory.** What matters is what ended up in the image, and a package can be present in the source tree and absent from the shipped filesystem, or the reverse.

Exercise 1.6 is to build that inventory for a real tree and say what you would have to publish. The Cross-check, Exercise 1.8, is to do a subdirectory by hand first, and find out where your script and your eyes disagree.

1.2.7 A worked example

A product ships:

Component	Licence	Modified
U-Boot	GPL-2.0-or-later	Yes, board port
Linux kernel	GPL-2.0-only	Yes, one driver
Your out-of-tree driver	Yours to choose	n/a
BusyBox	GPL-2.0-only	No
An MQTT client library	Apache-2.0	No
Your control application	Proprietary	n/a

What must you publish?

- **U-Boot**, complete corresponding source including your board port and your config.
- **The kernel**, complete corresponding source including your driver, your .config and your device tree. Note that the driver is in the kernel tree and is therefore part of the work.
- **BusyBox**, source at the version shipped. Unmodified does not mean exempt; the obligation is to the recipient of the binary, not conditional on your having changed it.

- **The MQTT library:** nothing, but carry its NOTICE file and its licence text in your documentation.
- **Your control application:** nothing. It is a separate process making system calls, and §1.2.3 is why.

And the one that is a decision rather than an obligation: **your out-of-tree driver**. If you keep it out of tree and declare it proprietary, you take on the taint, you lose every `EXPORT_SYMBOL_GPL` symbol, and you own the maintenance of it against a kernel API that has no stable interface and changes every release. Most companies that try this eventually stop, and the reason is usually the maintenance cost rather than the licence.

1.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Name the four pieces of an embedded Linux system, and say for any given file which one it belongs to.
- Walk the boot chain from power-on to the first userspace process, and say what each step does that the one before it could not.
- Say what -kernel skips, and therefore what this course does not teach you.
- Explain what a toolchain triplet names, and why a sysroot is not the same thing as the host's `/usr/include`.
- Answer the licence question for a given component: what must be published, to whom, and when.
- Say why an application that only makes system calls is not a derivative work of the kernel, and why a kernel module usually is.
- Predict whether a module will load, given its `MODULE_LICENSE` and the symbols it uses.

Questions to test yourself

1. A colleague says “we ship Linux, so all our code has to be open source”. What is wrong with that sentence, and what is the part of it that is right?
2. Your product has a bootloader, a kernel, a BusyBox root filesystem and one proprietary daemon you wrote. A customer asks for the source. What do you have to give them?
3. Why does the boot chain have an SPL at all, given that U-Boot could do the same job?
4. What does `aarch64-linux-gnu` tell you that `aarch64` alone does not?
5. A driver is licensed MIT and calls a function exported with `EXPORT_SYMBOL_GPL`. What happens, and at which point: compile, link, load, or run?
6. Why is the kernel image 24 MB when the machine it boots has one serial port and one timer?

Where to look

- The build targets are documented in the repository's root `README.md`, and `make help` lists them all.

Looking ahead

Chapter 2 is about:

- The command line, taken as the set of questions you are able to ask a running kernel.
- `/proc` and `/sys`: not directories, and not files either.
- What a two-megabyte userland gives up, measured against the one on your laptop.
- Why PID 1 is different from every other process, and what happens if it exits.

The machine you built in this chapter is the machine the other eleven chapters run on. Chapter 2 is about learning to ask it what it is doing.

1.4 Exercises

Eight, ending with the Cross-check. Do them in order; the last one needs the script from Exercise 1.6.

Several of these have a plausible wrong answer that is worth walking into before you find it. Where an exercise can be checked mechanically, a **Check yourself** line says how.

Everything here runs against the trees `make kernel` and `make rootfs` already downloaded, under `build/`. Nothing needs the network.

Solutions. The exercises that are not Code are answered in §A.1. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 1.1 The four pieces

RECALL

Without looking at §1.1, name the four pieces of an embedded Linux system and say in one sentence what each does.

Then, for each of the following, say which piece it belongs to:

- `/lib/modules/6.12.30/kernel/lib/kunit/kunit.ko`
- `u-boot.img`
- `aarch64-linux-gnu-gcc`
- `/init`
- The `.dtb` handed to the kernel at boot
- `/bin/busybox`

Exercise 1.2 Why there are two bootloaders

RECALL

A colleague looking at a board's flash layout asks why there is both an SPL and a `u-boot.img`, and suggests deleting the first to save space.

- Explain in two sentences why the SPL exists.
- What specifically would fail if you deleted it, and at what point in the boot?
- Name one thing about a board that makes bricking it recoverable, and one that does not.

Exercise 1.3 What the toolchain triplet promises

HAND CALCULATION

- Break `aarch64-linux-gnu` into its parts and say what each names.
- You are handed a binary and told only that it was built with `arm-none-eabi-gcc`. What can you say about the system it is intended to run on, and in particular, can it run on Linux? Justify it from the triplet alone.
- A program builds against `aarch64-linux-gnu-gcc` on a machine whose `sysroot` has `glibc 2.36`. It is deployed to a target whose root filesystem has `glibc 2.31`. Predict what happens, and say whether the failure is at build time or at run time.
- A colleague fixes a “missing header” error in a cross build by running `sudo apt install libfoo-dev`. Explain why this does not work, and what would.

Exercise 1.4 What leaves the building

DESIGN

You ship a product containing exactly these things:

Component	Licence	Modified by you
U-Boot	GPL-2.0-or-later	Yes, a board port
Linux kernel	GPL-2.0-only	Yes, one driver in-tree
BusyBox	GPL-2.0-only	No
<code>libcurl</code>	MIT-like (curl)	No
A logging daemon you wrote	Proprietary	n/a
A kernel module you wrote	You choose	n/a

A customer sends a written request for the source code.

- Go through the table and say, for each row, what you must publish.
- BusyBox is unmodified. Does that change the answer? Say why.
- Your logging daemon links dynamically against `libcurl` and talks to your kernel module through `ioctl`. Does either of those relationships oblige you to publish the daemon?
- What must you keep, and for how long, in order to be able to answer this request at all?

Exercise 1.5 Where the boundary falls

DESIGN

For each pair below, say whether the two things are one work or two, and name the argument that decides it. Where the answer is genuinely contested, say so rather than picking.

- a) A proprietary application and the kernel, communicating by system calls.
- b) A proprietary program statically linked against a GPL-2.0 library.
- c) A proprietary program dynamically linked against an LGPL-2.1 library.
- d) A proprietary program dynamically linked against a GPL-2.0 library.
- e) A proprietary kernel module and the kernel.
- f) A GPL program and a proprietary program on the same filesystem image, which never communicate.

Exercise 1.6 An SPDX inventory

CODE

Write a script, `lectures/L01/lab/spdx-audit.sh`, that takes a directory and reports what licences the files under it are under.

It should:

- a) Walk every `*.c` and `*.h` file under the directory it is given.
- b) For each, extract the SPDX identifier if there is one. The tag looks like `SPDX-License-Identifier: GPL-2.0-only` and appears in a comment in roughly the first few lines.
- c) Print a count per identifier, most common first.
- d) Print, separately and by name, every file that has no identifier at all. These are the interesting ones and they must not be silently folded into a total.
- e) Print the total number of files examined, so that the counts can be checked against it.

Two requirements that are easy to miss and are the point of the exercise:

- **Normalise the deprecated spellings.** SPDX renamed several identifiers; `GPL-2.0` is the old spelling of `GPL-2.0-only`, and `GPL-2.0+` is the old spelling of `GPL-2.0-or-later`. Both spellings are in the kernel tree, in quantity. A report that lists them separately is reporting two licences where there is one. Your script must report the current spelling and say how many of each it merged.
- **Do not stop at the first match in the file.** Some files carry more than one SPDX tag. Take the first one, but know that you have made that choice.

Check yourself: run it on `build/linux-6.12.30/drivers/rtc`, which has 184 `.c` and `.h` files at its top level. Your total must be 184.

Exercise 1.7 Measure the machine you built

CODE

Answer each of these with a command, run on the target after `make boot`, or on the host against `build/`. Write down the command as well as the number.

- What kernel version is running, what compiler built it, and when?
- What did the bootloader pass to the kernel on its command line?
- How large is the kernel image, and how large is the compressed `initramfs`?
- How many modules did the kernel build, and how many are installed on the target?
- How much memory does the target have, and how much of it is free once it has booted?
- What is the physical address of the `qa-dev` registers, according to `/proc/iomem`?

For f), note that nothing has claimed the region yet, because no driver for the device exists until Chapter 6. Say what you can see and what you cannot.

Exercise 1.8 An audit by hand against an audit by script

CROSS-CHECK

This is the exercise the other seven exist for. You will compute a set of numbers by hand, run your own code on the same problem, and reconcile the two. They will not agree, and the disagreement is the lesson.

- By hand, with no scripting, classify the licences of these ten files in `build/linux-6.12.30/drivers/rtc`:

```
rtc-ds1307.c  rtc-pl031.c  rtc-starfire.c  rtc-rs5c313.c  rtc-cmos.c
rtc-m41t80.c  rtc-pcf8563.c  rtc-abx80x.c  rtc-max77686.c  rtc-s35390a.c
```

For each, write down the identifier and where you found it. Do not run your script yet.

- Now predict, from that sample of ten, what the breakdown over the whole of `drivers/rtc` will be. Write down a number of distinct licences and a rough proportion for each.
- Run your script from Exercise 1.6 over the whole directory. Record the output.
- Reconcile. Specifically, answer these:
 - How many *distinct SPDX strings* did your script find, and how many *distinct licences* is that actually? What is the relationship between the two numbers?
 - Your script reports some number of untagged files. Find each one and determine its licence by another route. Say what that route was, and whether it was the same route for both.
 - Did your ten-file sample predict the whole? If your proportions were off, say which direction and why a sample drawn from files you recognised the names of would be biased.

e) A colleague runs your script, sees no Proprietary in the output, and concludes that the kernel tree contains nothing proprietary. Say what is wrong with that inference. There are at least two separate problems with it.

The Command Line, and the System Underneath It

The general Linux command line, taught as the set of questions you can ask a running kernel.

IN THIS CHAPTER

- The shell as a program: what a pipeline actually is, and what the kernel does to build one.
- The filesystem hierarchy, and why `/usr` looks the way it does for a reason nobody likes.
- Processes, signals, and the tree; what PID 1 is for and what happens if it exits.
- `/proc` and `/sys`: not directories, not files, and not the same thing as each other.
- `/dev`, device nodes, and what a major and minor number identify.
- BusyBox against `coreutils`, measured rather than asserted.
- `dmesg`, `ps`, `top` and `lsof`: the diagnostics this target actually has, and the ones it does not.
- Writing a script that reports the target's own configuration, out of `/proc` alone.

This chapter is the command line, taken as the set of questions you can ask a running kernel. It takes the shell apart as an ordinary program, down to the system calls a pipeline is built from, and follows processes through their tree, their states and the signals they are sent, to the one process that is different from all the others: PID 1, which cannot be killed and must not exit. It then opens the three directories through which a running system is inspected, `/proc`, `/sys` and `/dev`, none of which holds a file, and measures what a two-megabyte BusyBox userland gives up against the one on your laptop.

Read §2.1 first; it is the shell, the process tree and the filesystem hierarchy. §2.2 is `/proc`, `/sys` and `/dev` in detail, and §2.2.6, on reading those files from a script, is the background to the lab. The lab is specified in Exercise 2.6: a shell script that reports the machine's own configuration out of `/proc`, one key: value line per fact, checked by a test that ships with the course. The exercises in §2.4 end with the Cross-check: derive a context-switch rate by hand, have your script compute it, run the script again, and account for why the three do not agree. The lab is done when `make test L=L02` reports **PASSED**, with eighteen checks.

2.1 The Shell, the Process Tree, and Where Things Live

This section is the first half of this chapter's material: what the shell actually is, what it asks the kernel to do when you type a pipeline, how processes relate to one another, and what the filesystem hierarchy is for. The second half, the three pseudo-filesystems through which all of this is visible, is §2.2.

The framing for the whole chapter is worth stating once. **Every command you type is a question you are asking the kernel, or an instruction you are giving it.** The shell is not part of the kernel and has no special powers; it is an ordinary program that makes the same system calls your own programs make. Once that is genuinely believed rather than merely accepted, most of the command line stops needing to be memorised.

2.1.1 The shell is an ordinary program

`/bin/sh` on the target is a symlink to BusyBox, which is a normal ELF binary with no privileges of any kind. It reads a line, works out what you meant, and makes system calls. That is all.

The consequence people find surprising is that **most of what a shell appears to do is not done by the shell at all**. Wildcards are expanded by the shell before the program ever runs, so `rm *` hands `rm` a list of filenames and `rm` never sees a `*`. Redirection is done by the shell before `exec`, so a program has no idea its output is going to a file. And the program you ran received none of the syntax: by the time `main` starts, `argv` holds the expanded, redirection-free result.

There is a short list of things the shell *must* do itself, and the list is short for one reason: they change the shell's own state, and a child process cannot change its parent. `cd` is the canonical example. If `cd` were a program in `/bin`, the shell would fork, the child would change *its* directory, and the child would exit. Nothing would have happened. So `cd` is built in, and so are `export`, `exit`, `umask` and a handful of others.

```
$ type cd
cd is a shell builtin
```

2.1.2 What a pipeline actually is

`ps | grep sh | wc -l` looks like one thing. It is three processes, two pipes, and a specific sequence of system calls. This is the single most useful thing in the chapter to have drawn.

A **pipe** is a kernel object with two file descriptors: one you write to, one you read from. `pipe()` creates it. It has no name and no presence in the filesystem.

A process gets a new program by **forking** and then **exec'ing**: `fork()` produces a copy of the current process, and `exec()` replaces that copy's program while keeping its file descriptors. That last clause is what makes everything work. File descriptors survive `exec`, so the shell

can arrange a child's descriptors *before* the child becomes the program it will run, and the program starts life already plumbed.

For the two-stage pipeline `A | B`:

```

shell
|
+-- pipe()                creates fds [r, w]
|
+-- fork() -----> child A
|
|           dup2(w, 1)      stdout now writes into the pipe
|           close(r); close(w)
|           exec("A")      A starts, unaware of any of this
|
+-- fork() -----> child B
|
|           dup2(r, 0)      stdin now reads from the pipe
|           close(r); close(w)
|           exec("B")
|
+-- close(r); close(w)    the shell must close its own copies
+-- wait() for both

```

The closes are not tidiness. A pipe's read end returns end-of-file only when *every* copy of the write end is closed. If the shell keeps its copy open, B waits forever for an EOF that never comes, and the pipeline hangs. This is the most common bug in hand-written pipeline code and it is worth knowing before you write any.

Two consequences follow that explain things people find odd:

- **The stages run concurrently, not in sequence.** A is not finished when B starts. A pipeline reading a huge file starts producing output immediately.
- **The pipe has a fixed-size buffer**, 64 KB by default. When it fills, the writer blocks. That is the entire flow-control mechanism, and it is why `yes | head -1` never fills memory. What ends it is the reader leaving: once `head` exits, nothing holds the read end open, and `yes`'s next write kills it with `SIGPIPE`.

2.1.3 Exit status, and how the shell knows what happened

Every process returns one byte to its parent. Zero means success and anything else does not; `$?` holds the last one.

```

$ true; echo $?
0
$ false; echo $?
1

```

That single byte is the entire contract between a program and a script, and it is why `command && next` and `command || fallback` work. In a pipeline, `$?` is the status of the *last* stage only, which means `grep foo file | wc -l` reports success even when `grep` found nothing. This bites people writing CI scripts, and the fix is `set -o pipefail`, which this repository's own scripts use for exactly that reason.

A signal is reported differently. A process killed by signal `N` reports as `128 + N`, so a program killed by `SIGKILL` (9) gives 137. That number appears in `ci/test.sh`, which uses it to tell a target that hung from a target whose tests failed.

2.1.4 The filesystem hierarchy, honestly

The layout is conventional rather than enforced; the kernel cares about almost none of it.

Path	Holds	Notes
<code>/bin</code>	Essential user commands	On most systems now a symlink to <code>/usr/bin</code>
<code>/sbin</code>	Essential system commands	Same
<code>/lib</code>	Shared libraries and kernel modules	<code>/lib/modules/\$(uname -r)/</code>
<code>/etc</code>	Configuration, text, machine-local	Never binaries
<code>/dev</code>	Device nodes	Populated by <code>devtmpfs</code> , see §2.2.5
<code>/proc</code>	Process and kernel information	Not files, see §2.2.1
<code>/sys</code>	The driver model	Not files either, see §2.2.4
<code>/tmp</code>	Scratch, usually cleared at boot	
<code>/var</code>	State that changes: logs, spools	The part of the tree you cannot mount read-only
<code>/usr</code>	Everything not needed to boot	Historically a separate partition
<code>/root</code>	Root's home directory	Not /

The `/bin` against `/usr/bin` split is worth ninety seconds because people assume it encodes a principle. It does not. In 1971 the Unix authors filled a disk, mounted a second one at `/usr`, and moved the overflow there; the split has been rationalised ever since. Modern distributions have merged them back, and `/bin` is a symlink.

What matters for embedded is which directories must exist before `init` runs, and the answer is short: mount points for whatever you intend to mount, and the path named by `rdinit=`. Our own root filesystem creates `/proc`, `/sys`, `/dev`, `/tmp`, `/lab` and `/root` as empty directories in `ci/rootfs.sh`, and that is genuinely all that is needed. An empty directory is required because **mounting onto a path that does not exist fails**, and `/init` mounting `/proc` is one of the first things that happens.

2.1.5 Processes: the tree, and the states

Every process except PID 1 has a parent, so processes form a tree.

```
$ ps -o pid,ppid,stat,comm
PID  PPID  STAT  COMMAND
1      0   S      init
```



```
56      1 S    run.sh
70     56 R    ps
```

STAT is the state, and four of them matter:

State	Means	Consumes CPU
R	Running, or ready to run and waiting for a CPU	Yes
S	Interruptible sleep: waiting for something, killable	No
D	Uninterruptible sleep: usually waiting on I/O	No
Z	Zombie: exited, but its parent has not collected it	No

D is the one that alarms people. A process in D cannot be killed, not even with signal 9, because it is inside a system call that is not at a point where a signal can be delivered. It is almost always waiting for hardware. If it is stuck there permanently, something below it is broken, and the process is a symptom rather than the problem.

A zombie is not a leak of anything except a process ID. When a process exits, its memory is freed immediately, but its exit status has to be kept until its parent asks for it with `wait()`. That retained status is the zombie. A parent that never calls `wait()` accumulates them, and the fix is always in the parent.

An orphan is different and is not a problem. If a parent exits first, its children are re-parented to PID 1, which is expected to `wait()` for them. That is the mechanism by which daemons end up owned by `init`, and it is why PPID of 1 is normal rather than suspicious.

2.1.6 Signals

A signal is a number delivered to a process, which either has a handler for it, or gets the default behaviour.

Signal	Number	Default	Catchable
SIGINT	2	Terminate	Yes. This is Ctrl-C
SIGKILL	9	Terminate	No
SIGSEGV	11	Terminate and dump	Yes
SIGTERM	15	Terminate	Yes. The polite one, and what <code>kill</code> sends by default
SIGSTOP	19	Stop	No
SIGCHLD	17	Ignore	Yes. Sent to a parent when a child exits

SIGKILL and SIGSTOP cannot be caught, blocked or ignored, which is what makes them the last resort. Everything else can be handled, and a program that handles SIGTERM to flush its state before exiting is a program that behaves correctly when the system shuts down.

The relevance to the rest of this course is that **a driver's blocking `read()` has to cope with a signal arriving while it waits.** That is Chapter 9's subject, and the reason a user pressing Ctrl-C expects something to happen is this table.

2.1.7 PID 1, and two ways it is not an ordinary process

PID 1 is the first userspace program the kernel starts, named by `rdinit=` or `init=` on the kernel command line, or found at one of a few default paths. On our target it is `tools/target/init`, which is a shell script.

It is special in exactly two ways, and both are worth demonstrating.

It is immune to signals it has not handled. The kernel discards any signal sent to PID 1 for which PID 1 has installed no handler, including `SIGKILL`. So:

```
# kill -9 1
# echo "still here"
still here
```

Nothing happens. No error, no message, no dead system. This surprises nearly everyone, and it is the reason “just kill init” is not how you shut a machine down. The rule exists because the kernel cannot function without PID 1, so it refuses to let anything remove it by accident.

It may not exit. The one thing PID 1 must never do is return. If it does, the kernel stops immediately:

```
init: I am PID 1, and I am about to exit
[  3.183995] Kernel panic - not syncing: Attempted to kill init! exitcode=0
↳ x00000000
```

That is what a real `init`’s infinite loop is for, and it is why `tools/target/init` ends in either `exec /bin/sh` or `poweroff -f` and never simply falls off the end. **A script that runs to completion as PID 1 panics the machine**, which is a memorable way to discover you forgot the last line.

The pair together is the point. You cannot kill `init`, and `init` must not leave.

2.1.8 BusyBox, measured

The entire userland of this target is one binary. Not one binary per command; one binary:

```
# ls -l /bin/busybox
-rwxr-xr-x  1 root    root      2108608 ... /bin/busybox
# busybox --list | wc -l
402
```

402 commands in 2,108,608 bytes, statically linked, with no `libc` alongside it. Every command in `/bin` is a symlink to that one file, and BusyBox decides what to do by looking at `argv[0]`.

```
# ls -l /bin/ls
lrwxrwxrwx    1 root    root          12 ... /bin/ls -> /bin/busybox
```

Compare against a desktop: GNU coreutils alone is around 5 MB for roughly 100 commands, before libc, and a full /usr/bin is hundreds of megabytes. The ratio is not subtle.

What it costs is real and specific, and comes in three kinds.

Missing applets. Some tools simply are not there. On this target:

Tool	Present	Why
dmesg	Yes	BusyBox applet
ps, top	Yes	BusyBox applets, with fewer options than the ones you know
lsof	Yes	BusyBox applet, and much reduced: it lists fds and little else
free	Yes	BusyBox applet
strace	No	Not a BusyBox applet at all; it is a separate program
ss	No	Same
vmstat	No	Same
journalctl	No	It is systemd's, and this system has no systemd

Check for yourself with `busybox --list | grep -x vmstat`, which prints nothing.

The last two rows are two different absences and it is worth keeping them apart. `strace`, `ss` and `vmstat` are absent because nobody wrote a BusyBox applet for them and you could cross-compile and install one. `journalctl` is absent because it is part of a different init system; there is nothing to install, because there is no journal.

Reduced options. `busybox find` has no `-printf`. `busybox ps` does not take most of the flags you know. The applet is there and the flag is not, which is the more annoying failure because it shows up halfway through a script.

Different behaviour. The worst kind, because it is silent. This course has already been bitten by one: **BusyBox's `rmmmod` returns success even when the kernel refused to unload the module**, which is why Chapter 4's lab checks whether the module is still loaded instead of checking an exit status. A test written against coreutils behaviour passes on your laptop and lies on the target.

The engineering summary is that BusyBox is what you ship and coreutils is what you develop against, and the difference between them is a category of bug that only appears on the target. Test scripts on the target, not on your laptop.

2.2 /proc, /sys, and /dev

Three directories that are not what they look like. Between them they are how a running Linux system is inspected, and every diagnostic tool you will ever use is a program that reads one of them and formats the result. `top` reads /proc. `lsblk` reads /sys. Knowing that is the difference between using tools and being able to answer a question no tool answers.

Every listing in this section was produced on the target this course builds; you can reproduce each one with the command shown.

2.2.1 /proc is generated on read

Start with the demonstration, because the rest of this section follows from it:

```
# cat /proc/uptime
3.95 2.36
```

Two numbers: seconds since boot, and seconds spent idle summed across CPUs. Now the question that matters. **Where were those numbers a microsecond before you ran that command?**

Nowhere. There is no file. There is no disk. There is no buffer holding “3.95” waiting to be read. When cat opened the path, the kernel ran a function; when cat read, that function formatted a string from live kernel variables and returned it. Close the file and the string ceases to exist.

/proc is a filesystem interface to kernel data structures. It is mounted, like any filesystem:

```
# grep proc /proc/mounts
none /proc proc rw,relatime 0 0
```

and it is none because there is no device behind it. It is a filesystem in the sense that the kernel’s VFS layer can route open and read to it, and in no other sense.

Four consequences follow, and all four catch people:

- **Every file appears to have size 0.** The kernel does not know how long the answer will be until it generates it. So `ls -l /proc` is useless and `wc -c < /proc/uptime` is the only way to find out how big something is.
- **Reading twice gives different answers**, and there is no guarantee of consistency *between* two reads. This is the subject of the Cross-check, Exercise 2.9.
- **A seek regenerates it.** `lseek` from the start or from the current position works on most of it, but only by producing the text again up to the offset, so what follows the offset need not match what you read before it. A seek from the end fails with `EINVAL`, because there is no end until the text has been produced. Tools that `mmap` files do not work on `/proc`.
- **A read can be expensive.** Reading `/proc/slabinfo` walks kernel data structures under a lock. Polling a `/proc` file in a tight loop is not free, and on a busy system it is measurable.

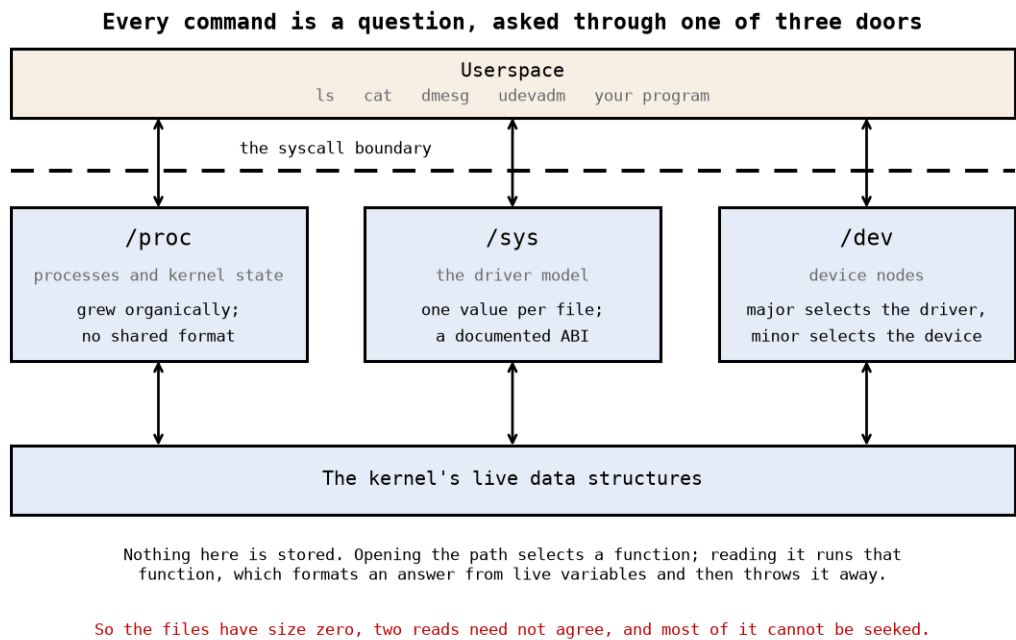


Figure 2.1 Userspace above a dashed syscall boundary, and below it three equally sized boxes for /proc, /sys and /dev, each connected by a two-way arrow to a single box holding the kernel’s live data structures. Nothing is stored: opening a path selects a function and reading it runs that function.

2.2.2 The files worth knowing

These nine answer most questions you will have about a running system.

File	Answers
/proc/version	Kernel version, compiler, build date, preemption model
/proc/cmdline	What the bootloader passed the kernel
/proc/uptime	Seconds since boot, and idle seconds
/proc/cpuinfo	One block per CPU
/proc/meminfo	Memory, in far more detail than free shows
/proc/stat	Counters since boot: CPU time, context switches, interrupts
/proc/interrupts	Interrupt counts per CPU per source. Chapter 8 lives here
/proc/devices	Which major numbers are claimed, and by what
/proc/iomem	The physical address map. Chapter 6 lives here

On the target:

```
# cat /proc/version
```

```
Linux version 6.12.30 (@f03d349e7b1d) (aarch64-linux-gnu-gcc (Debian 12.2.0-14) ↵
↵ 12.2.0,
GNU ld (GNU Binutils for Debian) 2.40) #1 SMP PREEMPT ...

# cat /proc/cmdline
console=ttyAMA0 rdinit=/init qa.test=L02
```

Three facts in the first line: the version, the compiler, and that this is an SMP PREEMPT build. That last word is a preemption model, it is CONFIG_PREEMPT, and Chapter 12 is about changing it.

The second line is the handover from bootloader to kernel. console= says where kernel messages go, rdinit= names PID 1, and qa.test=L02 is a parameter the kernel does not recognise. An unknown key=value is normally passed on to PID 1’s environment, but one with a dot in its name is taken for a parameter of a module, here one called qa, and passed to nobody. It stays in /proc/cmdline, which is where tools/target/init reads it to learn which lecture’s test to run.

/proc/stat, in detail

This is the one the lab uses.

```
# head -8 /proc/stat
cpu 16 0 341 252 0 75 25 0 0 0
cpu0 8 0 194 96 0 41 13 0 0 0
cpu1 8 0 146 156 0 34 11 0 0 0
intr 2274 0 76 635 0 0 0 3 0 0 0 ...
ctxt 1626
btime 1787646992
processes 63
procs_running 1
```

Line	Means
cpu	Aggregate CPU time, in USER_HZ units , not seconds. See below
cpu0, cpu1	The same, per CPU. There are two here because -smp 2
intr	Total interrupts, then a count per interrupt source
ctxt	Total context switches since boot. A monotonic counter
btime	Boot time, as a Unix timestamp
processes	Total processes ever forked since boot
procs_running	Processes currently in state R

The ten cpu fields are user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice. **ctxt is a counter, not a rate.** Nothing in /proc tells you context switches per second; to get a rate you read the counter twice, subtract, and divide by the elapsed time. That is what vmstat does internally, and doing it yourself is the lab.

The USER_HZ trap. The CPU times are in units of 1/100 s **regardless of CONFIG_HZ**. Our kernel has CONFIG_HZ=250, so a jiffy is 4 ms, but /proc/stat still reports in 10 ms units, because USER_HZ is fixed at 100 for ABI compatibility. Dividing by HZ instead of by 100 is a common and silent error, and it is worth 2.5x here.

2.2.3 /proc/<pid>/, and /proc/self

Every process has a directory named by its PID.

Path	Holds
cmdline	The full command line, arguments NUL-separated
comm	The short name, writable, 15 characters
status	Human-readable state, memory, UIDs, thread count
stat	The same in one machine-readable line
fd/	One symlink per open file descriptor
maps	The memory map
cwd, exe	Symlinks to the working directory and the binary

/proc/self is a magic symlink to the reading process's own directory, which is how a program finds out about itself without knowing its PID:

```
# grep -E '^(Name|State|Pid|PPid)' /proc/self/status
Name:   grep
State:  R (running)
Pid:    70
PPid:   56
```

Note that Name is grep and not the shell. The shell runs grep in a child process, and it is that child reading its own directory. This confuses people the first time and is worth reproducing.

fd/ is the most useful of these in practice. It is what lsof reads, and it will tell you which file a stuck process is holding open when nothing else will.

2.2.4 /sys, and why it is not /proc

/sys is also generated on read, so everything in §2.2.1 applies. The difference is where its contents come from and what the rules are.

/proc grew organically. It began as process information, then became the place anyone put anything, and its files have no common format. /proc/stat is a table, /proc/uptime is two floats, /proc/meminfo is key-value, and each was designed separately.

/sys is generated from the driver model. It is not written by hand at all: it is the kernel's internal tree of buses, devices, drivers and classes, exposed as directories. Its rule is **one value per file**, and that rule is the whole reason it is easy to script against.

```
# ls /sys/bus
amba      gpio      platform  spi
clockevents  i2c      pnp       spmi
clocksource mdio_bus  rpmsg     tee
container  memory   scmi_protocol virtio
cpu        memory_tiering serial     workqueue
...
```

Those are buses in the device model’s sense, which is a matching policy rather than a physical bus; platform is the one this course’s driver ends up on. Under each, devices/ and drivers/, and the match between them is what calls probe. **All of this is Chapter 10**, and the reason it appears here is that a reader should have walked the tree before being told what generates it.

The other top-level directories worth knowing now:

Path	Holds
/sys/class/	Devices grouped by what they are, regardless of bus
/sys/devices/	The real tree; everything else is symlinks into it
/sys/module/	One directory per loaded module. Chapter 4
/sys/firmware/devicetree/base/	The device tree, unpacked. Chapter 10
/sys/kernel/debug/	debugfs, if mounted. Not an ABI

The ABI distinction is the one that matters professionally. /sys is a documented, stable interface: a file that exists in one kernel version is expected to exist in the next, and Documentation/ABI/ records the promises. /sys/kernel/debug is explicitly *not*: debugfs may change or vanish between releases, and shipping a product whose userspace depends on a debugfs file is shipping a product that breaks on a kernel upgrade. Chapter 5 returns to this when deciding where a driver should expose something.

2.2.5 /dev, and the two numbers

```
# ls -l /dev/null /dev/console /dev/gpiochip0
crw-rw-rw-  1 root root    1,   3 ... /dev/null
crw-----  1 root root    5,   1 ... /dev/console
crw-----  1 root root 254,   0 ... /dev/gpiochip0
```

Where a regular file shows a size, a device node shows **two numbers**. The first character of the permissions says which kind:

- c is a **character device**: a stream of bytes, read and written in order. Serial ports, terminals, and everything this course writes.
- b is a **block device**: addressable in fixed-size blocks, cached by the kernel. Disks.

The two numbers are the **major** and **minor**. Major selects the *driver*; minor selects *which device of that driver’s*. /proc/devices lists the claimed majors:


```
# cat /proc/devices
Character devices:
 1 mem
 4 tty
 5 /dev/tty
10 misc
89 i2c
204 ttyAMA
252 rtc
254 gpiochip
...
```

So `/dev/null` at major 1 is handled by the `mem` driver, minor 3 selecting the null behaviour; `/dev/gpiochip0` at 254 is the GPIO subsystem, which Chapter 11 registers with. **A node is just a name carrying two numbers.** Nothing stops you creating one for a driver that does not exist:

```
# mknod /dev/nonsense c 42 0
# cat /dev/nonsense
cat: /dev/nonsense: No such device or address
```

The node was created happily. The open failed because no driver claims major 42. This is worth doing, because it separates two things people conflate: the node in the filesystem, and the driver registration in the kernel. **Chapter 5 creates the second, and something else creates the first.**

That something else on this target is **devtmpfs**:

```
# grep dev /proc/mounts
none /dev devtmpfs rw,relatime,size=221248k,...
```

`devtmpfs` is a filesystem the kernel populates itself: when a driver registers a device, a node appears. On a desktop, **udev** sits on top, applying naming rules, permissions and symlinks. This target has no `udev`, which is why every node is owned by root with default permissions, and why `/dev` here is shorter than the one on your laptop.

2.2.6 Reading these files from a script

The lab asks you to build a report out of `/proc`, and there are four traps.

Fields are positional and the separator is not consistent. `/proc/stat` is space-separated with a variable number of spaces after the label; `/proc/meminfo` is `key: then whitespace then value then a unit`. `awk` handles both because it splits on runs of whitespace by default:

```
ctxt=$(awk '/^ctxt/ { print $2 }' /proc/stat)
mem=$(awk '/^MemTotal:/ { print $2 }' /proc/meminfo)
```

Units are not always what they look like. `/proc/meminfo` is in kB and says so. `/proc/stat` CPU time is in `USER_HZ`, is always 1/100 s, and does *not* say so. `/proc/uptime` is in seconds with two decimals.

`/proc/cmdline` is space-separated; `/proc/<pid>/cmdline` is NUL-separated. The same-looking name, two different formats. `tr '\0' ' '` is the fix for the second.

Two reads are two moments. Anything computed from two files, or two reads of one file, is computed from a system that changed in between. For a rate this is exactly what you want, as long as you also measure the interval rather than assuming it:

```
a=$(awk '/^ctxt/ { print $2 }' /proc/stat)
t0=$(cut -d' ' -f1 /proc/uptime)
sleep 1
b=$(awk '/^ctxt/ { print $2 }' /proc/stat)
t1=$(cut -d' ' -f1 /proc/uptime)
```

Dividing `b - a` by 1 rather than by `t1 - t0` assumes `sleep 1` slept exactly one second. It did not, and the Cross-check, Exercise 2.9, is about how much that matters and why.

2.2.7 What these three do not tell you

Worth stating, because a reader who has just learned to read `/proc` tends to believe it answers everything.

- **`/proc` shows state, not history.** It cannot tell you what a process was doing a second ago, only what it is doing now. For history you need tracing, which is `ftrace` and `perf`.
- **Counters wrap and reset.** `ctxt` is 64-bit and will not wrap in practice, but plenty of counters are 32-bit, and all of them reset at boot. A rate computed across a reboot is nonsense.
- **`/proc/meminfo` does not tell you what is using memory.** It tells you how memory is classified. Attributing it to processes means walking every `/proc/<pid>/smaps`, and the numbers do not add up cleanly because pages are shared.
- **Nothing here is atomic.** There is no way to sample the whole system at one instant. Every tool that appears to, including `top`, is reading a sequence of files over a period of time.

2.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Describe what the kernel is asked to do when a shell builds a two-stage pipeline.
- Say what `/proc/uptime` is, and demonstrate that reading it opens no file on any disk.
- State the difference between `/proc` and `/sys` in terms of where their contents come from.
- Read `ls -l /dev/null` and say what each field means, including the two that are not a size.
- Find, on a running system, the kernel version, the boot command line, the CPU count, the memory size, and the number of context switches since boot.
- Say what BusyBox trades away, with a number rather than an adjective.
- Say what `kill -9 1` does, and what does end a Linux system.

Questions to test yourself

1. `cat /proc/uptime` prints two numbers. Where were those numbers a microsecond before you ran it?
2. Why can `cd` not be an ordinary program in `/bin`?
3. A file in `/sys` contains 1. What are you allowed to assume about what writing 0 to it does?
4. What is the difference between a process that is stopped, a process that is sleeping, and a process that is a zombie? Which of the three can you kill?
5. You run `kill -9 1` as root and the system carries on. Why, and what would have ended it?
6. Your target has `/dev/ttyAMA0` with major 204 and minor 64. What do those two numbers select?
7. Why does an `initramfs` need `/proc` to exist as an empty directory before anything mounts it?

Where to look

- §1.1.8 introduced four of these files; this chapter is the rest of them.
- `busybox --help` on the target lists what this particular userland can do.

Looking ahead

Chapter 3 is about:

- The kernel as a program: what it is made of and how its source is arranged.
- `Kconfig`, and what “configuring a kernel” actually chooses between.
- Why the kernel you built is 24 MB, and what that buys.

- Measuring what a single config option costs, in bytes and in modules.

This chapter was about interrogating a system somebody else configured. Chapter 3 is about becoming the person who configured it.

2.4 Exercises

Nine, ending with the Cross-check. Do them in order; the last one needs the script from Exercise 2.6.

Several have a plausible wrong answer that is worth walking into before you find it. Where an exercise can be checked mechanically, a **Check yourself** line says how.

Everything here runs on the target. Get a shell with `make boot`, and remember that Ctrl-A X quits QEMU. A few parts ask you to compare against your own machine, and say so where they do.

Solutions. The exercises that are not Code are answered in §A.2. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 2.1 What the shell does and what it does not

RECALL

For each of the following, say whether the **shell** does it or the **program** does it, and in one sentence why.

- Expanding `*.c` into a list of filenames.
- Deciding what to do when given the argument `-l`.
- Sending output to a file because the line ended in `> out.txt`.
- Changing the current directory.
- Reporting `No such file or directory` when a file is missing.
- Substituting the value of `$HOME`.

Then: `grep foo *.log` is run in a directory containing no `.log` files at all. What does `grep` receive as its arguments, and what happens?

Exercise 2.2 Pipelines

RECALL

- a) How many processes does `ps | grep sh | wc -l` create, and how many pipes?
- b) In the two-stage pipeline `A | B`, the shell closes both ends of the pipe after forking both children. Name what breaks if it does not close the *write* end, and say which process suffers.
- c) `yes | head -1` terminates promptly rather than filling memory. Explain the mechanism, in terms of what happens to the writer.
- d) `grep pattern file | wc -l` returns exit status 0 even when `grep` matched nothing. Why, and what would you add to a script to make that a failure?

Exercise 2.3 Reading /proc/stat

HAND CALCULATION

Here are two consecutive samples of `/proc/stat` from a target, taken with the given uptimes:

```
sample 1, /proc/uptime says 3.90 2.52
cpu  16 0 341 252 0 75 25 0 0 0
ctxt 1626
processes 63
procs_running 1

sample 2, /proc/uptime says 6.40 6.99
cpu  19 0 383 699 0 81 27 0 0 0
ctxt 1901
processes 71
procs_running 2
```

- a) How many context switches per second, over this interval?
- b) How many processes were created per second?
- c) The `cpu` line's fourth field is idle time. By how many units did it increase, and how many seconds is that? State the unit you used and why.
- d) There are two CPUs. Over the 2.50 s interval, how much CPU time was available in total, and how much of it was spent idle? Does your answer to c) exceed the wall-clock interval, and if so, why is that not a mistake?
- e) Which of the four numbers you have computed would be different if the kernel had been built with `CONFIG_HZ=1000` instead of 250?

Exercise 2.4 Device numbers

HAND CALCULATION

From a target's `/proc/devices` and `/dev`:

```

1 mem
4 tty
5 /dev/tty
10 misc
204 ttyAMA
254 gpiochip

crw-rw-rw-  1 root root    1,  3 /dev/null
crw-----  1 root root    5,  1 /dev/console
crw-----  1 root root  254,  0 /dev/gpiochip0
crw-rw-rw-  1 root root    1,  5 /dev/zero

```

- a) `/dev/null` and `/dev/zero` have the same major and different minors. What does that tell you, and what does it not?
- b) Which driver handles `/dev/console`?
- c) You run `mknod /dev/spare c 204 64`. Does the command succeed? Does opening the node succeed? Are those the same question?
- d) You run `mknod /dev/bogus c 42 0`. Same two questions.
- e) A colleague deletes `/dev/null` and says the null device is gone. What is actually gone, and what is not?

Exercise 2.5 Which file answers the question

DESIGN

For each question, name the single file under `/proc` or `/sys` that answers it, and say why the obvious alternative is worse.

- a) Which preemption model was this kernel built with?
- b) How many CPUs does this machine have?
- c) What did the bootloader pass to the kernel?
- d) How many times has the timer interrupt fired on CPU 1?
- e) Which physical addresses has a driver claimed?
- f) What is the name of the program running as PID 1?
- g) Which files does process 56 currently have open?

Exercise 2.6 The machine report

CODE

Write `lectures/L02/lab/machine-report.sh`, a POSIX shell script that runs on the target and prints one `key: value` line per fact, in any order. It is checked by the shipped `lectures/L02/lab/run.sh`, so the key names are a contract.

Key	Value
kernel	The kernel version alone, for example <code>6.12.30</code>
cpus	The number of CPUs
memtotal-kb	Total memory in kB, as an integer
uptime-s	Seconds since boot
cmdline	The kernel command line, verbatim
init	The name of the program running as PID 1
chardev-majors	How many character device majors are claimed
ctxt-per-s	Context switches per second , measured

Rules, all of which the test enforces:

- **Every key appears exactly once.** A report with a key twice is a report whose reader gets whichever line comes first.
- **kernel is the version alone**, not the whole of `/proc/version`.
- **chardev-majors counts character devices only.** `/proc/devices` has two sections and counting both is the plausible wrong answer.
- **ctxt-per-s is a rate, not a counter.** `/proc/stat`'s `ctxt` is a total since boot; nothing in `/proc` gives you a rate, so you must sample twice and divide by the time that actually elapsed rather than by the time you asked for.
- Use only what is on the target. In particular there is no `vmstat` and no `strace`. `awk` can do arithmetic.

Check yourself: `make build L=L02` does nothing for this lab, since there is no module. Run `make test L=L02`; before you have written the script it reports `SKIPPED`, and afterwards it should report `PASSED` with eighteen checks.

Exercise 2.7 PID 1 is not an ordinary process

CODE

Two short experiments on the target. Predict the outcome of each **before** running it, and write your prediction down.

- Run `kill -9 1`. What did you predict, what happened, and what does that tell you about the protection PID 1 has?
- Now the other half. You cannot easily make the real `init` exit, so reason it out instead: `tools/target/init` ends with either `exec /bin/sh` or `poweroff -f`. What would happen if you deleted that last line and the script simply ran off the end? Name the exact kernel message.

c) A daemon's parent exits while the daemon is still running. What is the daemon's PPID afterwards, and who is now responsible for calling `wait()` on it?

Exercise 2.8 What a two-megabyte userland costs you

DESIGN

- a) On the target, find the size of `/bin/busybox` and the number of applets it provides. Then work out the average bytes per command, and say why that number is misleading.
- b) Check whether each of `strace`, `ss`, `vmstat`, `lsof` and `journalctl` is present. For each absent one, say which of two reasons applies: *no BusyBox applet exists*, or *the subsystem it belongs to is not on this system*.
- c) You need to find out which files a stuck process has open, on a target with no `lsof` at all. What do you do instead?
- d) Name a class of bug that only appears on the target and never on your development machine, and give the concrete example this course has already hit.

Exercise 2.9 A rate measured three ways

CROSS-CHECK

The signature exercise. You will compute a number by hand, have your script compute it, and then run the script repeatedly, and the three will not agree. The disagreement is the exercise.

- a) **By hand.** On the target, run this twice with a pause you time yourself, using a watch or a phone rather than the machine:

```
cat /proc/uptime; awk '/^ctxt/ { print $2 }' /proc/stat
```

Compute context switches per second from your two readings. Write the number down.

- b) **By script.** Run your `machine-report.sh` from Exercise 2.6 and record its `ctxt-per-s`.
- c) **Three more times.** Run the script three times in a row and record all three values.
- d) **Reconcile.** Answer each of these:
 - By how much do your three script runs differ from each other, as a percentage of the largest? Is `ctxt-per-s` a property of the machine, or a property of the moment you measured it?
 - Your script prints a value with a decimal point. How many of those digits are meaningful? What would be an honest way to present this number?
 - Compare the hand figure to the script figure. Your hand interval was several seconds and the script's was about one. Which would you expect to be steadier, and did that hold?

- Now remove the `sleep` from your script so the two samples are taken back to back, and run it again. The measured interval is roughly 0.14 s. What does the rate do, and why is a short interval worse rather than better, given that it is closer to “now”?
- e) You would like to check your answer against `vmstat`, which reports exactly this number. Try it. Then say what happened, what it tells you about this target, and how you would check the answer without it.
- f) A colleague reads your report, sees `ctxt-per-s: 73.3`, and puts it in a datasheet as the machine’s context-switch rate. Give two separate reasons that is wrong.

The Kernel: Architecture, Source Tree, and Configuration

What the kernel is as a program, how its source is arranged, and how you choose what goes into the one you build.

IN THIS CHAPTER

- The kernel as a program: one address space, no libc, and the only public interface it has.
- The subsystem map, and where in the tree each of them lives.
- Mainline, stable, long-term stable, and the vendor tree your SoC actually shipped with.
- Kconfig: what a symbol is, what `select` does that `depends on` does not, and `=y` against `=m`.
- Kbuild: `ARCH`, `CROSS_COMPILE`, `O=`, and what Image is that `vmlinux` is not.
- Turn one config option off and measure what it was worth.

This chapter is about the kernel as a program and as a source tree, and about how you choose what goes into the one you build. It explains why a program with no `main` and one shared address space fails differently from a process, why the system call interface never changes while everything behind it may, and how twenty-four million lines of C are arranged so that you can find the one file you need. Then it turns to configuration: what a Kconfig symbol is, why `select` and `depends on` behave oppositely, when a driver has to be built in rather than as a module, and how a `.config` becomes an Image.

Read §3.1 first; it is the kernel as a program, and its source tree. §3.2 is Kconfig and Kbuild, and ends with the method for predicting what one option costs. The lab is specified in Exercise 3.6: a shell script that reports what the running kernel was configured with, and admits it when the configuration is not there to read. The exercises in §3.4 end with the Cross-check: predict what `CONFIG_IKCONFIG` costs from the object it builds, remove it, rebuild, and measure three artefacts that give three different answers, all of them correct. Start it early, because the rebuild takes about twenty minutes. The lab is done when `make test L=L03` reports **PASSED** with seventeen checks, and still does once the Cross-check has turned `/proc/config.gz` off.

3.1 The Kernel as a Program, and Its Source Tree

This section is the first half of the chapter: what kind of program the kernel is, what its one public interface is and what follows from that, and how twenty-four million lines of C are arranged so that you can find things in them. The configuration half is §3.2.

3.1.1 What kind of program the kernel is

You have written programs. The kernel is a program, and almost every assumption you carry from the first to the second is wrong. Five differences matter.

It has no main and never returns. It has an entry point that the bootloader jumps to, it brings itself up, and then it stops being a thing that is “running” at all. After boot the kernel is a body of code that gets entered: by a system call, by an interrupt, by a fault. Between those it is not executing. Chapter 4’s modules are the same shape, which is why `module_init` returning does not mean the module is finished.

It is one address space, shared by everything in it. There is no isolation between subsystems. A null pointer dereference in a sound driver corrupts the same address space the filesystem code is using. This is what “monolithic” means, and it is worth being precise: **monolithic is a claim about the address space, not about modularity.** The kernel is intensely modular in its source organisation; it just does not enforce any of those boundaries at run time.

It cannot fault the way a process can. A userspace segfault kills one process and the system carries on, because the kernel is there to clean up. When the kernel dereferences a bad pointer there is nobody above it. What happens instead is an **oops**: the kernel prints a register dump and a backtrace and tries to kill the offending task. If it cannot, because the fault was in something no task owns, it panics.

It has no C library. Compiled `-nostdinc`, `-ffreestanding`, against its own headers. There is no `printf`, no `malloc`, and no floating point. Chapter 4 covers the consequences; the point here is that “the toolchain” and “the kernel” are two of Chapter 1’s four pieces and not one, and this is why.

Its stack is small and fixed. 16 KB on arm64, for the entire call chain, and nothing grows it. Run off the end and the best you can hope for is a guard page and a panic.

3.1.2 The one public interface, and the one that is not

This is the most consequential fact in the chapter, and it is two facts that people routinely merge into one wrong one.

The system call interface is stable forever. A binary from 1998 still runs. Linus enforces this with unusual ferocity, and the rule is short: *we do not break userspace*. If a change makes a working program stop working, the change is reverted, however correct it was.

The internal API has no stability guarantee whatsoever. Function signatures change, structures gain and lose fields, whole subsystems are replaced. Between two point releases, code inside the kernel may be rearranged freely.

Both are deliberate, and together they explain almost everything about how kernel development works:

- **An in-tree driver is maintained for you.** When somebody changes an internal API they update every caller in the tree, including your driver, because the tree must always build.
- **An out-of-tree driver is maintained by you, forever.** Nobody changing an API knows your driver exists. This is the real cost of staying out of tree, and it is a much larger cost than the licensing argument in Chapter 1. `vermagic` and `CONFIG_MODVERSIONS` in Chapter 4 are the mechanisms that turn the resulting mismatch into an error message rather than into corruption.
- **“Why is there no stable driver ABI” is a settled question**, and the documented answer is in `Documentation/process/stable-api-nonsense.rst`. The short version is that a stable internal ABI would freeze design mistakes permanently, and the project has decided it would rather move the callers.

3.1.3 The subsystem map

Roughly, the kernel is:

Subsystem	Responsible for	Source
Process management	Creating, scheduling and destroying tasks	<code>kernel/</code>
Memory management	Address spaces, page tables, allocation	<code>mm/</code>
Virtual filesystem	The common file interface over many filesystems	<code>fs/</code>
Networking	Sockets, protocols, packet handling	<code>net/</code>
Device drivers	Everything that talks to hardware	<code>drivers/</code>
Architecture	The parts that differ per CPU family	<code>arch/</code>

Userspace reaches all of it through system calls. Drivers reach hardware directly. The layer that matters most to this course is the one that is not in the table: **the driver model**, which is how a driver is matched to a device, and which lives partly in `drivers/base/` and partly in every bus. Chapter 10 is about it.

3.1.4 The tree, by the numbers

The pinned source for this course is Linux 6.12.30. Measured:

	Count
Files	86,660
C and header files	59,998
Lines of C	24,631,623
Lines of C and headers	34,693,754

Twenty-four million lines is not a number to be impressed by; it is a number to have a strategy about. The strategy is that **you never read the kernel, you read one file in it**, and the tree is arranged so that you can find that file.

Where the bulk actually is:

Directory	Files	Size	What is in it
drivers/	34,970	1.1 GB	Device drivers. Two thirds of everything
arch/	17,417	153 MB	One subdirectory per CPU architecture
Documentation/	10,123	74 MB	Genuinely worth reading, and usually current
tools/	7,700	85 MB	Userspace tools shipped with the kernel
include/	6,206	56 MB	Headers, including the ones a module compiles against
sound/	2,781	52 MB	ALSA
fs/	2,480	51 MB	Filesystems, one subdirectory each
net/	1,992	38 MB	The network stack
kernel/	592	15 MB	Scheduler, timers, modules, tracing
mm/	196	5.8 MB	Memory management

Two observations are worth making.

drivers/ is two thirds of the kernel, and within it, drivers/gpu alone is 553 MB, half of drivers/ and a third of the whole tree. A course about embedded Linux will never open that directory. This is why arm64 defconfig takes an hour to build, and why kernel/trim.config exists.

arch/ is where portability lives, and where it stops. arch/arm64/ holds the entry code, the page table format, the memory model and the syscall table. Everything above it is written once. When something behaves differently on your target than on your laptop, arch/ is usually why.

3.1.5 Finding things

Four techniques, in the order you should reach for them.

Know the layout. A driver for hardware of type X is in drivers/X/. A filesystem is in fs/<name>/. An architecture detail is in arch/<arch>/. This answers most questions with no searching at all.

Search for the string the user sees. If dmesg printed something, that string is in the tree:

```
grep -rn "loading out-of-tree module taints kernel" kernel/
```

This is the single most effective kernel debugging technique there is, and it works because the kernel prints in English and does not localise.

Search for the interface, not the implementation. To find who implements something, find who registers it. `grep -rl "struct platform_driver" drivers/rtc/` finds every RTC platform driver. Searching for `platform_driver_register` finds three, because the rest register through the `module_platform_driver()` macro and never spell the function out.

Read Documentation/. It is unusually good and unusually current, and Documentation/driver-api/ in particular is the reference for most of what this course teaches.

For reading rather than searching, Bootlin's Elixir¹ cross-references every identifier in the tree, in a browser, without cloning anything.

3.1.6 Reading one real driver

The shape of a Linux driver is worth meeting before you write one. `drivers/rtc/rtc-digicolor.c` is **224 lines**, drives the real-time clock on an SoC none of us will ever own, and is a good first driver to read for exactly that reason: there is nothing special about it. It is the ordinary shape.

Its skeleton, with the real line numbers from the pinned tree:

```
static int __init dc_rtc_probe(struct platform_device *pdev)    /* 176: found my↵
↳ hardware */
{
    rtc->regs = devm_platform_ioremap_resource(pdev, 0);        /* 185: map ↵
↳ registers */
    rtc->rtc_dev = devm_rtc_allocate_device(&pdev->dev);         /* 189: join a ↵
↳ framework */
    irq = platform_get_irq(pdev, 0);                            /* 193: which ↵
↳ interrupt? */
    ret = devm_request_irq(&pdev->dev, irq, dc_rtc_irq, 0,       /* 196: take it ↵
↳ */
                          pdev->name, rtc);
    return devm_rtc_register_device(rtc->rtc_dev);             /* 205: go live ↵
↳ */
}

static const __maybe_unused struct of_device_id dc_dt_ids[] = { /* 208: what I ↵
↳ handle */
    { .compatible = "cnxt,cx92755-rtc" },
    { }
};

static struct platform_driver dc_rtc_driver = {
    .driver = {
        .of_match_table = of_match_ptr(dc_dt_ids),            /* 217: how I am↵
↳ matched */
    },
};
```

¹<https://elixir.bootlin.com/linux/latest/source>

```
module_platform_driver_probe(dc_rtc_driver, dc_rtc_probe); /* 220
    */
```

Every line of that maps onto a later chapter, in order. `devm_` and `ioremap` are Chapter 6. `platform_get_irq` and `devm_request_irq` are Chapter 8. `of_match_table` and `compatible` are Chapter 10. `devm_rtc_register_device` is Chapter 11. There is no `read()` and no `/dev` node written by hand, and that absence is Chapter 11’s argument.

Notice what is *not* in it: no addresses, no interrupt numbers, no unwind path. The addresses come from the device tree, and the `devm_` prefix means the cleanup is registered rather than written. A driver written the way Chapter 5 teaches would be half again as long and would have a ladder of `goto` labels at the bottom. Both of those disappearances are deliberate and both are later chapters.

The RTC actually on this target is a different one, `drivers/rtc/rtc-pl031.c`, 470 lines. It is worth knowing that it is matched by a different mechanism: it is an ARM PrimeCell, so it is identified by ID registers in the hardware itself rather than by a compatible string, and it uses `amba_driver` rather than `platform_driver`. Bus-specific matching is Chapter 10’s subject; the reason to mention it now is that if you go looking at the driver for the clock in your own machine, you will find it does not look like the skeleton above, and that is not because one of them is wrong.

You can see it running:

```
# dmesg | grep pl031
rtc-pl031 9010000.pl031: registered as rtc0
# grep pl031 /proc/interrupts
15:          0          0 GIC-0 34 Level      rtc-pl031
```

Zero interrupts on both CPUs, because nothing has set an alarm. Chapter 8 is where that column starts moving.

3.1.7 Versions, and which tree you are on

Tree	Meaning	Lifetime
Mainline	Linus’s tree. New features land here first	Continuous
-stable	Backported fixes for the current release	~2 months
LTS	A release designated long-term stable	2 to 6 years
Vendor	An SoC vendor’s fork of some older release	Until they stop

This course pins 6.12.30, which is an LTS release. The `.30` is the stable increment: 6.12 was the feature release, and 6.12.30 is that plus thirty rounds of backported fixes and no new features. Stable releases take only fixes, judged against documented rules in `Documentation/process/stable-kernel-rules.rst`, of which the important one is that a fix must already be in mainline before it can be backported.

The vendor tree is where embedded work actually happens, and it is the source of most of the pain. A typical SoC ships with a vendor kernel that is a fork of a release three or four years old, with several thousand out-of-tree patches. It boots your board and mainline does not. What follows from that:

- A fix from mainline may not apply, because the code around it has moved.
- An upgrade is a port, not an update.
- Everything you write against vendor-specific APIs is stranded when you do upgrade.

The engineering response is to push whatever you can upstream and to prefer mainline interfaces even when the vendor offers a shortcut. This course teaches mainline interfaces throughout, and that is the reason.

3.1.8 What “24 MB” is made of

Chapter 1 measured the kernel this course builds at **about 24 MB** and left the size unexplained. The explanation belongs here.

vmlinux is the linked kernel as an ELF object: **31,468,440 bytes**. Image is that stripped of ELF structure and laid out for the bootloader to load: **24,943,104 bytes**. The difference is symbol tables and ELF metadata that the boot process does not need.

Inside it, the largest contributors on this build:

Section	Size	What it is
.text	13,021,184	Executable code
.rodata	4,369,606	Constants, string literals, tables
.rela.dyn	3,341,232	Relocations, applied once at boot and then freed
.init.*	1,029,519	Code and data used once at boot and then freed

The last two rows are worth a moment, because you can watch them happen:

```
[ 5.001253] Freeing unused kernel memory: 4672K
```

Every function marked `__init` and every variable marked `__initdata` is placed in a section that is discarded once boot is over, and on arm64 so is the table of relocations the kernel applies to itself when it boots at a randomised address. On this build that is **4,672 KB returned to the system**, the whole stretch from `__init_begin` to `__init_end`, which is more than the entire BusyBox userland. Chapter 4 uses the same annotations.

And the reason it is 24 MB rather than 3 MB is configuration. Nothing in the size is mysterious: it is the sum of what was compiled in, and what was compiled in was chosen by a `.config` with **5,725 lines** in it. §3.2 is about that file, and the Cross-check is about learning to predict what one line of it is worth.

3.2 Kconfig and Kbuild

Configuring a kernel is choosing, from about eighteen thousand options, the few thousand that describe your machine. This section is how that choice is expressed, how it is enforced, and how the result becomes a kernel. It ends with what a single option costs, which is the Cross-check, Exercise 3.9.

3.2.1 What a configuration is

A `.config` is a plain text file of assignments. The one this course builds has **5,725 lines**:

```
CONFIG_ARM64=y
CONFIG_HZ=250
CONFIG_MODULES=y
CONFIG_IIO=m
# CONFIG_BTRFS_FS is not set
```

Three forms appear, and the third is not the same as absence:

Form	Means
CONFIG_X=y	Built into the kernel image
CONFIG_X=m	Built as a loadable module
# CONFIG_X is not set	Explicitly off
(absent entirely)	The symbol does not exist in this tree, or its dependencies are unmet so it was never offered

That last row is the one that costs people an afternoon. If you add `CONFIG_F00=y` to a fragment and it does not appear in the final `.config`, the symbol was not *rejected*; it was never *asked about*, because something it depends on is off. This course’s `ci/kernel.sh` checks for exactly this after merging and warns by name, because the failure is otherwise silent.

The `.config` is generated, not authored. It is produced by the Kconfig tools from the Kconfig files scattered through the tree, and hand-editing it is a mistake: set one symbol without its dependencies and the next make will either revert it or produce a tree that does not build.

3.2.2 Kconfig: the language

Every directory that offers options has a Kconfig file. An entry looks like this, from `drivers/gpio/Kconfig`:

```
config GPIO_SYSFS
    bool "/sys/class/gpio/... (sysfs interface)" if EXPERT
    depends on SYSFS
```

```
select GPIO_CDEV # We need to encourage the new ABI
help
    Say Y here to add the legacy sysfs interface for GPIOs.
```

Line by line, because all five lines do something:

Line	Effect
config GPIO_SYSFS	Declares the symbol, which becomes CONFIG_GPIO_SYSFS
bool "..."	Its type, and the prompt shown in menuconfig
if EXPERT	The <i>prompt</i> only appears when EXPERT is on
depends on SYSFS	The symbol cannot be enabled unless SYSFS is
select GPIO_CDEV	Turning this on turns GPIO_CDEV on too
help	The text menuconfig shows when you press ?

Types are bool (y or n), tristate (y, m or n), int, hex and string. Only tristate symbols can be modules, which is why CONFIG_GPIO_LIB cannot be `=m` however much you would like it to be.

depends on against select

These two are constantly confused and behave oppositely.

depends on B means “do not offer me unless B is on”. It is a precondition. If B is off, the symbol is hidden and cannot be set.

select B means “if I am on, force B on”. It is an imperative, and critically **it does not check B’s own dependencies**. You can select a symbol into a state its own `depends on` line forbids, producing a `.config` that is internally inconsistent and a build that fails somewhere unrelated.

This is why kernel maintainers discourage `select` for anything with dependencies of its own, and why Documentation/kbuild/kconfig-language.rst says it should be used only for non-visible symbols. It is also the answer to “how did an option I never chose end up in my `.config`”: something selected it.

GPIO_SYSFS above is a live example of both. Adding `CONFIG_GPIO_SYSFS=y` to a fragment in this course **does not work**, because its prompt is behind `if EXPERT` and `EXPERT` is off, so the symbol is never offered. That is not hypothetical; it happened while this course was being written, `ci/kernel.sh` reported `CONFIG_GPIO_SYSFS=y` did not survive `olddefconfig`, and the fix was to use `CONFIG_GPIO_CDEV` instead, which is the modern interface and did not need `EXPERT` at all.

3.2.3 The tools

Command	Does
<code>make defconfig</code>	The architecture's default configuration
<code>make menuconfig</code>	Interactive, with search: press /
<code>make olddefconfig</code>	Take the existing <code>.config</code> , answer new questions with defaults
<code>make savedefconfig</code>	Write a <i>minimal</i> config: only what differs from the defaults
<code>make allnoconfig</code>	Everything off that can be
<code>make tinyconfig</code>	The smallest kernel that builds

Two of these matter more than the others for real work.

olddefconfig is what you run after changing anything. It resolves the consequences of your change: symbols that are now offered get their defaults, symbols that are now impossible get removed. Every merge in this course is followed by one.

savedefconfig is how you store a configuration. A full `.config` is 5,725 lines, of which perhaps a hundred are choices anyone made; the rest are defaults. `savedefconfig` strips it to the difference, which is what `arch/arm64/configs/*_defconfig` files are and why they are readable. Committing a full `.config` to a product repository is a mistake for the same reason as committing generated code.

Config fragments

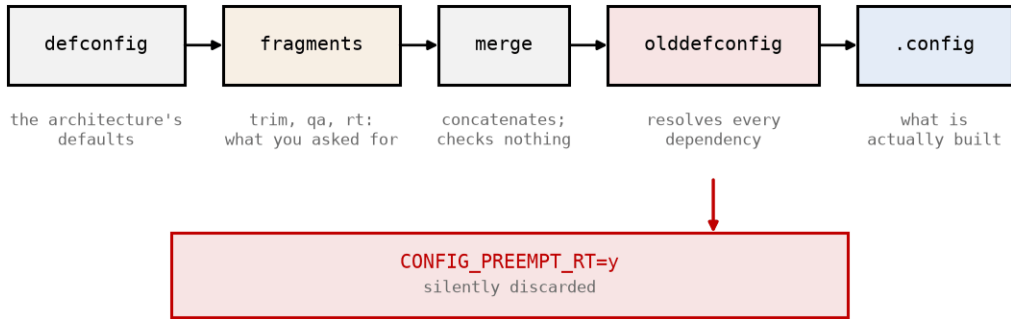
A fragment is a partial `.config` merged over a base:

```
scripts/kconfig/merge_config.sh -m -O $OUT $OUT/.config frag1.config frag2.config
make olddefconfig
```

This is how this course configures its kernel, and the reason is maintainability: the base is `defconfig`, and the course's own choices are three small files where each line can carry a comment saying why it is there. `kernel/qa.config` names the lecture that wanted each option; `kernel/trim.config` says what each removal was for. A 5,725-line `.config` can say neither.

Later fragments override earlier ones, which is why `kernel/local.config` is merged last: it is yours, and it wins.

A fragment is a request, and olddefconfig decides



It depends on EXPERT, and EXPERT is off. A symbol whose dependencies are unmet has no prompt; a symbol with no prompt cannot be set, and takes its default instead.

The kernel then builds cleanly, boots, and reports PREEMPT. Nothing failed, so nothing was reported: this is why `ci/kernel.sh` re-reads every fragment after the merge.

Figure 3.1 Five boxes in a row: `defconfig`, `fragments`, `merge`, `olddefconfig` and `.config`. A red arrow drops from `olddefconfig` to a callout reading `CONFIG_PREEMPT_RT=y`, silently discarded, because it depends on EXPERT and a symbol whose dependencies are unmet has no prompt.

3.2.4 Kbuild: turning a configuration into a kernel

The build reads the same `CONFIG_` symbols, from two directions.

In **Makefiles**, the symbol chooses whether an object is built and how:

```
obj-$(CONFIG_RTC_DRV_PL031) += rtc-pl031.o
```

`obj-y` is built into the kernel, `obj-m` becomes a module, and when the symbol is not set the line adds to `obj-`, which nothing reads. So that one line expands to one of three things depending on a symbol, and this is the entire mechanism by which a configuration selects code.

In **C**, the same symbols are available as macros, generated into `include/generated/autoconf.h`:

```
#ifndef CONFIG_SMP
/* ... */
#endif

if (IS_ENABLED(CONFIG_IIO)) { ... } /* true for =y and =m */
```

IS_ENABLED is preferred over #ifdef where possible, because the code inside it is still compiled and type-checked even when the option is off, and then discarded by the optimiser. Code behind #ifdef rots, because nobody builds it.

The variables that matter

Variable	Meaning
ARCH	Target architecture; selects arch/\${ARCH}/
CROSS_COMPILE	Toolchain prefix, for example aarch64-linux-gnu-
O=	Build output directory, keeping the source tree clean
M=	Build an out-of-tree module in this directory (Chapter 4)
-j	Parallel jobs

O= is worth insisting on. It puts every generated file in one directory, so two configurations can share one source tree; this course builds build/kernel and build/kernel-rt from the same build/linux-6.12.30, which is what lets Chapter 12 boot one and then the other without a second download.

What comes out

Artefact	What it is
vmlinux	The linked kernel as an ELF object, with symbols. 31,468,440 bytes
arch/arm64/boot/Image	The same, stripped and laid out for a bootloader. 24,943,104 bytes
*.ko	Loadable modules, scattered through the tree
System.map	Symbol addresses, for decoding an oops
*.dtb	Compiled device trees, when the platform has them

The bootloader loads Image, not vmlinux. vmlinux is what you point a debugger at.

make modules_install INSTALL_MOD_PATH=<dir> stages the modules into a directory tree and runs depmod to generate modules.dep, which is what modprobe reads. This course stages into build/kernel/modules-install and then copies a curated handful into the initramfs; **458 modules are built and 3 are installed**, because the target can use three of them.

3.2.5 Built in or module

The choice is per-symbol and it is not free either way.

	=y built in	=m module
Available at	Boot, immediately	After the root filesystem is mounted
Costs image size	Always	Only when loaded
Can be updated	No, rebuild the kernel	Yes, replace the .ko
Can be unloaded	No	Yes
Boot time	Included	Deferred

The rule that follows from row one is absolute: anything needed to reach the root filesystem cannot be a module, because the modules live on the root filesystem. The driver for your storage controller, its bus, and the filesystem type must all be `=y`, or must be in an `initramfs`, which is a root filesystem in RAM that exists precisely to break this circularity.

Everything this target uses is built in except three modules, and the target's root filesystem is an `initramfs`, so the circularity never arises. On a product with real storage it is the first thing to get right.

3.2.6 What one option costs

The Cross-check asks you to predict this, so the method matters more than the answer.

Step 1: find the objects. An option builds one or more objects. `CONFIG_IKCONFIG` builds `kernel/configs.o`, which embeds a compressed copy of the `.config` so the running kernel can be asked what it was built with.

Step 2: measure them. `size -A` gives the per-section sizes:

```
$ size -A build/kernel/kernel/configs.o
section              size  addr
.text                 76     0
.data                 0     0
.bss                  0     0
.rodata              34048    0
.initcall6.init        4     0
.rodata.str1.8         10    0
.exit.text            40     0
.init.text           108     0
__patchable_function_entries 16    0
.discard.addressable   8     0
.exitcall.exit         8     0
.modinfo             150     0
.comment              32     0
.note.gnu.stack        0     0
.note.gnu.property     32     0
Total                34532
```

Four of these are the option's content: its code (`.text`), its read-only data and strings (`.rodata`, `.rodata.str1.8`), and the 4-byte entry that has its init function called at boot (`.initcall6.init`). That is **34,138 bytes**, and nearly all of the 34,048 is the compressed configuration itself. The rest is 148 bytes of `__init` and `__exit` code, a few bookkeeping entries of 8 to 16 bytes, and sections that describe the object rather than add to the kernel, such as `.modinfo` and `.comment`; Total counts all of them. You can check the 34,048 independently two ways: `gzip -9 -c build/kernel/.config | wc -c` gives 33,944, and on the running

target `ls -l /proc/config.gz` shows 33,936. Three routes, agreeing to within a fraction of a percent.

Step 3: change one line and rebuild.

```
echo '# CONFIG_IKCONFIG is not set' > kernel/local.config
make kernel
```

Step 4: measure the result, and be specific about what you measured. This is where the exercise turns, because there are three artefacts and they give three different answers. Working that out is Exercise 3.9.

The general lesson, which applies well beyond this option: **“how big is it” is not a question with one answer.** Content, sections and image are three different measurements of the same change, they differ by roughly a factor of two here, and quoting one without saying which is how size budgets go wrong.

3.2.7 Where this course’s configuration comes from

Reading the three fragments is the fastest way to see the whole mechanism in use.

File	Purpose
kernel/trim.config	Removes what the emulated machine cannot have
kernel/qa.config	Adds what the course needs, naming the lecture per option
kernel/rt.config	PREEMPT_RT, for Chapter 12. Four lines
kernel/local.config	Yours. Gitignored, merged last, and the lab

`trim.config` is the interesting one for this chapter. `arm64 defconfig` is not a minimal configuration; it is the union of every `arm64` machine anyone has upstreamed, and it enables **44 SoC platform families**, compiling clock, pinctrl, DMA and PHY drivers for forty-four vendors’ silicon. QEMU’s `virt` machine is none of them.

Turning those off is what took this course’s build from **about an hour and roughly two thousand modules to eighteen minutes and 458**. That is one fragment, and its effect is larger than every other configuration decision in the course put together. It is also the answer to why the Image is 24 MB and not 3 MB: it is still not a minimal kernel, and the exercises ask you to find out how much further it could go.

3.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Say what “monolithic kernel” claims, and what it does not claim.
- Explain why the syscall interface is stable and the internal API is not, and what follows for a driver you maintain out of tree.
- Navigate to a named subsystem in the source tree without searching.
- Read a Kconfig entry and say under what circumstances the option will and will not appear.
- Say what select does that depends on does not, and why that can produce a broken .config.
- Choose `=y` or `=m` for a given driver and defend the choice.
- Cross-build a kernel with a config fragment, and say where each of Image, vmlinux and the modules ends up.
- Measure what a config option costs, in image bytes and in modules.

Questions to test yourself

1. Why can the driver for the device holding your root filesystem not be a module?
2. Your .config contains an option you never selected and whose depends on line is not satisfied. How did it get there?
3. What is the difference between vmlinux and Image, and which one does the bootloader load?
4. `make defconfig for arm64` produces a configuration for how many machines? What does that tell you about how much of your build you are going to throw away?
5. You backport a fix from mainline into a vendor tree that is three years old and it does not apply. What are your options, in order of increasing cost?
6. Why does the kernel build its own headers rather than using the ones in `/usr/include`?
7. You remove a config option contributing 34 KB and the Image shrinks by 64 KB. Where did the other 30 KB come from, and would removing a second 34 KB option shrink it by another 64 KB?

Where to look

- `kernel/trim.config` is where this course’s build lost most of its time and size, with a comment on every line saying what it removed and why. It is worth reading before the lab.

Looking ahead

Chapter 4 is about:

- The smallest thing you can write for a kernel, and how to load it into a running one.
- `module_init`, and what “init” means for code with no `main`.
- Why the kernel refuses to load some modules, and what a licence has to do with it.
- The C you are allowed to write up here, which is less than you are used to.

This chapter was about the kernel somebody else wrote. Chapter 4 is the first where you add to it.

3.4 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 3.9 needs a kernel build and is the one to start early, because it takes about twenty minutes of waiting.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

The kernel source is in `build/linux-6.12.30` and the built tree in `build/kernel`; both are there after `make kernel`. Nothing here needs the network.

Solutions. The exercises that are not Code are answered in §A.3. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 3.1 What kind of program the kernel is

RECALL

- Name three things that are true of a userspace program and false of the kernel.
- “Linux is a monolithic kernel.” What exactly does that claim, and what does it **not** claim? In particular, does it say anything about how the source is organised?
- A userspace program dereferences a null pointer and the system carries on. The kernel dereferences a null pointer. Describe what happens instead, and say why the difference is structural rather than a matter of the kernel being better tested.
- Why is there no `printf` in the kernel, given that there is obviously a way to print things?

Exercise 3.2 The two interfaces

RECALL

- a) State the stability guarantee on the system call interface, and the stability guarantee on the kernel's internal API.
- b) Those two answers are opposites. For each, give the practical consequence for someone maintaining an out-of-tree driver.
- c) A colleague proposes that Linux should have a stable driver ABI so that vendors can ship binary drivers. Give the strongest argument against, and name the file in the tree that makes it.
- d) Your driver is in the mainline tree. An internal API it uses changes. Who updates your driver, and why?

Exercise 3.3 Reading Kconfig

HAND CALCULATION

From drivers/gpio/Kconfig in the pinned tree:

```
config GPIO_SYSFS
    bool "/sys/class/gpio/... (sysfs interface)" if EXPERT
    depends on SYSFS
    select GPIO_CDEV # We need to encourage the new ABI

config GPIO_CDEV
    bool
    prompt "Character device (/dev/gpiochipN) support" if EXPERT
    default y
```

- a) Your .config has CONFIG_SYSFS=y and # CONFIG_EXPERT is not set. You add CONFIG_GPIO_SYSFS=y to a fragment and run olddefconfig. What is in the final .config, and why?
- b) Same question with CONFIG_EXPERT=y.
- c) GPIO_CDEV has default y and a prompt behind EXPERT. With EXPERT off, is CONFIG_GPIO_CDEV set? Explain how a symbol nobody was asked about gets a value.
- d) Explain, in one sentence each, what depends on does and what select does. Then say which of the two can produce a .config that violates a depends on line somewhere else, and how.
- e) You want /dev/gpiochipN on a kernel with EXPERT off. Which symbol do you set, and does it work?

Exercise 3.4 Built in or module

HAND CALCULATION

For each, say whether the option must be `=y`, may be `=m`, or cannot be `=m` at all, and give the reason in one sentence.

- a) The driver for the SATA controller holding the root filesystem, on a system with no `initramfs`.
- b) The same, on a system with an `initramfs` containing that driver.
- c) `CONFIG_GPIOLIB`, which is declared `bool` rather than `tristate`.
- d) A driver for a USB device the user may or may not plug in.
- e) The filesystem type of the root filesystem, with no `initramfs`.
- f) `CONFIG_PREEMPT_RT`.

Exercise 3.5 Navigating the tree

DESIGN

For each, say which directory you would look in **first**, and give the search you would run if that did not find it.

- a) The code that decides which task runs next.
- b) The driver for an I2C temperature sensor.
- c) The definition of `struct file_operations`.
- d) The arm64 exception entry path.
- e) The implementation of the `openat` system call.
- f) The kernel message `Attempted to kill init!`.

For f), actually run your search on the pinned tree and give the file and line.

Exercise 3.6 The kernel report

CODE

Write `lectures/L03/lab/kernel-report.sh`, a POSIX shell script that runs on the target and reports what the running kernel was configured with. It is checked by the shipped `lectures/L03/lab/run.sh`, so the key names are a contract.

Key	Value
version	The kernel version alone, for example 6.12.30
smp	yes or no
preempt	One of none, preempt, preempt_dynamic, preempt_rt
hz	The value of CONFIG_HZ, or unknown (see below)
modules	yes or no: can this kernel load modules at all
filesystems	How many filesystem types the kernel knows
char-majors	How many character device majors are claimed
config-source	config.gz or inferred (see below)

The point of the exercise is the last two rows. Some kernels are built with `CONFIG_IKCONFIG_PROC`, which embeds the configuration and exposes it as `/proc/config.gz`; on those, every question above can be answered by looking it up. Most kernels you meet in the field are not, and then the same questions must be answered from other evidence, or admitted to be unanswerable.

Your script must handle both:

- If `/proc/config.gz` is readable, report `config-source: config.gz` and read `hz` from it.
- If it is not, report `config-source: inferred`, and report `hz: unknown`.

Report unknown rather than guessing. There is no honest way to read `CONFIG_HZ` off a running kernel without the config, and a script that prints a confident number it cannot justify is worse than one that admits the gap. `run.sh` checks for exactly this.

Two more traps, both of which `run.sh` checks:

- **char-majors counts character devices only.** `/proc/devices` has two sections.
- **Test for PREEMPT_RT before PREEMPT.** The string `PREEMPT` is a prefix of `PREEMPT_RT`, so a case that tests the shorter one first reports an RT kernel as a plain preemptible one.

Chapter 12 depends on telling them apart.

Check yourself: make `test L=L03` reports `SKIPPED` before you write it and `PASSED` after, with seventeen checks. Then do Exercise 3.9, which turns `/proc/config.gz` off, and run `make test L=L03` again; it must still pass, with `config-source` now reading `inferred`.

Exercise 3.7 What is in this kernel that need not be

CODE

Using the running target and the built tree, answer with a command and a number.

- a) How many modules did the build produce, and how many are installed on the target?
- b) How many symbols does the kernel export to modules? Compare `/proc/kallsyms` with the number of *exported* symbols in `build/kernel/Module.symvers`.
- c) How much memory was returned to the system after boot by discarding `__init` code? Find the kernel message that says so.
- d) Image is 24 MB. How large is `vmlinux`, and account for the difference in one sentence.
- e) Name three subsystems that are compiled into this kernel that the emulated machine cannot possibly use, and find them in `/proc/filesystems` or `/proc/devices` to prove they are there.

Exercise 3.8 A configuration for a real product

DESIGN

You are configuring a kernel for a product: an ARM64 SoC, 256 MB of RAM, one Ethernet interface, an eMMC holding an ext4 root filesystem, an I2C sensor, and a watchdog. It boots from U-Boot with no `initramfs`. It must support field updates of the sensor driver without a reboot.

- a) For each of the seven components, say `=y`, `=m` or `=n`, and why.
- b) Which of the decisions above would, if you got them wrong, produce a kernel that does not boot at all? Describe the symptom.
- c) The product later gains a second hardware variant with a different sensor. What changes about your configuration strategy, and what would you use `savedefconfig` for?
- d) Your build is taking forty minutes. Name the first thing to look at, and say roughly what you expect to find, based on what `kernel/trim.config` does for this course.

Exercise 3.9 What one config option costs

CROSS-CHECK

Predict a number, change one line, measure, and reconcile. The three measurements disagree with each other and with your prediction, and all of them are correct.

`CONFIG_IKCONFIG` embeds a compressed copy of the `.config` into the kernel so that the running system can be asked what it was built with. It is a good option to measure because its contribution is mostly one blob whose size you can find out in advance.

- a) **Predict.** With the current kernel built, run:

```
size -A build/kernel/kernel/configs.o
```

Add up the four sections that §3.2.6 counts as the option’s content. That is what `CONFIG_IKCONFIG` contributes. Write the number down.

b) Check the prediction two other ways. The bulk of it is the compressed config itself, and you can measure that independently:

```
gzip -9 -c build/kernel/.config | wc -c      # on the host
ls -l /proc/config.gz                        # on the target, after make boot
```

Do all three agree? Should they be identical, and if not, why not?

c) Record the baseline. Note the exact byte counts of both `build/kernel/arch/arm64/boot/Image` and `build/kernel/vmlinux`, and the section sizes from `size -A build/kernel/vmlinux`.

d) Change one line. Create `kernel/local.config`, which is merged last and is gitignored:

```
printf '# CONFIG_IKCONFIG is not set\n' > kernel/local.config
make kernel
```

e) Measure the same three things again. You now have, for the same one-line change:

- a change in `vmlinux`’s **section sizes**,
- a change in the **`vmlinux` file**,
- a change in the **`Image`**.

f) Reconcile. Answer each:

- Which of the three measurements is closest to your prediction from **a)**, and which is furthest? By what factor is the furthest one out?
- Look at the per-section deltas. Two sections account for nearly all of it. For each, compare the section’s delta against the bytes `configs.o` contributed to it. They are not equal. What is the relationship between the two numbers, and what does 4096 have to do with it?
- The `Image` delta is a suspiciously round number. What is it a multiple of, and what does that tell you about how the `Image` is laid out?
- Given all that: if you now removed a *second* option contributing another 34 KB, would the `Image` shrink by the same amount again? Explain.

g) Boot the smaller kernel and run `make test L=L03`. Your report from Exercise 3.6 should still pass, with `config-source` now reading `inferred`. If it does not, Exercise 3.6’s requirement to handle both worlds is the part you skipped.

h) A colleague asks “how much does `CONFIG_IKCONFIG` cost?”. Give them a single number and a sentence, and say which of your measurements you chose and why.

Restore the kernel afterwards with `rm kernel/local.config && make kernel`, so that the labs of later chapters start from the configuration they were written against.

PART II

Writing a Driver

Kernel Modules

The smallest useful thing you can write for a kernel, and the environment it runs in.

IN THIS CHAPTER

- `module_init` and `module_exit`: code with no `main`, that runs once and then waits.
- Building out of tree, and what `M=` actually does.
- `insmod`, `rmmod`, `modprobe`, `depmod`, and why the first and third are not the same command.
- Module parameters, and the `sysfs` directory you get without asking.
- `EXPORT_SYMBOL` against `EXPORT_SYMBOL_GPL`, and a licence check that happens at load time.
- `printk`, the log levels, and `pr_debug` that costs nothing until you turn it on.
- The C environment: no `libc`, no floating point, and a stack you can overflow.
- A module that loads, then a second one that uses a symbol from the first.

This chapter is about the smallest thing you can add to a running kernel, and the environment it runs in. It explains why a module's `init` function is not a `main`, how an out-of-tree module is built by the kernel's own build system and why the result belongs to one kernel, and what `insmod`, `modprobe`, `depmod` and `rmmod` each do. It shows where the licence you declare is checked, twice and with two very different messages, how one line of parameter declaration produces a `sysfs` file nobody wrote, how to `print`, and what the C inside a kernel may not do.

§4.1 is the whole of the material, and ends with why this repository's code is not formatted the way the kernel's is. An out-of-tree module cannot be built until `make kernel` has finished. The lab is specified by two of the exercises in §4.3: Exercise 4.6 is a first module with two parameters, and Exercise 4.7 is a pair of modules, one calling a function the other exports. The exercises end with the Cross-check: predict how much memory a module occupies from its sections, load it, and find out why the kernel's figure is five times larger. The lab is done when `make test L=L04` reports **PASSED** with twelve checks: the load order that fails, the `holders` directory, and the kernel refusing to unload a module that is in use.

4.1 Kernel Modules, and the C You Are Allowed to Write

This section is the whole of the chapter's material: what a module is, how it is built and loaded, what the kernel does with the licence you declare, and what the C environment inside a kernel is actually like. It ends with the coding style question, which this repository answers differently from the kernel and owes you an explanation for.

4.1.1 A module is not a program

The smallest useful module is six lines of substance:

```
// SPDX-License-Identifier: GPL-2.0-only
#include <linux/init.h>
#include <linux/module.h>

static int __init qa_hello_init(void)
{
    pr_info("qa_hello: loaded\n");
    return 0;
}

static void __exit qa_hello_exit(void)
{
    pr_info("qa_hello: unloaded\n");
}

module_init(qa_hello_init);
module_exit(qa_hello_exit);
MODULE_LICENSE("GPL");
```

There is no main, and qa_hello_init is not one. The distinction is worth being precise about because it shapes everything you write afterwards.

main runs, and while it runs, your program exists. When it returns, your program is over. qa_hello_init runs **once, at load**, and when it returns the module is **resident and idle**. Nothing of yours is running. The module is a body of code and data sitting in the kernel's address space, waiting to be called by something else: a file operation, an interrupt, a timer, a probe.

So the shape of a module is *registration*. init announces what you can do and then gets out of the way; everything after that is callbacks. A module whose init function contains a loop is almost always a module that should have started a thread, and a module whose init never returns has hung the process that called insmod.

__init and __exit are instructions to the linker. Code marked __init is placed in a section that is freed once initialisation is done, which is why you see Freeing unused kernel memory: 4672K at the end of boot. __exit code can never run when the module is built into

the kernel rather than as a module, because a built-in module can never be removed, so it is thrown away: at link time on most architectures, and on arm64 at the end of boot, with the `__init` code.

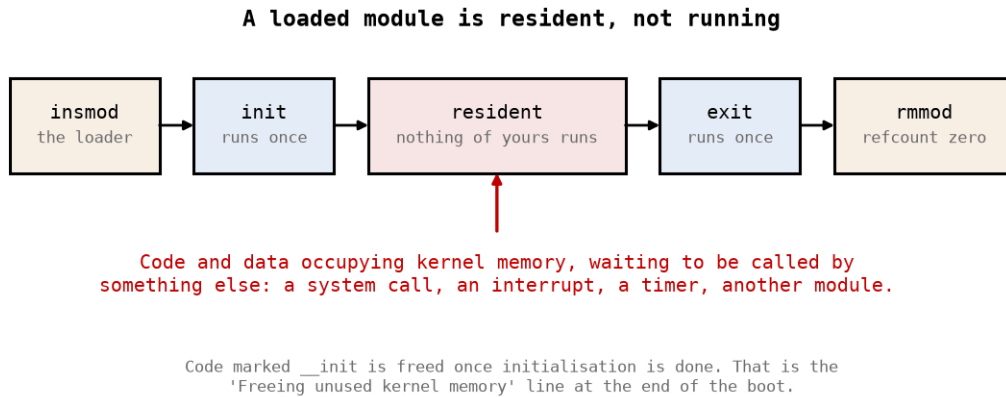


Figure 4.1 Five boxes in a row: `insmod`, `init`, `resident`, `exit` and `rmmod`. The middle box is highlighted and annotated to say that nothing of yours runs there; the module is code and data occupying kernel memory, waiting to be called by a system call, an interrupt, a timer or another module.

4.1.2 The four macros

Macro	Required	What it is for
<code>MODULE_LICENSE</code>	Yes	Read by the loader. See §4.1.5. Its absence is an error at build time
<code>MODULE_AUTHOR</code>	No	Convention, and useful when a bug report reaches a maintainer
<code>MODULE_DESCRIPTION</code>	No	Shown by <code>modinfo</code>
<code>MODULE_VERSION</code>	No	Free text; not used for dependency resolution

`MODULE_LICENSE` is the one that is not optional. Omit it and `modpost` fails the build with missing `MODULE_LICENSE()`, which is a good error, because the alternative is a module that loads as if it were proprietary: it taints the kernel and cannot see a single GPL-only symbol.

All of them are readable without loading anything:

```

$ modinfo qa_hello.ko
filename:      qa_hello.ko
license:      GPL
depends:
vermagic:      6.12.30 SMP preempt mod_unload modversions aarch64
  
```

That `vermagic` line is §4.1.3.

4.1.3 Building out of tree, and why a .ko is not portable

An out-of-tree module is built by running the *kernel's* build system on your directory:

```
make -C /path/to/kernel M=/path/to/your/module modules
```

You are not compiling against kernel headers with your own build system. You are asking kbuild to compile your file with exactly the flags the kernel was compiled with, which is why the kernel tree has to be configured and built first, and why there is no shorter way.

The consequence is *vermagic*. Every module records the kernel version, whether that kernel was SMP, which preemption model it used, and the architecture. The loader checks all of it and refuses a mismatch. This is a module built against this course's build/kernel, loaded into the real-time kernel Chapter 12 builds from the same source:

```
qa_hello: version magic '6.12.30 SMP preempt mod_unload modversions aarch64' ↵
↵ should be
    '6.12.30 SMP preempt_rt mod_unload modversions aarch64'
```

This looks like pedantry and is not. The kernel has no stable internal ABI; a structure that gained a field between two point releases would be read at the wrong offsets by a module compiled against the old one, and the failure would be silent corruption rather than a refusal to load. *vermagic* turns that into an error message.

With `CONFIG_MODVERSIONS`, which this course enables, the check is finer: a checksum is computed per exported symbol from its argument types, a module is refused if any symbol it uses has changed shape, and the release number at the front of *vermagic* is no longer compared at all. That is what lets a distribution ship third-party modules across a stable series.

This course's Kbuild files build every .c in the lab directory as its own module:

```
obj-m += $(patsubst $(src)/%.c,%.o,$(filter-out %.mod.c,$(wildcard $(src)/*.c)))
```

That is unusual for real kernel code, where the object list is written out explicitly. It is done here because the repository ships the build file and you write the sources, so a fixed list would mean editing the build every time you start an exercise. Note `$(src)` rather than a bare wildcard: kbuild runs the file from the kernel's output directory, and a wildcard over the current directory silently matches nothing, producing a build that succeeds having compiled zero modules. The `filter-out` is the other half: kbuild generates a `foo.mod.c` beside every `foo.c` it builds, and without it the second build tries to make a module of that as well.

4.1.4 Four commands, and the two people confuse

Command	Takes	Does
<code>insmod</code>	A path	Loads exactly that file. Resolves nothing
<code>modprobe</code>	A name	Looks the name up in <code>modules.dep</code> , loads dependencies first, then it
<code>depmod</code>	Nothing	Scans <code>/lib/modules/\$(uname -r)</code> and writes <code>modules.dep</code>
<code>rmmod</code>	A name	Unloads, if nothing holds a reference

The demonstration that makes the difference concrete is to load a module with a dependency the wrong way:

```
# insmod ./qa_user.ko
qa_user: Unknown symbol qa_answer (err -2)
insmod: can't insert './qa_user.ko': unknown symbol in module or invalid parameter
```

`insmod` did what it was told. It loaded that file, found an unresolved symbol, and gave up. It never occurred to it to look for a module providing `qa_answer`, because looking things up is `modprobe`'s job and `modprobe` reads a file that `depmod` generated.

Unloading is refcounted. While `qa_user` is loaded, `qa_export` cannot be removed:

```
# rmmod qa_export
rmmod: remove 'qa_export': Resource temporarily unavailable
# cat /sys/module/qa_export/refcnt
1
# ls /sys/module/qa_export/holders/
qa_user
```

The `holders` directory is the useful one: it names who is holding you, rather than merely how many. Note also that **BusyBox's `rmmod` returns success even when the removal failed**, which is why the lab for this chapter checks whether the module is still loaded rather than checking an exit status. A test that trusts that exit status gets the same answer on a kernel that refused and on a kernel that did not.

4.1.5 The licence, and where the check happens

[§1.2.4](#) made the argument. This is the mechanism.

Symbols leave a module in one of two ways:

```
EXPORT_SYMBOL(qa_answer);          /* any module */
EXPORT_SYMBOL_GPL(qa_answer);      /* GPL-compatible modules only */
```

The accepted values of `MODULE_LICENSE` are a fixed list: "GPL", "GPL v2", "GPL and additional rights", "Dual BSD/GPL", "Dual MIT/GPL", "Dual MPL/GPL", and "Proprietary". Anything else is treated as proprietary, including a typo.

Now the part worth doing rather than reading. Take a module that calls a `_GPL` symbol, declare its licence "Proprietary", and build it. **The build fails**, and the message is unusually good:

```
ERROR: modpost: GPL-incompatible module qa_proprietary.ko uses GPL-only symbol '↵
↵ qa_answer'
```

That is `modpost`, a build step that runs after every module is linked and before the `.ko` is finished. It reads the exports of everything it can see, and the check is three lines in `scripts/mod/modpost.c`:

```
if (!mod->is_gpl_compatible && exp->is_gpl_only)
    error("GPL-incompatible module %s.ko uses GPL-only symbol '%s'\n",
          basename, exp->name);
```

The licence is enforced twice, at two different stages, and it is worth knowing both because they fail differently.

At build time, as above, when `modpost` can see that the symbol is GPL-only. That is the usual case: the exports of the built-in kernel come from its `Module.symvers`, and modules built together see each other. You get a clear message naming the licence, the module and the symbol, and no `.ko` is produced at all.

At load time, as a backstop, and here the failure is much less obvious. The module loader does not check the licence of a symbol you asked for and refuse it; instead it **hides GPL-only symbols from the search entirely**. From `kernel/module/main.c`:

```
static bool find_exported_symbol_in_section(...)
{
    if (!fsa->gpl_ok && syms->license == GPL_ONLY)
        return false;
```

where `gpl_ok` is set from the loading module's own taint:

```
.gpl_ok = !(mod->taints & (1 << TAIN_T_PROPRIETARY_MODULE)),
```

So a proprietary module looking up a GPL-only symbol is told the symbol **does not exist**, and `insmod` reports:

```
qa_user: Unknown symbol qa_answer (err -2)
```

Nothing in that message mentions a licence. It is the same message you get for a genuine typo or a missing dependency, which makes it one of the more confusing failures in kernel work. The load-time path is the one you hit when `modpost` could not have known: a module built with `KBUILD_MODPOST_WARN=1`, which turns an unresolved symbol from an error into

a warning, or a module built against a kernel where the symbol was `EXPORT_SYMBOL` and loaded into one where it has since become `EXPORT_SYMBOL_GPL`. That second case is real and happens on kernel upgrades.

Tainting is separate and additive. Loading any out-of-tree module taints the kernel, even a GPL one:

```
qa_hello: loading out-of-tree module taints kernel.
```

`/proc/sys/kernel/tainted` is a bitmask; `Documentation/admin-guide/tainted-kernels.rst` decodes it. Loading an out-of-tree module sets bit 12 (`TAINT_OOT_MODULE`, value 4096); loading a proprietary one *also* sets bit 0 (`TAINT_PROPRIETARY_MODULE`), so a proprietary out-of-tree module leaves `/proc/sys/kernel/tainted` reading **4097** and shows as (P0) in `/proc/modules`.

What a taint means is mostly social: an oops from a tainted kernel is one that upstream maintainers will decline to debug, because they cannot see the code that might have caused it.

One taint has a concrete technical consequence, and it is worth knowing before Chapter 7. Loading a proprietary module prints this as well:

```
qa_prop_ok: module license 'Proprietary' taints kernel.
Disabling lock debugging due to kernel taint
```

The lock validator is switched off, for the rest of that boot, because it cannot reason about locking in code it cannot see. Chapter 7 is built entirely on that validator. So a single proprietary module loaded anywhere on the system silently removes the tool you would use to find the deadlock it might be causing, and the rule that follows is to keep development kernels free of proprietary modules even when the production system has one.

4.1.6 Parameters, and sysfs you did not write

One line:

```
static char *who = "world";
module_param(who, charp, 0444);
MODULE_PARM_DESC(who, "who to greet");
```

buys a load-time argument and a sysfs file:

```
# insmod ./qa_hello.ko who=embedded
# cat /sys/module/qa_hello/parameters/who
embedded
```

Nobody wrote code to create that file. The third argument is its mode, and 0 means do not create one at all. If the mode is writable, userspace can change the variable while the module

is loaded, which is worth pausing on: that is a shared variable being written by another context, and everything Chapter 7 says about concurrency applies to it.

This is the first appearance of the pattern the whole driver model rests on, which is that you describe something and the kernel generates the interface. Chapter 10 is where it stops looking like a convenience and starts looking like the architecture.

4.1.7 Printing

printk with a level, or the wrappers, which is what you should actually use:

Wrapper	Level	Use for
pr_emerg	0	The system is unusable
pr_alert	1	Action must be taken immediately
pr_crit	2	Critical conditions
pr_err	3	An error your driver could not handle
pr_warn	4	Something unexpected that you did handle
pr_notice	5	Normal but significant
pr_info	6	Normal. Most of what you write during development
pr_debug	7	Compiled out unless dynamic debug is on

Three practical notes.

Set **pr_fmt** once, at the top of the file, and every message gets a prefix for free:

```
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt
```

pr_debug is not pr_info you comment out. With CONFIG_DYNAMIC_DEBUG, which this course enables, each call site can be switched on at run time:

```
echo 'module qa_hello +p' > /sys/kernel/debug/dynamic_debug/control
```

Until then it costs nothing but a few bytes of metadata. That is what makes it reasonable to leave debug printing in shipped code.

Prefer dev_info once you have a struct device, from Chapter 10 onwards. It prints which device the message came from, which matters the moment there are two of them.

4.1.8 The C you are allowed to write

The kernel is compiled -nostdinc, -ffreestanding, against its own headers. There is no C library. The practical list:

- **No stdio.h, stdlib.h, string.h.** There are kernel equivalents with the same names for some things (memcpy, strlen, snprintf) and no equivalent for others.
- **No malloc.** kmalloc and friends, which take a flag saying what context you are in. Chapter 6.

- **No floating point.** The kernel does not save FPU state on entry, so using a double corrupts whatever userspace task was interrupted. On arm64 the compiler is not allowed to try: the kernel is built with `-mgeneral-regs-only`, so a double is a compile error. If you genuinely need it there is `kernel_fpu_begin()`, and needing it is nearly always a sign the calculation belongs in userspace. Fixed point is the normal answer.
- **A small, fixed stack.** 16 KB on arm64, for the whole call chain, and nothing grows it; interrupts run on a separate per-CPU stack of the same size. This kernel puts an unmapped guard page below each stack (`CONFIG_VMAP_STACK`, on by default), so running off the end is a kernel stack overflow panic rather than a diagnostic you can recover from. Without the guard page, a large local array silently corrupts whatever lies below the stack, and the crash comes somewhere else entirely. `-Wframe-larger-than=2048` warns about the obvious cases at compile time.
- **No exceptions and no unwinding.** Every function returns an error code, and the caller checks it. The convention is zero for success and a negative `-Exxx` for failure, and pointer-returning functions use `ERR_PTR`, `IS_ERR` and `PTR_ERR` to pack an error into a pointer.
- **Never trust a userspace pointer.** Chapter 5.

The unifying idea is that a mistake in kernel C does not produce a segfault in your program, because there is no “your program”. It produces an oops, and often it produces one somewhere else, later, in code you did not write.

4.1.9 Coding style, and why this repository does not use the kernel’s

The Linux kernel has a documented coding style, in `Documentation/process/coding-style.rst`, and it is not a matter of taste: patches are rejected for violating it, and `scripts/checkpatch.pl` exists to check.

It is tabs, eight columns wide, with braces in K&R placement:

```
static int qa_fifo_put(struct qa_fifo *fifo, const u8 *data, size_t len)
{
    size_t n = 0;

    if (!fifo || !data)
        return 0;

    while (n < len && fifo->level < fifo->capacity) {
        fifo->data[(fifo->head + fifo->level) % fifo->capacity] = data[n];
        ++n;
        fifo->level++;
    }

    return n;
}
```

This repository is formatted differently, with four spaces and braces on their own line:

```
static int qa_fifo_put(struct qa_fifo* fifo, const u8* data, size_t len)
{
    size_t n = 0;

    if (!fifo || !data) { return 0; }

    while (n < len && fifo->level < fifo->capacity)
    {
        fifo->data[(fifo->head + fifo->level) % fifo->capacity] = data[n++];
        fifo->level++;
    }

    return n;
}
```

That is the QAcademy house style, shared with every other course in the series and enforced by `.clang-format`. The trade is deliberate and it is worth being honest about both halves of it.

What it buys is that a reader moving between the Rust, C++ and Linux courses sees one style, and that `format` does the same thing everywhere.

What it costs is that none of the code in this repository would be accepted upstream, and that a reader who goes from here to `drivers/` will find every file looks different. That second cost is real, which is why this section exists, and why the two versions above are printed side by side. The difference is entirely mechanical: eight-column tabs against four spaces, brace placement, and `char *p` against `char* p`.

If you write a driver you intend to submit, run `scripts/checkpatch.pl --file` on it and follow what it says. Nothing in this course’s material depends on the formatting, and everything in it would be correct in either style.

4.2 Review

What you should be able to do now

After this chapter, you should be able to:

- Write, build and load an out-of-tree module, and say what each of the four macros in it is for.
- Explain why `module_init` returning is not the module finishing.
- Say what `insmod` does that `modprobe` does not, and produce the failure that distinguishes them.
- Add a module parameter and read it back from `sysfs` without writing any `sysfs` code.

- Predict whether a given module will load, from its `MODULE_LICENSE` and the symbols it uses.
- Choose a `printk` level, and say why `pr_debug` is not simply `pr_info` you comment out.
- List three things you may not do in kernel C that you do without thinking in userspace C.

Questions to test yourself

1. What is resident in memory after `module_init` returns, and what is running?
2. Why is a `.ko` built against one kernel usually refused by another?
3. A proprietary module using a GPL-only symbol can fail at two different points, with two very different messages. Name both, and say which one you get when the symbol changed from `EXPORT_SYMBOL` to `EXPORT_SYMBOL_GPL` after your module was built.
4. You add `module_param(depth, int, 0644)`. Who wrote the code that makes `/sys/module/yours/parameters/depth` appear?
5. Why is there no float in the kernel? What would have to happen for there to be one?
6. `rmmod` refuses with “Module is in use”. What is holding it, and where can you see that?

Where to look

- `lectures/L04/lab/Kbuild` is one line and eighteen lines of comment explaining it: why it builds every `.c` in the directory rather than a fixed list, and what goes wrong with each half of the line if you write it the obvious way.

Looking ahead

Chapter 5 is about:

- The oldest interface a driver can offer userspace, and the one you will write first.
- `open`, `read`, `write`, and what each of them has to return.
- Why dereferencing a pointer from userspace is a bug and not merely bad style.
- A ring buffer, and a KUnit suite that knows one thing your `read()` does not.

4.3 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 4.6 onwards need the modules from the lab.

Where an exercise can be checked mechanically, a **Check yourself** line says how. Several have a plausible wrong answer that is worth walking into before you find it.

Everything here runs against the target from `make boot`, or against the built tree under `build/kernel`. Nothing needs the network.

Solutions. The exercises that are not Code are answered in §A.4. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 4.1 A module is not a program

RECALL

- `qa_hello_init` returns 0. What is running afterwards, and what is resident?
- Why is it a mistake to put a `while (1)` loop in a module's init function? What should you have written instead, and what happens to `insmod` if you do it anyway?
- What do `__init` and `__exit` actually do? Neither is a hint to the reader.
- The boot log says `Freeing unused kernel memory: 4672K`. What relationship does that message have to `__init`?
- Name the four `MODULE_*` macros from the lab's module, and say which one is not optional and what happens if you leave it out.

Exercise 4.2 Four commands

RECALL

- State in one sentence each what `insmod`, `modprobe`, `depmod` and `rmmod` do.
- `insmod` and `modprobe` are often described as the same command. Give the concrete difference, and describe the experiment that demonstrates it using the lab's `qa_export` and `qa_user`.
- What file does `modprobe` read to decide what to load first, and what generates it?
- `rmmod qa_export` fails while `qa_user` is loaded. Where, in `/sys`, can you see *which* module is holding it, rather than merely how many are?

Exercise 4.3 Reading `vermagic`

HAND CALCULATION

`modinfo` on the lab's module reports:

```
vermagic: 6.12.30 SMP preempt mod_unload modversions aarch64
```

- Name what each of the six fields is asserting about the kernel this module may be loaded into.
- For each of the following changes to the kernel, say whether this module would still load, and why:
 - The kernel is rebuilt from the same source with no config change.
 - The kernel is rebuilt with `# CONFIG_SMP` is not set in `kernel/local.config`. Read `arch/arm64/Kconfig` before answering.

- The kernel is upgraded from 6.12.30 to 6.12.31.
 - The kernel is rebuilt with `CONFIG_PREEMPT_RT=y` instead of `CONFIG_PREEMPT=y`.
- c) `modversions` appears in the list. What does `CONFIG_MODVERSIONS` add beyond the version string, and what problem does it solve that `vermagic` alone does not?
- d) Explain why a mismatch is reported as an error rather than being allowed with a warning. What is the failure mode that policy is protecting you from?

Exercise 4.4 The licence, at two stages

HAND CALCULATION

A module declares `MODULE_LICENSE("Proprietary")` and calls a function exported with `EXPORT_SYMBOL_GPL`.

- a) You build it in the same directory as the module that exports the symbol. What happens, at which build step, and what exactly does the message say?
- b) You build it in a directory of its own, where the build cannot see the exporting module's symbols. What happens now, and why is it a *different* message?
- c) Neither of those produced a `.ko`. Describe the circumstances in which a proprietary module using a GPL-only symbol does build successfully and then fails at `insmod`.
- d) In that case, what does `insmod` print? Read §4.1.5 and say why the message does not mention a licence, in terms of what the module loader actually does with GPL-only symbols.
- e) Why is that the worst of the three messages to receive?

Exercise 4.5 Choosing a log level

DESIGN

For each, choose from `pr_debug`, `pr_info`, `pr_warn`, `pr_err`, and justify it in one sentence.

- a) The driver has probed successfully and is ready.
- b) A device returned a value outside its documented range, and you clamped it.
- c) `ioremap` returned `NULL` and probe is about to fail.
- d) You want to print the value of every register during development.
- e) The device tree specified a clock rate the hardware cannot produce, so you used the nearest.

Then: f) why is `pr_debug` not simply `pr_info` that you comment out before shipping? Give two reasons, one about the binary and one about the field.

Exercise 4.6 Your first module

CODE

Write `lectures/L04/lab/qa_hello.c`. It must:

- Print a greeting from its `init` function and a farewell from its `exit` function.
- Take a string parameter `who`, defaulting to `world`, readable but **not** writable via `sysfs`.
- Take an integer parameter `count`, defaulting to 1, readable **and** writable via `sysfs`, that controls how many times the greeting is printed.
- Carry `MODULE_LICENSE`, `MODULE_AUTHOR`, `MODULE_DESCRIPTION` and `MODULE_VERSION`.
- Use `pr_fmt` so every message is prefixed with the module name, and contain one `pr_debug` that says something only useful when switched on.

Check yourself: `insmod ./qa_hello.ko who=embedded count=2` prints the greeting twice; `cat /sys/module/qa_hello/parameters/who` returns `embedded`; writing to that file fails with `Permission denied`, and writing to `count` succeeds.

Exercise 4.7 One module using another

CODE

Write `qa_export.c`, which exports `qa_answer()` returning 42 with `EXPORT_SYMBOL_GPL`, and `qa_user.c`, which calls it from its `init` function and prints the result.

- Load them in the wrong order deliberately. Record the exact error.
- Load them in the right order, then try to `rmmod` them in the wrong order. Record what happens, and find the holder in `/sys`.
- `qa_export.c` also exports `qa_answer_open()` with plain `EXPORT_SYMBOL`. Write a third module declaring `MODULE_LICENSE("Proprietary")` that calls **only** that one. Does it build? Does it load?
- Read `/proc/sys/kernel/tainted` before loading anything, after loading `qa_export`, and after loading the proprietary one. You get three different numbers. Decompose each into bits, and name the two taints involved using `include/linux/panic.h`. Then look at the flags in `/proc/modules` and in `/sys/module/<name>/taint`: which letters appear against which module, and why does the proprietary module carry two?
- Loading the proprietary module prints one further line that is not about licensing at all:

```
Disabling lock debugging due to kernel taint
```

Find out what was disabled. Then say what that means for Chapter 7, which is built entirely on that facility, and state the rule it implies for a development kernel.

Check yourself: `make test L=L04` passes, with twelve checks. Note that its check on `rmmod` deliberately asserts on `lsmod` rather than on `rmmod`'s exit status; Exercise 4.8 is why.

Exercise 4.8 A test that cannot fail

DESIGN

The lab's `run.sh` checks that a module in use cannot be unloaded. It does so by attempting the `rmmmod` and then looking at `lsmod`, rather than by checking `rmmmod`'s exit status.

- a) Run `rmmmod qa_export` while `qa_user` is loaded, then immediately `echo $?`. What do you get?
- b) Given that, what would a test written the obvious way conclude, on a kernel that correctly refused *and* on one that wrongly allowed it?
- c) Name the general rule this is an instance of, and give one other example from §2.1.8 of the same class of problem.

Exercise 4.9 How big is a module

CROSS-CHECK

Three measurements of the same module, differing by a factor of five. Predict, measure, reconcile.

- a) **The file.** With the lab built, record the size of `qa_hello.ko` on disk.
- b) **The content.** Run `size -A` on it, in the container so you get the cross toolchain's version:

```
aarch64-linux-gnu-size -A lectures/L04/lab/qa_hello.ko
```

Record the Total. Then record the sum of only those sections that will actually be **loaded**: leave out `.modinfo`, `__versions` and `.comment`, which are metadata the loader reads and does not keep, and `.symtab` and `.strtab`, of which it keeps only a small copy for `/proc/kallsyms`. The `.note.*` sections are kept, and are small.

- c) **Predict.** From b), write down what you expect the module to occupy in kernel memory once loaded.
- d) **Measure.** Boot, load the module, and read its actual size two ways:

```
grep qa_hello /proc/modules
cat /sys/module/qa_hello/coresize
```

- e) **Reconcile.** Answer each:

- By what factor does the measured size exceed your prediction?
- The measured number is round. What is it a multiple of, and how many of them?
- Load `qa_export` and compare *its* `coresize` with `qa_hello`'s. They are not the same multiple. Before reading further, say what would have to differ between two modules for one to need more of these units than the other.
- Now find out why. List the section addresses and sort them:

```
for f in /sys/module/qa_hello/sections/.*; do
    echo "$(cat "$f" 2>/dev/null) $(basename "$f")"
done | sort
```

The addresses fall into distinct groups. How many groups, what is in each, and what is the spacing between them? What does that tell you about how a module is laid out in memory?

- `include/linux/module.h` defines `enum mod_mem_type`. How many of its entries are *core* types rather than *init* types, and how does that number relate to what you measured? The allocation itself is one line in `kernel/module/main.c`; find it.
- Why is the module laid out this way rather than packed into one contiguous region? The answer is about permissions, and it is the same reason the kernel's own `.text` and `.data` are separate.

f) `/sys/module/qa_hello/initsize` reads 0 after the module has finished loading, even though the module plainly has an `init` function. What happened to it, and what does that have to do with Exercise 4.1(d)?

g) Someone asks how much memory your driver uses. Give a single number and a sentence, and say which of your three measurements you chose and why. Then say what your answer would be if the driver also allocated a 64 KB buffer with `kmalloc` in its `init` function, and which of the three numbers would have changed.

Character Device Drivers

The oldest and simplest interface a driver can present to userspace, and everything it implies.

IN THIS CHAPTER

- Major and minor numbers, and what each of them selects.
- `struct file_operations`, and the fact that a driver is a table of callbacks and nothing else.
- `open`, `release`, `read`, `write`: what each returns, and what userspace does with it.
- `copy_to_user`, and why dereferencing a userspace pointer is a bug rather than a shortcut.
- Which `-Exxx` to return, and why the choice is part of the interface.
- The `misc` device, and when the shortcut is the right answer.
- `ioctl`, and why adding a new one is usually the wrong answer.
- A ring buffer, and the KUnit suite that tests it.

This chapter is about the oldest and simplest interface a driver can present to userspace: a node in `/dev`, a table of callbacks behind it, and what each of those callbacks owes its caller. It explains what the two numbers on a node select, how a driver registers a `struct file_operations` against them, and why the value `read()` returns is the interface rather than a detail of it. It shows why a pointer handed in from userspace must never be followed directly, which error code answers which failure, when the `misc` device is the right shortcut, and why a new `ioctl` is usually the wrong answer.

Read §5.1 first; it is the character device interface. §5.2 then specifies the lab in prose, every function, what it returns and what it must do at the boundaries: a byte ring buffer, checked by a KUnit suite that ships with the course, and a character device that exposes it at `/dev/qa_fifo`, checked by a userspace tester. The header that declares the ring buffer is part of that specification, and is printed in §5.2.1. The exercises in §5.4 end with the Cross-check: work out by hand what a sequence of five calls must return, then write a program that performs it and reconcile the two. One of the five is not what almost anyone predicts. The lab is done when `make test L=L05` reports **PASSED**, with the KUnit suite reporting **10 of 10**.

5.1 Character Device Drivers

This section is how a driver presents itself to userspace: what a device node is, how the kernel routes a `read()` on it into your function, and what each of those functions owes its caller. §5.2 is the specification of the driver you write.

The framing worth keeping: **a character driver is a table of callbacks and nothing else.** You fill in a `struct file_operations`, you register it against a number, and from then on the kernel calls you. There is no main loop, no thread of yours, and nothing running between calls.

5.1.1 The node, and the two numbers

§2.2.5 established that a device node carries a **major** and a **minor** where a regular file has a size. Major selects the driver; minor selects which of that driver's devices.

Three facts follow, and all three surprise people the first time.

A node is just a name. `mknod` writes a directory entry containing a type and two numbers, and consults nothing. You can create a node for a driver that does not exist; the failure comes later, at open, as `ENXIO`.

A node is not unique. Two nodes with the same major and minor are the same device. Deleting one removes a name and nothing else.

The driver does not create the node. Your driver registers a *number*. Something else makes a file appear in `/dev`. On this target that something is `devtmpfs`, prompted by your call to `device_create`; on a desktop, `udev` sits on top and applies naming and permission rules.

5.1.2 Getting a number

Two ways, and you want the second.

```
/* Static: you pick the major and hope nobody else has it. */
register_chrdev_region(MKDEV(240, 0), 1, "qa_fifo");

/* Dynamic: the kernel picks, and tells you. */
alloc_chrdev_region(&devno, 0, 1, "qa_fifo");
```

Static allocation is how it was done for years and is a bad idea now: the major space is shared, the “local/experimental” range is small, and a collision produces `-EBUSY` at load time on somebody else's machine and not on yours. **Always allocate dynamically**, then find out what you got:

```
# grep qa_fifo /proc/devices
243 qa_fifo
```

243 is not a constant. It is whatever was free, it can differ between boots, and any script that hardcodes it is wrong. This is precisely why `device_create` and `udev` exist: userspace should find the device by *name*, and the number should be nobody's business.

5.1.3 The table

```
static const struct file_operations qa_fops = {
    .owner    = THIS_MODULE,
    .open     = qa_open,
    .read     = qa_read,
    .write    = qa_write,
};
```

Every member is optional except `.owner`. A missing `.read` is not an error; the kernel substitutes a default, and for read the default returns `-EINVAL`. So a driver that forgets to implement something fails at the point userspace tries to use it, with an error that says nothing about what is missing.

`.owner = THIS_MODULE` is what stops the module being unloaded while a process has the device open. It is one line, it is easy to leave out, and leaving it out means `rmmod` succeeds while a process still holds a file whose operations have just been freed. The result is not an error message.

Registering the table against the number takes two more calls:

```
cdev_init(&cdev, &qa_fops);
cdev.owner = THIS_MODULE;
cdev_add(&cdev, devno, 1);
```

After `cdev_add` the device is **live**: a `read()` on a matching node reaches your function, even though you have not finished your `init` function yet. Everything your callbacks depend on has to be ready before this call, not after it. That ordering constraint is the single most common source of a crash-on-load in a first driver.

5.1.4 The functions, and what they owe

open and release

```
static int qa_open(struct inode *inode, struct file *file);
static int qa_release(struct inode *inode, struct file *file);
```

`open` returns 0 or a negative error. `release` is called when the **last** descriptor referring to the open file is closed, which is not once per `close()`: a fork duplicates descriptors, and `release` runs when the last of them goes.

inode identifies the device; file identifies this particular opening of it, and its private_data is where per-open state belongs.

read and write

```
static ssize_t qa_read(struct file *file, char __user *buf, size_t count, loff_t *pos);
static ssize_t qa_write(struct file *file, const char __user *buf, size_t count, loff_t *pos);
```

The return value is the interface, and there are four cases:

Return	Means
n, 0 < n <= count	This many bytes were transferred. A short count is normal, not an error
0 from read	End of file. Nothing more will ever come
0 from write	Nothing was written, and it is not an error. Rarely what you mean
negative	An error; the value is -Exxx and userspace sees it as errno

The 0 row is the one this chapter exists for. If your read returns 0 because it has nothing *right now*, every reader concludes the device is finished. cat exits, a loop terminates, and a program waiting for data quietly stops waiting. Nothing reports an error, because nothing went wrong as far as userspace can tell.

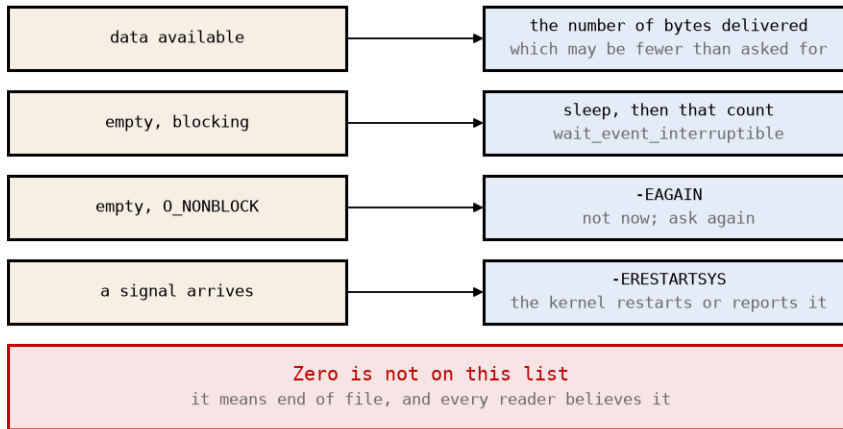
Until Chapter 9 gives you wait queues, the honest answer when a driver has nothing to give is **-EAGAIN**: “not now, ask again”. Chapter 9 replaces it with blocking, and -EAGAIN then becomes the answer only for 0_NONBLOCK openers.

Which error code

The code you return is part of the interface, because userspace acts on it.

Code	Use for
-EAGAIN	No data now; try again
-ENOSPC	No room; the write cannot be satisfied
-EFAULT	A userspace pointer was bad. Return this when copy_to_user fails
-EINVAL	The arguments do not make sense
-ENOMEM	An allocation failed
-EIO	The hardware failed
-ENODEV	The device is gone
-ERESTARTSYS	A signal arrived while sleeping. Chapter 9

What read must return, in each of the four cases it faces



cat exits. A while (read(...) > 0) loop stops. The program concludes the device is finished when it has merely not produced anything yet.

Figure 5.1 Four situations a read faces, each with an arrow to what it must return: data available gives a byte count that may be short, empty and blocking sleeps, empty with O_NONBLOCK gives -EAGAIN, and a signal gives -ERESTARTSYS. A red box below states that zero is not on the list, because it means end of file.

5.1.5 The user pointer, which is not a pointer you may follow

buf is an address **in the calling process's address space**. It looks like a char * and the compiler will let you dereference it. Do not.

```
memcpy(kbuf, buf, count);           /* WRONG */
if (copy_from_user(kbuf, buf, count)) /* right */
    return -EFAULT;
```

Three separate things are wrong with the first line, and it is worth having all three rather than a rule to memorise:

- **The page may not be present.** Userspace memory is demand-paged and can be swapped or not yet faulted in. A direct dereference from kernel context faults where no handler expects it.
- **The address may not belong to the caller at all.** It may point into kernel memory, and a driver that copies from it hands kernel memory to a userspace program that asked for it. That is not a theoretical class of bug; it is a large fraction of real kernel CVEs.

- **The mapping can change under you.** Another thread in the same process can munmap it between your check and your copy.

copy_to_user and copy_from_user check the range, handle the fault, and **return the number of bytes they could not copy**, so zero means success, which reads backwards the first time:

```
if (copy_to_user(buf, kbuf, n))
    return -EFAULT;
```

They may also **sleep**, because faulting in a page can block. That is fine here and is not fine in an interrupt handler, which is Chapter 8.

5.1.6 Making a node appear

cdev_add makes the device reachable. It does not put anything in /dev.

```
cls = class_create("qa_fifo");
device_create(cls, NULL, devno, NULL, "qa_fifo");
```

class_create makes a directory under /sys/class/, and device_create adds a device to it, which emits a uevent that devtmpfs (or udev) turns into /dev/qa_fifo.

Both of these have changed signature within recent memory, which matters because most tutorials you will find are older than the change:

- class_create took an owner argument until 6.4 and takes only a name now.
- no_llseek was a real function for decades and **does not exist in 6.12**; code using it will not compile.

This is §3.1.2's "the internal API has no stability guarantee" arriving in the first driver you write. When a tutorial does not compile, the tutorial is usually right and old.

5.1.7 Unwinding, in reverse

Every one of those calls can fail, and each that succeeded must be undone in reverse order:

```
...
cls = class_create("qa_fifo");
if (IS_ERR(cls)) { ret = PTR_ERR(cls); goto err_cdev; }
...
err_class: class_destroy(cls);
err_cdev: cdev_del(&cdev);
err_region: unregister_chrdev_region(devno, 1);
err_fifo: qa_fifo_destroy(fifo);
return ret;
```


The goto ladder is the kernel's house idiom for this and is not the sin it would be elsewhere: each label undoes exactly one thing, and the order falls out of the order they were acquired. Write it once here, because Chapter 6 shows you `devm_` and you delete the whole ladder.

`IS_ERR` and `PTR_ERR` are the other idiom: functions returning pointers pack an error into the pointer rather than using a separate output parameter. `IS_ERR(p)` tests it and `PTR_ERR(p)` extracts the `-Exxx`. Checking for `NULL` instead is a bug that survives testing, because the failure it misses is the rare one.

5.1.8 The shortcut

Everything above is about forty lines. The misc device is six:

```
static struct miscdevice qa_misc = {
    .minor = MISC_DYNAMIC_MINOR,
    .name  = "qa_fifo",
    .fops  = &qa_fops,
};
misc_register(&qa_misc);
```

One shared major (10), a dynamically allocated minor, and the node created for you. **Most small drivers should use it.** It is taught second because it hides the number, the `cdev`, the class and the unwind path, and those are the parts worth having seen once.

5.1.9 `ioctl`, and why to think twice

```
long qa_ioctl(struct file *file, unsigned int cmd, unsigned long arg);
```

`cmd` is an encoded number built with `_IO`, `_IOR`, `_IOW` or `_IOWR`, packing a direction, a type, a number and the argument's size, so that a wrong call is usually caught rather than misinterpreted.

The argument against adding one:

- **It is an ABI you now own forever.** The moment a second program uses it, the numbers and the structure layout are frozen.
- **It is a portability trap.** A 32-bit process on a 64-bit kernel passes a differently sized structure, and handling that needs `compat_ioctl` and a struct with no implicit padding. Getting this wrong is a standing source of bugs.
- **There is usually a subsystem that already defines the operation.** That argument is Chapter 11, and it is the reason this course's driver eventually deletes its character device entirely.

Use one when the operation genuinely is device-specific and no framework covers it. Reach for `sysfs` for single values and `debugfs` for things that are not an ABI at all.

5.2 The Driver, Specified

This is what to build. Everything here is prose rather than code, because the code is yours; what the course provides is the contract, the header that declares it, and the tests that check it.

Two modules, in this order:

1. **qa_fifo.c**, a byte ring buffer with no kernel API in it beyond allocation. Declared by `lectures/L05/lab/qa_fifo.h` and checked by `lectures/L05/lab/qa_fifo_kunit.c`, both of which ship with the course.
2. **qa_chardev.c**, a character device that exposes one of those buffers at `/dev/qa_fifo`. Checked by `lectures/L05/lab/user/qa_fifo_test.c`, which also ships.

Write them in that order. The first is pure logic and can be got completely right before anything touches the kernel's device model; the second is where the kernel API is, and debugging both at once is what makes a first driver miserable.

5.2.1 The ring buffer

The declarations are in `lectures/L05/lab/qa_fifo.h`, and the prose in that header is part of the specification rather than a comment on it. Here it is, typeset straight from the repository:

```
lectures/L05/lab/qa_fifo.h
/**
 * @file Byte ring buffer implementation.
 */
#ifndef QA_FIFO_H
#define QA_FIFO_H

#include <linux/types.h>

/** Byte ring buffer structure. */
struct qa_fifo;

/**
 * @brief Create a new FIFO.
 *
 * @param[in] capacity FIFO capacity. Must be greater than 0.
 *
 * @returns The new FIFO, or NULL on allocation failure.
 */
struct qa_fifo* qa_fifo_create(size_t capacity);

/**
 * @brief Destroy a FIFO.
 *
 * @param[in] fifo FIFO to destroy.
 */
void qa_fifo_destroy(struct qa_fifo* fifo);
```

```

/**
 * @brief Get the capacity the FIFO was created with.
 *
 * @param[in] fifo FIFO to read.
 *
 * @returns The capacity in bytes.
 */
size_t qa_fifo_capacity(const struct qa_fifo* fifo);

/**
 * @brief Get how many bytes the FIFO currently holds.
 *
 * @param[in] fifo FIFO to read.
 *
 * @returns The number of bytes held.
 */
size_t qa_fifo_level(const struct qa_fifo* fifo);

/**
 * @brief Copy up to len bytes into the FIFO.
 *
 * @param[in] fifo FIFO to write to.
 * @param[in] data Bytes to copy in.
 * @param[in] len Number of bytes to copy.
 *
 * @returns The number of bytes copied, which is less than len when the FIFO fills.
 */
size_t qa_fifo_put(struct qa_fifo* fifo, const u8* data, size_t len);

/**
 * @brief Copy up to len bytes out of the FIFO, removing them.
 *
 * @param[in] fifo FIFO to read from.
 * @param[out] data Buffer the bytes are copied into.
 * @param[in] len Number of bytes to copy.
 *
 * @returns The number of bytes copied, or zero if it was already empty.
 */
size_t qa_fifo_get(struct qa_fifo* fifo, u8* data, size_t len);

/**
 * @brief Discard everything the FIFO holds, without freeing or reallocating.
 *
 * @param[in] fifo FIFO to reset.
 */
void qa_fifo_reset(struct qa_fifo* fifo);

#endif /* QA_FIFO_H */

```

What follows is what the header cannot say.

The struct is opaque. `struct qa_fifo` is declared in the header and defined only in your .c. Nothing outside may look inside it, which means you choose the layout, and the tests cannot accidentally depend on your choices. It also means every access goes through a

function, which is the habit that makes Chapter 7's locking tractable: there is exactly one place per operation to put a lock.

Every function must be exported. The KUnit suite is a separate module, so each function in the header needs an `EXPORT_SYMBOL_GPL`, exactly as in Chapter 4. Forgetting this produces `Unknown symbol` at `insmod` time for the `test` module, and the lab's `run.sh` distinguishes that case from a missing KUnit framework and says which it is.

The capacity is what you asked for. A FIFO created with capacity 4 must hold 4 bytes, not 3. The usual ring-buffer implementation keeps head and tail indices and cannot distinguish full from empty, so it wastes one slot; that implementation fails `qa_fifo_uses_its_whole_capacity`. Keeping a *level* alongside the head, rather than a tail, is one way out and there are others.

Short is not an error. `qa_fifo_put` given more than fits copies what fits and returns that count; `qa_fifo_get` on a partly filled buffer copies what is there. Neither is a failure, and the caller is expected to read the return value. This is the same contract `read()` and `write()` have, deliberately, so that the character device can pass its arguments almost straight through.

Wrapping is the part that gets written wrong. Fill, drain, and fill again: the second fill straddles the end of the storage. An implementation that tracks positions without wrapping passes every test in the suite up to the seventh and fails the eighth.

5.2.2 The character device

`qa_chardev.c` registers one device, backed by one `qa_fifo`.

What it must provide

Member	Required	Behaviour
<code>.owner</code>	Yes	<code>THIS_MODULE</code> . See §5.1.3
<code>.open</code>	Yes	Succeeds. Marks the file as a stream
<code>.read</code>	Yes	Below
<code>.write</code>	Yes	Below
<code>.release</code>	No	There is no per-open state to release

Module parameter

One parameter, `capacity`, an `int`, default **256**, readable but not writable through `sysfs`. It is the capacity of the FIFO created at load time. The userspace tester does not assume a value; it measures what it can write and reports it, so changing the default does not break the test.

open

Succeed, and mark the file as a stream. `stream_open()` does exactly this and is the one line the function needs. Note that `no_llseek`, which most older material uses for this, **was removed and does not exist in 6.12**.

A FIFO has no position, and a driver that silently accepts an offset is lying about what it is. Two different things refuse one, and it is worth knowing which does what. Since 6.0 the kernel refuses `lseek` with `-ESPIPE` on any file whose `file_operations` has no `.llseek`, which yours does not; the userspace tester checks that, and it passes with or without `stream_open`. **`stream_open` is what refuses the rest.** It makes `pread` and `pwrite` fail with `-ESPIPE` too, where without it they would succeed and read the FIFO as though the offset meant something, and it tells the VFS there is no file position at all, so the `pos` your read and write are handed is `NULL` and must not be touched.

read

Given a userspace buffer and a count:

- A count of 0 returns 0. There is nothing to do and it is not an error.
- **If the FIFO is empty, return `-EAGAIN`.** Not 0.
- Otherwise take up to count bytes out of the FIFO and copy them to userspace, and return how many. Fewer than count is normal.
- If the copy fails, return `-EFAULT`.

You cannot copy straight from the FIFO to the user pointer, because §5.1.5 says you may not touch that pointer directly. So there is a bounce buffer in the middle: allocate, take from the FIFO into it, `copy_to_user` out of it, free. Cap the allocation at something sensible; a program is allowed to ask for a gigabyte and you are not obliged to allocate one.

The `-EAGAIN` is the whole chapter. Returning 0 means end of file, and every reader believes it. Chapter 9 replaces this with blocking, at which point `-EAGAIN` becomes the answer only for a caller that opened with `O_NONBLOCK`.

write

The mirror image:

- A count of 0 returns 0.
- **If the FIFO is full, return `-ENOSPC`.**
- Otherwise copy up to count bytes in from userspace and return how many were accepted.
- If the copy fails, return `-EFAULT`.

Note the asymmetry with `read`: a full FIFO is `-ENOSPC` and an empty one is `-EAGAIN`. Both say “not now”, but a reader can reasonably wait for data to arrive, whereas a writer facing a buffer nobody is draining has a different problem. The codes are different because userspace does different things with them.

Init, in the right order

Five things to acquire, and the order is a constraint rather than a preference:

1. Create the FIFO.
2. Allocate a device number **dynamically**, with `alloc_chrdev_region`.

3. `cdev_init` and `cdev_add`.
4. `class_create`.
5. `device_create`, which is what makes `/dev/qa_fifo` appear.

Everything your callbacks touch must exist before step 3. `cdev_add` makes the device live; from that instant a `read()` can arrive, and it will arrive before your `init` function has returned if anything is watching `/dev`. Creating the FIFO after adding the `cdev` is a race you will not reproduce on demand and will hit eventually.

Print the major and minor you were given at the end, with `pr_info`. You will want it, and it is different on every boot.

Exit, in reverse

Undo all five, in the opposite order, and free the FIFO last. Any error path in `init` must unwind only what it actually acquired; see §5.1.7 for the `goto` ladder that expresses this.

5.2.3 What the tests check, and what they do not

The KUnit suite (`lectures/L05/lab/qa_fifo_kunit.c`) is ten cases against the ring buffer, run inside the kernel. It says so at the top, and it is worth repeating here: **it checks nothing about concurrency**. Every case runs on one thread. A ring buffer that passes all ten and is destroyed by two writers passes all ten. That is Chapter 7.

The userspace tester (`lectures/L05/lab/user/qa_fifo_test.c`) is a statically linked ARM64 program that opens the node and makes one system call at a time. It exists as a program rather than as shell because several of the things a driver must get right are invisible from a shell: a `read` returning 0 and a `read` returning `-EAGAIN` both produce no output from `cat`, and only a program can look at `errno`.

Between them they do **not** check: concurrency, what happens under memory pressure, whether your error paths unwind correctly, or whether you leak the FIFO on an `init` failure. Reading the tests to find out what they are blind to is a habit worth forming, and one of the exercises asks you to do exactly that.

5.2.4 Working order

1. Write `qa_fifo.c` and get the KUnit suite to **10 of 10**. Nothing below matters until it does.
2. Write `qa_chardev.c` with `open`, `read` and `write`, and load it. Check `/proc/devices` for your major and `/dev/qa_fifo` for the node.
3. Run `./qa_fifo_test` on the target by hand before running `make test`; its output is more informative than the harness's summary.
4. `make test L=L05` reports **PASSED**, with eighteen checks.

If the KUnit module refuses to load with Unknown symbol `qa_fifo_create`, you have not exported the functions. If it refuses with Unknown symbol `kunit_...`, the framework is not loaded and `run.sh` will say so.

5.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Say what a major and a minor number each select, and find both for any node in `/dev`.
- Write a `struct file_operations` and register it, both the long way and with `misc_register`.
- Explain the three separate things `copy_to_user` checks, and what goes wrong if you skip it.
- Say what `read()` must return when it has nothing to give, and why zero is the wrong answer.
- Choose the right `-Exxx` for a given failure and say what userspace will print for it.
- Give one reason to add an `ioctl` and two reasons not to.
- Write a ring buffer that survives its own wraparound, and prove it with a test rather than by inspection.

Questions to test yourself

1. Two device nodes have the same major and different minors. What do they have in common?
2. Why is `copy_to_user` a function rather than a macro that expands to `memcpy`?
3. Your `read()` returns 0 whenever the buffer is empty. Describe what `cat /dev/yours` does, and say why it is not what you wanted.
4. What does `open()` on your device return if you never implemented `open` in your `file_operations`?
5. Where does the node in `/dev` come from, given that your driver only registered a number?
6. A ring buffer passes every test except one about wraparound. What is almost certainly wrong with it, and why did the other tests not catch it?

Where to look

- `lectures/L05/lab/qa_fifo.h` is the contract, printed in §5.2.1, and `lectures/L05/lab/qa_fifo_kunit.c` is what checks it. Reading the suite before you start is allowed and recommended; it is a specification written in a second language.
- The suite tests nothing about concurrency, and says so at the top. That is Chapter 7.

Looking ahead

Chapter 6 is about:

- Where a driver gets memory, and the flag that says which context you are in.
- How a driver reaches its hardware, and why the compiler must not be trusted with it.
- `ioremap`, `readl`, and the address you compute by hand before you map it.
- The error path you write four times and then delete.

5.4 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 5.6 onwards need the modules from §5.2.

Where an exercise can be checked mechanically, a **Check yourself** line says how. Several have a plausible wrong answer that is worth walking into before you find it.

Solutions. The exercises that are not Code are answered in §A.5. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 5.1 The node and the number

RECALL

- Your driver calls `alloc_chrdev_region` and gets major 243. A colleague hardcodes 243 in a deployment script. Give two separate reasons that will eventually break.
- What does `mknod /dev/spare c 243 0` do, given that `/dev/qa_fifo` already exists with the same numbers? What does opening it do?
- Your driver loads, `/proc/devices` shows your name against a major, and `/dev/qa_fifo` does not exist. Which of the five init steps did you skip?
- Conversely: the node exists, but opening it returns `ENXIO`. What does that tell you, and what does it rule out?

Exercise 5.2 The table

RECALL

- `.owner = THIS_MODULE` is one line and easy to omit. Describe precisely what can go wrong without it, in terms of what a process is holding and what `rmmod` frees.
- You implement `.read` and `.write` but not `.release`. Is that a bug? What is called instead?
- You implement `.read` but not `.open`. Can userspace open the device?
- After `cdev_add` returns, but before your init function returns, what can happen? Name the ordering rule this implies.

e) `release` is not called once per `close()`. When is it called, and construct a case where two `close()` calls produce one release.

Exercise 5.3 Return values

HAND CALCULATION

For each, say what the driver should return, and what the userspace program will observe.

- a) `read(fd, buf, 100)` with 40 bytes in the FIFO.
- b) `read(fd, buf, 100)` with the FIFO empty.
- c) `read(fd, buf, 0)` with 40 bytes in the FIFO.
- d) `write(fd, buf, 100)` with 60 bytes of space free.
- e) `write(fd, buf, 100)` with the FIFO full.
- f) `read(fd, buf, 100)` where `buf` points at an unmapped page.

Then: g) for b), suppose the driver returns 0 instead. Write down what each of these does: `cat /dev/qa_fifo`; a program looping while `((n = read(fd, b, 64)) > 0)`; and a program checking if `(n < 0) perror(...)`. Which of the three reports a problem?

Exercise 5.4 A sequence of reads

HAND CALCULATION

A FIFO of capacity 8, created empty. Trace this sequence and give the return value of every call and the FIFO's level after it.

```
1. write(fd, "abcdefghij", 10)
2. read(fd, buf, 3)
3. write(fd, "klm", 3)
4. read(fd, buf, 100)
5. read(fd, buf, 100)
```

- a) Fill in the ten numbers.
- b) After step 4, exactly which bytes are in `buf`, in order?
- c) Step 3 succeeds even though step 1 could not fit everything. Explain why in terms of what step 2 did.
- d) Which steps would behave differently if the ring buffer wasted one slot to distinguish full from empty? Give the old and the new return value for each.

Exercise 5.5 Where does this belong

DESIGN

For each piece of information a driver might expose, choose `read()` on the device node, a **sysfs attribute**, **debugfs**, or an **ioctl**, and justify it in one sentence.

- a) The stream of bytes the device produces.
- b) The FIFO's configured capacity.
- c) How many bytes have been dropped since load, for a field engineer to read.
- d) A count of internal retries, useful only while you are debugging the driver.
- e) An instruction to discard everything currently buffered.
- f) The device's serial number.

Then: **g)** two of your answers are an ABI you will maintain indefinitely and one is explicitly not. Which are which, and what follows for a product that ships?

Exercise 5.6 The ring buffer

CODE

Write `lectures/L05/lab/qa_fifo.c` to the specification in `lectures/L05/lab/qa_fifo.h` and §5.2.1.

Check yourself: load `qa_fifo.ko` and then `qa_fifo_kunit.ko` on the target and read `dmesg`. The suite must report `pass:10 fail:0`. `make test L=L05` does this for you and prints the summary line.

Two things worth knowing before you start, both of which the suite checks and neither of which is obvious:

- A FIFO of capacity 4 must hold **four** bytes.
- Test 8 is the one that fails first for most people, and it is the eighth of ten, so seven passing tests tell you very little.

Exercise 5.7 The character device

CODE

Write `lectures/L05/lab/qa_chardev.c` to §5.2.2.

Check yourself: `make test L=L05` reports **PASSED** with eighteen checks, and the userspace tester prints `qa_fifo_test: 0 failure(s)`.

- a) Before running the tester, load your module and record the major from `/proc/devices` and from `dmesg`. Reboot and do it again. Are they the same? Should a script rely on that?
- b) Load the module twice without unloading. What happens, and where does the failure come from?
- c) Deliberately omit `stream_open` from your `open` and rerun the tester. Which checks fail? Explain the result from §5.2.2, then find a system call that does behave differently without

`stream_open`, and say what the difference means for a program that assumes a file offset means something.

Exercise 5.8 Reading the tests for what they miss

DESIGN

Read `lectures/L05/lab/qa_fifo_kunit.c` and `lectures/L05/lab/user/qa_fifo_test.c` before answering.

- a) The KUnit suite says at the top that it checks nothing about concurrency. Construct a specific interleaving of two `qa_fifo_put` calls that corrupts the buffer, and say which of the ten tests would notice. (The answer is none, which is the point.)
- b) Name three other properties neither test suite checks. For each, say whether it would be worth testing and how you would do it.
- c) The userspace tester exists as a compiled program rather than as shell. Name two checks in it that a shell script genuinely could not make, and say why.
- d) Your driver leaks the FIFO if `class_create` fails. Would either suite catch it? What would?

Exercise 5.9 A byte count worked out twice

CROSS-CHECK

Predict a sequence of return values by hand, then have the machine produce them, and reconcile.

- a) **By hand.** Your FIFO has the default capacity of 256. Predict the exact return value of every call in this sequence, and the level after each:

1. write of 200 bytes
2. write of 100 bytes
3. read of 512 bytes
4. read of 512 bytes
5. write of 300 bytes

Write down all five return values, all five levels, and the total number of bytes that will have passed through the device when the sequence ends.

- b) **By machine.** Write a short program in `lectures/L05/lab/user/` that performs exactly that sequence against `/dev/qa_fifo` and prints each return value and `errno`. Anything you put in that directory is cross-compiled by `make build` and copied to `/lab` on the target, so you can run it there directly.
- c) **Reconcile.** Compare your ten numbers against the machine's. Then answer:
 - Step 2 is the one people get wrong. What did you predict, what happened, and which contract in §5.2.2 decides it?

- Did any call return a value you had classed as an error, or an error you had classed as a value?
- The total bytes through the device: does your figure count step 2's outcome the same way the machine does?

d) **Now change one thing.** Reload the module with `insmod ./qa_chardev.ko capacity=128` and run the same program without changing it. Which of the five return values change, and which do not? Explain each unchanged one.

e) Your program hardcodes 256 in its predictions; the shipped `qa_fifo_test` does not, and measures the capacity instead. Say what the shipped tester would report on a `capacity=128` device and what yours would, and draw the general lesson about tests that encode a configuration.

f) A colleague reports “the device only accepted 56 bytes of my 300-byte write, it is broken”. Given the sequence above, say what actually happened, what their program should have done, and which single line of §5.1.4 they had not read.

Memory, MMIO, and Resource Management

Where a driver gets memory, how it reaches its hardware, and how it gives both back.

IN THIS CHAPTER

- The kernel's memory map, and the difference between an address and a mapping.
- `kmalloc`, `vmalloc`, `kvmalloc`, `alloc_pages`: four allocators and how to pick.
- GFP flags as a statement about context, not about urgency.
- `request_mem_region`, `/proc/iomem`, and what claiming a region does and does not prevent.
- `ioremap` and `__iomem`: why a register is not memory and must not be treated as such.
- `readl` and `writel`, ordering, and when `_relaxed` is safe.
- `devm_*`, the error path it deletes, and why you cannot use it yet.
- Map `qa-dev`, read its identity, and prove the mapping is right.
- The DMA API, for the device that reads memory itself; there is nothing to run, because `qa-dev` cannot do DMA.

This chapter is about the two things a driver needs that its callbacks do not give it: memory, and a way to reach its hardware. It explains the difference between a physical and a virtual address, which of the four allocators to ask and what each of them fails like, and why the GFP flag is a statement about the context you are in rather than about how badly you want the memory. It then follows one device from the numbers in its device tree node to a register read: claiming the region, mapping it, reading it with accessors the compiler cannot rearrange, and unwinding all of it in reverse when something fails. The unwinding is written by hand, because `devm_` needs a device and a bare module does not have one.

Read §6.1 first; it is kernel memory and the allocators. Its §6.1.6, the DMA API, is reading only: `qa-dev` cannot do DMA, so nothing in it is measured or tested, and the section says what that costs. §6.2 is MMIO and resource management, and prints the device's register map, which every driver from here on is written against. The lab is specified in Exercise 6.7: a module, `qa_mmio`, that takes the device's physical address as a parameter, claims and maps the region, checks the identity register, and writes and reads back SCRATCH. The exercises in

§6.4 end with the Cross-check: translate qa-dev’s reg through its parent’s ranges by hand, predict what the first two registers will read, then map them and find out. The lab is done when `make test L=L06` reports **PASSED**, with nine checks.

6.1 Kernel Memory

Where a driver gets memory, which allocator to ask, and what each of them fails like. §6.2 is how a driver reaches its hardware.

The one idea to carry through: **in the kernel, choosing an allocator is choosing a failure mode**. Userspace has `malloc`, which either works or the process dies. Up here you pick between allocators that fail in different ways at different sizes under different conditions, and the flag you pass is a statement about where your code is running.

6.1.1 Two things called an address

A **physical address** is where something is on the bus. A **virtual address** is what your code may dereference. The MMU maps the second to the first.

In userspace you never see a physical address and rarely think about the distinction. In the kernel you see both, and confusing them is the most common cause of a crash in a first driver. The types help a little:

Type	Holds	Dereference it?
<code>void *</code>	A kernel virtual address	Yes
<code>phys_addr_t</code>	A physical address	No . It is a number
<code>resource_size_t</code>	A physical address or a size	No
<code>void __iomem *</code>	A mapped device register window	Only with <code>readl/writel</code>
<code>char __user *</code>	A userspace virtual address	No . <code>copy_*_user</code> only

Four of the five are not pointers you may follow, and two of them are spelled like pointers. `__iomem` and `__user` are annotations that compile to nothing; what makes them useful is sparse, a static checker that does understand them:

```
make C=1 M=$PWD modules      # run sparse over your module
```

/proc/iomem is a map of the first kind. It lists physical ranges that a driver has claimed. Exercise 1.7 looked there for qa-dev and found nothing, because nothing had claimed it. §6.2.2 is where that changes.

Two is the whole story while the CPU moves every byte, which is true of everything this course builds. A device that reads memory by itself introduces a third, and §6.1.6 is that one.

6.1.2 The allocators

Four, and the differences that matter are contiguity, size limit and failure mode.

Function	Gives you	Physically contiguous	Practical limit	Can sleep
<code>kmalloc</code>	A small object	Yes	About 4 MB, fragmenting well before that	Depends on flag
<code>kzalloc</code>	The same, zeroed	Yes	Same	Depends on flag
<code>vmalloc</code>	A large buffer	No	Large	Always
<code>kvmalloc</code>	Tries <code>kmalloc</code> , falls back	Only if it got lucky	Large	Yes
<code>alloc_pages</code>	Whole pages	Yes	Order-limited	Depends on flag

`kmalloc` is what you want, almost always. It comes from the slab allocator, it is fast, and it returns physically contiguous memory, which is what hardware doing DMA needs.

Its size limit is not the one documented. The hard cap is a few megabytes, but the practical limit is much lower and it is not a constant: `kmalloc` must find *physically contiguous* pages, and after a system has been up for a while, physical memory is fragmented. An allocation of 128 KB that succeeds every time on your desk can fail on a machine that has been running for a month. **This is the failure that does not appear in testing**, and it is why anything large should be `vmalloc` or `kvmalloc`.

`vmalloc` gives virtually contiguous memory built from pages that are not physically adjacent. That is fine for a big software buffer and useless for DMA, which sees physical addresses. It is also slower to allocate and costs page table entries.

`kvmalloc` is the sensible default for “large, and I do not need contiguity”: it tries `kmalloc` and falls back to `vmalloc`. Note that you therefore do not know which you got, so it must not be used for anything that needs physical contiguity.

Free with the matching call: `kfree`, `vfree`, `kvmfree`, `__free_pages`. Mismatching them is a bug that usually survives testing.

6.1.3 GFP flags say where you are, not how much you want

The flag is the second argument to `kmalloc` and it is the part people treat as decoration. It is not a priority. **It tells the allocator what it is allowed to do to satisfy you**, and the thing that matters most is whether it may put you to sleep.

Flag	Means	Use in
<code>GFP_KERNEL</code>	May sleep: may reclaim, may swap, may wait for I/O	Process context only
<code>GFP_ATOMIC</code>	May not sleep. Takes from an emergency pool	Interrupt handlers, code holding a spinlock
<code>GFP_NOWAIT</code>	May not sleep and may not use the emergency pool	Where failure is fine
<code>GFP_DMA32</code>	Restrict to memory a 32-bit device can reach	DMA with an addressing limit

Two consequences follow, and they are the whole of why this matters:

GFP_KERNEL in the wrong place is a bug even when it works. Calling it while holding a spinlock, or from an interrupt handler, means the kernel may try to sleep in a context where sleeping deadlocks the machine. It will often get away with it, because memory was available and it never needed to sleep. It will fail on the day the system is under pressure. CONFIG_DEBUG_ATOMIC_SLEEP, which this course enables, turns that into a loud complaint instead; Chapter 7 is where you make it complain on purpose.

GFP_ATOMIC is more likely to fail, not less. It cannot reclaim, so it takes from a small reserve. Code using it must handle failure, and must not use it merely because it seemed safer.

The rule that follows is short: **choose the flag from the context you are in, not from how badly you want the memory.** If you cannot say whether your code can sleep, that is the question to answer first, and Chapter 7 is about answering it.

6.1.4 What a driver actually allocates

For the driver this course builds, in order of appearance:

- **A small state structure**, one per device. `kzalloc(sizeof(*priv), GFP_KERNEL)` in probe.
- **A bounce buffer** for `copy_to_user`, as in Chapter 5. Small, short-lived, `GFP_KERNEL`, because read runs in process context.
- **Nothing at all for the registers.** Device registers are not memory and are not allocated. §6.2.3 is what happens instead, and it is the distinction this chapter turns on.

That last point is worth being explicit about, because the syntax hides it. After `ioremap` you hold something spelled like a pointer to memory, and it is not memory: there is no storage behind it, reads have side effects, and the number of bytes you “allocated” is zero.

6.1.5 The page allocator underneath

`kmalloc` is built on the **slab allocator**, which is built on the **page allocator**. You can see all three:

```
# head -5 /proc/meminfo
MemTotal:      481248 kB
MemFree:       448524 kB

# head -3 /proc/slabinfo      # what the slab is caching, and how much
# cat /proc/buddyinfo        # free pages, by order
```

`/proc/buddyinfo` is the one worth knowing, because it is where the fragmentation in §6.1.2 becomes visible: it lists how many free blocks exist at each order, from single pages up. A machine with plenty of free memory and nothing above order 4 cannot satisfy a large `kmalloc`, and `/proc/meminfo` will not tell you that.

`alloc_pages(gfp, order)` asks for 2^{order} contiguous pages directly. Drivers rarely need it; it appears when you want a whole page for DMA or a ring, and it returns a `struct page` * rather than an address, which is a different currency again.

6.1.6 A third thing called an address

§6.1.1 said there were two. There are two until a device reads memory by itself, and then there is a third: the **bus address**, which is the number the device puts on the bus to reach your buffer. It has its own type, `dma_addr_t`, and the rule about it is the same rule as for `phys_addr_t`. **It is a number, the CPU may not dereference it, and it is not necessarily the physical address.**

On a simple SoC it usually is the physical address, and that is the trap. An IOMMU between the device and memory translates it; a bus can apply a fixed offset. So a driver that computes `virt_to_phys(buf)` and writes the result into a device register works on the board you have and corrupts memory on the one with an IOMMU, and it corrupts it silently, somewhere else, later. **Never hand a device an address you calculated. Ask for one.**

Asking is the DMA API, and there are two ways to ask.

Coherent, for what both sides touch constantly

```
void *cpu;
dma_addr_t handle;

cpu = dma_alloc_coherent(dev, size, &handle, GFP_KERNEL);
/* cpu is yours to dereference; handle is what you write into the device. */

/* ... the device's lifetime happens here ... */

dma_free_coherent(dev, size, cpu, handle);
```

One call allocates and maps. You get both addresses, and coherency stops being your problem: what the CPU writes, the device sees. The cost is that the memory is usually mapped uncached or otherwise made special, so the CPU reads it slowly. Use it for descriptor rings and status words, which both sides poke at constantly and neither reads in bulk. `dma_alloc_coherent` is the `devm_` version, and §6.1.7 is about why that matters.

Coherent does not mean ordered, and the two get confused. It means the device will not read a stale value; it does not mean your writes reach memory in the order you made them. Filling a descriptor and then writing the register that tells the device to look at it is exactly the pattern §6.2.5 is about, and it needs the same barrier here as it does there.

Streaming, for a buffer you already had

```

dma_addr_t handle = dma_map_single(dev, buf, size, DMA_TO_DEVICE);

if (dma_mapping_error(dev, handle))
    return -ENOMEM;
/* ... the transfer happens ... */
dma_unmap_single(dev, handle, size, DMA_TO_DEVICE);

```

Here the buffer already exists and you are lending it out. The direction is one of `DMA_TO_DEVICE`, `DMA_FROM_DEVICE` and `DMA_BIDIRECTIONAL`, and it is not documentation: it tells the kernel whether to flush the CPU's caches, invalidate them, or both. Getting it wrong is a data bug, not an error.

And mapping can fail, which is the line first drivers leave out. `dma_map_single` returns `DMA_MAPPING_ERROR` rather than setting an `errno`, so the check is `dma_mapping_error` and there is no other way to spot it.

The rule the section is for

Between the map and the unmap, the buffer belongs to the device. Reading it or writing it from the CPU in that window is a bug. If you have to look, `dma_sync_single_for_cpu` borrows it back and `dma_sync_single_for_device` returns it.

This is the same shape of rule as `__iomem` in §6.2.3 and as atomic context in Chapter 7: a statement about which side owns something, enforced by nothing at compile time. It is worth taking seriously precisely because breaking it is so quiet. A machine whose caches happen to be coherent with its devices runs the broken code correctly forever, which means your desk, your CI and your emulator can all agree it is fine.

Why any of this exists

The cache. Your store may still be sitting in a dirty line the device cannot see, so the device reads memory and gets the old value. In the other direction the CPU holds a clean line, the device overwrites the memory underneath it, and the CPU never notices. Mapping and unmapping are where the flush and the invalidate happen, and the direction is how the kernel knows which it needs.

Note what this says about §6.2. MMIO needs none of it, because `ioremap` maps device registers uncached by construction: there is no cache line to be stale. A DMA buffer is ordinary cached RAM, and that single difference is the whole of this section.

What you may not hand a device

- **`vmalloc` or `kvmalloc` memory.** §6.1.2 said it in a line; here is why. The buffer is virtually contiguous and physically is not, so a device given the address of the first page walks straight off the end of it into whatever page happens to follow. The kernel refuses this one rather than letting it through: `dma_map_single_attrs` checks `is_vmalloc_addr` and

returns `DMA_MAPPING_ERROR` with a warning. Use `kmalloc`, `alloc_pages`, or `dma_alloc_coherent`.

- **Anything on the kernel stack.** Two reasons, and the first is the one above wearing a disguise: this course's kernel sets `CONFIG_VMAP_STACK=y`, as arm64 does by default, so the stack is itself virtually mapped and `is_vmalloc_addr` catches a stack buffer too. The second reason outlives that config. A DMA buffer wants a cache line to itself, and a local variable shares its line with whatever the compiler put next to it, so an invalidate that is correct for your buffer destroys a neighbour's data. Allocate it; do not declare it.
- **Memory the device cannot reach.** A device with 32 address lines cannot address anything above 4 GB, whatever the CPU can. `dma_set_mask_and_coherent(dev, DMA_BIT_MASK(32))` in probe is how you say so, and after it the kernel allocates low or bounces through a temporary buffer for you. That is where `GFP_DMA32` in the §6.1.3 table comes from, and why you should not normally be reaching for that flag yourself.

Two more, in a sentence each

Scatter-gather. For a buffer that is many pages and need not be one run, `dma_map_sg/dma_unmap_sg` map a whole struct `scatterlist` at once. The count it returns can be smaller than the count you passed, because the kernel merges entries that turned out to be adjacent, and iterating over the number you passed instead of the number you got back is the standard bug.

dmaengine. On many SoCs your device does not master the bus at all; a separate DMA controller does, and you request a channel from it with `dma_request_chan()` and describe a transfer. That controller already has a driver, and the argument for using it instead of poking its registers is exactly the argument Chapter 11 makes about every framework.

There is no lab for this, and that is worth saying

qa-dev has no DMA engine. It is a sixteen-deep FIFO and an interrupt line, and every sample in this course arrives through `readl`. So nothing above is measured on the target and nothing above is in `make test`; this section is reading, held to the same standard §12.2 holds its histograms to.

Where you meet it first is the first real device whose FIFO is shallower than the burst it produces. At that point the interrupt-per-sample loop you build in Chapter 8 stops keeping up, and the answer is not a faster handler. It is to stop moving the data with the CPU.

6.1.7 devm_ and what it is attached to

The kernel has a managed version of most allocation functions:

```
priv = devm_kzalloc(dev, sizeof(*priv), GFP_KERNEL);
```

Nothing frees that explicitly. It is released automatically when **the device it is attached to is unbound from its driver**, which is the exact event: not when the module unloads, not when the process exits, but when the driver stops being responsible for that device.

That is what makes it valuable, because it turns the error ladder in §5.1.7 into nothing: if probe fails halfway through, everything acquired so far is released for you, in reverse order, without your writing any of it.

And it is why you cannot use it in this chapter. Every `devm_` function takes the device it attaches to, nearly always as a `struct device *` first argument:

```
void __iomem *devm_ioremap(struct device *dev, resource_size_t offset, ↵
↵ resource_size_t size);
```

A bare module is not a device. It has no `struct device`, nothing is bound to it, and there is no unbind event for a destructor to hang from. So this chapter's lab writes the unwind path by hand, knowing it is temporary. Chapter 10 turns the module into a platform driver, which arrives with a `struct device *` in its probe, and the ladder is deleted there.

Writing it once by hand first is deliberate. `devm_` is easy to use and easy to misunderstand as “the kernel tidies up after you”; it is not, it is a destructor list attached to one specific object's lifetime, and knowing what that object is matters the first time something is released earlier or later than you expected.

6.2 MMIO and Resource Management

How a driver reaches its hardware: finding the address, claiming it, mapping it, and reading it without the compiler or the processor helpfully rearranging what you asked for.

Every listing here was produced on this course's target, against the `qa-dev` device. Its register map is `tools/qa-dev/qa-dev.h`, and that header is the only place the map exists: the QEMU device model includes it, and so does every driver from here on, through a copy in its lab directory. Read it as you would a datasheet. Here it is, typeset straight from the repository:

```
tools/qa-dev/qa-dev.h
/**
 * @file QAcademy teaching device.
 */
#ifndef QA_DEV_H
#define QA_DEV_H

/** The QOM type name, which is what "-device qa-dev" names. */
#define TYPE_QA_DEV "qa-dev"

/** The MMIO window. One page, which is the smallest thing an MMU can map. */
#define QA_DEV_MMIO_SIZE 0x1000U
```

```

/** Number of samples the device buffers before it starts dropping them and setting OVERRUN. */
#define QA_DEV_FIFO_DEPTH 16U

/** The shortest period the device accepts. */
#define QA_DEV_MIN_PERIOD_NS 1000U

/** Register offsets. Every register is 32 bits and must be accessed as 32 bits. */
#define QA_DEV_ID 0x00U          /**< R0. Magic. */
#define QA_DEV_VERSION 0x04U     /**< R0. Major in bits 15:8, minor in bits 7:0. */
#define QA_DEV_CTRL 0x08U        /**< RW. */
#define QA_DEV_STATUS 0x0CU       /**< R0. */
#define QA_DEV_PERIOD_NS 0x10U   /**< RW. Sample period in nanoseconds. */
#define QA_DEV_SAMPLE 0x14U      /**< R0. Reading pops one sample from the FIFO. */
#define QA_DEV_FIFO_LEVEL 0x18U  /**< R0. Samples currently queued. */
#define QA_DEV_IRQ_STATUS 0x1CU  /**< RW1C. Write a one to a bit to clear it. */
#define QA_DEV_TS_LO 0x20U       /**< R0. Low 32 bits of the last interrupt's timestamp. */
#define QA_DEV_TS_HI 0x24U       /**< R0. High 32 bits. Latched by reading TS_LO. */
#define QA_DEV_GPIO_OUT 0x28U    /**< RW. Outputs in bits 7:0; bits 3:0 loop back to GPIO_IN. */
#define QA_DEV_GPIO_IN 0x2CU     /**< R0. Bits 7:4 read back GPIO_OUT bits 3:0; bits 3:0 read 0. */
#define QA_DEV_SCRATCH 0x30U     /**< RW. Reads back what was written. Nothing else uses it. */

/** The value of QA_DEV_ID: "QADV" in ASCII, big end first. */
#define QA_DEV_ID_MAGIC 0x51414456U

/** QA_DEV_CTRL. */
#define QA_DEV_CTRL_ENABLE (1U << 0U) /**< Start sampling. */
#define QA_DEV_CTRL_IRQ_EN (1U << 1U) /**< Let a pending IRQ_STATUS bit raise the line. */
#define QA_DEV_CTRL_RESET (1U << 2U) /**< Self-clearing. Empties the FIFO and the status bits. */
#define QA_DEV_CTRL_MODE_SHIFT 4U
#define QA_DEV_CTRL_MODE_MASK (3U << QA_DEV_CTRL_MODE_SHIFT)

/** Sample modes. */
#define QA_DEV_MODE_COUNTER 0U /**< Sample n is n. */
#define QA_DEV_MODE_SAWTOOTH 1U /**< Sample n is n modulo 4096. */
#define QA_DEV_MODE_NOISE 2U /**< A linear congruential sequence from a fixed seed. */

/** QA_DEV_STATUS. */
#define QA_DEV_STATUS_READY (1U << 0U)
#define QA_DEV_STATUS_FIFO_EMPTY (1U << 1U)
#define QA_DEV_STATUS_FIFO_FULL (1U << 2U)
#define QA_DEV_STATUS_OVERRUN (1U << 3U) /**< Sticky. Cleared by clearing IRQ_STATUS_OVERRUN. */

/** QA_DEV_IRQ_STATUS. Write-one-to-clear, both bits. */
#define QA_DEV_IRQ_SAMPLE (1U << 0U) /**< At least one sample is waiting. */
#define QA_DEV_IRQ_OVERRUN (1U << 1U) /**< A sample was dropped because the FIFO was full. */

/** The device tree node this device generates, and the string a driver matches on. */
#define QA_DEV_DT_COMPATIBLE "qacademy,qa-dev-1.0"

#endif /* QA_DEV_H */

```

6.2.1 Where the address comes from

On an embedded system, nothing discovers it. Somebody wrote it down, and in a device tree that somebody is the board's author. `qa-dev`'s node, as QEMU generated it:

```
platform-bus@c0000000 {
    ranges = <0x00 0x00 0xc0000000 0x20000000>;
    #address-cells = <0x01>;
    #size-cells = <0x01>;

    qa-dev@0 {
        compatible = "qacademy,qa-dev-1.0";
        reg = <0x00 0x1000>;
        interrupts = <0x00 0x70 0x04>;
    };
};
```

reg = <0x00 0x1000> does not say the device is at address zero. It says the device begins at offset 0 on *its parent bus*, and is 0x1000 bytes long. To get a physical address you translate through the parent's ranges, which reads: child 0x0 maps to parent 0xC000000, for 0x2000000 bytes. So:

$$0x0 + 0xC000000 = \mathbf{0xC000000}$$

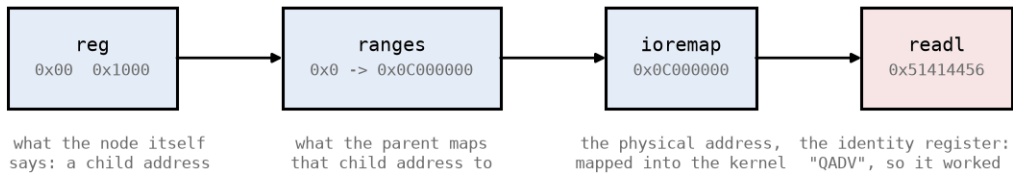
A reader who takes `reg` at face value maps address 0 and reads nothing useful. That translation is the Cross-check, and Chapter 10 is where `platform_get_resource` does it for you.

The cells are 32-bit big-endian, whatever the machine's own endianness. On the target the properties are raw bytes:

```
od -An -tx1 -v /sys/firmware/devicetree/base/platform-bus@c0000000/qa-dev@0/reg
```

Reading them with `od -tx4` is the obvious shortcut and is wrong: it decodes four bytes as a *native-endian* integer, so on this little-endian target every cell comes back byte-swapped and 0xC0000000 reads as 0x0000000C. That number is plausible enough to act on, which is what makes it worth knowing. The lab's `run.sh` reassembles bytes by hand for exactly this reason.

reg is not an address until the parent has translated it



Take reg at face value and you map address 0. A wrong mapping fails two ways and only one of them is loud: an unbacked address raises a synchronous external abort, and an address that is merely not the device reads plausible zeros with no error at all.

Which is why probe reads the identity register and refuses to continue unless it matches.

Figure 6.1 Four boxes in a row showing one address being translated: reg holding 0x00 and 0x1000, ranges mapping child 0x0 to 0x0C000000, ioremap taking that physical address, and readl returning 0x51414456. Below, a note that a wrong mapping fails two ways and only one of them is loud.

6.2.2 Claiming the region

```

region = request_mem_region(base, QA_DEV_MMIO_SIZE, "qa_mmio");
if (!region)
    return -EBUSY;
  
```

This maps nothing and prevents nothing. It is an entry in a registry. What it buys you is that a *second* driver calling it for an overlapping range gets NULL, so two drivers cannot both believe they own the same device and work intermittently.

It is bookkeeping that the hardware knows nothing about. A driver that skips it and calls ioremap directly works fine, right up until the day a second driver does the same.

The registry is visible, and this is the payoff for Exercise 1.7, which looked for qa-dev in /proc/iomem and found nothing:

```

# grep qa_mmio /proc/iomem
0c000000-0c000fff : qa_mmio
  
```

The name in that listing is the string you passed, so make it the driver's name and not something generic. Release with `release_mem_region(base, size);` failing to means the next insmod gets -EBUSY from your own previous load, which is a confusing way to discover a leak.

6.2.3 Mapping it

```
regs = ioremap(base, QA_DEV_MMIO_SIZE);
if (!regs)
    return -ENOMEM;
```

`ioremap` creates a kernel virtual mapping for a physical range and returns `void __iomem *`.

It is not memory, and the distinction is not pedantry. Four things are true of device registers and false of memory:

- **Reads have side effects.** Reading `QA_DEV_SAMPLE` removes a sample from the FIFO. Reading it twice is not the same as reading it once.
- **Writes may not be merged or reordered.** Two writes to the same register are two operations.
- **It must not be cached.** `ioremap` maps it uncached; a cached mapping would let the processor satisfy a read from a stale line and postpone a write indefinitely.
- **There is no storage.** Nothing was allocated. `ioremap` returns the mapping, not any memory.

Which is why the compiler must not be trusted with it. Given an ordinary pointer, a compiler may eliminate a read whose value you ignore, merge two writes, or hoist a read out of a loop. All three are correct for memory and all three break a driver.

6.2.4 `readl` and `writel`

```
u32 id = readl(regs + QA_DEV_ID);
writel(0xdeadbeef, regs + QA_DEV_SCRATCH);
```

These are the only legal way to touch an `__iomem` pointer. They compile to a single load or store plus the barriers the architecture needs, and they are not optimised away.

Accessor	Width
<code>readb/writeb</code>	8 bits
<code>readw/writew</code>	16 bits
<code>readl/writel</code>	32 bits
<code>readq/writeq</code>	64 bits

Use the width the device documents. `qa-dev` accepts 32-bit accesses only and rejects anything else rather than quietly widening it, which is deliberate: a driver that gets away with a byte write to a word register here would not get away with it on real hardware.

The address arithmetic is plain byte arithmetic on the returned pointer, so `regs + QA_DEV_ID` with the offsets from the register map is exactly right, and casting to a struct of `u32` fields is a common alternative that works until a register map has a gap in it.

6.2.5 Ordering, and the `_relaxed` variants

`writel` carries a barrier; `writel_relaxed` does not. What the barrier orders is the part worth getting right.

Accesses from one CPU to one device's registers, through a mapping `ioremap` made, arrive at the device in program order either way. Write a control register and then read the status register, relaxed or not, and the device sees the write first. What the barrier orders is a register access against *ordinary memory*: `writel` waits for every earlier store to RAM before its own store, and `readl` completes before any later load from RAM begins. That matters the moment the device reads memory itself. Fill a descriptor in RAM, then write the register that tells the device to fetch it:

```
desc->len = len;                                /* ordinary memory */
writel(DOORBELL_G0, regs + DOORBELL);           /* ordered after it */
```

Without the barrier the processor may let the register write overtake the store to the descriptor, and the device fetches a descriptor from before your own update. `qa-dev` reads no memory, so every access this course's labs make is to one device, and the difference never shows on this target.

The relaxed forms exist because barriers cost time, and a driver writing a burst of registers where only the last one matters can use `_relaxed` for the burst and an ordered access at the end. **Use the ordered forms until you have measured a reason not to.** A missing barrier produces a driver that works on one machine and fails on another, which is the single worst class of bug to debug.

This is the same memory ordering that is a property of the processor on its own; here there is hardware on the other end that also has an opinion.

6.2.6 What a wrong address does

Worth knowing before you guess one, and the answer is not “you read a wrong number”.

A wrong-but-mapped address reads plausibly. Pointing the lab's module at `0xC` rather than `0xC0000000` produces:

```
qa_mmio: id=0x00000000 version=0x0000
```

No error, no fault, just zeros. A driver that did not check its identity register would carry on and misbehave later.

A wrong-and-unmapped address aborts the machine. Pointing it one page past the device, at `0xC001000`, produces this:

```
Internal error: synchronous external abort: 0000000096000010 [#1] PREEMPT SMP
Modules linked in: qa_mmio(0+)
CPU: 0 UID: 0 PID: 67 Comm: insmod Tainted: G          0      6.12.30
```

```
pc : qa_mmio_init+0x98/0xff8 [qa_mmio]
Call trace:
  qa_mmio_init+0x98/0xff8 [qa_mmio]
  do_one_initcall+0x84/0x360
```

insmod is killed, the module is left half-loaded so it can be neither used nor removed, and the sensible next step is a reboot. On this target that costs two seconds; on a board on a bench it costs a walk across the room, and on a product in the field it costs a truck.

Three lessons, and the third is the one worth carrying:

- **Always check an identity register** if the device has one. qa-dev has QA_DEV_ID for this, and reading 0x51414456 is what tells you the mapping landed where you meant.
- **The failure mode depends on the address**, so “it did not crash” is not evidence that a mapping is right.
- **This is what the emulator is for.** §1.1.4 said the course cannot teach you what a bricked board feels like; the other side of that trade is that you can point a driver at a wrong address here and find out what happens, which is not an experiment anyone runs twice on real hardware.

6.2.7 The whole sequence, and its unwind

Acquire in order, release in reverse, and unwind only what you actually got:

```
region = request_mem_region(base, QA_DEV_MMIO_SIZE, "qa_mmio");
if (!region)
    return -EBUSY;

regs = ioremap(base, QA_DEV_MMIO_SIZE);
if (!regs) {
    ret = -ENOMEM;
    goto err_region;
}

if (readl(regs + QA_DEV_ID) != QA_DEV_ID_MAGIC) {
    ret = -ENODEV;
    goto err_map;
}
return 0;

err_map:
    iounmap(regs);
err_region:
    release_mem_region(base, QA_DEV_MMIO_SIZE);
```

```
return ret;
```

Two acquisitions, two labels, in reverse. Add a third and you add a third label; the pattern does not get cleverer, it gets longer, and by the time a real driver has a clock, a regulator, a reset line and three mappings it is twenty lines of unwinding that must stay in step with the acquire path.

That is the argument for `devm_`, which §6.1.7 explains and which this chapter cannot use, because every `devm_` function needs a `struct device *` and a bare module does not have one. Chapter 10 is where one appears and this ladder is deleted.

6.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Tell a physical address from a virtual one in unfamiliar driver code, from the types alone.
- Choose an allocator from the size and the contiguity requirement, and say what it fails like.
- Choose a GFP flag from the context you are in, and say what GFP_ATOMIC costs you.
- Say what `request_mem_region` prevents and what it does not.
- Map a device's registers and read them without the compiler reordering or eliding the access.
- Explain what `__iomem` is for, given that it compiles to nothing.
- Say when `writel_relaxed` is safe and when it is not.
- Say what `devm_*` buys, what it is attached to, and why a bare module cannot use it.
- Say why a `vmalloc` buffer cannot be handed to a device, and which two calls give you one that can.

Questions to test yourself

1. A device tree node says `reg = <0x0 0x1000>` and its parent's `ranges` is `<0x0 0x0 0xC0000000 0x20000000>`. What physical address do the registers start at?
2. You `ioremap` a region and read the ID register; it returns `0xFFFFFFFF`. Name three different causes, and say how you would tell them apart.
3. Why can `kmalloc(GFP_KERNEL)` not be called from an interrupt handler?
4. What does `__iomem` do at run time?
5. Your driver fills a descriptor in memory and then tells the device to fetch it with `writel_relaxed`. It works on one machine and not another. What is the likely cause?
6. `devm_kzalloc` memory is freed when? Name the exact event, and say what has to exist before you can call it at all.

7. You map a buffer with `dma_map_single(..., DMA_FROM_DEVICE)` and read it before unmapping. On your desk it works every time. What have you actually done, and where does it stop working?

Where to look

- §1.1.8 is where the device tree node was first shown, with the numbers you now have to translate.

Looking ahead

Chapter 7 is about:

- Every way two pieces of kernel code can run at once, which is more ways than you expect.
- Why the ring buffer you wrote in Chapter 5 is already broken.
- Spinlocks and mutexes, and the question that chooses between them.
- A deadlock you build on purpose, and the tool that reports it.

6.4 Exercises

Ten, ending with the Cross-check. Do them in order; Exercise 6.7 onwards need the target running.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

One warning before you start. Exercise 6.9 asks you to point a driver at addresses that are not the device. One of them will oops the machine. That is intended, it is survivable here, and §6.2.6 says what to expect. Do not run that exercise against anything you cannot reboot in two seconds.

Solutions. The exercises that are not Code are answered in §A.6. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 6.1 Two kinds of address

RECALL

- For each of `void *`, `phys_addr_t`, `void __iomem *` and `char __user *`, say whether you may dereference it, and if not, what you must use instead.
- `__iomem` and `__user` compile to nothing. What are they for, and what tool makes them useful?
- `/proc/iomem` lists physical ranges. Does an entry there mean the range is mapped? Does its absence mean the hardware is not present?
- Your driver stores the value returned by `ioremap` in a `void *` rather than a `void __iomem *`. It compiles and works. Name what you have lost.

Exercise 6.2 The allocators

RECALL

- a) Choose an allocator for each, and give the reason in one sentence:
- A 48-byte per-device state structure.
 - A 2 MB buffer for software processing, no hardware involvement.
 - A 16 KB buffer a device will DMA into.
 - A 1 MB buffer that hardware never sees.
- b) `kmalloc(131072, GFP_KERNEL)` succeeds on your desk every time and fails on a customer's machine after three weeks of uptime. Explain, and name the file that shows you why.
- c) Why can `kvmalloc` not be used for a DMA buffer, given that it might have returned `kmalloc` memory?
- d) Which free function goes with which allocator? What happens if you mismatch them?

Exercise 6.3 GFP flags

HAND CALCULATION

For each, say which GFP flag is correct and what goes wrong with `GFP_KERNEL`:

- a) Allocating a bounce buffer inside `read()`.
- b) Allocating inside an interrupt handler.
- c) Allocating while holding a spinlock.
- d) Allocating in a module's init function.
Then:
- e) `GFP_ATOMIC` is described in some places as "high priority". In what sense is that backwards? Which of the two is *more* likely to return `NULL`, and why?
- f) A driver uses `GFP_KERNEL` inside a spinlock and has run in production for two years. Is it correct? What would make the problem appear, and which config option this course enables would have reported it on the first run?

Exercise 6.4 Translating a device tree address

HAND CALCULATION

A device tree contains:

```
/ {
    #address-cells = <2>;
    #size-cells = <2>;

    bus@40000000 {
        #address-cells = <1>;
```

```

        #size-cells = <1>;
        ranges = <0x0 0x0 0x40000000 0x1000000>;

        widget@2000 {
            reg = <0x2000 0x100>;
        };
};

```

- What physical address do widget's registers start at, and how many bytes long are they?
- How many 32-bit cells does widget's reg property occupy, and how do you know?
- How many cells does the ranges property occupy? Account for each one.
- A colleague reads `reg = <0x2000 0x100>` and maps `0x2000`. What do they map, and what will they most likely see when they read it?
- The bus node's ranges is `<0x0 0x0 0x40000000 0x1000000>`. Rewrite it as a sentence.

Exercise 6.5 Ordering

DESIGN

- Your driver writes CTRL to start a conversion and immediately reads STATUS to see whether it finished. Which accessors do you use, and what happens if you use `_relaxed` for both?
- Your driver writes sixteen configuration registers and then a single G0 bit. Where does a barrier actually need to be, and what does that buy over using ordered accessors throughout?
- Why is a missing barrier one of the worst classes of bug to diagnose? Answer in terms of what you observe rather than what is happening.
- qa-dev rejects byte accesses to its 32-bit registers rather than widening them. Argue for that decision from the point of view of somebody writing a driver for real silicon.

Exercise 6.6 Coherent or streaming

DESIGN

None of this runs on qa-dev, which has no DMA engine. Argue it on paper from §6.1.6.

For each buffer, say whether you would reach for `dma_alloc_coherent` or for `dma_map_single`, and give the reason in terms of how often each side touches it:

- a) A sixteen-entry descriptor ring. The driver writes an entry, the device reads it, the device writes a status word back, and this happens thousands of times a second.
- b) A 64 KB block of samples that the device fills once and the driver then hands to userspace.
- c) A four-byte doorbell the driver writes and the device polls.
- d) Your driver maps a buffer `DMA_FROM_DEVICE`, and while the transfer is running it reads the first word “just to see whether data has started arriving”. Nothing ever goes wrong in testing. What did the code do, why did testing not catch it, and on what kind of machine does it break?
- e) A colleague proposes `kvmalloc(65536, GFP_KERNEL)` for the buffer in (b), reasoning that 64 KB is large enough to risk `kmalloc` failing on a fragmented system. They are right about the fragmentation. Why is the suggestion still wrong, and what does the kernel do about it?

Exercise 6.7 Map the device

CODE

Write `lectures/L06/lab/qa_mmio.c`. It must:

- Take an unsigned long module parameter `base`, the physical address of the registers, readable through `sysfs` and with no default that works. **Refuse to load if it was not given**, with `-EINVAL` and a message saying so, rather than mapping address 0.
- Claim the region with `request_mem_region`, naming it after the module.
- Map it with `ioremap`.
- Read `QA_DEV_ID` and `QA_DEV_VERSION` and print them as `id=0x%08x version=0x%04x`.
- **Refuse to continue if the identity register is wrong**, with `-ENODEV`.
- Write `0xDEADBEEF` to `QA_DEV_SCRATCH`, read it back, and print `scratch ok (0x...)` if it matches.
- Unwind correctly from every failure point, and release everything on unload.

The register offsets are in `lectures/L06/lab/qa-dev.h`, which is a copy of the device’s own register map, printed in §6.2; read it as you would a datasheet.

Check yourself: make `test L=L06` reports **PASSED**, with nine checks. It works the address out of the device tree itself and passes it to your module, so if your arithmetic in Exercise 6.10 disagrees with the harness, one of you is wrong and it is worth finding out which.

Exercise 6.8 Prove the mapping, three ways

CODE

Reading a correct constant is weaker evidence than it looks. With your module loaded:

- a) Find your region in `/proc/iomem`. What name does it appear under, and where did that name come from?
- b) `QA_DEV_ID` reads `0x51414456`. Decode those four bytes as ASCII. Why is an identity register conventionally something you can recognise by eye?
- c) Reading a constant correctly does not prove the mapping is writable, or that it is not aliased onto something else. Which part of your module tests that, and what exactly would fail if `ioremap` had returned a read-only mapping?
- d) Unload and reload the module without rebooting. Then comment out `release_mem_region` in your exit path, rebuild, and try again. What does the second `insmod` say, and why is that error message a good one?

Exercise 6.9 What a wrong address does

DESIGN

Read the warning at the top of this section first.

- a) Load your module with `base=0xC` instead of the correct address. What does it read, and what does your identity check do about it? What would a driver *without* that check have done next?
- b) Now load it with `base=0xC001000`, one page past the device. Record what happens: the message, what happened to `insmod`, and whether the module can afterwards be used or removed.
- c) You now have two different failure modes from two wrong addresses. State the general lesson about using “it did not crash” as evidence that a mapping is correct.
- d) Both experiments cost you a reboot at most. Write two sentences on what the same two experiments would cost on a board on your bench and on a product in the field, and relate that to Chapter 1’s argument about what this course gives up by having no hardware.

Exercise 6.10 An address worked out by hand

CROSS-CHECK

The signature exercise: derive a physical address on paper, predict what is at it, then look.

- a) **Read the node.** On the target, find the device tree node and print its reg, and its parent’s ranges, as raw bytes:

```
dt=/sys/firmware/devicetree/base
node=$(find $dt -name 'qa-dev@*' -type d | head -1)
od -An -tx1 -v "$node/reg"
```



```
od -An -tx1 -v "${dirname "$node"}/ranges"
```

Write out the cells by hand, four bytes each, most significant byte first.

b) Translate. From those cells alone, compute the physical address of the registers and the size of the window. Show the arithmetic; it is one addition, and the point is knowing which two numbers to add.

c) Predict. From `lectures/L06/lab/qa-dev.h`, say what you expect to read at offset 0 and at offset 4, as 32-bit values, before you run anything.

d) Measure. Load your module with the address you computed and read the two values off `dmesg`.

e) Reconcile. Answer each:

- Did your address match what the harness computed? If not, which of you translated wrongly, and what does `run.sh` do that you did not?
- Now do part **a)** again with `od -An -tx4` instead of `-tx1`. You get different numbers. Which is right, what is `-tx4` doing, and why is the wrong answer *plausible* rather than obviously broken?
- Compare `reg`'s first cell against the address you ended up using. They differ by a large constant. Where did that constant come from, and which node declares it?
- You now know the address. Nothing in your driver's source contains it. Where is it, and what has to happen for a driver to stop being told and start finding out? Name the chapter.

f) Suppose the device tree said `reg = <0x1000 0x1000>` instead. Recompute the address, and say what you would now expect `QA_DEV_ID` to read and why.

PART III

Concurrency, Interrupts and Time

Concurrency and Locking

Every way two pieces of kernel code can run at once, and what to do about each.

IN THIS CHAPTER

- Five sources of concurrency in a kernel, only one of which is another CPU.
- Atomics: what they promise, and the much larger thing they do not.
- Memory barriers, and why the compiler is as much of a problem as the processor.
- Spinlocks, `_irq` and `_irqsave`; what each one disables and at what cost.
- Mutexes and semaphores, and the single question that chooses between them and a spinlock.
- Atomic context, `might_sleep()`, and why “do not sleep here” is not a style rule.
- Deadlock, lock ordering, and lockdep.
- Race a shared counter, watch it lose data, then fix it.

This chapter is about every way two pieces of kernel code can run at once, and what to do about each. It measures what an unprotected counter loses on the course’s two-CPU target, and how hard that loss was to make appear at all. It explains why atomics fix a counter and not a data structure, what the compiler does to a loop that waits on a flag, how a spinlock and a mutex differ and the one question that chooses between them, what atomic context is and why sleeping in it is a bug even on the runs where nothing goes wrong, and how lockdep reports a deadlock that has not happened.

Read §7.1 first; it is the sources of concurrency and the primitives. §7.2 is deadlock, lock ordering and lockdep, and reads a real lockdep report from the lab line by line. The lab is two modules, specified in Exercises 7.6 and 7.7: `qa_race`, which races a counter with and without a spinlock, and `qa_abba`, which takes two locks in both orders on a single thread so that lockdep objects. The exercises in §7.4 end with the Cross-check: compute the expected counter by hand, measure it locked and unlocked, account for the difference, then find the iteration count at which the broken program is correct every time. The lab is done when `make test L=L07` reports **PASSED**, which includes producing a lockdep report on purpose.

7.1 Where Concurrency Comes From, and What to Do About It

Five ways two pieces of kernel code can run at once, what each of the locking primitives costs, and the single question that chooses between them. §7.2 is deadlock and the tool that finds it.

The idea to carry: **every locking decision in the kernel is downstream of one question, which is whether the code holding the lock is allowed to sleep.** Get that right and the rest follows. Get it wrong and the failure is a deadlock under load on a machine you do not own.

7.1.1 Five sources, two of which you have already met

Source	Another thread of control appears because	First met in
SMP	Another CPU is running your code at the same time	Chapter 3
Preemption	The scheduler stops you mid-function and runs something else	Chapter 3
Interrupts	A device raised a line and your handler runs	Chapter 8
Timers	Something you scheduled earlier fires	Chapter 9
Workqueues	Deferred work you queued runs on a kernel thread	Chapter 8

On a single-CPU kernel built with `PREEMPT_NONE`, only the interrupt is left, with the timers that fire on the way out of one, and a great deal of old driver code is written for exactly that world. **Copying it is dangerous**, because the two assumptions it makes silently, that nothing else runs your code and that you cannot be stopped mid-function, are both false on the target this course boots.

That target has two CPUs and `CONFIG_PREEMPT=y`, which is the ordinary configuration of a modern embedded Linux, and it is enough to lose data in a way you can measure in about a second.

7.1.2 What a race actually costs

The lab's `qa_race` module runs four kernel threads, each incrementing a shared counter twenty thousand times, so 80,000 increments are expected. Ten consecutive unlocked runs on this target:

```
expected=80000 actual=78436 lost=1564
expected=80000 actual=73529 lost=6471
expected=80000 actual=70292 lost=9708
expected=80000 actual=67722 lost=12278
expected=80000 actual=63562 lost=16438
expected=80000 actual=63195 lost=16805
expected=80000 actual=62715 lost=17285
expected=80000 actual=62525 lost=17475
```

```
expected=80000 actual=53455 lost=26545
expected=80000 actual=49953 lost=30047
```

Between 2% and 38% of the work vanished, and no two runs agree. With the spinlock switched on, every run returns exactly 80,000.

counter++ is not one operation. It is a load, an add, and a store, and two CPUs interleaving those three steps lose one of the two increments. That is all a lost update is.

The part worth dwelling on is how hard this was to make happen. The first version of that module lost *nothing*, three runs in a row, on the same hardware with the same code. The threads were being created one at a time, and the first finished before the last was created, so they never overlapped. Adding a starting barrier, so all four threads begin together, took the loss from zero to a third of the total.

Nothing about the code changed. The bug was there the whole time. **A race that does not reproduce is not a race that is absent**, and “it works on my machine” is usually a statement about timing rather than about correctness.

The same point again, from the other direction. With the barrier in place and everything else identical, varying only how much work each thread does:

Increments per thread	Expected	Loss, three runs
200	800	0%, 0%, 0%
2,000	8,000	0%, 0%, 0%
20,000	80,000	23.3%, 20.8%, 6.9%
200,000	800,000	31.7%, 34.9%, 26.5%

The first two rows are the same broken program, returning the right answer every time. Below about ten thousand iterations the threads do not overlap for long enough to collide, even though they were released together; the work finishes inside the window it takes the other threads to get going. There is no size at which the code is correct, only sizes at which it has not yet been caught.

Anyone testing this would reasonably pick a small iteration count, because it is faster. They would get a clean run, every time, and be wrong.

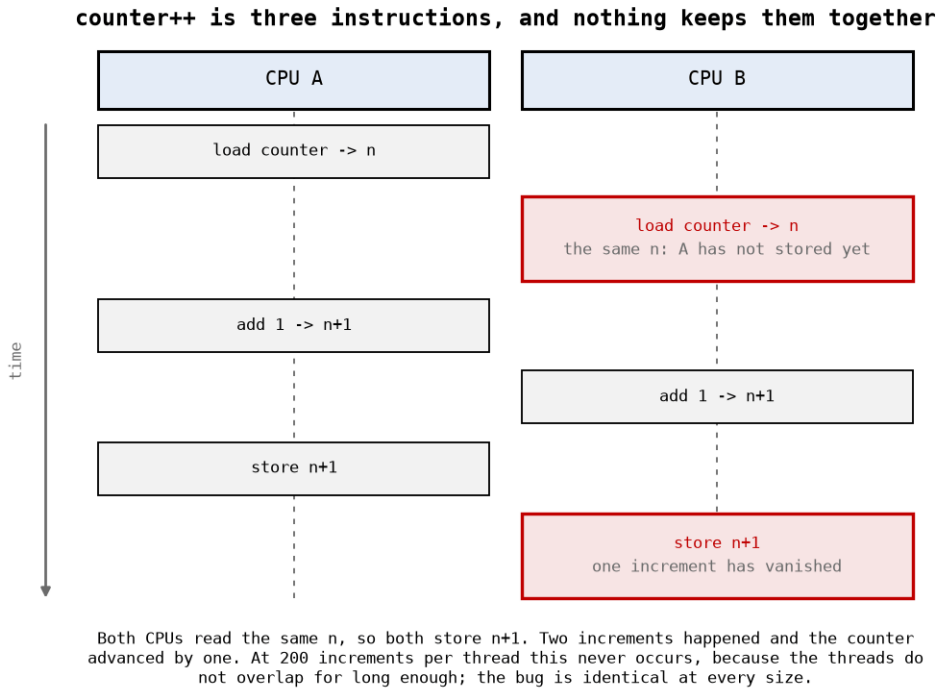


Figure 7.1 A sequence diagram with a lane for each of two CPUs, time running downwards. Both load the counter and get the same n , both add one, and both store $n+1$. Two increments happened and the counter advanced by one.

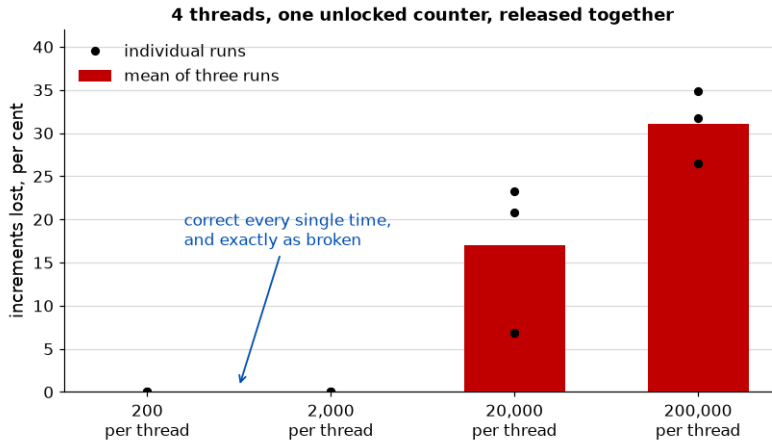


Figure 7.2 A bar chart of increments lost against increments per thread, with the three individual runs marked as dots on each bar. At 200 and 2,000 per thread the loss is zero in every run; at 20,000 and 200,000 it ranges from about 7 to 35 per cent.

7.1.3 Atomics, and the thing they do not fix

```
atomic_t counter = ATOMIC_INIT(0);
atomic_inc(&counter);
```

`atomic_inc` makes the load-add-store indivisible, and it fixes §7.1.2's counter completely.

It does not fix a data structure, and this is the misconception worth the most time. A ring buffer has a head, a level, and an array, and its invariant spans all three. Making each of them `atomic_t` makes each individual update indivisible and leaves the invariant exactly as broken:

```
/* Still wrong, with every field atomic. */
if (atomic_read(&level) < capacity) { /* another CPU can act here */
    data[(atomic_read(&head) + atomic_read(&level)) % capacity] = byte;
    atomic_inc(&level);
}
```

Two writers can both pass the test, both compute the same slot, and both write to it; one byte is lost and `level` says two arrived. Atomicity per field buys nothing when the thing that must be atomic is the *sequence*.

The rule: **atomics protect one variable, locks protect an invariant**. If you can state what must be true before and after, and it mentions more than one variable, you need a lock.

Useful atomics beyond the counter: `atomic_inc_return`, `atomic_dec_and_test` (returns true when the result is zero, which is how refcounts are implemented), and `atomic_cmpxchg`.

7.1.4 The compiler is also a problem

Before the CPU gets a chance to reorder anything, the compiler already has.

```
while (!flag) { }           /* may never re-read flag */
```

flag is not volatile, nothing in the loop changes it, so the compiler may load it once and spin on a register forever. This is a correct optimisation and an infinite loop.

```
while (!READ_ONCE(flag)) { cpu_relax(); }
```

READ_ONCE and WRITE_ONCE force the access to happen exactly once, in the right place. They are not barriers and they do not order anything against other accesses; they stop the compiler from inventing, eliding or splitting the access itself.

For ordering between accesses there are memory barriers, `smp_mb()`, `smp_rmb()`, `smp_wmb()`, and their acquire/release forms. **Most driver code should not be using them directly.** A lock provides both mutual exclusion and the ordering, and code that reaches for bare barriers is code that has to be right about the memory model of every architecture it will ever run on. §6.2.5 covers the same problem where the other party is a device rather than a CPU.

7.1.5 Spinlocks

```
static DEFINE_SPINLOCK(my_lock);

spin_lock(&my_lock);
/* critical section */
spin_unlock(&my_lock);
```

A waiting CPU **spins**, burning cycles until the lock is free. It does not sleep, which is the whole point: a spinlock is usable from contexts where sleeping is forbidden, and an interrupt handler is one.

Two consequences:

- **The critical section must be short.** Every cycle spent holding it is a cycle another CPU spends doing nothing. Microseconds, not milliseconds.
- **You may not sleep while holding one.** No `kmalloc(GFP_KERNEL)`, no mutex, no `copy_to_user`, no `msleep`. Sleeping with a spinlock held can deadlock the machine outright, and `CONFIG_DEBUG_ATOMIC_SLEEP`, which this course enables, turns it into a complaint instead.

The interrupt variants, and why they exist

Consider a lock taken both by a driver's `read()` and by its interrupt handler. Process context takes the lock; before it releases, the device interrupts *the same CPU*; the handler runs and tries to take the same lock; it spins, waiting for a lock that will be released by code that cannot run until the handler returns. The machine stops.

```
unsigned long flags;

spin_lock_irqsave(&my_lock, flags);    /* disables interrupts on this CPU, saves ←
    ← the old state */
/* critical section */
spin_unlock_irqrestore(&my_lock, flags);
```

Variant	Disables	Use when
<code>spin_lock</code>	Preemption only	The lock is never taken in interrupt context
<code>spin_lock_irq</code>	Preemption and interrupts	You know interrupts were enabled
<code>spin_lock_</code> <code>irqsave</code>	The same, restoring the prior state	Anywhere. The one to default to
<code>spin_lock_bh</code>	Preemption and softirqs	The lock is shared with a softirq or tasklet

`irqsave` rather than `irq` because you may be called with interrupts already disabled, and blindly re-enabling them on unlock would be a bug in the caller's code that you introduced.

Note that this only guards against the handler on the **same** CPU. Another CPU's handler is excluded by the lock itself; that is what the lock is for.

7.1.6 Mutexes

```
static DEFINE_MUTEX(my_mutex);

mutex_lock(&my_mutex);
/* critical section, may sleep */
mutex_unlock(&my_mutex);
```

A waiting task **sleeps**: it leaves the run queue and the CPU does something else. Which means:

- It **cannot** be used in interrupt context or anywhere else that may not sleep.
- The critical section **may** sleep. `kmalloc(GFP_KERNEL)`, `copy_to_user`, and I/O are all fine.
- It scales better for anything long, because waiters cost nothing while they wait.

`mutex_lock_interruptible` returns `-EINTR` if a signal arrives while waiting, which is what you want in a driver reachable from a system call; a user pressing Ctrl-C expects something to happen.

Choosing

One sentence decides it: **can the code inside the critical section sleep, and can the code waiting for it afford to wait?**

	Spinlock	Mutex
Usable from an interrupt handler	Yes	No
The critical section may sleep	No	Yes
Waiter burns CPU	Yes	No
Short critical section	Good	Overhead dominates
Long critical section	Terrible	Good

If the answer is “an interrupt handler touches this”, it is a spinlock and the critical section has to be short enough to live inside one. If the answer is “this copies to userspace”, it is a mutex and no interrupt handler may touch that data.

Also available: **semaphores**, which are a counting generalisation and are mostly legacy in new code; and **reader-writer locks** (`rwlock_t`, `rw_semaphore`), which allow concurrent readers. Reader-writer locks are less of a win than they look, because they are slower than a plain lock in the uncontended case, and the uncontended case is most of them.

7.1.7 Atomic context

“Atomic context” means **any context in which the code may not sleep**. You are in it when:

- You are in an interrupt handler.
- You are in a softirq, tasklet or timer callback.
- You hold a spinlock.
- You have disabled preemption or interrupts explicitly.

What is forbidden there is anything that might call `schedule()`:

Forbidden	Because
<code>kmalloc(GFP_KERNEL)</code>	May sleep to reclaim memory. Use <code>GFP_ATOMIC</code>
<code>mutex_lock</code>	Sleeps when contended
<code>copy_to_user</code>	May fault, and faulting may sleep
<code>msleep</code> , <code>schedule</code>	Obviously
<code>wait_event</code>	Chapter 9

This is not a style rule and there is no “it usually works”. Sleeping in atomic context is a bug even on the runs where nothing goes wrong, because whether it goes wrong depends on whether the allocator happened to have memory, or whether the lock happened to be free.

The kernel will tell you, if you ask it to. `CONFIG_DEBUG_ATOMIC_SLEEP`, enabled by `kernel/qa.config` for this course, makes any sleep in atomic context print a BUG: sleeping

function called from invalid context with a backtrace. `might_sleep()` is the annotation that produces it, and it is worth putting at the top of any function of yours that can sleep, so that a caller who gets it wrong finds out immediately rather than eventually.

7.1.8 RCU and per-CPU, in one section

Two mechanisms for the case where a lock is the wrong shape. Both are worth recognising; neither is something to reach for in a first driver.

RCU (read-copy-update) is for data read constantly and written rarely. Readers take no lock at all and pay almost nothing; a writer builds a new version, publishes it with one atomic pointer store, and waits until every reader that might still hold the old pointer has finished before freeing it. That waiting is the cost, and it is why RCU suits routing tables and module lists and not a driver's ring buffer.

Per-CPU data removes the sharing instead of protecting it. `DEFINE_PER_CPU` gives each CPU its own copy, so there is nothing to race over; statistics counters are the classic use, summed across CPUs only when somebody reads them. `/proc/stat`'s per-CPU lines in §2.2.2 are exactly this.

The unifying observation: **the fastest lock is the one you did not need**, and both of these are ways of arranging not to need one.

7.2 Deadlock, Lock Ordering, and Lockdep

Deadlock is the failure mode that testing does not find, because it depends on two things happening in an order that almost never occurs. This section is what deadlock is, the discipline that prevents it, and the tool that finds violations of that discipline **without the deadlock having to happen**.

That last clause is the whole reason lockdep matters, and it is worth being precise about it up front: lockdep does not detect deadlocks. It detects *the conditions* for one, the first time it sees them, in a program that ran perfectly.

7.2.1 The four conditions

A deadlock needs all four of these at once:

Condition	Meaning
Mutual exclusion	The resource cannot be shared
Hold and wait	A holder may request another resource
No preemption	The resource cannot be taken away
Circular wait	A cycle exists in the “waits for” graph

In kernel locking the first three are given: locks are exclusive, code does take a second lock while holding a first, and nothing revokes a held lock. **So the only one you can attack is the fourth**, and every practical anti-deadlock discipline is a way of making a cycle impossible.

In the general case that is a theorem about resource graphs; here it collapses to one rule.

7.2.2 The rule: a global order

Define an order over your locks, and always take them in that order.

If every thread takes locks in the same order, no cycle can form, because a cycle requires at least one thread to have gone the other way. The order can be anything as long as it is total and written down: address order, an order documented in a comment, or the structural order implied by a hierarchy (device lock before channel lock before buffer lock).

The classic violation is called **ABBA**:

```
/* Thread 1 */           /* Thread 2 */
spin_lock(&a);            spin_lock(&b);
spin_lock(&b);            spin_lock(&a);
```

Each holds one and waits for the other. Both stop, and so does everything that later needs either lock, which on a busy system is soon everything.

The reason this survives testing is that it needs the two threads to interleave in a narrow window. Thread 1 must acquire a, then thread 2 must acquire b, then both must ask for the other, all before either releases. Miss the window and both sequences complete happily. Code with this bug can run for years.

7.2.3 Lockdep watches the order, not the outcome

CONFIG_PROVE_LOCKING, enabled for this course in kernel/qa.config, makes the kernel record, for every lock acquisition, which locks were already held. From that it builds a graph of “lock X has been taken while holding lock Y” and checks each new edge for a cycle.

So it does not need the deadlock to happen. It needs only to see a taken while holding b, having previously seen b taken while holding a, at any earlier point, on any CPU, in any thread. The first time the second ordering appears, it reports.

The lab’s qa_abba module demonstrates this in the least dramatic way possible: **one thread, no concurrency at all, two sequential critical sections.**

```
spin_lock(&lock_a);
spin_lock(&lock_b);
spin_unlock(&lock_b);
spin_unlock(&lock_a);

spin_lock(&lock_b);
spin_lock(&lock_a);           /* lockdep objects here */
spin_unlock(&lock_a);
spin_unlock(&lock_b);
```

Nothing deadlocks. Nothing can: there is one thread, and each lock is released before the next pair is taken. The module loads successfully and prints a cheerful message afterwards. Lockdep reports a deadlock anyway, because it is reporting the *ordering*, and the ordering is wrong.

7.2.4 Reading the report

This is the real output from the lab on this target, abridged where it repeats. It is worth reading in order, because the parts are in a deliberate sequence.

The header: what was being attempted, and what was already held.

```
=====
WARNING: possible circular locking dependency detected
6.12.30 #6 Tainted: G          0
-----
insmod/121 is trying to acquire lock:
ffffbfa4a6bc020 (lock_a){+..}-{2:2}, at: qa_abba_init+0x6c/0xff8 [qa_abba]

but task is already holding lock:
ffffbfa4a6bc060 (lock_b){+..}-{2:2}, at: qa_abba_init+0x64/0xff8 [qa_abba]
```

Two locks, named, with the address of each and **the source location where each was taken**. That is usually enough on its own.

The `{+..}` is the lock's usage signature, recording the contexts it has been used in, which is how lockdep distinguishes a lock used in interrupt context from one that is not. `-{2:2}` is its wait type, outer and inner, and 2 means a lock that spins; it is how lockdep catches a lock that may sleep being taken inside one that may not. You rarely need either.

The chain: how it already knew about the other order.

```
the existing dependency chain (in reverse order) is:

-> #1 (lock_b){+..}-{2:2}:
    _raw_spin_lock+0x50/0x70
    qa_abba_init+0x40/0xff8 [qa_abba]
    ...

-> #0 (lock_a){+..}-{2:2}:
    __lock_acquire+0x12f4/0x1f48
    lock_acquire+0x1fc/0x330
    _raw_spin_lock+0x50/0x70
    qa_abba_init+0x6c/0xff8 [qa_abba]
```

Note the two different offsets into the same function: `qa_abba_init+0x40` is where `lock_b` was taken the first time, and `+0x6c` is where `lock_a` is being taken now. **Lockdep is showing you both halves of the contradiction, with a line number for each.**

The diagram: the interleaving that would deadlock.

Possible unsafe locking scenario:

CPU0	CPU1
----	----
<code>lock(lock_b);</code>	
	<code>lock(lock_a);</code>
	<code>lock(lock_b);</code>
<code>lock(lock_a);</code>	
*** DEADLOCK ***	

This is the part people misread. **Those two CPUs are hypothetical.** Nothing ran on CPU1; the module is single-threaded. Lockdep is telling you what *would* happen given the orderings it has observed, if two threads ever hit them together.

The *backtrace*, which names `print_circular_bug` and `check_noncircular`, the functions that found it. Seeing those two in a trace is how you recognise a lockdep report at a glance.

7.2.5 What lockdep does and does not find

It finds, without the failure occurring:

- Lock ordering inversions (ABBA), across threads, CPUs and time.
- Taking a sleeping lock in atomic context.
- A lock used both with and without interrupts disabled, in a way that could deadlock against its own interrupt handler.
- Recursive acquisition of a non-recursive lock.

It does not find:

- **Missing locks.** A data structure with no lock at all has no ordering to violate. `qa_race` loses up to 38% of its increments with lockdep fully enabled and lockdep says nothing, because nothing did anything wrong with a lock. This is the single most important limitation and the reason the lab has two modules rather than one.
- Deadlocks not built from lock ordering: two threads each waiting for the other to signal a completion, or a lock against a hardware condition that never arrives.
- Anything on a code path that never executed. It is a runtime tool, so an ordering that only happens in an error path nobody exercised is an ordering lockdep has not seen.

That last point has a practical consequence: **lockdep finds what your tests reach**. It turns “we would have to be unlucky” into “we ran the path once”, which is an enormous improvement and is not the same as proof.

7.2.6 The cost, and what to do about it

Lockdep is expensive. Every acquisition does bookkeeping, and the graph grows. Kernels built with `CONFIG_PROVE_LOCKING` are measurably slower, which is why production kernels do not have it and why `qa.config` says so in a comment: “that contrast is part of the lecture”.

The working practice that follows:

- **Development and CI kernels: on.** The cost is irrelevant next to finding an ordering bug.
- **Production: off**, and accept that you are now relying on having exercised the paths.
- **Do not measure latency on a lockdep kernel.** Chapter 12 needs numbers, and a kernel doing this bookkeeping in every lock is not the kernel whose latency you care about. This course breaks its own rule: both of its kernels keep the validator on, and §12.2.5 says what that costs the figures it publishes.

And one interaction worth remembering from Chapter 4: **loading a proprietary module switches lockdep off for the rest of that boot**, with the single line `Disabling lock debugging due to kernel taint`. A development kernel with a proprietary module on it has silently lost the tool you would use to find the deadlock that module might be causing.

7.2.7 Practical discipline

Ordered by how much they are worth:

1. **Hold locks for as little code as possible.** A short critical section cannot contain a call into something that takes another lock, which is where orderings come from.
2. **Do not call out of a critical section into unfamiliar code.** You do not know what it locks. This is why callbacks are usually invoked with no lock held.
3. **Write the order down**, in a comment beside the lock definitions, the first time a second lock appears.
4. **Prefer one lock.** Two locks are more than twice the trouble of one, and a driver’s data is usually small enough that one lock protecting all of it costs nothing measurable.
5. **Run the error paths.** Lockdep only sees what executes, and error paths are where the unusual orderings live.

7.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Name five sources of concurrency in a kernel and say which of them a single-CPU kernel has.
- Say why making every field of a structure atomic does not make the structure thread-safe.
- Choose between a spinlock and a mutex, and give the one-sentence reason.
- Say what `spin_lock_irqsave` saves and restores, and what goes wrong without it.
- Recognise atomic context in unfamiliar code, and list what may not be done there.
- Read a lockdep report and say which two locks, taken in which orders, it is complaining about.
- Explain why lockdep can report a deadlock that has not happened.

Questions to test yourself

1. Your driver works perfectly on a single-CPU target and corrupts data on a dual-CPU one. Name three things that changed, only one of which is “two things run at once”.
2. Why is `atomic_read` followed by `atomic_set` not an atomic read-modify-write?
3. You hold a spinlock and call `kmalloc(GFP_KERNEL)`. What is wrong, and when will you find out?
4. What exactly does a spinlock do to the CPU that is waiting for it? Why is that acceptable for short sections and catastrophic for long ones?
5. Two functions take locks A and B in opposite orders, and the code has run in production for two years without deadlocking. Is it correct?
6. Why does `CONFIG_PROVE_LOCKING` make a kernel slower, and why would you still enable it during development?

Where to look

- `kernel/qa.config` turns on `CONFIG_PROVE_LOCKING` and `CONFIG_DEBUG_ATOMIC_SLEEP` for this course, with a comment on why. Those two options are the reason the exercises in this chapter produce output rather than silence.

Looking ahead

Chapter 8 is about:

- The other source of concurrency, and the one you cannot schedule.
- What an interrupt handler may not do, and what it must do before it returns.
- Four places to put work that does not belong in a handler.
- Why the default answer is now a thread.

7.4 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 7.6 onwards need the target running.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

Solutions. The exercises that are not Code are answered in §A.7. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 7.1 Where concurrency comes from

RECALL

- a) Name the five sources of concurrency from §7.1.1. Which of them exist on a single-CPU kernel built with `PREEMPT_NONE`?
- b) A driver written in 2004 for a single-CPU non-preemptible kernel is copied into a modern one. Name the two assumptions it makes silently, and say what breaks.
- c) Your driver's `read()` and its interrupt handler both touch the same buffer. Which of the five sources is in play, and does the answer change on a single-CPU machine?
- d) `counter++` is one line of C. How many machine operations is it, and which of them can be interleaved with another CPU's copy of the same three?

Exercise 7.2 Atomics and their limits

RECALL

- a) `atomic_inc` fixes a shared counter completely. Give a data structure it does not fix, and state the invariant that spans more than one variable.
- b) Rewrite this so it is correct, and say which primitive you used and why:

```
if (atomic_read(&level) < capacity) {
    data[(atomic_read(&head) + atomic_read(&level)) % capacity] = byte;
    atomic_inc(&level);
}
```

- c) State the rule that separates “use an atomic” from “use a lock”, in one sentence.
- d) `while (!flag) { }` never terminates even though another thread sets `flag`. Explain, and give the fix. Is the fix a memory barrier?

Exercise 7.3 Choosing a primitive

HAND CALCULATION

For each, choose a spinlock, a spinlock with `_irqsave`, or a mutex, and justify it in one sentence.

- a) Protecting a counter touched only by two kernel threads.
- b) Protecting a buffer touched by `read()` and by an interrupt handler.
- c) Protecting a structure while copying 4 KB of it to userspace.
- d) Protecting a device register sequence that takes about 2 microseconds.
- e) Protecting a list that is walked in a timer callback and modified from `write()`.
- f) Protecting a structure while allocating memory that may have to be reclaimed.

Then: g) two of your answers would deadlock the machine if you had chosen a plain `spin_lock`. Which, and describe the deadlock in each case.

Exercise 7.4 What the race costs

HAND CALCULATION

Four threads each increment a shared counter 20,000 times, unlocked.

- a) What is the expected final value if nothing is lost?
- b) A run reports `actual=63562`. How many increments were lost, and what percentage is that?
- c) Ten runs on the same machine with the same code produced losses of 1,564; 6,471; 9,708; 12,278; 16,438; 16,805; 17,285; 17,475; 26,545; and 30,047. Compute the range as a percentage of the expected total. What does the spread tell you that any single run does not?
- d) Can the loss ever be zero? Can it ever be 60,000? Justify both from what a lost update is.
- e) A colleague runs the unlocked version once, gets exactly 80,000, and concludes the lock is unnecessary. Name the two separate errors in that reasoning.

Exercise 7.5 Atomic context

DESIGN

For each, say whether the code is in atomic context, and whether the call is legal there.

- a) `kmalloc(64, GFP_KERNEL)` inside `read()`.
- b) `kmalloc(64, GFP_KERNEL)` inside an interrupt handler.
- c) `kmalloc(64, GFP_ATOMIC)` inside an interrupt handler.
- d) `mutex_lock` while holding a spinlock.
- e) `spin_lock` while holding a mutex.
- f) `copy_to_user` while holding a spinlock.

g) `msleep(1)` inside a timer callback.

Then: h) on a kernel without debugging options, three of the illegal ones will usually work in testing. Which, why, and what makes each fail later? i) Which config option that this course enables would have reported them the first time?

Exercise 7.6 Race a counter, then fix it

CODE

Write `lectures/L07/lab/qa_race.c`. It must:

- Take module parameters `threads` (default 4), `increments` (default 20000) and `locked` (default 0), all read-only through `sysfs`.
- Start `threads` kernel threads with `kthread_run`, each incrementing a single shared unsigned long `increments` times.
- With `locked=0`, increment it without any protection. With `locked=1`, protect it with a spinlock.
- **Release all the threads from a barrier**, so that they start together. Without this the first thread often finishes before the last is created and the race does not happen at all; see §7.1.2.
- Wait for all of them to finish, then print exactly: `locked=%d threads=%d increments=%d expected=%lu actual=%lu lost=%lu`.

Check yourself: with `locked=1` the loss is 0 every time. With `locked=0` it is not. make `test L=L07` runs the unlocked version three times and requires that at least one of them loses something.

- Before adding the barrier, run the unlocked version three times and record the losses. Then add it and run three more. Report both sets.
- Try `threads=1`. What loss do you get, and why is that not evidence of anything?
- Try `threads=8` on a two-CPU target. Does the loss go up in proportion to the threads? Explain what actually limits it.

Exercise 7.7 Make lockdep object

CODE

Write `lectures/L07/lab/qa_abba.c`. It must define two spinlocks, take them in one order and release them, then take them in the opposite order and release them, all from its `init` function on a **single thread**, and print a message before and after.

Check yourself: the module loads successfully, `lsmod` shows it, and `dmesg` contains `WARNING: possible circular locking dependency detected` and a `*** DEADLOCK ***` diagram.

- The module loaded and did not hang. Explain why lockdep reported a deadlock anyway.
- The report's diagram has a CPU0 column and a CPU1 column. How many CPUs actually executed your code? What are those two columns, then?

- c) Find the two `qa_abba_init+0x...` offsets in the report's dependency chain. What is each one telling you, and why are they different?
- d) Reload the module a second time without rebooting. Does the report appear again? Explain in terms of what lockdep is storing.
- e) Now fix the module so both sections take the locks in the same order. Confirm the report is gone. Has your module's behaviour changed in any observable way other than the absence of the warning?

Exercise 7.8 What the tools do not see

DESIGN

- a) `qa_race` loses up to 38% of its increments with lockdep fully enabled, and lockdep says nothing at all. Explain why, precisely.
- b) State the general limitation that follows, and say what it implies about a codebase that "passes with lockdep enabled".
- c) Name two kinds of deadlock lockdep cannot find, and say why each is outside its model.
- d) Lockdep only sees code that runs. Which parts of a driver are least likely to be exercised by a test suite, and why does that matter more for lock ordering than for most bugs?
- e) A development kernel has a proprietary module loaded. What has silently happened to lockdep, what is the single line of `dmesg` that said so, and which chapter did you first meet it in?

Exercise 7.9 A number computed twice

CROSS-CHECK

Predict a total by arithmetic, measure it, and account for the difference. Unlike the earlier Cross-checks the disagreement here is not a rounding artefact; it is the bug.

- a) **Predict.** With `threads=4` and `increments=20000`, compute the expected final counter. This is one multiplication and it is not the interesting part; write it down anyway, because it is the number everything else is measured against.
- b) **Measure, locked.** Load with `locked=1` and record `actual`. Run it three times.
- c) **Measure, unlocked.** Load with `locked=0` and record `actual` five times.
- d) **Reconcile.** Answer each:
 - The locked runs agree with your prediction exactly, every time. Why is "exactly" the right word here, rather than "to within measurement error"? What is different about this measurement compared to Chapter 3's Image size or Chapter 2's context-switch rate?
 - The unlocked runs disagree with your prediction and with each other. Compute the mean and the range. Is the mean a useful number? What would you have to say alongside it for it to mean anything?

- In the interleaving drawn in §7.1.2, a collision loses exactly one increment. If every collision were of that kind, how many times did two threads collide in your worst run? A thread preempted between its load and its store erases every increment made while it was off the CPU; what does that make your figure, a count or a bound? Is it plausible given the run took a few hundred milliseconds?

e) **Change one variable at a time.** Rerun the unlocked version three times each at increments=200, 2000, 20000 and 200000, keeping threads=4. Report the loss as a *percentage* of the expected total in all twelve runs.

The result is not the smooth curve most people predict. Answer these:

- At which values does the loss appear at all, and what happens below that?
- You now have a configuration in which the unlocked program is **correct every single time**. Is the bug absent at that size? What is actually different?
- The threads are released from a barrier, so they genuinely start together. Given that, explain why a short run still does not collide. How long does the overlap have to last, and what is it competing against?
- Someone testing this code would naturally use a small number of iterations, because it is faster. Say what they would conclude and how confident they would be.

f) **The prediction that matters.** You now have a measurement whose value you cannot predict, and a measurement whose value you can. Say which of the two you would put in a bug report and which you would put in a test, and why they are not the same choice.

g) A colleague proposes fixing qa_race by making the counter atomic_t instead of adding a lock. For this specific program, would that work? Now suppose the threads also had to keep a running maximum alongside the counter. Would it still work? Which section answers this, and what is the one-sentence rule?

Interrupts and Deferred Work

The half of the mechanism the hardware drives: a line goes high, and whatever was running stops.

IN THIS CHAPTER

- The path from a wire going high to your function being called, and the three numbers along it.
- `/proc/interrupts`, and what each column is actually counting.
- `request_irq`, the handler signature, and `IRQ_HANDLED` against `IRQ_NONE`.
- Level and edge triggering, and an acknowledge that has to happen in the right order.
- What a handler may not do, and what happens when it does it anyway.
- Four places to put the work that does not belong in the handler.
- Threaded handlers, and why they are the default answer now.
- Take `qa-dev`'s interrupt, acknowledge it correctly, and move the work out.

This chapter is about the half of a driver the hardware drives: a device raises a line, and whatever the CPU was doing stops while your function runs. It follows an interrupt from the device's wire to your handler, through the three numbers it has on the way and the tool that prints each, and shows what `/proc/interrupts` is counting. It explains `request_irq` and what a handler owes the system: `IRQ_NONE` when the interrupt is not its own, and an acknowledgement of a level-triggered device in the right order, without which the machine never runs anything else again. It derives what a handler may not do from one fact, that it runs on time borrowed from whatever it interrupted, and then gives the four places to put the work that does not belong in it, and the reason the threaded handler is the default answer now.

Read §8.1 first; it is the interrupt path and the rules for the handler. §8.2 is deferred work in its four forms, with the course's measurement of a threaded handler against a hard one. The lab is specified by Exercises 8.6 and 8.7: a module that finds `qa-dev`'s interrupt in the device tree, takes it, acknowledges it, and moves the drain into a threaded handler, checked on the target by `lectures/L08/lab/run.sh`. The exercises in §8.4 end with the Cross-check: predict how many interrupts `qa-dev` raises in two seconds from `PERIOD_NS`, read the delta

out of `/proc/interrupts`, and account for the shortfall. Your first explanation will be partly right, and the exercise is about finding out which part. The lab is done when `make test` reports **PASSED**.

8.1 Interrupts

The path from a wire going high to your function being called, what that function owes the system, and what happens when it does not pay. §8.2 is where the work goes instead.

The idea to carry: **an interrupt handler runs on time borrowed from whatever was running**. Every rule about handlers follows from that one fact, and none of them has to be memorised separately.

8.1.1 Three numbers, not one

This is where an afternoon goes if you do not know it in advance. The same interrupt has three different identities.

On the target, with the lab’s module loaded:

```
# grep qa_irq /proc/interrupts
17:          1789          0  GIC-0 144 Level      qa_irq
```

and the device tree that produced it says:

```
interrupts = <0x00 0x70 0x04>;
```

Number	Where it comes from	Value here
The DT cell	Written by whoever described the board. $0 \times 70 = 112$	112
The GIC number	Hardware. SPI 112 plus the 32-entry SPI base	144
The Linux IRQ	Allocated at run time when the mapping is created	17

$112 + 32 = 144$, and 17 is not derived from either: it is an index into the kernel’s own table, handed out in the order mappings are created. **Boot the machine with a different set of devices and it changes.**

The table has a name worth knowing, because the kernel’s documentation uses it and nothing else does: an **IRQ domain** (`struct irq_domain`) is one controller’s mapping from its own hardware numbers to Linux ones. The GIC registers a domain at boot; asking that domain to map hardware 144 is what produced 17. A driver never creates a domain or looks inside one, and this is the last time this course mentions it. It matters only so that the word is not new when you meet it in `Documentation/core-api/irq/`, where the two numbers are called the `hwirq` and the `virq`.

The three cells of the DT property are, for a GIC: interrupt type (0 = SPI, 1 = PPI), the number within that type, and the flags (4 = level-triggered, active high). Chapter 10 is where a driver stops parsing these and lets `platform_get_irq` do it.

So when a datasheet says interrupt 112 and `/proc/interrupts` says 17, nothing is wrong. They are answers to different questions.

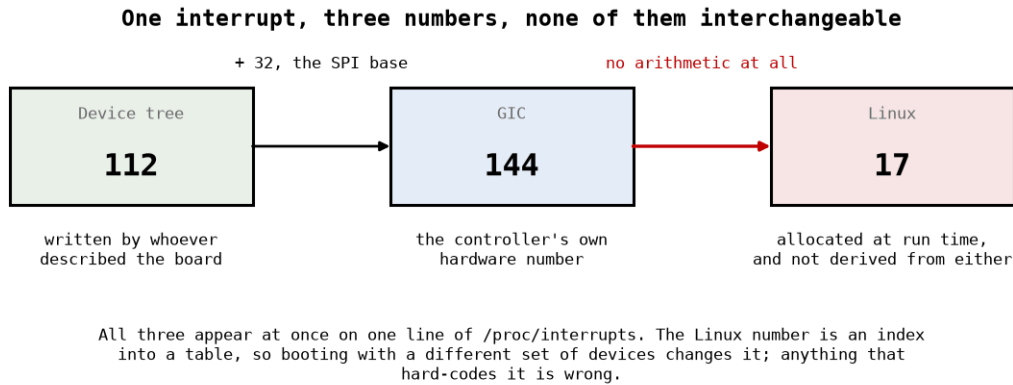


Figure 8.1 Three boxes holding 112, 144 and 17: the device tree cell, the GIC hardware number reached by adding the SPI base of 32, and the Linux number, which is allocated at run time and reached by no arithmetic at all.

8.1.2 Reading `/proc/interrupts`

```

17:      1789      0 GIC-0 144 Level      qa_irq
^         ^         ^   ^   ^   ^         ^
|         |         |   |   |   |         +-- the name you passed to ↵
|         |         |   |   |   |         ↵ request_irq
|         |         |   |   |   |         +----- trigger type
|         |         |   |   |   |         +----- hardware number
|         |         |   |   |   |         +----- which interrupt controller
|         |         |   |   |   |         +----- count on CPU1
|         |         |   |   |   |         +----- count on CPU0
|         |         |   |   |   |         +----- the Linux IRQ number
+-----+

```

Two diagnostics come almost free from that line:

- **A count that is not moving, with a device that is not working**, means the interrupt is not reaching the CPU at all: wrong number, wrong trigger type, controller not configured, device not actually asserting. Nothing in your handler is at fault, because it has not run.

- A **count racing away** means the opposite: it is arriving and you are not clearing it (§8.1.6).

The counts belong to the interrupt descriptor, not to your handler. They are not reset by `free_irq`, so unloading and reloading a module continues from where it left off:

```
after the first load:           919
immediately after loading again: 993    <- not zero
one second later:             1885
```

The line disappears while nothing is registered and comes back with its history. Any measurement of an interrupt rate therefore has to take a **delta**, and the Cross-check is built on that.

8.1.3 Getting the number, and the handler

```
ret = request_irq(irq, qa_hard, 0, "qa_irq", &private);
```

Argument	Is
<code>irq</code>	The Linux number, not the hardware one
<code>handler</code>	Your function
<code>flags</code>	<code>IRQF_SHARED</code> , <code>IRQF_ONESHOT</code> , trigger overrides
<code>name</code>	What appears in <code>/proc/interrupts</code> . Use the driver's name
<code>dev_id</code>	An opaque cookie handed back to your handler, and the key <code>free_irq</code> uses to identify which registration to remove

`dev_id` matters on shared lines: several drivers register handlers for one number, and it is how the kernel and your own handler tell which device is being talked about. It must be unique per registration, which is why drivers pass their private structure and never `NULL` when sharing.

In this chapter the number comes from the device tree directly, because the module is not yet a driver bound to a device:

```
np = of_find_compatible_node(NULL, NULL, "qacademy,qa-dev-1.0");
irq = irq_of_parse_and_map(np, 0);
```

That is a small taste of Chapter 10, which replaces both lines with `platform_get_irq(pdev, 0)` and deletes the search.

The return value

```
static irqreturn_t qa_hard(int irq, void *dev_id);
```

Return	Means
IRQ_HANDLED	This was mine and I dealt with it
IRQ_NONE	Not mine. Some other device on this shared line
IRQ_WAKE_THREAD	Mine; run the threaded half. See §8.2.4

IRQ_NONE is not an error and returning IRQ_HANDLED unconditionally is a real bug. On a shared line the kernel uses these to detect a stuck interrupt: if nobody ever claims responsibility it eventually disables the line and prints `nobody cared`. A handler that always says “mine” prevents that diagnosis for every other device sharing the line, and the symptom appears in somebody else’s driver.

The first thing a handler should do is read its own status register and return `IRQ_NONE` if nothing is pending.

8.1.4 Level and edge

	Level-triggered	Edge-triggered
The device	Holds the line asserted until told to stop	Pulses it
To stop it	Your handler must clear the source	Nothing; the pulse is over
If you forget	The handler is re-entered immediately, forever	The interrupt is simply lost
Missed events	Cannot be missed while asserted	Can be, if two arrive too close

qa-dev is **level-triggered**, which the device tree records as the 4 in `interrupts = <0 112 4>`. So its line stays high until the guest clears `IRQ_STATUS`, and §8.1.6 is what happens if you do not.

Neither is better. Level triggering cannot lose an event and requires an acknowledgement; edge triggering needs no acknowledgement and can coalesce two events into one. Real hardware uses both, and a driver written for one and configured for the other fails in a characteristic way: with edge configured for a level device, you get exactly one interrupt and then silence.

8.1.5 Acknowledging, and the order

qa-dev’s `IRQ_STATUS` is **write-one-to-clear**: writing a bit clears it, writing zero to a bit leaves it alone. So an acknowledgement is a read followed by writing back what you read.

```
u32 status = readl(regs + QA_DEV_IRQ_STATUS);
if (!(status & (QA_DEV_IRQ_SAMPLE | QA_DEV_IRQ_OVERRUN)))
    return IRQ_NONE;

writel(status, regs + QA_DEV_IRQ_STATUS); /* acknowledge exactly what we saw */

/* then do the work: drain the FIFO */
```

```
return IRQ_HANDLED;
```

Write back the value you read, not a mask of everything. Writing 0xFFFFFFFF clears bits that were set after your read, and the events they represented are gone.

And acknowledge before the work, not after. The order matters for a reason worth working through: if you clear the status and then read the FIFO, an event arriving in between sets the status bit again *and* adds to the FIFO you are about to drain. You then service it, and on a device whose bit only the write clears you leave the status bit set, so you get one spurious interrupt with nothing to do. That is survivable. The opposite arrangement, draining and then acknowledging with the value read on entry, clears the bit of an event that arrived after the drain, and its sample waits in the FIFO with no interrupt to collect it, which is not.

There is a second valid strategy for this device: draining the FIFO completely clears IRQ_SAMPLE on its own, because the device clears it when the FIFO empties, which is also why qa-dev does not even produce the spurious interrupt. It covers IRQ_SAMPLE only. IRQ_OVERRUN is cleared by the write and by nothing else, so a driver that relies on the drain alone wedges the machine the first time a sample is dropped. Knowing *which* mechanism you are relying on is the difference between a driver you can reason about and one that happens to work.

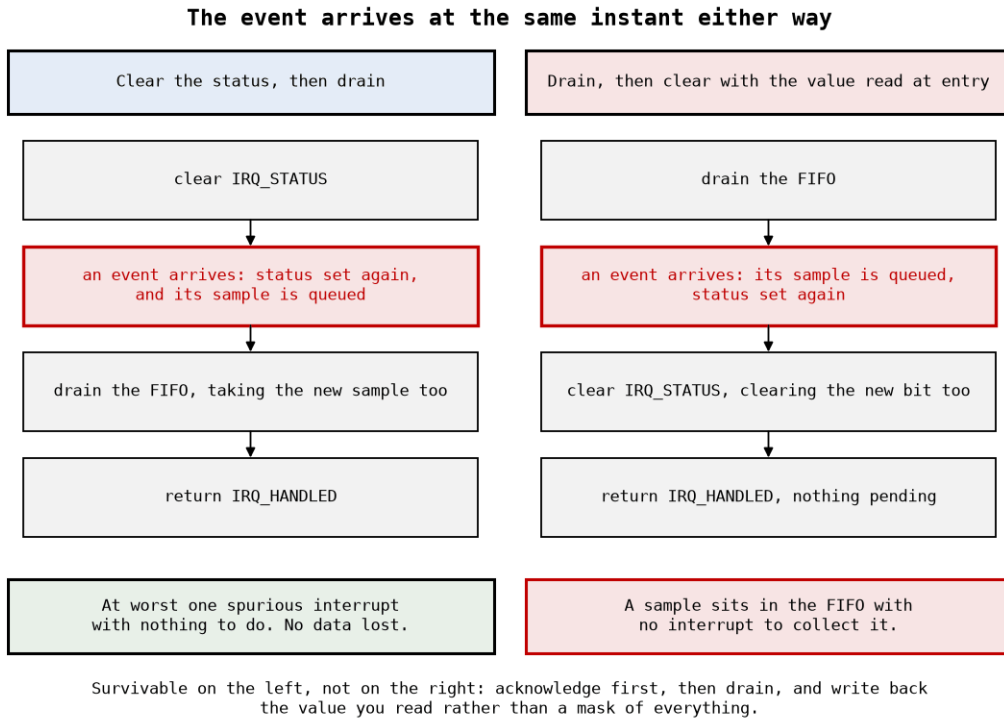


Figure 8.2 Two columns comparing handler orderings, with the same event arriving at the same instant in both. Clearing the status and then draining costs at most one spurious interrupt with nothing to do and loses no data; draining and then clearing with the value read at entry leaves a sample in the FIFO with no interrupt to collect it.

8.1.6 What a missing acknowledgement does

Not a warning, and not a slow driver. The lab's module with nothing in its handler that clears `IRQ_SAMPLE`, neither the `writel` to `IRQ_STATUS` nor the drain, since on this device emptying the FIFO clears it too (§8.1.5), loaded on the target:

```
qa_irq: phys=0xc0000000 irq=17
qa_irq: running, period_ns=1000000 threaded=0
[ 29.847211] rcu: INFO: rcu_preempt detected stalls on CPUs/tasks:
[ 29.848268] rcu: 0-....: (0 ticks this GP) idle=3cd4/1/0x4000000000000002
[ 29.850382] Sending NMI from CPU 1 to CPUs 0:
```

insmod never returned. The line stays asserted, so the handler is re-entered the instant it returns, forever. CPU0 does nothing else from that moment on; twenty-six seconds later the

RCU stall detector notices that a grace period has made no progress and CPU1 sends an NMI to find out what CPU0 is doing.

The machine is gone. It does not recover, and this course's test harness reports it as a **timeout** rather than a failure, which is the distinction `ci/test.sh` draws precisely because this class of bug does not produce a failing test, it produces no test result at all.

Worth noticing: the symptom is an RCU stall, which mentions neither interrupts nor your driver. The first line of a real diagnosis is `/proc/interrupts` from another CPU, where the count is astronomical.

8.1.7 What a handler may not do

One rule, from which the list follows: **a handler may not sleep**.

It runs in interrupt context, on a CPU borrowed from whatever it interrupted (on arm64, on that CPU's own interrupt stack), with its own line masked and every other interrupt on that CPU disabled. There is no task to put to sleep and nothing to schedule in its place.

Forbidden	Use instead
<code>kmalloc(GFP_KERNEL)</code>	<code>GFP_ATOMIC</code> , or allocate in advance
<code>mutex_lock</code>	<code>spin_lock_irqsave</code> , or defer the work
<code>copy_to_user</code>	Defer it; there is no user context here
<code>msleep</code> , <code>schedule</code>	Defer it
<code>wait_event</code>	Defer it

The second rule follows from the first being about *borrowed time*: **be quick**. Every microsecond in a handler is a microsecond stolen from a task that had done nothing wrong, and on a shared or high-rate line it is multiplied. That is what §8.2 is for, and what Chapter 12 measures.

`CONFIG_DEBUG_ATOMIC_SLEEP`, which this course enables, turns a sleep in interrupt context into a BUG: sleeping function called from invalid context with a backtrace instead of an intermittent hang. §7.1.7 is the same rule seen from the locking side.

8.1.8 Sharing data with a handler

The handler and the rest of your driver touch the same data, and the handler can arrive between any two instructions of the rest.

```
spin_lock_irqsave(&priv->lock, flags);
/* ... */
spin_unlock_irqrestore(&priv->lock, flags);
```

`irqsave` and not plain `spin_lock`, for the reason §7.1.5 gives: process context takes the lock, the device interrupts the same CPU, the handler spins for a lock that cannot be released

until the handler returns, and the machine stops. Disabling interrupts on this CPU for the length of the critical section is what prevents it.

Inside the handler itself, plain `spin_lock` is enough: interrupts are already masked there.

The lightest option is often no lock at all. A counter the handler increments and userspace only reads can be an `atomic_t`; the lab’s module uses exactly that, and §7.1.3 is the rule for when that stops being enough.

8.2 Deferred Work

A handler must be quick and must not sleep. Most real work is one or the other. This section is the four places to put it, what each costs, and why one of them is the answer nearly every time now.

8.2.1 The split, and why it has two names

The kernel’s traditional vocabulary is **top half** and **bottom half**. The top half is the handler that runs in interrupt context: acknowledge the device, take the data out of it, and get out. The bottom half is everything else, run later, in a context with fewer restrictions.

The names are old and slightly misleading, because “bottom half” also names one specific mechanism (`local_bh_disable` still refers to it). Modern material says **hard IRQ handler** and **deferred work**, and this section does too.

The question is not whether to split, it is **what to split at**. The dividing line is: the hard handler does what only it can do, which is whatever must happen before the device is quiet and whatever would be lost if it waited. Everything else is deferred.

For the lab’s device that means: read `IRQ_STATUS`, acknowledge, drain the FIFO so no samples are dropped, return. Formatting, copying to userspace, waking readers and anything that allocates belongs on the other side.

8.2.2 Four places to put it

Mechanism	Context	May sleep	Priority	Use it
Softirq	Interrupt-ish	No	Fixed, very high	Almost never. Networking and timers own it
Tasklet	Interrupt-ish	No	Fixed	Deprecated. Recognise it in old code
Workqueue	Kernel thread	Yes	Schedulable	Work that sleeps and is not urgent
Threaded handler	Kernel thread	Yes	Settable, per-IRQ	The default answer for a driver

Softirqs

A fixed, compile-time set of ten or so, defined in the kernel and not extensible by modules. They run after the hard handler returns, still in a context that may not sleep, and they can run concurrently with themselves on different CPUs. Networking and the timer wheel use them. **You will not add one**, and the reason to know they exist is that they are what `local_bh_disable` and `spin_lock_bh` are protecting against.

Tasklets

A softirq with a nicer interface, historically the standard bottom half for drivers. **They are deprecated**, and the kernel's own documentation says so. Two reasons: they run at a fixed high priority nobody can tune, and they still cannot sleep, so they solve only half the problem. Code using them is not broken, it is old; new code should not add any.

```
/* You will meet this. Do not write it. */
DECLARE_TASKLET(my_tasklet, my_func);
tasklet_schedule(&my_tasklet);
```

Workqueues

Work runs on a kernel thread, so it **may sleep**: allocate with `GFP_KERNEL`, take mutexes, do I/O.

```
static void my_work_fn(struct work_struct *work) { /* may sleep */ }
static DECLARE_WORK(my_work, my_work_fn);

/* from the hard handler */
schedule_work(&my_work);
```

`schedule_work` puts it on the shared system queue, which is convenient and means your work waits behind everyone else's. A driver that cares creates its own with `alloc_workqueue`, and one that cares about ordering uses a single-threaded one.

The catch is that **the same work item cannot be queued twice**. If it is already pending, a second `schedule_work` does nothing, so a burst of interrupts produces one run. Sometimes that is exactly right; when it is not, you need a queue of your own between the two halves, which is what the FIFO in this course's driver is.

8.2.3 What runs when

Worth having straight, because the ordering explains a class of confusing latency.

```
device asserts line
-> CPU takes the interrupt, masks it
    -> hard IRQ handler runs          (atomic, fast, must not sleep)
```

```

-> handler returns
-> softirqs run                (still atomic)
   -> tasklets are a softirq
-> scheduler may now run something else
   -> workqueue thread runs when scheduled (may sleep)
   -> threaded IRQ handler runs when scheduled (may sleep)

```

The hard handler is the only part guaranteed to run promptly. Everything below it runs *when the scheduler gets to it*, and how soon that is depends on load, priority and the preemption model, which is Chapter 12’s subject entirely.

8.2.4 Threaded handlers

The modern answer, and the one to reach for by default:

```
request_threaded_irq(irq, qa_hard, qa_thread, IRQF_ONESHOT, "qa_irq", priv);
```

Two functions. `qa_hard` runs in interrupt context and returns `IRQ_WAKE_THREAD`; `qa_thread` then runs in a dedicated kernel thread and may sleep.

You can see the thread:

```
# ps | grep irq
85 root      0:00 [irq/17-qa_irq]
```

It is an ordinary task, with a PID, that the scheduler can preempt, prioritise and account for. That is the whole advantage over a workqueue: **the deferred work has an identity**. You can give it a real-time priority, pin it to a CPU, and see it in `top` when it misbehaves.

IRQF_ONESHOT is not optional for a level-triggered device. It keeps the interrupt masked from the moment the hard handler returns until the thread has finished. Without it the line is unmasked as soon as the hard handler returns, and since a level-triggered device is still asserting until acknowledged, the hard handler is re-entered immediately, which is §8.1.6. Either the hard handler acknowledges before returning, or you use `IRQF_ONESHOT`.

Passing `NULL` as the hard handler is allowed and means “wake the thread for every interrupt”; `IRQF_ONESHOT` is then mandatory.

Measured, on this target

The lab’s module both ways, at a one-millisecond period, over two seconds:

	Elapsed	Interrupts serviced	Shortfall against ideal
Hard handler only	2.11 s	1789	15.2%
Threaded handler	2.10 s	1792	14.7%

The threaded version is not slower, which surprises people who expect the extra context switch to cost. For work of this size the scheduler absorbs it, and what you bought is a handler that may sleep and a thread you can prioritise. Under PREEMPT_RT almost every handler becomes one of these whether you asked or not, which is why Chapter 12 revisits this table.

8.2.5 Choosing

In order of preference:

1. **Do it in the hard handler**, if it is a few register accesses and cannot sleep.
2. **Threaded handler**, if it needs to sleep or is long enough to matter. This is the default.
3. **Workqueue**, if the work is not per-interrupt: something periodic, or triggered from several places, or genuinely low priority.
4. **Tasklet**, never in new code.
5. **Softirq**, not available to you.

Two questions decide between 2 and 3. *Is this work one-per-interrupt?* If yes, a threaded handler already has the plumbing. *Does it need its own priority?* If yes, a threaded handler has one and the shared workqueue does not.

8.2.6 What deferring does not fix

It moves work; it does not make it free, and three things stay broken.

The device still has to be serviced promptly. qa-dev has a FIFO of sixteen samples and sets OVERRUN when it overflows. Deferring the drain does not slow the device down; it just means the FIFO fills while you wait. If the hard handler does not empty it, no amount of deferred work will recover the samples that were dropped.

A high interrupt rate is still a high interrupt rate. At a hundred-microsecond period the lab's module services about 13,000 interrupts in two seconds against an ideal of 21,300, a **39% shortfall**, and no arrangement of deferred work improves that: the cost is in taking the interrupt at all.

What *does* happen at that rate is visible in the driver's own two counters:

```
period=1000000    hard=1813    samples=1813    <- one sample per interrupt
period=100000     hard=13708   samples=14389   <- more samples than interrupts
```

At 1 ms the handler keeps up exactly. At 0.1 ms it does not, and the device's level-triggered line does the only sensible thing: a sample arriving while the line is still asserted does not generate a second interrupt, it simply joins the FIFO that the handler is about to drain. **The interrupts coalesced, and no data was lost.**

That is level triggering paying for itself, and it is the answer to why a device under load generates fewer interrupts than its nominal rate rather than more. Push harder and the FIFO

fills, `OVERRUN` sets, and samples do start being lost; at that point the answer is hardware, in the shape of a bigger FIFO, explicit coalescing, or DMA.

The total work is unchanged. Deferring improves *latency for everything else* and worsens *latency for this device's data*, which is a trade rather than a win. Knowing which of the two you are optimising is the point, and Chapter 12 is where it gets measured rather than argued about.

8.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Trace an interrupt from the device's line to your handler, naming the three numbers involved.
- Read `/proc/interrupts` and say what a stuck count and a racing count each indicate.
- Register a handler, and say what the `dev_id` argument is for on a shared line.
- Say when `IRQ_NONE` is the correct return value, and what breaks if you never return it.
- Acknowledge a level-triggered interrupt in the right order, and say what the wrong order loses.
- List what a hard interrupt handler may not do, and derive the list from one fact rather than memorising it.
- Choose between a workqueue and a threaded handler, and defend the choice.

Questions to test yourself

1. The device tree says interrupt 112 and `/proc/interrupts` says 47. Are they the same interrupt?
2. Your handler runs once and the machine then stops responding. What did you forget?
3. Why can a hard interrupt handler not call `copy_to_user`, given that it is only copying memory?
4. On a shared line, your handler always returns `IRQ_HANDLED`. Describe the symptom on the *other* device sharing that line.
5. What is the difference between a tasklet and a workqueue, in terms of what each may do?
6. You measure 9,847 interrupts where you predicted 10,000. Name two mechanisms that could account for the difference, and say how you would tell which it was.

Where to look

- `tools/qa-dev/qa-dev.h` is the register map, and documents `IRQ_STATUS` as write-one-to-clear.
- The device's own model, `tools/qa-dev/qa-dev.c`, is readable and shows exactly when the line is raised and lowered. Reading the hardware's source is a luxury you will not have again.

Looking ahead

Chapter 9 is about:

- The other half: a reader that wants data which does not exist yet.
- Wait queues, and the lost wakeup that is a bug rather than bad luck.
- `poll`, and how one thread waits on your device and a socket at once.
- Jiffies, `ktime`, and the difference between delaying and sleeping.

8.4 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 8.6 onwards need the target running.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

A warning about Exercise 8.8. It asks you to load a handler that never acknowledges its device. That wedges the machine, on purpose, and the only way out is a reboot. A few seconds here; do not run the equivalent against anything you cannot power-cycle.

Solutions. The exercises that are not Code are answered in §A.8. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 8.1 The three numbers

RECALL

- The device tree says `interrupts = <0 112 4>` and `/proc/interrupts` says `17 ... GIC-0 144`. Explain all three numbers and where each comes from.
- $112 + 32 = 144$. What is the 32?
- Which of the three changes if you boot the same kernel on the same machine with one extra device present? Why?
- A datasheet documents the device's interrupt as 112. A colleague greps `/proc/interrupts` for 112, finds nothing, and concludes the driver is not loaded. What has gone wrong with their reasoning?
- What does the third cell, 4, mean, and what would 1 have meant instead?

Exercise 8.2 Reading /proc/interrupts

RECALL

17:	1789	0	GIC-0 144	Level	qa_irq
-----	------	---	-----------	-------	--------

- Name every field.
- The second CPU's count is 0. Give two different explanations, and say how you would tell them apart.
- The count is not moving and the device is not working. What have you ruled out, and what is still possible?
- The count is increasing by millions per second. What is almost certainly wrong?
- You unload your module and load it again. Does the count restart from zero? What does that imply for any measurement of an interrupt rate?

Exercise 8.3 The acknowledgement

HAND CALCULATION

qa-dev's `IRQ_STATUS` is write-one-to-clear. At the moment your handler runs, it holds `0b01` (`QA_DEV_IRQ_SAMPLE`). For **c**) to **e**), suppose the bit is cleared only by a write to it; **f**) is about the other way this device has.

- Your handler writes `0xFFFFFFFF` to it. What happens to the `SAMPLE` bit? What happens to an `OVERRUN` that the device set one microsecond after your read?
- Your handler writes back exactly the value it read. Same two questions.
- Your handler acknowledges first and then drains the FIFO. Describe the sequence of events that produces a spurious interrupt, and say whether any data is lost.
- Your handler drains the FIFO and then acknowledges with a value it read *before* draining. Describe the sequence that loses an event.
- Of **c**) and **d**), one is survivable and one is not. Which, and why?
- For this device there is a second way to clear `IRQ_SAMPLE` that involves no write to `IRQ_STATUS` at all. What is it, and how would you find that out from `tools/qa-dev/qa-dev.c`?

Exercise 8.4 An interrupt budget

HAND CALCULATION

A device raises an interrupt every 250 microseconds. Its handler takes 8 microseconds.

- How many interrupts per second, and what fraction of one CPU does the handler consume?
- The handler grows to 40 microseconds. Recompute both.
- At what handler duration does the CPU do nothing but service this device?
- The device's FIFO holds 16 samples and it produces one per interrupt. How long can the system ignore the device before samples are dropped?
- You move 30 of the 40 microseconds into a threaded handler. Which of your answers change, and which does not? In particular, does the *system's* total work change?

Exercise 8.5 Where does the work go

DESIGN

For each, choose hard handler, threaded handler, or workqueue, and justify it in one sentence.

- Reading a status register and clearing an interrupt.
- Copying 4 KB out of the device's FIFO into a driver buffer.
- Waking a userspace reader that is blocked on `read()`.
- Allocating a buffer with `GFP_KERNEL`.
- Retrying a failed transfer after a 10 ms delay.
- Updating a statistics counter.
- Logging an error with `dev_err`.

Then: **h)** two of your answers would be illegal in a hard handler. Which, and what exactly goes wrong? **i)** Which config option this course enables would tell you?

Exercise 8.6 Take the interrupt

CODE

Write `lectures/L08/lab/qa_irq.c`. It must:

- Find the device's node with `of_find_compatible_node` using `QA_DEV_DT_COMPATIBLE`, get its address with `of_address_to_resource`, and its interrupt with `irq_of_parse_and_map`. Print both.
- Claim and map the registers as in Chapter 6, and check the identity register.
- Register a handler with `request_irq`, named `qa_irq`.
- Program `PERIOD_NS` from a module parameter (default 1000000) and set `ENABLE | IRQ_EN`.
- In the handler: read `IRQ_STATUS`, **return `IRQ_NONE` if nothing is pending**, acknowledge with the value you read, then drain the FIFO counting samples.

- Count hard interrupts and samples in `atomic_ts` and print them on unload.
- Take a threaded module parameter which, when set, uses `request_threaded_irq` with `IRQF_ONESHOT` and does the draining in the threaded half.
- Disable the device and `free_irq` on unload, in that order.

Check yourself: `make test L=L08` reports **PASSED**, with nine checks. `/proc/interrupts` gains a `qa_irq` line whose count climbs at roughly a thousand per second.

- Why must the device be disabled before `free_irq`, and not after?
- Your handler returns `IRQ_HANDLED` unconditionally instead of checking. On this target nothing appears to break. Explain what you have broken anyway, and on what kind of system it would show up.

Exercise 8.7 The threaded half

CODE

With `threaded=1`:

- Find the kernel thread with `ps`. What is it called, and what does the name tell you?
- The hard handler now returns `IRQ_WAKE_THREAD`. Where must the acknowledgement happen, and what happens if you leave it in the threaded half without `IRQF_ONESHOT`?
- Measure the interrupt rate both ways over two seconds. Is the threaded version slower? Was that your prediction?
- Name two things the threaded handler can do that the hard one cannot, and one thing it gives you that a workqueue does not.

Exercise 8.8 The interrupt that never stops

DESIGN

Read the warning at the top of this section.

- Remove everything in your handler that clears `IRQ_SAMPLE`: the acknowledgement, and the drain with it, because emptying the FIFO clears the bit too (Exercise 8.3, part f)). Rebuild, load it, and record what happens to `insmod`, and how long it takes before the console says anything.
- The message that eventually appears mentions neither interrupts nor your driver. Quote it, and explain the connection.
- This produces no failing test. `make test L=L08` reports something other than a failure; what, and why does the harness draw that distinction?
- From another machine you cannot get, but suppose you could run one command on this one before it died. Which, and what would it have shown?
- `qa-dev` is level-triggered. Would the same missing acknowledgement wedge an edge-triggered device? What would happen instead?

Exercise 8.9 Interrupts counted two ways

CROSS-CHECK

Predict a count from the period, measure it, and account for a shortfall that is larger than measurement error and entirely explicable.

a) **Predict.** Your device is programmed with `PERIOD_NS=1000000`. How many interrupts should it raise in exactly two seconds? Write the number down.

b) **Measure.** Load your module, then take a **delta** across a measured interval:

```
n0=$(grep qa_irq /proc/interrupts | awk '{ print $2 + $3 }')
t0=$(cut -d' ' -f1 /proc/uptime)
sleep 2
n1=$(grep qa_irq /proc/interrupts | awk '{ print $2 + $3 }')
t1=$(cut -d' ' -f1 /proc/uptime)
```

Compute the actual elapsed time and the actual count. Do this three times.

c) **Reconcile.** The measured count is short by something like 15%, consistently. Answer each:

- Why a delta rather than reading the count once? What did §8.1.2 say happens across a reload?
- Your elapsed time is not 2.00 s. Does using the measured elapsed time rather than 2.00 remove the shortfall? By how much does it move it?
- The remaining shortfall is not noise: three runs agree to within a few percent of each other. So it is systematic. **Before reading further, write down a hypothesis.**

d) **Find out.** The device is not a black box; its source is `tools/qa-dev/qa-dev.c`. Find `qa_dev_arm_timer` and read the one line that decides when the next interrupt happens.

- Is the next interrupt scheduled relative to *when the last one fired*, or to an absolute schedule?
- Given that, what is the actual interval between two interrupts, in terms of the programmed period and anything else?
- Does that account for a 15% shortfall at a 1 ms period? Estimate the missing term and say whether it is a plausible number for this system.

e) **Test the explanation, and watch it fail.** A relative re-arm means each interval is period + overhead, so the model predicts a **constant** overhead: a large shortfall at short periods and a small one at long ones.

Run it at `period_ns=10000000` (10 ms), `1000000` (1 ms) and `100000` (0.1 ms). For each, compute the shortfall *and* the overhead your model implies, which is $(\text{elapsed} / \text{count}) - \text{period}$.

The measured result on this target:

Period	Count in ~2.1 s	Shortfall	Implied overhead
10 ms	190	8.7%	947 us
1 ms	1731	17.6%	213 us
0.1 ms	13085	38.6%	63 us

The implied overhead is not constant. It is larger at longer periods, which is backwards. The relative re-arm is real and is part of the answer, and it is plainly not the whole answer.

Do not resolve this by inventing a mechanism. Instead, answer:

- What would the table look like if a fixed per-interrupt overhead were the only effect?
- Your driver is one of at least three things in the measurement: the driver, the kernel's timer and scheduler, and QEMU. Which of the three does your `/proc/uptime` measurement include?
- At 10 ms the guest is idle almost all the time and at 0.1 ms it is not. Why might *that* affect how promptly an emulated timer is serviced?
- What experiment would separate the device's behaviour from the emulator's? Name one you could run, and one you could not run here at all.

f) **The fast end is a different effect.** At 0.1 ms compare the driver's own two counters:

```
hard=13708 samples=14389
```

More samples arrived than interrupts were taken.

- How can one interrupt deliver more than one sample? Answer in terms of level triggering and when the line is re-asserted.
- At 1 ms the same two counters are equal. What changed?
- Which status bit would tell you the device had given up and started dropping samples, and did it get set?

g) A colleague reports "the device is dropping 15% of its interrupts, the driver is broken". There are at least three things wrong with that sentence. Rewrite it so that every claim in it is supported by a measurement you have actually taken, and say what you would still not know.

h) You are asked for "the interrupt latency of this device" for a datasheet. Given everything above, say what you would put, what conditions you would state alongside it, and what you would refuse to claim. Chapter 12 is the chapter that measures this properly, using a register that does not exist on real hardware; say why that register is necessary.

Sleeping, Waiting, and Time

The half the software drives: a reader that wants data which does not exist yet.

IN THIS CHAPTER

- What sleeping is: a task state, a queue, and a scheduler that stops choosing you.
- Wait queues, and the condition that is re-tested after every wake.
- The lost wakeup: why the obvious flag-and-sleep is wrong every time, not rarely.
- Completions, and when they are the simpler answer.
- Blocking `read()`, `O_NONBLOCK`, and `-EAGAIN` at the right moment.
- `poll`, and what a driver must implement for `select` to work on it.
- Signals, `-ERESTARTSYS`, and an interruptible sleep that gets interrupted.
- Jiffies, `HZ`, `ktime`; delaying against sleeping; timers against `hrtimers`.

This chapter is the other half of Chapter 8: a program has asked for data that does not exist yet, and something has to happen to it in the meantime. It explains what sleeping is, a task state, a queue, and a scheduler that stops choosing you, and why the obvious test-then-sleep loses its wakeup; what `wait_event_interruptible` expands to, and which line of it closes the window; how a blocking `read()` must answer `O_NONBLOCK` and a signal; and what `poll` buys. Then it turns to time: jiffies and `HZ`, `ktime` for measuring, delaying against sleeping, timers against `hrtimers`, and why `msleep(1)` on this course's kernel takes eight milliseconds.

Read §9.1 first; it is sleeping and wait queues. §9.2 is time, delays and timers, with the course's measurement of what each kind of sleep actually costs. The lab is specified by Exercises 9.6 and 9.7: the driver from Chapter 8 with a character device added, whose `read` blocks until the interrupt has data for it and which implements `poll`, checked by the userspace tester `qa_wait_test` that ships with the course; and a module that measures the sleeps. The exercises in §9.4 end with the Cross-check: predict what `msleep(1)`, `msleep(4)` and `msleep(10)` cost given `HZ`, then measure them. One call takes eight times what it asks for, and two of the three answers are the same number. The lab is done when `make test L=L09` reports **PASSED**, including the `poll` test driven from userspace.

9.1 Sleeping and Waiting

The other half of Chapter 8. That chapter was about the half the hardware drives: a line goes high and a handler runs. This one is about the half the software drives: a program has asked for data that does not exist yet, and something has to happen to it in the meantime.

§9.2 is time, delays and timers.

The idea to carry: **sleeping is not waiting**. A sleeping task is off the run queue entirely and costs nothing; a spinning task is on it and costs a CPU. The whole of this section is machinery for getting into the first state safely.

9.1.1 What sleeping is

A task has a state. The ones that matter:

State	ps shows	On the run queue	Woken by
TASK_RUNNING	R	Yes	It is already running or ready
TASK_INTERRUPTIBLE	S	No	An explicit wake, or a signal
TASK_UNINTERRUPTIBLE	D	No	An explicit wake only

To sleep, a task sets its state to something other than TASK_RUNNING and calls `schedule()`. The scheduler then does not consider it again until something sets it back to TASK_RUNNING. That is all a wait queue is built out of.

Use **TASK_INTERRUPTIBLE in a driver, essentially always**. The uninterruptible state exists for short, uninterruptible hardware operations, and a task stuck in D cannot be killed, not even with signal 9, which is §2.1.5's observation arriving with a cause.

9.1.2 The wrong version, and why it is wrong every time

The obvious implementation of “wait until there is data”:

```
/* WRONG */
while (fifo_empty())
    schedule();
```

Two things are wrong. It never sets its state, so it stays TASK_RUNNING and the scheduler keeps picking it: this is a busy-wait with extra steps, and it burns a CPU doing nothing. And it is on no wait queue, so even if it did sleep, no wake-up could ever find it.

The obvious fix is worse, because it looks right. Take it that the reader has already joined the queue the handler wakes; the snippet leaves that line out, because what is wrong is elsewhere:

```
/* WRONG, and subtly */
if (fifo_empty()) {
    set_current_state(TASK_INTERRUPTIBLE);
```

```

    schedule();
}

```

Consider the interleaving. The reader evaluates `fifo_empty()` and finds it true. **The interrupt fires here.** The handler adds data and calls `wake_up_interruptible`, which finds nothing asleep on the queue and does nothing at all. The reader then sets its state and calls `schedule()`, and sleeps waiting for a wakeup that has already happened: until the next one, if there is a next one, and forever if there is not.

This is the lost wakeup, and it is not a rare race. The window is two instructions wide, but with an interrupt arriving every millisecond it is hit within seconds. It is not the kind of bug that waits for production; it is the kind that fails in your first test and then works on the second, which is worse.

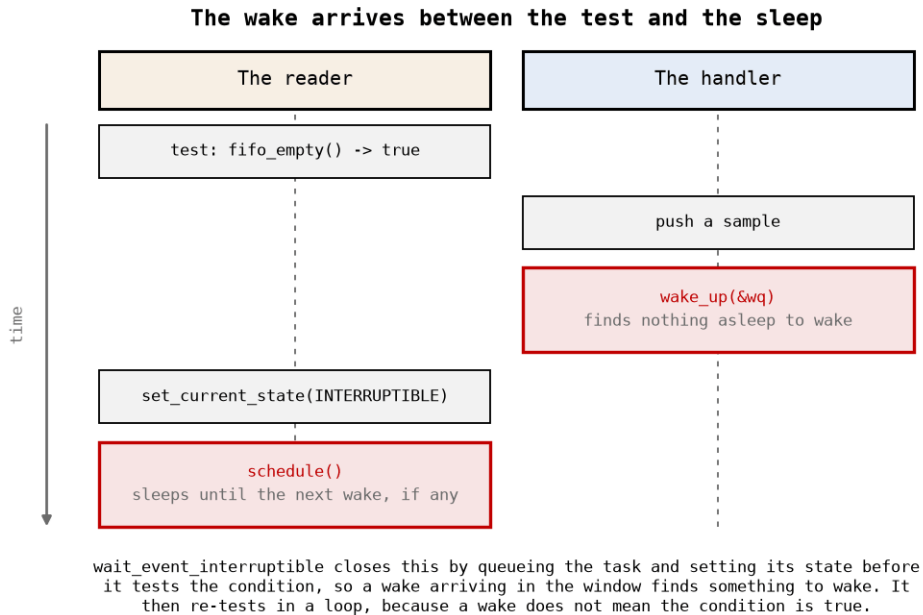


Figure 9.1 A sequence diagram with a lane for the reader and a lane for the handler. The reader tests `fifo_empty` and finds it true; the handler then pushes a sample and calls `wake_up`, which finds nothing asleep to wake; the reader then sets its state and calls `schedule`, and sleeps until the next wake, if one ever comes.

9.1.3 What the macro actually does

`wait_event_interruptible` is the correct version, and reading its expansion is worth more than memorising its usage. From `include/linux/wait.h`, abridged:

```
for (;;) {
    long __int = prepare_to_wait_event(&wq_head, &__wq_entry, state);

    if (condition)
        break;

    if (__wait_is_interruptible(state) && __int) {
        __ret = __int;
        goto __out;
    }

    schedule();
}
finish_wait(&wq_head, &__wq_entry);
```

Three things close the window in §9.1.2, in this order:

1. **prepare_to_wait_event first.** It adds the task to the queue *and* sets its state, before the condition is looked at. From that moment a wake_up cannot be missed: it will find this task on the queue and set it runnable.
2. **The condition is tested after being queued.** If the data arrived during step 1, the test sees it and breaks out without ever sleeping.
3. **It is a loop.** After `schedule()` returns, the condition is tested again.

Step 3 is why the macro takes a *condition* and not a flag. **A wake does not mean the condition is true.** Several readers may be woken and the first to run may take all the data; a wake may be spurious. The sleeper must re-check, and the loop does it for you.

Two consequences for how you write the condition: it is evaluated **many times**, so it must be cheap and free of side effects, and it is evaluated **from the waiting task's context**, so it must be safe to call there.

9.1.4 Using it

```
static DECLARE_WAIT_QUEUE_HEAD(qa_readq);

/* the reader */
for (;;) {
    n = take_the_data();          /* under the lock; 0 if there was none */
    if (n)
```



```

        break;
    if (file->f_flags & O_NONBLOCK)
        return -EAGAIN;
    if (wait_event_interruptible(qa_readq, !fifo_empty()))
        return -ERESTARTSYS;
}

/* the interrupt handler */
if (got_data)
    wake_up_interruptible(&qa_readq);

```

The reader loops for the same reason the macro does. A wake says there was data when the waker looked; with two readers, the other may have taken it by the time this one runs. A reader that takes nothing goes back to waiting, because returning 0 would tell its caller the device had finished.

Function	Wakes
wake_up_interruptible	Tasks in TASK_INTERRUPTIBLE
wake_up	Those and TASK_UNINTERRUPTIBLE
wake_up_interruptible_all	The same, and every exclusive waiter, not one

An *exclusive* waiter is one that asked, with `wait_event_interruptible_exclusive`, to be woken on its own. The first two wake every other waiter on the queue and at most one exclusive one.

Waking is safe from interrupt context and does not sleep, which is what makes this the natural join between Chapter 8's handler and this chapter's reader.

A name to avoid. Calling the queue `readq` does not compile: the kernel already has `readq`, the 64-bit MMIO accessor from §6.2.4, and the error is 'readq' redeclared as different kind of symbol. The kernel namespace is flat and short names are taken; prefix yours with the driver's name.

9.1.5 Blocking read(), properly

Four cases, and a driver has to get all four right:

Situation	Return
Data available	The bytes, and how many
Empty, blocking	Sleep until there is data
Empty, O_NONBLOCK	-EAGAIN
Signal arrives while asleep	-ERESTARTSYS

Never 0. Zero means end of file, which is Chapter 5's subject and the reason that chapter returned `-EAGAIN` unconditionally. This chapter upgrades that: `-EAGAIN` is now the answer only for a caller who asked for it with `O_NONBLOCK`.

-ERESTARTSYS

`wait_event_interruptible` returns non-zero when a signal arrived. The driver returns `-ERESTARTSYS`, which never reaches userspace: the kernel sees it and either restarts the system call transparently or converts it to `-EINTR`, depending on how the signal handler was installed.

Getting this wrong makes your device un-interruptible with Ctrl-C, and users notice. A driver that returns `-EINTR` directly is close but takes the restart decision away from the kernel; one that ignores the return value and carries on has a reader that cannot be killed.

toggling `O_NONBLOCK`

Worth knowing because it catches people writing tests. A descriptor's blocking behaviour is a property of the open file, changeable at run time:

```
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

The course's own tester needs this: draining a blocking descriptor attached to a device that produces a sample every millisecond never terminates, because there is always more data a millisecond from now.

9.1.6 poll, and what it buys

Two lines of driver code, and every multiplexing interface in userspace works on your device.

```
static __poll_t qa_poll(struct file *file, struct poll_table_struct *wait)
{
    poll_wait(file, &qa_readq, wait);
    return fifo_empty() ? 0 : (EPOLLIN | EPOLLRDNORM);
}
```

poll_wait does not wait. It registers the caller's interest in your wait queue and returns immediately; the sleeping is done by the `select/poll/epoll` machinery above you, on all the descriptors at once. Your function is then called a second time after any wake, to ask again.

So the function must do exactly two things: register on every queue that could make it ready, and return a mask describing readiness *now*.

Bit	Means
EPOLLIN	Readable
EPOLLRDNORM	Readable, normal data. Conventionally set with EPOLLIN
EPOLLOUT	Writable
EPLLERR	An error
EPOLLHUP	Hung up

What it buys is that **one thread can wait on your device and a socket and a timer together**, which is the whole architecture of most userspace daemons. Without `poll`, a program that wants to watch your device and anything else needs a thread per descriptor.

Measured on the target: with the device running, `select` reports the descriptor readable within a second; with the device stopped, `select` given a 300 ms timeout returns 0 after 305 ms.

9.1.7 Completions

For the common special case of “wait for this one thing to finish once”.

```
static DECLARE_COMPLETION(setup_done);

wait_for_completion(&setup_done);      /* the waiter */
complete(&setup_done);                 /* whoever finished */
```

A completion is a wait queue plus a “has it happened” flag, so it has no lost-wakeup problem to solve: completing before anyone waits is fine, and the waiter returns immediately.

Use a completion for a one-shot event (initialisation finished, a DMA transfer completed) and a wait queue for a recurring condition (data is available). Choosing a wait queue for a one-shot means writing the flag yourself, which is where the lost wakeup comes back.

`wait_for_completion_interruptible` and `wait_for_completion_timeout` exist and are usually what you want in a driver, for the same reasons as their `wait_event` equivalents.

9.1.8 What may sleep, and where

The rules from §7.1.7 and §8.1.7, stated once more because this is the chapter where you deliberately sleep:

Context	May sleep
A system call, so read	Yes
A threaded IRQ handler	Yes
A workqueue	Yes
A hard IRQ handler	No
Holding a spinlock	No
A timer callback	No

The one that catches people in this chapter: **you may not sleep while holding a spinlock**, and `wait_event_interruptible` sleeps. So the condition is checked under the lock, the lock is dropped, and the wait happens outside it. Doing it the other way round deadlocks the machine, and `CONFIG_DEBUG_ATOMIC_SLEEP` turns that into a BUG: sleeping function called from invalid context rather than a hang.

9.2 Time, Delays and Timers

Four ways to say “later”, how much later each of them actually means, and why the difference is larger than anyone expects.

Every number here was measured on this course’s target with the lab’s `qa_timing` module, at `HZ=250`. Yours will match, because they are properties of the configuration rather than of the machine.

9.2.1 Jiffies, and why not to quote them

The kernel counts ticks in a global jiffies. The tick rate is `HZ`, a compile-time constant:

```
# zcat /proc/config.gz | grep '^CONFIG_HZ='
CONFIG_HZ=250
```

so on this kernel **one jiffy is 4 milliseconds**. Common values are 100, 250, 300 and 1000, and which one a distribution picks is a trade between timer resolution and tick overhead.

A jiffy is not a unit of time. It is a unit of ticks, and the same code on a `HZ=1000` kernel waits a quarter as long. Any duration you care about should be written in real units and converted:

```
unsigned long timeout = jiffies + msecs_to_jiffies(500);
if (time_after(jiffies, timeout)) { /* ... */ }
```

`time_after` and `time_before` rather than `<` and `>`, because jiffies wraps. On a 32-bit counter at `HZ=250` that is once every 199 days, and the comparison macros handle it; a naive comparison produces a driver that misbehaves twice a year.

9.2.2 ktime, for measuring

Jiffies are for *scheduling*. For *measuring*, use `ktime`, which is nanoseconds:

```
ktime_t start = ktime_get();
/* ... */
64 us = ktime_us_delta(ktime_get(), start);
```

Clock	Reads	Use for
mtime_get	Monotonic since boot	Intervals. Always this one
mtime_get_real	Wall-clock time	Timestamps a human will read
mtime_get_boottime	Monotonic, includes suspend	Intervals across a sleep

Measure intervals with the monotonic clock. Wall-clock time can jump forwards or backwards when NTP corrects it or somebody sets the date, and an interval computed across that is nonsense, occasionally negative. This is the same rule userspace has with `CLOCK_MONOTONIC`, and it is the one Chapter 12 depends on entirely.

9.2.3 Delaying against sleeping

Two families, and confusing them is a correctness bug rather than a performance one.

Function	Mechanism	Context	Costs
udelay(n)	Busy-wait	Any , including atomic	A CPU, entirely
mdelay(n)	Busy-wait	Any	A CPU, for milliseconds
msleep(n)	Timer wheel	Process context only	Nothing
usleep_range(a, b)	hrtimer	Process context only	Nothing

The delay family burns a CPU and is legal in atomic context precisely because it never sleeps. **The sleep family gives the CPU up** and cannot be used where sleeping is forbidden.

The rule: **use udelay only for short hardware delays in code that cannot sleep**, and only for a few microseconds. `udelay(5000)` is legal, compiles, works, and spends five milliseconds of a CPU doing nothing, which is a real-time problem that Chapter 12 will come back to. `mdelay` exists mostly so that its name can warn you.

9.2.4 What the sleeps actually cost

This is the measurement that surprises people. From the lab's `qa_timing`, 200 repetitions of each, at `HZ=250` where one jiffy is 4000 us:

Call	Asked for	Measured, each	Ratio
<code>msleep(1)</code>	1,000 us	8,000 us	8.0×
<code>msleep(4)</code>	4,000 us	8,000 us	2.0×
<code>msleep(10)</code>	10,000 us	15,998 us	1.6×
<code>usleep_range(1000)</code>	1,000 us	1,443 us	1.4×
<code>usleep_range(100)</code>	100 us	307 us	3.1×
<code>udelay(100)</code>	100 us	112 us	1.1×

`msleep(1)` and `msleep(4)` take exactly the same time. Asking for a quarter as long got nothing. That single row is the most useful thing in this section: `msleep` is quantised to jiffies, and below one jiffy the argument stops carrying information.

The arithmetic is exact and you can reproduce it. `msecs_to_jiffies` rounds **up**:

Call	msecs_to_jiffies	Plus the guarantee	Total
msleep(1)	1 jiffy	+1	2 jiffies = 8 ms
msleep(4)	1 jiffy	+1	2 jiffies = 8 ms
msleep(10)	3 jiffies	+1	4 jiffies = 16 ms

Where the extra jiffy comes from is worth knowing, because it is not in `msleep`'s source. `schedule_timeout`'s own documentation says:

at least `@timeout` jiffies are guaranteed to pass before the routine returns

“At least” is the whole answer. You call it partway through a jiffy, and the kernel does not know how far through. To guarantee that *N* jiffies have *elapsed*, it must wait for *N*+1 ticks. The overshoot is the price of a guarantee that never returns early.

usleep_range is the fix, and the table shows why: it uses hrtimers rather than the jiffy wheel, so a 1 ms request costs 1.4 ms instead of 8 ms. It takes a range because giving the kernel slack lets it coalesce your timer with others already due, which saves wakeups on a system trying to stay idle. Use it for anything between about 10 us and 20 ms; `msleep` above that, where the jiffy quantum stops mattering.

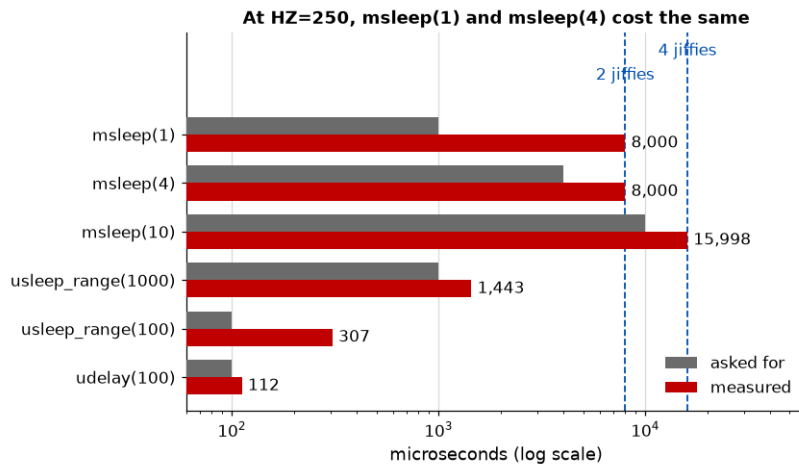


Figure 9.2 Horizontal bars comparing what each call asked for against what it measured, on a logarithmic axis, with dashed lines at two and four jiffies. `msleep(1)` and `msleep(4)` both land on 8,000 microseconds, and `msleep(10)` on 15,998.

9.2.5 Timers

Two kinds, and the difference is the same jiffy-against-nanosecond split.

timer_list, on the jiffy wheel

```
static struct timer_list my_timer;

timer_setup(&my_timer, my_callback, 0);
mod_timer(&my_timer, jiffies + msecs_to_jiffies(500));
```

Resolution is a jiffy, and the wheel deliberately batches timers into buckets, so a long timeout may be rounded by a noticeable fraction. That is a feature: it lets an idle system stay idle.

The callback runs in softirq context, so it may not sleep, may not take a mutex, and may not call `copy_to_user`. This catches people, because the code around a timer is usually process context and the callback looks like it belongs to it.

timer_delete_sync before freeing anything the callback touches, and note the *sync*: it waits for a callback already running on another CPU. timer_delete alone can return while the callback is still executing on the memory you are about to free. Most code you will read calls them `del_timer_sync` and `del_timer`, the old names, which 6.12 keeps as wrappers marked not for new code.

hrtimer, on real time

```
hrtimer_init(&my_hrtimer, CLOCK_MONOTONIC, HRTIMER_MODE_REL);
my_hrtimer.function = my_callback;
hrtimer_start(&my_hrtimer, ms_to_ktime(500), HRTIMER_MODE_REL);
```

Note that the callback is assigned to the structure rather than passed to the initialiser. That is the 6.12 interface; a later kernel introduces `hrtimer_setup`, which takes the callback as an argument and is what newer material will show you. This is §3.1.2's unstable internal API again, in a function you would meet in your first week.

Nanosecond resolution, backed by a real hardware timer rather than the tick. The callback returns `HRTIMER_NORESTART` or `HRTIMER_RESTART`, which makes periodic timers straightforward.

hrtimers cost more per timer, so the kernel uses the wheel for the many timeouts that never expire (network retransmits, mostly) and hrtimers for the few that must be accurate.

Choosing

Use	Because
timer_list for a timeout	It will usually be cancelled, and precision is irrelevant
hrtimer for a deadline	You need the resolution, and it will fire
Neither, if an interrupt will do	The device already knows when it has data

That last row is worth stating. This course's driver could poll the device on a timer; it does not, because Chapter 8 gave it an interrupt. **A timer is what you use when nothing tells**

you. The exercises ask you to build the timer version anyway and compare, because the comparison is the argument.

9.2.6 What none of this gives you

None of it is a guarantee about the maximum. Everything above is “at least”, and the numbers in §9.2.4 are averages over 200 repetitions on an idle machine. What happens to the worst case under load is not in this section, is not what `msleep` documents, and is the subject of Chapter 12.

And the measurements themselves contain the emulator. The 1,443 us for a 1 ms `usleep_range` is this target’s number, and part of it belongs to QEMU rather than to Linux. That is the same caution the Cross-check of Chapter 8, Exercise 8.9, arrives at, and Chapter 12 states it in full.

9.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Say what state a sleeping task is in, and what a spinning one is in.
- Describe the lost wakeup, and say which line of `wait_event_interruptible` prevents it.
- Explain why the condition passed to a `wait` macro is evaluated more than once.
- Implement a blocking `read()` that handles `O_NONBLOCK` and signals correctly.
- Implement `poll`, and say what your driver must do for `select` to work on it.
- Choose between `udelay` and `msleep` from the context and the duration.
- Say what a jiffy is on a given kernel, and why quoting a duration in jiffies is a mistake.
- Choose between a `timer_list` and an `hrtimer`, and say what resolution each gives you.

Questions to test yourself

1. Why does `wait_event_interruptible` take a condition rather than a variable to wait on?
2. Your reader wakes, finds no data, and returns 0 to userspace. What did the caller conclude, and was it true?
3. A user presses Ctrl-C while your `read()` is blocked and nothing happens. What did you fail to return?
4. What is HZ on the kernel you built, and how many jiffies is a 3 ms delay?
5. `msleep(1)` in a loop of 1000 takes noticeably more than one second. Why?
6. Why can `udelay(5000)` be a bug even in code that is allowed to busy-wait?
7. Your driver implements `read` but not `poll`. What happens when a program calls `select` on it?

Where to look

- Chapter 8 produced the events this chapter waits for. The two labs are one driver.

Looking ahead

Chapter 10 is about:

- How a driver finds its hardware without an address in its source.
- Bus, device and driver, and the match that puts the three together.
- Device tree syntax, properly, and the ranges you already translated by hand in Chapter 6.
- probe, and the moment your driver stops being a module that pokes at an address.

9.4 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 9.6 onwards need the target running.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

Solutions. The exercises that are not Code are answered in §A.9. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 9.1 Sleeping and spinning

RECALL

- A task is sleeping and another is spinning on a flag. For each, say what state it is in, whether it is on the run queue, and what it costs.
- Why can a task in `TASK_UNINTERRUPTIBLE` not be killed with signal 9? What is it usually waiting for, and what does its presence in `ps` output tell you?
- Which of the two sleeping states should a driver use, and why essentially always?
- `schedule()` is called by the incorrect versions in §9.1.2 and by the correct one in §9.1.3. What else has to happen for it to be a sleep rather than a yield?

Exercise 9.2 The lost wakeup

RECALL

The second version from §9.1.2, with the reader already on the queue the handler wakes:

```

if (fifo_empty()) {
    set_current_state(TASK_INTERRUPTIBLE);
    schedule();
}

```

- Write out the interleaving that loses the wakeup. Be precise about where the interrupt lands.
- How wide is the window, in instructions? Given a device interrupting every millisecond, roughly how long before you hit it?
- Which line of `__wait_event`'s expansion closes it? Quote it.
- `wait_event_interruptible` takes a **condition**, not a variable. Give two separate reasons, one about the lost wakeup and one about what happens after a wake.
- The condition is evaluated many times. What two properties must it therefore have?
- A colleague argues the bug is rare because the window is two instructions. What is wrong with that reasoning?

Exercise 9.3 Jiffies

HAND CALCULATION

This kernel is HZ=250.

- How long is a jiffy, in microseconds?
- `msecs_to_jiffies(1)`, `msecs_to_jiffies(4)`, `msecs_to_jiffies(10)`: give all three. Does it round up or down, and why does that choice matter?
- `msleep` sleeps for at least `msecs_to_jiffies(n)` jiffies and in practice one more. Predict the measured duration of `msleep(1)`, `msleep(4)` and `msleep(10)`.
- Two of those three are the same number. Which, and what does that mean for a driver that calls `msleep(1)` in a loop expecting a millisecond each time?
- The same driver is rebuilt on a HZ=1000 kernel. Recompute all three.
- `jiffies` is compared with `time_after` rather than `>`. What goes wrong with `>`, and how often, on a 32-bit counter at HZ=250?

Exercise 9.4 Choosing a delay

HAND CALCULATION

For each, choose `udelay`, `usleep_range`, `msleep`, or a timer, and justify it in one sentence.

- a) A hardware register needs 2 μ s to settle, inside a spinlock.
- b) Polling a status bit that takes about 500 μ s, in probe.
- c) Retrying a failed transfer in 100 ms.
- d) Waiting 5 ms in a hard interrupt handler.
- e) Giving up on a device that has not responded within 2 seconds.
- f) Waiting 50 μ s in process context, as accurately as possible.

Then: g) one of the six is a trick. Which, and what is the right answer?

Exercise 9.5 The four cases of read

DESIGN

Your driver's `read` faces four situations. For each, give the return value and say what the calling program observes.

- a) 40 samples available, 100 requested.
- b) Nothing available, descriptor is blocking.
- c) Nothing available, descriptor has `O_NONBLOCK`.
- d) Nothing available, blocking, and the user presses Ctrl-C.

Then:

- e) For b), Chapter 5 returned `-EAGAIN` and this chapter blocks instead. What changed, and why was `-EAGAIN` the right answer then?
- f) For d), what happens if you return `-EINTR` instead of `-ERESTARTSYS`? What if you ignore the return value of `wait_event_interruptible` entirely?
- g) What does returning 0 mean, and name a program that would silently do the wrong thing.

Exercise 9.6 Block until there is data

CODE

Write `lectures/L09/lab/qa_wait.c`. Start from your Chapter 8 driver and add a character device.

- Keep the interrupt handler from Chapter 8, but have it copy samples into a driver-side FIFO of 256 entries under a spinlock, and call `wake_up_interruptible` when it added anything.
- Add `/dev/qa_wait`, registered the long way as in Chapter 5.
- `read` returns whole samples. Empty and blocking: `wait`. Empty and `O_NONBLOCK`: `-EAGAIN`. Interrupted: `-ERESTARTSYS`.
- Implement `poll`.
- Take a `period_ns` parameter; **zero means create the device but leave it stopped**, which is how the tests reach the idle-device cases.

Check yourself: make `test L=L09` reports **PASSED**, with eighteen checks in total: ten from the harness and eight from the userspace tester across its two modes. The shipped `qa_wait_test` exercises both a running and a stopped device.

- You will be tempted to hold the spinlock across `wait_event_interruptible`. What happens, and which config option this course enables tells you?
- Name your wait queue `readq` and try to build. Quote the error and explain it.
- With the device running, drain the device and time the next `read`. How long did it block, and what woke it?

Exercise 9.7 Measure the sleeps

CODE

Write `lectures/L09/lab/qa_timing.c`. It must print HZ and the jiffy length, then measure the average duration of `msleep(1)`, `msleep(4)`, `msleep(10)`, `usleep_range(1000, 1000)`, `usleep_range(100, 100)` and `udelay(100)` over a configurable number of repetitions, using `ktime_get`.

Check yourself: `msleep(1)` and `msleep(4)` come out within a few hundred microseconds of each other, and `usleep_range(1000)` comes out below one jiffy.

- Why `ktime_get` and not `ktime_get_real`?
- `udelay(100)` measures about 112 us and `usleep_range(100)` about 307 us. Explain both the ordering and the gap.
- Your module measures `msleep` by calling it 200 times in a loop. Name one thing that measures which a single call would not, and one thing it hides.

Exercise 9.8 A timer instead of an interrupt

DESIGN

Your driver gets its data because the device interrupts. Suppose it did not.

- a) Rewrite the design using a `timer_list` that polls the device every 4 ms. What is the callback allowed to do, and what would you have to move?
- b) With `hrtimer` instead, what changes about resolution and about where the callback runs?
- c) For each design, say what happens to a sample that arrives just after a poll.
- d) The device's FIFO holds 16 samples. At a 1 ms sample rate, what is the slowest poll interval that cannot lose data? What does that number become if the FIFO were 4 deep?
- e) State the general rule for when polling is the right answer and when an interrupt is.

Exercise 9.9 A millisecond that is not a millisecond

CROSS-CHECK

Predict a duration from HZ, measure it, and find that one call takes eight times as long as it asks for.

a) **Predict.** This kernel is HZ=250. Using `msecs_to_jiffies` rounding up, and the rule that `schedule_timeout` guarantees *at least* the requested number of jiffies, predict the measured duration of:

- `msleep(1)`
- `msleep(4)`
- `msleep(10)`
- `usleep_range(1000, 1000)`

Write all four down before running anything.

b) **Measure.** Load `qa_timing` with `reps=200` and record all four.

c) **Reconcile.** Answer each:

- Which call came out furthest from what it asked for, and by what factor? Did your prediction expect it?
- Two of the four measurements are the same number to within a rounding error. Which two, and is that a bug in the measurement, in `msleep`, or in your expectation?
- `msleep(4)` asks for exactly one jiffy and takes two. Where does the second jiffy come from? `msleep`'s source does not add one; find the sentence in `schedule_timeout`'s documentation that explains it, and say why a guarantee of "at least" costs an extra tick.
- `usleep_range(1000)` asks for the same duration as `msleep(1)` and takes a fifth as long. What is it using instead, and what did you give up to get that?

d) **Make a prediction and test it.** Your account of the extra jiffy implies something specific about `msleep(5)` on this kernel. Predict it, measure it, and say whether you were right.

- e) **Change the constant.** Recompute all four predictions for HZ=1000. Which of the four measurements would change, and which would not? What does that tell you about which of these functions are quantised and which are not?
- f) A driver contains `for (i = 0; i < 1000; i++) msleep(1);` in a loop, with a comment saying “wait one second”. How long does it actually take on this kernel? On a HZ=1000 kernel? Rewrite the line so it does what the comment says on both.
- g) You are asked to document your driver’s worst-case response time. Everything measured here was an average over 200 repetitions on an idle machine. Say what you would put in the document, what you would refuse to claim from these numbers, and what you would have to measure instead. Chapter [12](#) is that measurement.

PART IV

Fitting into the Kernel

The Device Model and the Device Tree

How a driver finds its hardware without being told where it is.

IN THIS CHAPTER

- Bus, device and driver, and the match that puts the three together.
- kobjects and sysfs, and the reference counting under a device's lifetime.
- probe and remove: what may be done in each, and what must not.
- The platform bus, and what “platform device” really means.
- of_match_table, module aliases, and how autoloading actually happens.
- Device tree syntax: compatible, reg, interrupts, phandles, #address-cells, ranges.
- Bindings, dt-schema, and where the DTB enters the boot flow.
- Delete every address from your driver and make it probe anyway.

This chapter is about how a driver finds its hardware without containing a single address. It explains the device model's three parts, the bus, the device and the driver, and the match that joins them; what probe and remove may do, and why everything taken with devm_ is released when the device is unbound rather than when the module goes; and the four steps that turn a device appearing into a module loading. It then reads the device tree that describes the hardware: compatible, reg and the cells its parent declares, the translation through ranges that Chapter 6 did by hand, and an interrupt specifier that only its controller can decode.

Read §10.1 first; it is the device model. §10.2 is the device tree, and ends where the driver takes its address and its interrupt out of it. The lab is specified in Exercise 10.6: the Chapter 8 driver rewritten as a platform driver, qa_platform.c, with no address and no interrupt number anywhere in its source; and Exercise 10.7 unbinds and rebinds it to show what devm_ is attached to. The exercises in §10.4 end with the Cross-check: work out the device's physical address and interrupt by hand from the raw bytes of the device tree, then compare them with the name the kernel gave the device and with what probe was handed. The lab is done when `make test L=L10` reports **PASSED**, with nineteen checks.

10.1 The Device Model

How a driver finds its hardware without being told where it is, and what the kernel does with the three-way relationship between buses, devices and drivers.

§10.2 is where the description comes from.

The idea to carry: **you do not write code that goes looking for hardware**. You register a driver that describes what it can handle, something else registers a device that describes what exists, and the kernel calls your probe when the two match. Everything in this section is a consequence of that inversion.

10.1.1 Three things and a match

	Is	Registered by
Bus	A matching policy	The kernel, once
Device	Something that exists	Whatever discovered it
Driver	Code that can handle a class of devices	Your module

Each bus keeps two lists, its devices and its drivers. Whenever either list changes, the bus tries to match the new entry against every entry on the other list, and calls probe on each success.

A bus is a matching policy, not a wire. platform is a bus and there is no platform bus in any schematic; it is the bus for devices that cannot be discovered, which is most of an SoC. PCI is a bus and can enumerate itself. USB is a bus and enumerates on hotplug. What they share is the matching machinery, not the electricals.

The consequence people find surprising is the ordering: **the device and the driver can arrive in either order**. On the target, the device for qa-dev exists from boot and sits there unbound:

```
# ls /sys/bus/platform/devices/c0000000.qa-dev
driver_override modalias of_node power subsystem uevent
# ls /sys/bus/platform/devices/c0000000.qa-dev/driver
ls: no such file
```

Loading a module that registers a matching driver binds it immediately. Load the module first on a machine where the device appears later, and it binds then instead. Your probe is written the same way either way.

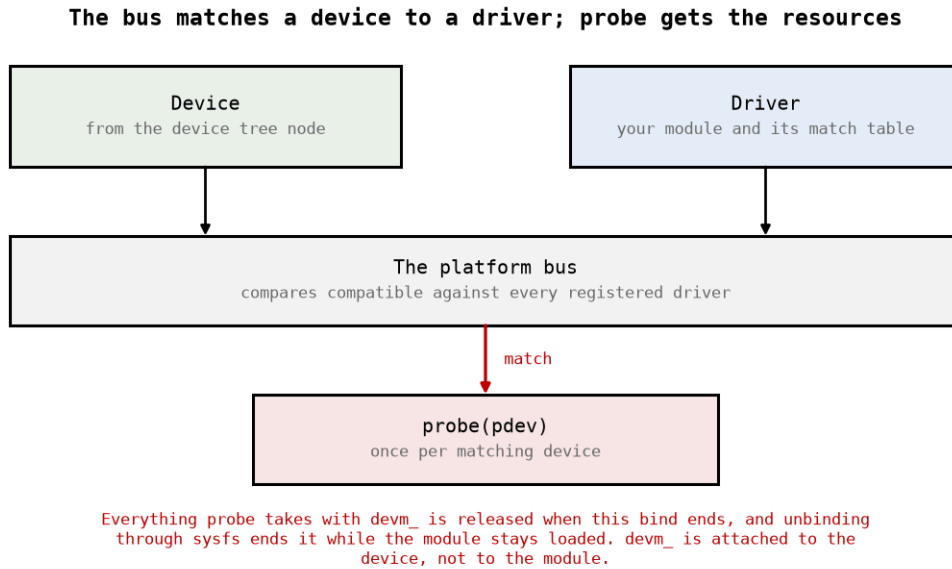


Figure 10.1 A device from the device tree and a driver with its match table, both pointing down at the platform bus, which compares compatible against every registered driver. A red arrow marked match leads to probe, with a note that everything probe takes with devm_ is released when the bind ends.

10.1.2 The tree, in sysfs

/sys is the device model made visible, which is why §2.2.4 could describe it as generated rather than written.

Path	Holds
/sys/devices/	The real tree, by physical topology
/sys/bus/*/devices/	Symlinks, grouped by bus
/sys/bus/*/drivers/	One directory per registered driver
/sys/class/*/	Symlinks, grouped by what a device is

The distinction between the last two is worth having. /sys/bus groups by *how it is attached*; /sys/class groups by *what it does*. A serial port reached over USB appears under /sys/bus/usb and under /sys/class/tty, and a program looking for serial ports wants the second. Chapter 11 is about getting into /sys/class at all.

Every device and driver directory there is a **kobject**, and every kobject is reference counted. That is what makes it safe for a process to hold a device open while somebody unbinds its driver: the memory is not freed until the last reference goes.

10.1.3 probe and remove

```
static int qa_probe(struct platform_device *pdev);
static void qa_remove(struct platform_device *pdev);
```

probe is called once per matching device, and **it can be called more than once**: two of the same chip on a board means two calls. So everything it allocates must hang off the device it was given, not off a static in your module. A driver with a global regs pointer works perfectly until the second device appears.

That is what platform_set_drvdata and dev_get_drvdata are for: one private structure per device, retrieved by any callback that is handed the struct device.

remove must undo what probe did, and returns void in 6.12: **it is not allowed to fail**. By the time it is called the decision has been made, and there is nothing sensible for the driver core to do with an error.

What may be done in probe

probe runs in process context and may sleep, which means GFP_KERNEL, mutexes and I/O are all available. Two things to know:

- **Do not assume a fixed order between devices.** If your device needs a clock provided by another driver that has not probed yet, return -EPROBE_DEFER and the core will call you again later. Returning a real error instead makes your driver depend on link order, which is not a thing you control.
- **Publish last.** Anything that makes the device reachable from userspace, a character device or a sysfs attribute, should be the last thing probe does, for the same reason §5.1.3 gave about cdev_add.

10.1.4 devm_ and what it is really attached to

§6.1.7 introduced these and could not use them, because a bare module has no struct device. probe is handed one, so this chapter can.

```
priv      = devm_kzalloc(&pdev->dev, sizeof(*priv), GFP_KERNEL);
priv->regs = devm_platform_ioremap_resource(pdev, 0);
ret       = devm_request_irq(&pdev->dev, irq, qa_isr, 0, dev_name(&pdev->dev), ↵
    ↵ priv);
```

Three acquisitions, no error labels, and no remove code for any of them. Compare §6.2.7, which is the same work with a goto ladder underneath it.

The release happens when the device is unbound from the driver. Not at module unload, and not at process exit. That distinction is testable, and the lab tests it:

```
# echo c000000.qa-dev > /sys/bus/platform/drivers/qa_platform/unbind
qa_platform c000000.qa-dev: removed after 1296 interrupts, 1297 samples

# grep -c c000000.qa-dev /proc/iomem
0
# lsmod | grep qa_platform
qa_platform 12288 0 - Live
```

The module is **still loaded**. The region and the interrupt are gone, released by the driver core walking the device's list of managed resources in reverse, and so are the sysfs attributes, which the core removes itself because they came from the driver's `.dev_groups`. Bind it again and probe runs again and they come back.

That is the whole mechanism: `devm_` builds a list of destructors attached to one device's binding. Knowing that is what stops the two surprises, which are things released earlier than expected (because something unbound) and things not released at all (because they were attached to the wrong device, or acquired without `devm_` at all).

10.1.5 Matching, and how a module gets loaded

```
static const struct of_device_id qa_of_match[] = {
    { .compatible = "qacademy,qa-dev-1.0" },
    { }
};
MODULE_DEVICE_TABLE(of, qa_of_match);
```

The table is the driver's statement of what it handles, terminated by an empty entry. The `MODULE_DEVICE_TABLE` line does something different and is easy to omit: it copies those strings into the module's binary metadata, so that they can be found **without loading the module**.

Four steps turn a device appearing into a module loading, and none of them is magic:

1. `MODULE_DEVICE_TABLE` puts the compatible strings in the `.ko`'s `.modinfo` section.
2. `depmod` reads every module's metadata and writes `/lib/modules/<version>/modules.alias`.
3. The device's uevent carries a `MODALIAS=` string built from its own compatible.
4. `udev` matches one against the other and runs `modprobe`.

You can see step 3 on the target:

```
# cat /sys/bus/platform/devices/c000000.qa-dev/uevent
DRIVER=qa_platform
OF_NAME=qa-dev
```

```
OF_FULLNAME=/platform-bus@c0000000/qa-dev@0
OF_COMPATIBLE_0=qacademy,qa-dev-1.0
OF_COMPATIBLE_N=1
MODALIAS=of:Nqa-devT(null)Cqacademy,qa-dev-1.0
```

This target has no udev, so nothing performs step 4 and modules are loaded by hand. That does not change steps 1 to 3, and it is worth knowing which part is missing rather than concluding that autoloading is mysterious.

10.1.6 Whose name is it

A detail that makes the inversion concrete. In Chapter 6 the driver chose the name it registered its region under:

```
0c000000-0c000fff : qa_mmio
```

As a platform driver it does not choose, and the entry reads:

```
0c000000-0c000fff : c000000.qa-dev qa-dev@0
17:          243          0 GIC-0 144 Level    c000000.qa-dev
```

The device is named from its **translated address** and its node name, by the driver core, from the device tree. Two consequences: the same driver bound to two instances produces two distinguishable entries, which a hand-chosen string could not; and `dev_name(&pdev->dev)` is the right thing to pass wherever a name is wanted, including to `devm_request_irq`.

The same applies to messages. `dev_info(&pdev->dev, ...)` prints `qa_platform c000000.qa-dev: probed, irq 17` and identifies which device spoke; Chapter 4's `pr_info` cannot, and that stops being a stylistic preference the moment there are two of anything.

10.1.7 Binding by hand

Every driver directory has `bind` and `unbind`:

```
# echo c000000.qa-dev > /sys/bus/platform/drivers/qa_platform/unbind
# echo c000000.qa-dev > /sys/bus/platform/drivers/qa_platform/bind
```

This is genuinely useful and not just a demonstration. It is how you hand a device to `vfioplatform` for passthrough, how you reset a wedged driver without rebooting, and how you test that your remove path actually works, which is otherwise the least-exercised code in a driver.

`driver_override` in the device's directory forces a specific driver regardless of matching, which is the other half of the passthrough workflow.

Unbind while userspace holds the device open is the case that catches people. The object reference count keeps the memory alive, but your `remove` has already run, so any pointer the file operations still hold is pointing at released hardware state. Handling that properly is what a driver's reference counting is for, and it is why frameworks (Chapter 11) are worth using: they have already solved it.

10.2 The Device Tree

Where the description of the hardware comes from, how to read one, and how a driver gets its addresses out of it without containing any.

§6.2.1 did the translation by hand once. This is the rest of the format, and the machinery that does it for you.

10.2.1 Why it exists

An x86 PC can discover itself. PCI is enumerable, ACPI describes the rest, and one kernel image boots on any machine.

An SoC can do none of that. There is no bus to walk that reveals a UART at `0x09000000` raising interrupt 33. Somebody has to say so, and before 2011 that somebody was C code in `arch/arm/`, one file per board, thousands of them, all of which had to be compiled into the kernel and maintained by the kernel community. The device tree moved that description out of the kernel and into a data file.

The consequence is the one that matters commercially: **one kernel image, many boards**. The bootloader passes a different DTB and the same binary boots different hardware.

10.2.2 The syntax

```
/ {
    #address-cells = <2>;
    #size-cells = <2>;

    platform-bus@c0000000 {
        compatible = "qemu,platform", "simple-bus";
        #address-cells = <1>;
        #size-cells = <1>;
        ranges = <0x00 0x00 0xc0000000 0x20000000>;
        interrupt-parent = <0x8003>;

        qa-dev@0 {
            compatible = "qacademy,qa-dev-1.0";
            reg = <0x00 0x1000>;
            interrupts = <0x00 0x70 0x04>;
```

```
};  
};  
};
```

Element	Is
name@address	A node. The part after @ matches the first address in reg
property = value	A property. Values are byte strings, printed as cells or text
<...>	Cells: 32-bit big-endian words
"..."	A string, or a list of them separated by nulls
<&label>	A phandle: a pointer to another node

Node names are conventional and carry no meaning to the kernel; **compatible is what matters**.

10.2.3 compatible

The match key, and a list rather than a string:

```
compatible = "qemu,platform", "simple-bus";
```

Most specific first. A driver matches if its of_match_table contains any of the strings, and where its table contains several, the entry for the earliest string wins. So a node can say “I am exactly this chip, and failing that I behave like this generic thing”, and a kernel with only the generic driver still works. The order does not rank one driver above another: if a driver for each string is loaded, whichever the bus tries first binds.

The convention is vendor,model. The vendor prefix is registered in Documentation/devicetree/bindings/vendor-prefixes.yaml, and making one up is how a binding gets rejected upstream.

A driver matches on the string, not on the node name. Change qa-dev@0 to widget@0 and everything still works; change one character of qacademy, qa-dev-1.0 and probe is never called. That is the most common reason a driver silently does not load, and the diagnosis is to compare the two strings by eye:

```
# tr -d '\0' < /sys/firmware/devicetree/base/platform-bus@c0000000/qa-dev@0/↵  
↵ compatible  
qacademy,qa-dev-1.0
```

10.2.4 reg, cells, and ranges

reg is a list of address-and-size pairs, but **how many cells each takes is declared by the parent**, not by the node itself:


```
parent: #address-cells = <1>; #size-cells = <1>;
child:  reg = <0x00 0x1000>;          -> one region: address 0x0, size 0x1000
```

With `#address-cells = <2>` the same property would be one address of two cells and no size, or an error. **You cannot read a reg without looking at its parent**, and that is the single most common mistake in reading a device tree by hand.

`ranges` translates a child address into the parent's address space, as a list of `<child-address parent-address size>`:

```
ranges = <0x00 0x00 0xc0000000 0x20000000>;
```

Here the child address takes 1 cell, the parent address 2 (the root declares `#address-cells = <2>`), and the size 1. So: child `0x0` maps to parent `0x0_0c000000`, for `0x20000000` bytes. The device at child offset `0x0` is therefore at physical **0xC0000000**.

An empty `ranges`; means the child addresses *are* the parent's, with no translation. No `ranges` property at all means the node is not memory-mapped through its parent, and children cannot be translated.

The cells are big-endian, always, regardless of the machine. Reading them with `od -tx4` on a little-endian host byte-swaps every one, so `0x0C000000` reads as `0x0000000C`, which is plausible enough to act on. Use `-tx1` and reassemble, or use a tool that knows.

10.2.5 Interrupts

```
interrupts = <0x00 0x70 0x04>;
interrupt-parent = <0x8003>;
```

`interrupts` is interpreted **by the interrupt controller**, not by the kernel generally, and how many cells it takes and what they mean is that controller's business. For a GIC it is three:

Cell	Meaning	Here
0	Type: 0 = SPI, 1 = PPI	0, a shared interrupt
1	Number within that type	0x70 = 112
2	Flags: 1 = rising edge, 4 = level high	4, level triggered

The GIC's own numbering adds the 32-entry SPI base, so this is hardware interrupt **144**, and Linux allocates its own number on top of that. All three appear in one line of `/proc/interrupts`, which §8.1.1 takes apart.

`interrupt-parent` is a phandle naming which controller, inherited by children if not overridden.

10.2.6 Getting it into your driver

The point of the whole format:

```
priv->regs = devm_platform_ioremap_resource(pdev, 0);
priv->irq = platform_get_irq(pdev, 0);
```

`devm_platform_ioremap_resource` does, in one call, what §6.2.7 spent a page on: takes reg entry 0, which the kernel already translated through every ranges between the node and the root when it created the device, claims the region, maps it, and registers a destructor for all of it.

`platform_get_irq` parses interrupts, finds the controller through `interrupt-parent`, creates the mapping and returns the Linux number. **It returns a negative error, not zero, on failure**, so it is checked with `< 0`. Treating 0 as the failure value is a bug that never shows on a board where the call succeeds, and on the first board where it fails, passes the negative error on to `devm_request_irq` as if it were a number, deferral included.

Other properties are read with the `of_property_*` family:

```
u32 depth;
if (of_property_read_u32(pdev->dev.of_node, "fifo-depth", &depth))
    depth = 16; /* absent: use the default */
```

Note the idiom: a missing optional property is not an error, it is a default. A missing *required* property is an error and should say which property, by name, in `dev_err`.

10.2.7 Bindings

A **binding** is the documentation of what properties a compatible string implies. They live in `Documentation/devicetree/bindings/`, and since about 2018 they are YAML schemas rather than prose, which means they can be checked mechanically:

```
make dt_binding_check      # are the schemas themselves valid
make dtbs_check            # do the shipped device trees match them
```

This matters more than it sounds. **A device tree is an ABI**. A DTB written for one kernel is expected to work on later ones, because the bootloader that supplies it may not be updated with the kernel. So a binding, once accepted, is a promise, and getting one reviewed upstream is deliberately harder than getting a driver reviewed.

Two consequences for a driver author:

- **Do not invent properties for things that are not hardware description.** A debug flag or a buffer size that is a policy choice is a module parameter, not a device tree property.
- **Read the binding before the driver.** For an unfamiliar device the binding tells you what the hardware is; the driver tells you only what one author did about it.

10.2.8 Where the DTB comes from, and overlays

At boot, the bootloader loads the DTB into memory and passes its address to the kernel in a register. §1.1.3 drew that handover; this is the file it was handing over.

On this course's target there is no bootloader: **QEMU generates the DTB itself**, including the qa-dev node, and hands it to the kernel exactly as U-Boot would. The node's address and interrupt are whatever QEMU assigned when it placed the device, which is why nobody typed them anywhere. You can dump what it produced:

```
qemu-system-aarch64 -machine virt,dumpdtb=virt.dtb ... -device qa-dev
dtc -I dtb -O dts virt.dtb
```

Overlays are fragments applied to a base DTB at run time, which is how an expansion board or a cape describes itself without a new base tree. They are genuinely useful for that and are commonly reached for as a way to hand-patch a tree during development, which works and is worth knowing is a development technique rather than a shipping one.

10.2.9 Reading one on a running system

The whole tree is under `/sys/firmware/devicetree/base`, one directory per node and one file per property:

```
/sys/firmware/devicetree/base/platform-bus@00000000/qa-dev@0/
compatible      "qacademy,qa-dev-1.0"
reg             00 00 00 00 00 00 10 00
interrupts      00 00 00 00 00 00 00 70 00 00 00 04
```

Strings come out with trailing nulls, so `tr -d '\0'` is usually wanted. Numeric properties are raw big-endian bytes and need reassembling by hand, per §10.2.4.

The other view is `/sys/bus/platform/devices/`, where each node with a `compatible` has become a device named from its translated address. Those two views are the same information from the two sides of §10.2.1: what the firmware said, and what the kernel made of it.

10.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Say what a bus is, in the device model's sense, without naming a physical bus.
- Follow a device from its device tree node to the probe call that handled it.
- Write a device tree node for a memory-mapped, interrupt-driven device.
- Say how many cells a reg entry has, and where that number is declared.

- Translate a child address through a `ranges` property.
- Write a `platform_driver` with an `of_match_table` and no addresses in its source.
- Explain the four steps between plugging in a device and its module being loaded.
- Say what `remove` must undo, and why `devm_` changes the answer.

Questions to test yourself

1. Your driver's probe is never called. List, in order, the five things you would check.
2. `compatible` holds three strings. Which one does the kernel match on, and what are the other two for?
3. A node has `reg = <0x1 0x2 0x3>`. Is that one region or an error? What do you need to know to answer?
4. Why is `interrupts = <0 112 4>` meaningless without knowing the interrupt parent?
5. What is the difference between probe failing and probe deferring, and when would you defer?
6. You unbind a driver from a device by writing to `sysfs` while a process holds the device node open. What stops the memory being freed underneath it?

Where to look

- `tools/qa-dev/integrate.py` contains the code that generates `qa-dev`'s device tree node inside QEMU, in the function `add_qa_dev_fdt_node`. It is short, and reading the thing that *writes* a node is an unusual and useful angle on reading nodes.

Looking ahead

Chapter 11 is about:

- Why your driver should probably not be a character device at all.
- What registering with a subsystem gives you, for free, that you have been writing by hand.
- The `ioctl` you would invent, and the ABI you would then maintain forever.
- Doing without a `/dev` node of your own, and getting something better in exchange.

10.4 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 10.6 onwards need the target running.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

Solutions. The exercises that are not Code are answered in §A.10. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 10.1 Bus, device, driver

RECALL

- Define each of the three, in one sentence, without naming a physical bus.
- The platform bus corresponds to nothing on any schematic. What is it for, and what property do its devices share?
- On the target, `/sys/bus/platform/devices/c000000.qa-dev` exists before any driver is loaded. Who created it, from what, and when?
- Does a device have to exist before its driver is registered? Describe both orders and say what the driver author has to do differently for each.
- A device can appear under both `/sys/bus` and `/sys/class`. What is the difference in what they group by, and which one would a program looking for “all the serial ports” use?

Exercise 10.2 Probe and remove

RECALL

- `probe` can be called more than once for one loaded module. When, and what does that rule out about where a driver keeps its state?
- A driver keeps static `void __iomem *regs;` at file scope and sets it in `probe`. It works. Describe the machine on which it stops working.
- `remove` returns `void` in this kernel. Why is it not allowed to fail?
- What is `-EPROBE_DEFER` for, what does the core do with it, and what is the alternative that makes your driver depend on link order?
- In `probe`, what should be done last, and why? Relate it to §5.1.3’s rule about `cdev_add`.

Exercise 10.3 Reading a device tree

HAND CALCULATION

```
/ {
    #address-cells = <2>;
    #size-cells = <2>;

    bus@10000000 {
        #address-cells = <1>;
        #size-cells = <1>;
        ranges = <0x1000 0x0 0x10000000 0x100000>;
    }
}
```

```

        thing@2000 {
            compatible = "acme,thing-v2", "acme,thing";
            reg = <0x2000 0x40>;
            interrupts = <0 45 1>;
        };
    };
};

```

- How many cells does thing's reg occupy, and how do you know?
- How many cells does the bus's ranges occupy? Account for each.
- What physical address do thing's registers start at? Show the arithmetic, and note that this ranges does not start its child range at zero.
- compatible has two entries. Which does the kernel prefer? What is the second for?
- Decode interrupts = <0 45 1> for a GIC. What hardware interrupt number is that, and what trigger type?
- A driver's of_match_table contains only "acme,thing". Does it bind? What if it contained only "acme,thing-v3"?

Exercise 10.4 What the helpers do

HAND CALCULATION

For each, say what it reads out of the device tree and what it returns on failure.

- devm_platform_ioremap_resource(pdev, 0)
- platform_get_irq(pdev, 0)
- of_property_read_u32(node, "fifo-depth", &v)
Then:
 - One of the three returns a negative error code as an int, one returns an error packed into a pointer, and one returns zero for success. Say which is which and how each is checked.
 - A driver checks if (!irq) after platform_get_irq. On which boards does that work, and on which does it silently do the wrong thing?
 - devm_platform_ioremap_resource replaces four separate calls from §6.2.7. Name all four.

Exercise 10.5 What belongs in a device tree

DESIGN

For each, say whether it belongs in the device tree, in a module parameter, or neither, and why.

- a) The physical address of the device's registers.
- b) Which interrupt it raises.
- c) How deep the driver's software FIFO should be.
- d) The number of hardware channels the chip has.
- e) Whether to enable verbose debugging.
- f) The crystal frequency feeding the device.
- g) The name of the /dev node to create.

Then: **h)** state the rule in one sentence. **i)** A device tree is an ABI. What follows for anything you put in one, and why is that a stronger constraint than it is for a module parameter?

Exercise 10.6 A driver that is told nothing

CODE

Rewrite your Chapter 8 driver as `lectures/L10/lab/qa_platform.c`, a platform driver.

- An `of_device_id` table matching `QA_DEV_DT_COMPATIBLE`, and `MODULE_DEVICE_TABLE`.
- probe uses `devm_kzalloc` for a per-device private structure, and `platform_set_drvdata` to attach it.
- Registers from `devm_platform_ioremap_resource`, interrupt from `platform_get_irq`, handler from `devm_request_irq` named with `dev_name(&pdev->dev)`.
- Check the identity register and fail with `-ENODEV` if it is wrong.
- Two read-only sysfs attributes, `interrupts` and `samples`, exposed through the driver's `.dev_groups` so that the core adds and removes them for you.
- `remove` stops the device. It must do nothing else.
- Use `dev_info` and `dev_err`, not `pr_info`.

There must be no address and no interrupt number anywhere in the file, and no `goto` ladder.

Check yourself: make test L=L10 reports **PASSED**, with nineteen checks. `grep -c '0xC000'` on your source returns 0.

- a) Compare the line count against your Chapter 8 driver. Where did the difference go?
- b) Your Chapter 6 driver named its own region and the entry in `/proc/iomem` read `qa_mmio`. What does it read now, and who chose that name?

c) Change one character of the compatible string in your `of_match_table` and reload. What happens, what does `dmesg` say, and how would you have diagnosed it without knowing what you had changed?

Exercise 10.7 Bind, unbind, and what `devm_` is attached to

CODE

With your driver loaded and bound:

a) Unbind it by hand:

```
echo c000000.qa-dev > /sys/bus/platform/drivers/qa_platform/unbind
```

Check `lsmod`, `/proc/iomem`, `/proc/interrupts` and the device's `sysfs` attributes. Which of them changed, and is the module still loaded?

b) From that result, state precisely what event releases `devm_` resources. Is it module unload? Give the evidence.

c) Bind it again. Did probe run? How do you know from the `sysfs` attributes alone?

d) Now unload the module without unbinding first. What happens, and in what order?

e) Why is `unbind` useful in real work? Give two uses that are not demonstrations.

f) A process has your device open when you unbind. What keeps the memory alive, and what is now dangerous about any pointer your file operations still hold?

Exercise 10.8 Autoloading

DESIGN

This target has no `udev`, so modules are loaded by hand. The rest of the mechanism is present.

a) Name the four steps between a device appearing and its module being loaded, and say which one is missing here.

b) Which of the four does `MODULE_DEVICE_TABLE` participate in? What breaks if you omit it, and does the driver still work when loaded by hand?

c) Find the `MODALIAS` or `OF_COMPATIBLE_0` line in your device's `uevent`. Where did that string come from?

d) Run `modinfo` on your `.ko` and find the alias it advertises. Which file would `depmod` have collected it into?

e) A USB device you plug in is autoloaded by the same four steps. Where does the string its `uevent` carries come from, and why does a device soldered to the board need a device tree to supply it?

Exercise 10.9 An address derived three ways

CROSS-CHECK

You have now computed this address by hand once, in Chapter 6. This time you compute it, the harness computes it, and the kernel computes it, and all three must agree.

a) **By hand, from the raw bytes.** On the target:

```
dt=/sys/firmware/devicetree/base
node=$(find $dt -name 'qa-dev*' -type d | head -1)
od -An -tx1 -v "$node/reg"
od -An -tx1 -v "${dirname "$node"}/ranges"
od -An -tx1 -v "$node/interrupts"
```

Write out every cell, four bytes each, most significant first. Then compute the physical address, the size, and the GIC hardware interrupt number.

b) **From the kernel's own naming.** Look at `ls /sys/bus/platform/devices/ | grep qa-dev`. The device's name contains a number. Which of your computed values is it, and what does that tell you about when the translation happened?

c) **From the driver.** Have your probe print what it was handed. `platform_get_resource(pdev, IORESOURCE_MEM, 0)` gives a struct `resource` with start and end; print both, and print the value `platform_get_irq` returned.

d) **Reconcile.** All three should agree on the address. Answer:

- Do they? If your hand figure differs, which ranges cell did you use as the parent base?
- The driver's IRQ number and the GIC number from a) are *different numbers*, and both are right. Explain, and say which one `/proc/interrupts` shows in which column.
- Repeat a) with `od -An -tx4` instead of `-tx1`. Every non-zero cell changes. Which is correct, what is `-tx4` doing, and why is the wrong answer plausible rather than obviously broken?

e) **The point of the whole chapter.** `grep` your driver's source for the address and for the interrupt number. Neither is there.

- Where is the address, then? Name the file, the node and the property.
- Give QEMU a **second** -device `qa-dev` on the command line and boot again. Two devices appear, and your driver probes both:

```
qa_platform c000000.qa-dev: probed, irq 17
qa_platform c001000.qa-dev: probed, irq 18
0c000000-0c000fff : c000000.qa-dev qa-dev@0
0c001000-0c001fff : c001000.qa-dev qa-dev@1000
17: 426 0 GIC-0 144 Level c000000.qa-dev
18: 420 0 GIC-0 145 Level c001000.qa-dev
```

Read the two devices' `interrupts` attributes. They differ. What would have happened if your private structure had been a file-scope `static` rather than `devm_kzalloc`ed per device? Would your Chapter 6 driver, with its `base=` parameter, have coped with this at all?

- Notice the second device's address. You have seen `0xC001000` before, in Exercise 6.9, where reading it took the machine down. What is different now, and what does that say about how much a driver can infer from an address alone?
- State what would have to change in your driver to move this device to a different SoC.

f) A colleague proposes putting the address back in the driver as a fallback, "in case the device tree is wrong". Give the argument against, in terms of what a device tree is and who owns it.

Kernel Frameworks

Why your driver probably should not be a character device.

IN THIS CHAPTER

- What a subsystem gives you: a stable userspace ABI, existing tools, and power management.
- The cost of the shortcut: an `ioctl` you invented is an ABI you maintain forever.
- GPIO and `pinctrl`, and what a `gpiochip` has to provide.
- `clk` and regulator, and why a driver should not be turning power on by writing a register.
- I2C and SPI, and `regmap` as the thing that makes a register map declarative.
- IIO for anything that samples; input, MTD and watchdog in a sentence each.
- How to tell which subsystem a device belongs to, and what to do when the answer is none.
- Register with two frameworks, and drop the character device Chapters 5 and 9 built.

This chapter is about why a driver should usually not be a character device at all. It sets out what a subsystem gives a driver in exchange for implementing its interface: a documented ABI, tools that already exist, and a place where userspace knows to look. It sets that against what an `ioctl` of your own costs, tours the subsystems worth recognising, and registers `qa-dev` with two of them, IIO and GPIO. It ends with how to choose a subsystem, what to do when none fits, and what a framework does not do for you, including what it costs to read a value through one.

§11.1 is the whole of the material, and the argument for it; read it first. The lab is specified in Exercise 11.6: the Chapter 10 driver rewritten as `qa_frameworks.c`, registering one IIO channel and an eight-line `gpio_chip`, and presenting no character device of its own; Exercise 11.7 then drives it with programs written without any knowledge of it. The exercises in §11.3 end with the Cross-check: predict what `in_voltage0_raw` reads in `qa-dev`'s counter mode, read it, and account for a sample rate that falls short of the programmed one by an amount you will have seen before. The lab is done when `make test L=L11` reports **PASSED**, with twenty-two checks, eight of them from a GPIO tester that contains no reference to your driver.

11.1 Kernel Frameworks

Why your driver probably should not be a character device, what a subsystem gives you in exchange for implementing its interface, and how to tell which one a device belongs to.

The argument in one sentence: **a character device is an interface you invent and then maintain forever; a framework is an interface that already exists and that other people's tools already speak.**

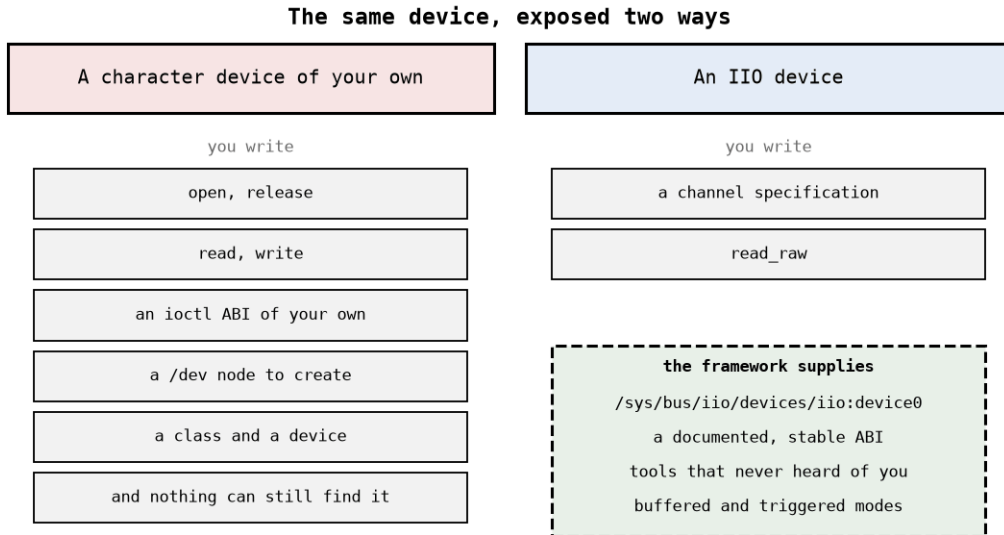
11.1.1 What you have, and what you do not

By Chapter 10 the driver works. It probes off the device tree and services its interrupt, and in Chapter 9 it presented `/dev/qa_wait` with a `read()` you wrote. Nothing is broken.

Now list what it does not have:

- No way for a program that does not know about it to find it.
- No documented interface. Yours exists in your head and in the source.
- No existing tools. Every user writes their own reader.
- No power management. Suspend and resume do nothing.
- No way to express what the device *is*, only what it does.

Each of those is something a subsystem provides, and provides identically for every driver in it. That is the trade: **you implement an interface someone else designed, and you stop having to design, document, version and support one of your own.**



Reading in_voltage0_raw costs 571 us from a C program, so sysfs is for inspection.
Keeping up with 881 samples a second is what the buffered modes are for.

Figure 11.1 Two columns for the same device. A character device of your own needs open and release, read and write, an ioctl ABI of your own, a /dev node, a class and a device, and still nothing can find it. An IIO device needs a channel specification and read_raw, and the framework supplies the sysfs path, a documented ABI, existing tools and buffered modes.

11.1.2 The cost of the shortcut

Adding an ioctl is always the path of least resistance. It is worth knowing what it costs before taking it.

It is an ABI the moment a second program uses it. The numbers and the structure layout are frozen; you may add but never change or remove. There is no deprecation period for something a customer’s binary calls.

It is a portability trap. A 32-bit process on a 64-bit kernel passes a differently laid-out structure, and handling that needs `compat_ioctl` and a structure with no implicit padding and no long. Getting this wrong is a standing source of kernel bugs, and it is invisible until somebody runs a 32-bit binary.

It is undiscoverable. `ls /sys/bus/iio/devices` finds every IIO device on a machine. Nothing finds every device that implements your ioctl.

And there is usually already a subsystem for it. The kernel has spent thirty years accumulating interfaces for classes of hardware, and “a thing that produces samples” or “a thing with controllable pins” are both very well-trodden.

11.1.3 The tour

Ten subsystems worth recognising, with the question each answers.

Subsystem	For	Userspace sees
gpio	Individually controllable pins	/dev/gpiochipN, libgpiod
pinctrl	What a pin is multiplexed <i>as</i>	Nothing directly; other drivers
clk	Clock sources and gates	Nothing directly; /sys/kernel/debug/clk
regulator	Power supplies	Read-only /sys/class/regulator
i2c, spi	Devices on those buses	/dev/i2c-N, /dev/spidevN.M
regmap	Register access over any of those	Nothing; it is a driver-side helper
iio	Anything that samples or converts	/sys/bus/iio/devices/, /dev/iio:deviceN
input	Keys, buttons, touch, motion	/dev/input/eventN
mtdev	Raw flash	/dev/mtdevN, /dev/mtdevblockN
watchdog	A timer that reboots the machine	/dev/watchdog

Four of them are worth more than a row.

pinctrl is the one people are surprised to need. Before a pin can be a GPIO it has to be *multiplexed* as one rather than as a UART line or an I2C signal, and that is a different piece of hardware in the SoC. Your driver should not be doing it: the pin configuration belongs in the device tree, and pinctrl applies it before your probe runs.

clk and regulator exist because the answer to “turn the device on” is shared and ref-counted. Two drivers on one clock is the normal case, and a driver that writes the gate register directly works until the second one disagrees with it. `devm_clk_get` and `clk_prepare_enable` cost two lines and are correct with any number of consumers.

regmap is a driver-side helper rather than a userspace interface. Most drivers for an I2C or SPI chip contain the same fifty lines of read-modify-write, byte-order and caching; regmap replaces them with a *description*, which registers exist, which are volatile, which are cacheable, and handles the rest. It is worth reaching for the moment a driver has more than a handful of registers.

iio is where anything that measures belongs. Not just ADCs: accelerometers, light sensors, pressure, temperature, DACs, anything with a value and a unit. It carries scale, offset and units so that userspace can convert raw counts into volts without knowing anything about the chip.

Two of the ten you will drive yourself and eight you will only have read about, which is worth knowing before the table starts to feel like competence: §11.1.4 and §11.1.5 register `qa-dev` with iio and gpio and the lab checks both, but `qa-dev` sits on no bus and has no clock or regulator to ask for, so nothing here can teach you what regmap feels like against a real I2C part.

11.1.4 Registering with IIO

The whole of it, for a device with one channel:

```

indio_dev = devm_iio_device_alloc(&pdev->dev, sizeof(*priv));
priv      = iio_priv(indio_dev);

indio_dev->name      = "qa_dev";
indio_dev->info      = &qa_iio_info;      /* .read_raw = qa_read_raw */
indio_dev->modes      = INDIO_DIRECT_MODE;
indio_dev->channels   = qa_channels;
indio_dev->num_channels = ARRAY_SIZE(qa_channels);

ret = devm_iio_device_register(&pdev->dev, indio_dev);

```

`devm_iio_device_alloc` allocates the IIO device *and* your private structure in one object; `iio_priv()` gets yours back out. That is the framework's idiom for per-device state and it replaces the `devm_kzalloc` from Chapter 10.

The channel says what the value *is*:

```

static const struct iio_chan_spec qa_channels[] = {
    {
        .type          = IIO_VOLTAGE,
        .indexed       = 1,
        .channel        = 0,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),
    },
};

```

and `.info_mask_separate` is the list of attributes to create. `IIO_CHAN_INFO_RAW` makes `in_voltage0_raw`; adding `IIO_CHAN_INFO_SCALE` would make `in_voltage0_scale`, and userspace multiplies the two to get millivolts, the unit the IIO ABI documents for a voltage.

`read_raw` is the one callback:

```

static int qa_read_raw(struct iio_dev *indio_dev, struct iio_chan_spec const *chan,
    int *val, int *val2, long mask)
{
    struct qa_priv *priv = iio_priv(indio_dev);

    switch (mask) {
    case IIO_CHAN_INFO_RAW:
        *val = atomic_read(&priv->last_sample);
        return IIO_VAL_INT;
    default:
        return -EINVAL;
    }
}

```

```
}
```

What that buys, on the target, immediately:

```
# ls /sys/bus/iio/devices/
iio:device0
# cat /sys/bus/iio/devices/iio:device0/name
qa_dev
# cat /sys/bus/iio/devices/iio:device0/in_voltage0_raw
195
```

No `/dev` node was written, no `read()` implemented, no `ioctl` invented. And because the device is in counter mode, reading the attribute twice measures the sample rate:

```
195 ... one second ... 1076
```

881 samples in a second, which is Chapter 8’s interrupt rate arriving through an interface that knows nothing about interrupts.

11.1.5 Registering a gpiochip

Five callbacks and a structure:

```
priv->gc.label      = dev_name(&pdev->dev);
priv->gc.parent      = &pdev->dev;
priv->gc.owner       = THIS_MODULE;
priv->gc.base        = -1;                /* let the core allocate the numbers ↵
↳ */
priv->gc.ngpio       = 8;
priv->gc.get         = qa_gpio_get;
priv->gc.set         = qa_gpio_set;
priv->gc.direction_input = qa_gpio_direction_input;
priv->gc.direction_output = qa_gpio_direction_output;
priv->gc.get_direction = qa_gpio_get_direction;

ret = devm_gpiochip_add_data(&pdev->dev, &priv->gc, priv);
```

`.base = -1` matters: a hardcoded global GPIO number is a legacy practice. This kernel still accepts one, with a warning that static allocation is deprecated, but two chips whose hardcoded ranges overlap cannot both register. `gpiochip_get_data(gc)` retrieves the pointer you passed.

Note that `.set` returns `void` in this kernel. It gained an `int` return later, so a driver copied from newer material will not compile here, and one copied from older material will not compile there. §3.1.2’s unstable internal API, again.

What appears:

```
# ls -l /sys/bus/gpio/devices/gpiochip1/of_node
of_node -> ../firmware/devicetree/base/platform-bus@c0000000/qa-dev@0
# ls -l /dev/gpiochip1
crw----- 1 root root 254, 1 /dev/gpiochip1
```

The chip is a child of your platform device, linked back to the device tree node it came from, with a character device the framework created and will remove when the driver unbinds.

There is no `/sys/class/gpio`. That interface is deprecated and this course's kernel does not enable it; the character device replaced it, and a driver written against the sysfs interface is a driver written against something on its way out. `kernel/qa.config` says so in a comment.

11.1.6 What userspace can now do

The tester shipped with the lab contains **nothing specific to this driver**. It finds the chip by the label the driver gave it and then speaks the standard ABI:

```
ioctl(fd, GPIO_GET_CHIPINFO_IOCTL, &info);      /* what is this chip */
ioctl(chip, GPIO_V2_GET_LINE_IOCTL, &req);      /* give me line 0 as an output ↵
↵ */
ioctl(line, GPIO_V2_LINE_SET_VALUES_IOCTL, &v); /* drive it */
```

```
found /dev/gpiochip1: label "c0000000.qa-dev", 8 lines
ok line 0 can be driven high
ok line 4 reads high, through the loopback
```

That is the argument, demonstrated: **a program written before your driver existed can drive your hardware**, because both sides implement an interface neither of them owns. `libgpiod` and its `gpioinfo`, `gpioget` and `gpioset` tools do exactly this and would work here unchanged.

11.1.7 Choosing a subsystem

The question is **what the device is**, not what it does.

The device is	Subsystem
Something that measures or converts a value	iio
Something with individually controlled pins	gpio
A source of human input	input
Raw flash	mtd
A timer that resets the board	watchdog
A clock source	clk
A power supply	regulator
On an I2C or SPI bus	i2c or spi, plus one of the above

A device is often several: a sensor on I2C that reports temperature is an **i2c** client and an **iio** device, and those are not competing answers.

When nothing fits, a character device is a legitimate answer and not a failure. Write it with `misc_register` rather than the long way, document the interface, use `_IOR/_IOW` properly, and be honest that you now own an ABI. What is not legitimate is reaching for it because implementing an interface looked like more work than inventing one.

11.1.8 What a framework does not do

Worth stating, because the argument above is one-sided by design.

It does not make your driver correct. Everything from Chapters 6 to 9 still applies: the locking, the interrupt acknowledgement, the sleeping rules. A framework changes what your driver presents, not what it has to get right underneath.

It constrains you to its model. If your device genuinely does not fit IIO's idea of a channel, forcing it in produces a driver that is harder to read than a character device would have been. The model is a good fit surprisingly often and is not a universal one.

It can be slower, and here is how much. Going through a subsystem's generic path costs something against a purpose-built `read()`. Measured on this target, reading `in_voltage0_raw` repeatedly:

From	Per read	Rate
A C program	571 us	1,748 reads/s
A shell loop of <code>cat</code>	29,700 us	34 reads/s

The shell figure is fifty times worse because each `cat` is a fork and an `exec`, and it is worth knowing before you write a monitoring script: **a shell reading sysfs cannot keep up with a device sampling at more than about thirty hertz.**

Even the C figure is only twice this device's 881 samples a second, so reading the attribute in a loop is already marginal at one kilohertz and hopeless above it. That is not a criticism of `sysfs`; it is what `sysfs` is for, which is configuration and occasional inspection. **IIO's buffered and triggered modes exist for exactly this**, delivering batches through `/dev/iio:deviceN`

instead, and a driver that needs them says so by allocating a buffer rather than by abandoning the framework.

And it does not remove the ABI problem, it moves it to someone competent. The GPIO character device is on its second version, `GPIO_V2_*`, because the first got some things wrong. The difference is that the migration was designed, documented and supported by the subsystem maintainers rather than by you.

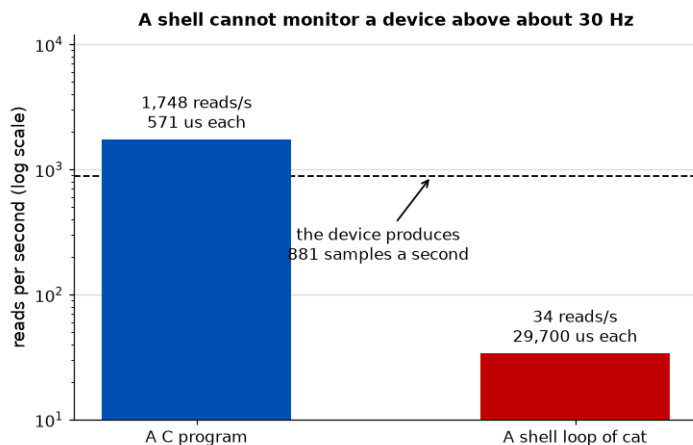


Figure 11.2 A logarithmic bar chart of reads per second: 1,748 from a C program at 571 microseconds each, against 34 from a shell loop of cat at 29,700 microseconds each, with a dashed line marking the 881 samples a second the device produces.

11.2 Review

What you should be able to do now

After this chapter, you should be able to:

- List four things a subsystem provides that a character device does not.
- Say why an `ioctl` you invent is a long-term cost, and name the specific portability trap.
- Implement a `gpio_chip` and drive it with `libgpiod` tools.
- Say what `pinctrl` is for, and why it is a separate subsystem from GPIO.
- Explain why a driver should acquire a clock rather than write a clock gate register.
- Say what `regmap` replaces, and when it is not worth it.
- Register an IIO device with one channel, and read it from `sysfs`.
- Given an unfamiliar device, name the subsystem it belongs to, or say honestly that none does.

Questions to test yourself

1. Your driver exposes a `/dev` node and two `ioctl`s. Name three things a `gpiochip` would have given you that you would otherwise write yourself.
2. Why is `/sys/class/gpio` deprecated, and what replaced it?
3. Two drivers share a clock. One disables it when it suspends. What goes wrong, and which subsystem exists to prevent it?
4. What does `regmap` know about your device that your read-modify-write code did not?
5. An IIO device with a buffer and a trigger delivers samples to userspace. Through what, given that you did not implement `read`?
6. You have a device that genuinely fits no subsystem. What do you write, and what do you owe the people who will use it?

Where to look

- `kernel/qa.config` enables `CONFIG_GPIO_CDEV` and not `CONFIG_GPIO_SYSFS`, with a comment saying why. That choice is the argument of §11.1.5 in one line.
- `gpioinfo`, `gpioget` and `gpiset` are not on the target, which carries BusyBox and the course's own programs and nothing else. The shipped tester, `lectures/L11/lab/user/qa_gpio_test.c`, speaks the same ABI they do, and is short enough to read in one sitting.

Looking ahead

Chapter 12 is about:

- What all of this costs in latency, and how you would know.
- Where latency actually comes from, which is three places and not one.
- What `PREEMPT_RT` changes about the driver you have just finished.
- A measurement that is honest about what it cannot see.

11.3 Exercises

Nine, ending with the Cross-check. Do them in order; Exercise 11.6 onwards need the target running.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

Solutions. The exercises that are not Code are answered in §A.11. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 11.1 What a framework gives you

RECALL

- a) Your Chapter 9 driver works and presents `/dev/qa_wait`. List five things it does not have that a subsystem would have provided.
- b) State the trade a framework offers, in one sentence.
- c) Name three separate costs of adding an `ioctl` of your own. One of them only appears on a particular kind of system; which, and what kind?
- d) `ls /sys/bus/iio/devices` finds every IIO device on a machine. What is the equivalent command for finding every device that implements your own `ioctl`?

Exercise 11.2 The tour

RECALL

For each device, name the subsystem or subsystems it belongs to.

- a) A temperature sensor on an I2C bus.
- b) A pin that drives an LED.
- c) A hardware timer that resets the board if not kicked.
- d) A rotary encoder a person turns.
- e) A NAND flash chip.
- f) A 16-bit ADC on SPI.
- g) A crystal oscillator feeding three peripherals.
Then:
- h) Two of your answers are “more than one subsystem”. Which, and why is that not a contradiction?
- i) Which of the ten subsystems in §11.1.3 presents **nothing** to userspace, and what is it for instead?

Exercise 11.3 Reading an IIO device

HAND CALCULATION

A device exposes `in_voltage0_raw` and `in_voltage0_scale`. Reading them gives:

```
in_voltage0_raw    = 2048
in_voltage0_scale  = 0.805664
```

- a) What is the voltage? What are the units of scale, and how do you know?
- b) Why does IIO split the value into a raw count and a scale, rather than having the driver report volts directly?

- c) The driver adds `II0_CHAN_INFO_OFFSET` and it reads `-100`. Recompute the voltage. In what order are raw, offset and scale applied?
- d) Your device reports a plain counter with no physical meaning at all. Which of raw, scale and offset should the driver implement, and what should it do about type?

Exercise 11.4 The pin that is not yet a GPIO

DESIGN

- a) Before a pin can be driven as a GPIO, something must have multiplexed it as one rather than as a UART line. Which subsystem, and where is the configuration written?
- b) Should your driver perform that multiplexing in probe? Give the reason.
- c) Your device needs a clock enabled before its registers respond. Write the two calls, and say what happens if you instead write the SoC's clock gate register directly and a second driver shares that clock.
- d) Your device has a regulator that must be on. Same question: what does the framework do that a direct register write does not?
- e) All three of these are dependencies that may not be ready when your probe runs. What should probe return, and what does the core do with it?

Exercise 11.5 When a character device is right

DESIGN

- a) Give a device that genuinely fits no subsystem, and say why.
- b) For it, would you register the long way as in Chapter 5, or use `misc_register`? Justify it.
- c) You add two `ioctl`s. Write down what you now owe the people who use them, and for how long.
- d) A colleague proposes an `ioctl` that returns a struct containing a long and a pointer. Name both problems.
- e) State the difference between “no subsystem fits” and “implementing a subsystem interface looked like more work”. Which is a reason and which is an excuse?

Exercise 11.6 Register with two frameworks

CODE

Rewrite your Chapter 10 driver as `lectures/L11/lab/qa_frameworks.c`.

IIO:

- Allocate with `devm_iio_device_alloc`, sized for your private structure, and get it back with `iio_priv`.
- One channel: `II0_VOLTAGE`, indexed, channel 0, with `II0_CHAN_INFO_RAW`.
- `read_raw` returns the most recent sample your interrupt handler stored, as `II0_VAL_INT`.

- Name the device `qa_dev`, mode `INDIO_DIRECT_MODE`, and register with `devm_iio_device_register`.

GPIO:

- A `gpio_chip` with eight lines, labelled with `dev_name(&pdev->dev)`, `.base = -1`, parented to your platform device.
- `get`, `set`, `direction_input`, `direction_output` and `get_direction`. Lines 0 to 3 are outputs; lines 4 to 7 are inputs wired back to them by the hardware.
- Register with `devm_gpiochip_add_data`.

And no character device: no `file_operations`, no `read`, no `poll` and no `/dev` node of your own, where your Chapter 9 driver had all four.

Check yourself: make `test L=L11` reports **PASSED**, with twenty-two checks, of which eight come from a GPIO test program that contains no reference to your driver.

- `devm_iio_device_alloc` allocates your private structure for you. Which call from your Chapter 10 driver does that replace, and why does the framework want to own the allocation?
- Your module fails to load with Unknown symbol `devm_iio_device_alloc`. What is wrong, and what is the one command that fixes it? Why is IIO a module here at all?
- `.base = -1`. What would happen on this target if you hardcoded 0 instead?

Exercise 11.7 Drive it with tools you did not write

CODE

- Read `/sys/bus/iio/devices/iio:device0/name` and `in_voltage0_raw`. Which of these did you write code to create?
- Find your `gpiochip` under `/sys/bus/gpio/devices/`. What is its parent, and what does its `of_node` symlink point at? Trace that path back to Chapter 10.
- Run the shipped `qa_gpio_test`. Read its source. Identify every line that would have to change if it were pointed at a completely different GPIO driver on different hardware.
- `ls /sys/class/gpio` fails. Explain why that is correct rather than a missing feature, and name the config option and the chapter that decided it.
- `libgpiod`'s `gpioinfo`, `gpioget` and `gpioset` are not on this target. Would they work if they were? What exactly would they be talking to?

Exercise 11.8 What you gave up

DESIGN

- a) Your Chapter 9 driver could block a reader until data arrived. Can a program using `in_voltage0_raw` do that? What has been lost?
- b) IIO has an answer to a), which this lab does not implement. Find it in the kernel documentation and describe in two sentences what it provides and what a driver must add.
- c) Name one thing about your driver that is now *harder* than it was with a character device.
- d) Name a device for which forcing it into IIO would produce worse code than a character device, and say what makes it a bad fit.
- e) The GPIO character device is on its second version, `GPIO_V2_*`. What does that tell you about the claim that a framework removes the ABI problem? Restate the claim accurately.

Exercise 11.9 A sample rate measured through an interface that knows nothing about it

CROSS-CHECK

The device is in counter mode, so sample n has the value n . That makes the value readable through IIO a running total, and the difference between two reads a measurement of something the IIO interface has no concept of.

- a) **Predict.** Your device is programmed with `PERIOD_NS=1000000`. Predict:
 - the value of `in_voltage0_raw` one second after the driver loads;
 - the difference between two reads taken one second apart;
 - what both would be at `period_ns=500000`.
- b) **Measure.** Read `in_voltage0_raw` twice, a second apart, and record both values and the difference. Do this three times.
- c) **Reconcile.** Answer each:
 - The difference is not 1000. What is it, roughly, and where have you seen that number before? Give the chapter and the effect.
 - Is the shortfall here the same as the one you measured in Chapter 8, or has the framework added something of its own? How would you tell?
 - You have measured a sample rate using an interface that exposes no rate, no timestamp and no counter. What made that possible, and what would you have had to do instead if the device were in `QA_DEV_MODE_NOISE`?
- d) **The reads are not free either.** Time a thousand reads of `in_voltage0_raw` from a shell loop and from a C program. Which is faster, by how much, and what does that tell you about the `sysfs` path as a way of moving samples?
- e) From d), state the sample rate above which reading `in_voltage0_raw` in a loop stops being a sensible way to get data out. What does IIO offer instead, and why does it exist?

f) A colleague reports “the IIO driver only delivers 880 samples a second when the device is configured for 1000”. Rewrite that as an accurate sentence, naming which component each part of the shortfall belongs to and which parts you have actually measured rather than inferred.

Real-Time Linux

What all of it costs in latency, and what a real-time kernel changes about the answer.

IN THIS CHAPTER

- What “real time” claims, and why it is a statement about the worst case and never the average.
- Three sources of latency, only one of which is the scheduler.
- Priority inversion, and the spacecraft it kept resetting.
- Four preemption models, and what each one is willing to interrupt.
- What PREEMPT_RT actually changes: sleeping spinlocks, threaded interrupts, inheritance.
- Which locks do not change, and why `raw_spinlock_t` exists.
- `cyclicttest`: what it measures and what it structurally cannot see.
- Measure your own driver’s latency both ways, interrupt to handler and handler to userspace, and plot it.

This chapter is about what everything the book has built costs in latency, and what a real-time kernel changes about the answer. It defines real time by the worst case rather than the average, names the three places latency comes from, and tells the Mars Pathfinder story as it happened. It sets out the four preemption models, what PREEMPT_RT, merged in the 6.12 kernel this course builds, does to spinlocks, interrupt handlers and mutexes, and which locks it leaves alone. It then turns to measurement: what `cyclicttest` measures and what it cannot, how a device that timestamps its own interrupt lets latency be subtracted rather than inferred, and why a number taken under an emulator is worthless on its own while the comparison between two kernels survives.

Read §12.1 first; it is the theory. §12.2 is the measurement, and the lab is a measurement too: Exercise 12.6 specifies a driver that timestamps every interrupt twice and delivers the record declared in the header printed in §12.2.3, and the course ships the program that reads it. The second kernel is `make kernel RT=1`, a full build of its own, and it needs an `initramfs` of its own, `make rootfs RT=1`. The exercises in §12.4 end with the Cross-check that closes the course: predict whether PREEMPT_RT lowers the mean and the maximum, measure both kernels idle and under load, and find that the two answers differ; then state at length what the measurement is blind to, which is the deliverable rather than the number. The lab is

done when `make test L=L12` reports **PASSED** with ten checks on the PREEMPT kernel, and `RT=1 make test L=L12` does the same on the PREEMPT_RT one.

12.1 Real Time

What the term claims, where latency comes from, and what PREEMPT_RT changes about the driver you have spent eleven chapters writing.

§12.2 is how to measure any of it without lying.

12.1.1 The definition, and the trap in it

A real-time system is one in which being late is being wrong. Not fast: correct within a stated deadline, every time.

The trap is that “real time” sounds like a performance claim and is not. Consider:

	Average	Worst observed
System A	50 us	30 ms
System B	5 ms	5.1 ms

System B is real time for a 6 ms deadline and system A is not, despite being a hundred times slower on average. A is faster; B is predictable. If your requirement is a 6 ms deadline, A fails it several times an hour and no amount of tuning its average will help.

Everything in this chapter is about the right-hand column. **An average is not evidence**, and a measurement without a stated worst case, a stated load and a stated duration is not a measurement of anything.

Two more distinctions worth having:

- **Hard real time:** a missed deadline is a system failure. An airbag, a motor commutation loop.
- **Soft real time:** a missed deadline degrades quality. Audio, video.

Linux with PREEMPT_RT is good soft real time and is used for hard real time by people who have measured their own system very carefully. It is not a hard real-time operating system in the sense that a small RTOS with a provable worst case is.

12.1.2 Where latency comes from

Three sources, and only three. Every real-time problem you will meet is one of them.

Interrupts disabled. Between `local_irq_disable()` and the matching enable, no interrupt is taken, including yours. `spin_lock_irqsave` does this for the length of its critical section, which is why §7.1.5 insisted that those be short.

Preemption disabled. The interrupt is taken but the scheduler cannot switch to your task afterwards. Holding any spinlock disables preemption, so a long critical section anywhere in the kernel delays every high-priority task on that CPU.

Priority inversion. Your high-priority task is runnable but waiting on a lock held by a low-priority task, which is itself not running because a medium-priority task is using the CPU. The high-priority task now waits behind the medium one, which is the inversion.

The first two are code somebody wrote. The third is structural and needs a mechanism, which is §12.1.3.

12.1.3 Priority inversion, and Mars

The canonical example is worth knowing accurately, because it is usually told badly.

Mars Pathfinder landed in 1997 and began resetting itself. A high-priority bus-management task needed a mutex held by a low-priority meteorological task; a medium-priority communications task was runnable and preempted the low-priority one; the low-priority task therefore never finished and never released the mutex; the high-priority task missed its deadline; a watchdog concluded the system had failed and reset it.

The fix was to enable **priority inheritance** on that mutex, and it was uploaded to Mars.

Priority inheritance is the mechanism: while a low-priority task holds a lock that a high-priority task is waiting for, it **temporarily inherits the higher priority**, so it cannot be preempted by anything in between. It finishes, releases the lock, and drops back.

The reason this story matters to a driver author is that the bug got through testing: it had caused one or two resets in months of pre-flight testing, and nobody could reproduce or explain them. It needed three tasks, one lock and a specific interleaving. It is the point §7.2 makes about ordering bugs, one level up: **a system that has never missed a deadline is not a system that cannot.**

12.1.4 The four preemption models

One question distinguishes them: **when a high-priority task becomes runnable, at what points can the kernel switch to it?**

Model	Switches at	For
PREEMPT_NONE	Only on return to userspace, or a voluntary sleep	Throughput. Servers
PREEMPT_VOLUNTARY	Those, plus explicit <code>might_sleep</code> points	Desktops, historically
PREEMPT	Those, plus almost anywhere in the kernel	Desktops and embedded. This course's default
PREEMPT_RT	Those, plus inside most spinlock sections	Real time

The progression is one of removing places where the kernel refuses to be interrupted. Each step costs throughput, because each adds preemption points that must be checked, and buys a lower worst case.

PREEMPT_RT was merged into mainline in 6.12, which is the kernel this course builds. That is recent and it matters: before it, running a real-time Linux meant applying an out-of-tree patch series that trailed the kernel it patched, sometimes did not exist for a given version, and was a standing porting cost. `kernel/rt.config` is now **four lines**, and the whole of the difference between the two kernels this chapter compares.

One of those four lines is `CONFIG_EXPERT=y`, and it is not decoration: **PREEMPT_RT** is declared depends on `EXPERT && ARCH_SUPPORTS_RT`, so without it the symbol is never offered and `CONFIG_PREEMPT_RT=y` is silently discarded, leaving a kernel that builds, boots, says **PREEMPT** and is not real time. That is exactly the hidden-prompt mechanism Exercise 3.3 makes you work through with `GPIO_SYSFS`.

12.1.5 What **PREEMPT_RT** actually changes

Four things, and the first is the one that reaches your code.

Most spinlocks become sleeping locks. `spinlock_t` is replaced by an `rt_mutex` underneath, so code holding one can be preempted and can even sleep. That removes the largest source of preemption-disabled time in the kernel at a stroke.

It also **changes what your driver is allowed to assume**. Code that took a `spinlock_t` and relied on preemption being disabled, for instance to touch per-CPU data safely, is wrong under **PREEMPT_RT** and was correct before.

Most interrupt handlers become threads. Every handler registered with `request_irq` runs in a kernel thread by default, exactly like the threaded handlers §8.2.4 introduced, unless it asks not to with `IRQF_NO_THREAD`; per-CPU interrupts, and a primary handler requested with `IRQF_ONESHOT`, stay in hard interrupt context as well. So the hard-interrupt path shrinks to almost nothing and your handler becomes schedulable, preemptible and prioritisable.

Mutexes get priority inheritance, which is §12.1.3's fix applied by default.

Softirqs are threaded too, removing another unbounded source of delay.

What does not change

`raw_spinlock_t` still spins, still disables preemption, and still cannot sleep. It exists for the places that genuinely cannot be preempted: the scheduler itself, the timer core, the interrupt entry path.

Reaching for `raw_spinlock_t` to make your driver faster reintroduces exactly the latency **PREEMPT_RT removed**, and does so in a way that is invisible until somebody measures the worst case. A driver should essentially never contain one.

12.1.6 What makes a driver hostile to real time

Four habits. Look for them in your own driver; at least two are in it.

Long critical sections. Anything held across a loop, a copy, or a call into unfamiliar code. The fix is to shorten it, not to change the lock type.

Interrupts disabled across real work. `spin_lock_irqsave` around a page of processing rather than around the three lines that touch shared state.

Busy-waiting. `udelay(50)` is legal, works, and spends fifty microseconds of a CPU doing nothing. §9.2.3 is where the alternatives are; the rule is that `udelay` is for a few microseconds of genuine hardware settling time and nothing else.

Per-CPU assumptions. Code that updates a per-CPU variable while holding a `spinlock_t`, relying on preemption being disabled to keep every other task on that CPU away from it. Correct on `PREEMPT`, wrong on `PREEMPT_RT`, where the lock still keeps the task on its CPU but no longer keeps other tasks off it. `local_lock_t` is the lock that does that on both kernels.

12.1.7 Priorities, and the throttle that saves you

Making a task real-time is one call, and the consequences deserve a paragraph.

Policy	Behaviour
<code>SCHED_OTHER</code>	The normal fair scheduler. Everything by default
<code>SCHED_FIFO</code>	Runs until it blocks or a higher priority becomes runnable
<code>SCHED_RR</code>	The same, with a time slice between equal priorities
<code>SCHED_DEADLINE</code>	Declares a period and a budget; the kernel admits or refuses

A `SCHED_FIFO` task that does not block will not be preempted by anything of lower priority, including the shell you would use to kill it. On a single-CPU machine that is an unrecoverable hang, and it is the classic first mistake.

Linux therefore throttles: `sched_rt_runtime_us` defaults to 950,000 in every 1,000,000, so real-time tasks get at most 95% of a CPU and something else can always run. That saves you, and it also means **a real-time task can be preempted by the throttle**, which people discover as an unexplained 50 ms gap. Turning the throttle off is possible and should be a deliberate, documented decision.

`SCHED_DEADLINE` is the interesting one for periodic work: you state a period and a worst-case execution time and the kernel performs an admission test, refusing the request if the deadline tasks together would need more CPU time than the system allows them. On one CPU that is a real guarantee rather than a priority; on several it bounds how late a task can be rather than promising it never is, which is still more than a priority says.

Two tools that belong next to this, and that this course does not teach. On a real system you would also reach for **CPU isolation** (`isolcpus, nohz_full`) to keep everything else off the core your task runs on, and **IRQ affinity** (`/proc/irq/N/smp_affinity`) to decide which core takes the interrupt. Both are absent here, and the reason is the target rather than the topic: this course runs on a two-CPU QEMU guest, and isolating one of two emulated CPUs from a scheduler that is itself emulated produces a number that says nothing about a real machine. That is the same standard §12.2.6 applies to everything else measured here. Know that the two knobs exist and that they are usually the next thing tried after `PREEMPT_RT`; measure them somewhere with real cores.

12.1.8 The comparison, measured

The lab measures handler-to-userspace latency, the interval §12.2.3 can give as an absolute figure, on both kernels, idle and under four spinning processes on two CPUs. One run of each:

Kernel	Load	Mean	Worst
PREEMPT	idle	449 us	11,321 us
PREEMPT_RT	idle	551 us	4,964 us
PREEMPT	busy	1,685 us	29,729 us
PREEMPT_RT	busy	2,393 us	18,559 us

Read the two columns separately, because they disagree.

The mean is worse under PREEMPT_RT, by about 23% idle and 42% loaded. That is the cost of the extra preemption points, the threaded handlers and the inheritance bookkeeping, and it is expected.

The worst case is better under PREEMPT_RT, by a factor of 2.3 idle and 1.6 loaded.

That is the whole trade, and it is the definition of real time in one table. A change that made the average worse and the maximum better would be a straightforward regression on a throughput system and is exactly what you are buying here.

And every number above is worthless on its own, for reasons §12.2 sets out in full. They were measured under an emulator, over 2,000 samples, on one run. Repeating the idle PREEMPT measurement gave a worst case of 2,502 us rather than 11,321. The *shape* of the comparison survives that; the values do not.

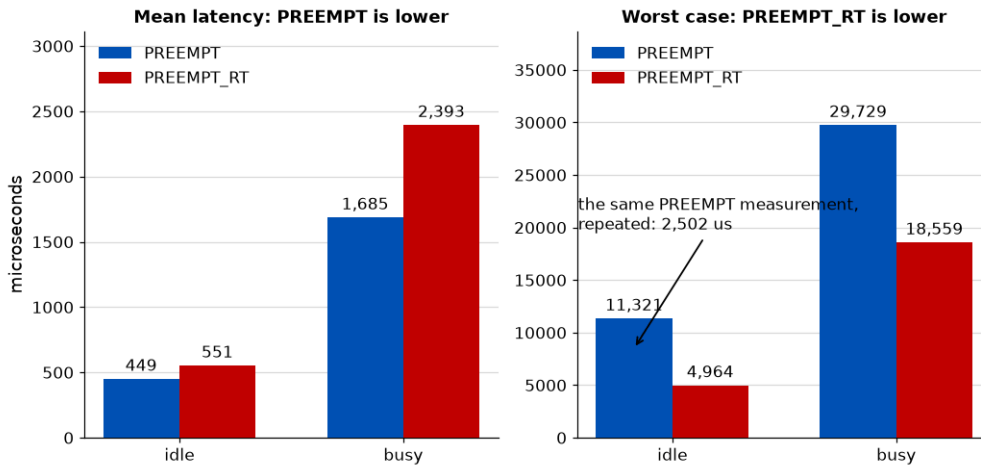


Figure 12.1 Two panels of grouped bars. On the left, mean latency, where PREEMPT is lower at 449 against 551 microseconds idle and 1,685 against 2,393 busy. On the right, the worst case, where PREEMPT_RT is lower at 4,964 against 11,321 idle and 18,559 against 29,729 busy, annotated with the same PREEMPT measurement repeated at 2,502.

12.2 Measuring Latency Honestly

How to measure interrupt latency, what `cyclictst` does and does not measure, and the caveats that make a number mean something. This section is as much about what to refuse to claim as about how to obtain the figure.

12.2.1 Which latency

“Latency” names at least four different intervals, and quoting one when you meant another is the most common error in the subject.

```

device asserts the line
|
v
hard IRQ handler runs
|
v
threaded handler / bottom half runs
|
v
userspace read() returns
|

```

(1) interrupt latency

(2) handler-to-thread latency

(3) wakeup latency

```
v                                (1)+(2)+(3) = interrupt-to-userspace latency
```

A datasheet that says “latency: 12 us” and does not say which of these it is has told you nothing. The lab measures (1), and only as a spread, and the stretch from the handler to userspace as an absolute figure; §12.2.3 is why it cannot give the sum. The exercises ask you to say which you would quote.

12.2.2 What `cyclicttest` measures, and what it cannot

`cyclicttest` is the standard real-time benchmark and it works like this: a thread asks to be woken in n microseconds, records the time when it actually wakes, and reports the difference.

```
T: 0 ( 700) P:80 I:1000 C: 10000 Min: 8 Act: 12 Avg: 14 Max: 67
```

Minimum, current, average and maximum wakeup lateness in microseconds, per thread.

It measures the timer wakeup path. That is genuinely the thing most real-time applications care about, it needs no hardware, and it is comparable across machines, which is why it is the standard.

It cannot measure your device’s interrupt path, because there is no device in it. Nothing in `cyclicttest` touches your driver, your handler or your interrupt line, so a good `cyclicttest` result says nothing about whether your device’s data reaches userspace on time. That is not a criticism; it is a statement of what the tool is for.

To measure your path you need a timestamp taken **at the device**, which real hardware almost never provides. That is why this course built a device that does.

12.2.3 Measuring by subtraction

`qa-dev` records the instant it asserted its interrupt line, in `TS_L0/TS_HI`. So the interval can be *subtracted* rather than inferred:

```
handler_ns = ktime_get_ns();                                /* first statement of the ↵
↵ handler */
lo = readl(regs + QA_DEV_TS_L0);                             /* reading L0 latches the pair ↵
↵ */
hi = readl(regs + QA_DEV_TS_HI);
device_ns = ((u64) hi << 32) | lo;
```

Take `ktime_get_ns()` first. Every instruction before it is counted as interrupt latency, and the two register reads are not free.

The two clocks, and why one interval is harder than the other

The device’s timestamp comes from QEMU’s virtual clock. The guest’s `ktime_get_ns()` counts from its own boot. **The two have different origins**, and the device publishes nothing

that relates them: recovering the constant between them would mean relying on how QEMU happens to implement its clocks. Measured on this target, `handler_ns - device_ns` sits at about **-1,768,000,000 ns**, and over 1,720 consecutive interrupts it ranged over 737 us.

So:

Interval	Both ends in	Usable as
<code>handler_ns - device_ns</code>	Different clocks	A spread above the smallest observed
<code>now - handler_ns</code>	Both <code>CLOCK_MONOTONIC</code>	An absolute figure, no correction

The second is exact because `clock_gettime(CLOCK_MONOTONIC)` in userspace and `ktime_get_ns()` in the kernel are the same clock. The first is not, and the honest treatment is to report it as “worst case, above the best case observed”, which is what the lab’s program prints and why it prints a sentence explaining that the absolute value is unknowable here.

The driver hands both timestamps to userspace in one record per interrupt. The record is declared in `lectures/L12/lab/qa_latency.h`, which ships with the lab, and the prose in that header is the argument above in brief. Here it is, typeset straight from the repository:

```
lectures/L12/lab/qa_latency.h
/**
 * @file The record the qa_latency driver delivers, one per interrupt.
 */
#ifndef QA_LATENCY_H
#define QA_LATENCY_H

#include <linux/types.h>

/** Latency record structure. */
struct qa_latency_record
{
    /** The instant the device asserted its interrupt line, on QEMU's virtual clock. */
    __u64 device_ns;

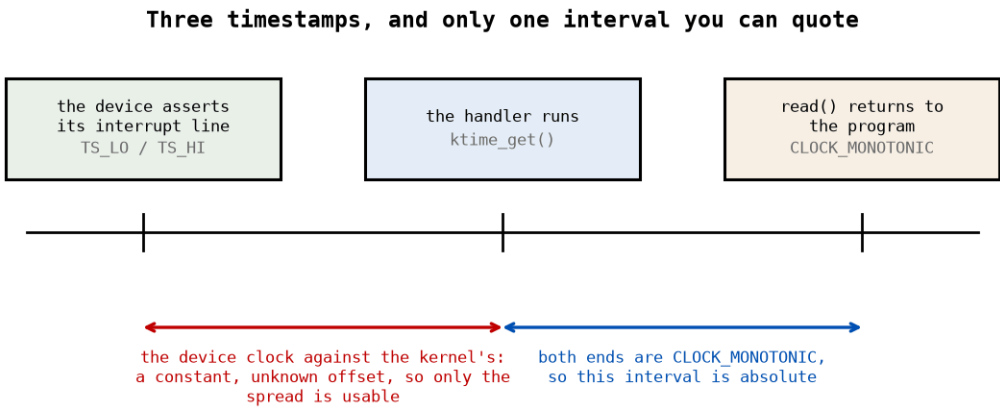
    /** ktime_get_ns() taken as the interrupt handler's first statement, on CLOCK_MONOTONIC. */
    __u64 handler_ns;

    /** Sequence number of this record, so a reader can tell which ones it did not see. */
    __u32 sequence;

    /** Records the driver dropped because the reader was behind. */
    __u32 overruns;
};

#endif /* QA_LATENCY_H */
```

On real hardware with a device that timestamps, the same subtraction gives an absolute figure, because the device and the CPU share a clock domain. The technique is the point; the caveat is specific to measuring inside an emulator.



Which is why the appendix reports the first as a spread above the best case observed,
and the second as a number. Quoting both the same way would be the mistake.

Figure 12.2 A timeline with three marks: the device asserting its line and recording TS_LO and TS_HI, the handler running and calling ktime_get, and read returning to a program holding CLOCK_MONOTONIC. The first interval spans two clocks and carries a constant unknown offset; the second has CLOCK_MONOTONIC at both ends and is absolute.

12.2.4 Reading a histogram

A mean and a maximum are not enough. From the lab, PREEMPT, idle, 2,000 samples:

latency	count	cum %	
32 us	1	0.05%	
64 us	12	0.65%	
128 us	209	11.10%	####
256 us	1309	76.55%	#####
512 us	399	96.50%	#####
1024 us	61	99.55%	#
2048 us	6	99.85%	
4096 us	2	99.95%	
8192 us	1	100.00%	

Four things to read off it:

The bulk. 77% of samples are under 512 us. That is the number a throughput argument would quote and it is the least interesting one here.

The tail. Seventy samples out of 2,000 are at 1,024 us or more, nine at 2,048 us or more, and one is above 8 ms. **The tail is the system**, for a real-time purpose: a deadline of 1 ms is missed at least 3.5% of the time, which is dozens of times a second at this rate.

The shape. A distribution with a long thin tail is a system with a rare blocking event. A distribution with a second hump is a system with two different behaviours, and finding out what distinguishes them is usually productive.

The buckets are logarithmic. Latency spans orders of magnitude, and linear microsecond buckets either lose the bottom or produce ten thousand rows. Anyone showing you a linear latency histogram that covers both 30 us and 30 ms has hidden something.

12.2.5 The caveats that make a number mean something

A latency figure without these is not a measurement, it is a number.

Under what load. The lab measures idle and under four spinning processes on two CPUs. Those differ by a factor of about four in the mean and about three in the maximum. An idle-system latency figure describes a system that is not doing anything, which is not the system you are shipping.

For how long. A worst case over 2,000 samples is two seconds of evidence. Real qualification runs for hours or days, because the interesting events are rare by definition. Running the lab's idle measurement twice on the same kernel gave worst cases of **11,321 us and 2,502 us**: more than a factor of four, from the same code on the same machine. **One run does not establish a worst case.**

On what kernel, with what configuration. CONFIG_PROVE_LOCKING, which this course enables for Chapter 7, does bookkeeping on every lock acquisition and is not something you measure latency on. Debug options, tracing and CONFIG_DEBUG_* all cost, and a latency number from a debug kernel is not the number your product has. **This course's own numbers are such numbers.** Both of its kernels keep lockdep and CONFIG_DEBUG_ATOMIC_SLEEP on, qa.config for Chapter 7 and rt.config so that a driver that sleeps in the wrong place says so, and every figure in this section carries their bookkeeping, on both sides of the comparison.

And on what machine. Which is the caveat this course cannot escape.

12.2.6 What this measurement is blind to

Stated plainly, because everything above is otherwise an invitation to quote a number that is not true.

QEMU is not a real-time host. It is a userspace process on a general-purpose Linux, scheduled by that kernel, competing with everything else on the machine, on a virtual clock that advances in steps the guest cannot see. The guest's "interrupt latency" therefore includes the host's scheduling of QEMU, which has nothing to do with the guest kernel at all and is frequently the largest term.

So the absolute numbers are worthless. Not approximate: worthless. A worst case of 4,964 us on this target predicts nothing whatever about the same kernel on real silicon, where the equivalent figure would typically be tens of microseconds.

What survives is the comparison. Both kernels run in the same emulator, under the same host conditions, with the same load, measured the same way. The emulator's contribution is a common term, and the *difference* between the two columns is still informative even though neither column is. That is why §12.1.8 presents them side by side and never on their own.

And even the comparison has a limit: it establishes the *direction* and roughly the *shape* of what PREEMPT_RT does, and it does not establish the factor. Do not carry “2.3x better worst case” out of this course.

What you would need instead

To measure this properly:

- Real hardware, with the device and CPU in one clock domain.
- A kernel with debug options off.
- Hours of running under a load that resembles the product's.
- A device or a scope providing the reference timestamp, or a GPIO toggled by the handler and watched externally, which is what people actually do.
- And the discipline to report the maximum, the duration, the load and the configuration together.

That last point is the one that generalises beyond this chapter. It is the same conclusion Chapter 3's Cross-check, Exercise 3.9, reached about an Image size and Chapter 2's, Exercise 2.9, reached about a context-switch rate: **a number is meaningless without the conditions it was measured under**, and the cost of forgetting that rises with how much the number matters.

12.3 Review

What you should be able to do now

After this chapter, you should be able to:

- Define a real-time system without using the word “fast”, and say why an average is not evidence.
- Name three sources of latency and say, for a given piece of code, which one it contributes to.
- Explain priority inversion, and say what priority inheritance does about it.
- Choose a preemption model for a stated requirement and defend it.
- Say what PREEMPT_RT does to a spinlock_t and to an interrupt handler.
- Say why raw_spinlock_t exists and when using one is a mistake.
- Find, in unfamiliar driver code, the four habits that make it hostile to real time.
- Run cyclictst, read its output, and state precisely what it did and did not measure.
- Present a latency measurement together with the conditions that make it meaningful.

Questions to test yourself

1. System A responds in 50 us on average and 30 ms at worst. System B responds in 5 ms every time. Which is real time, and for what requirement?
2. Your handler takes a `spinlock_t` and does 200 us of work. What changes under `PREEMPT_RT`, and is the change an improvement?
3. Why can `PREEMPT_RT` not simply make every lock a sleeping lock?
4. `cyclictst` reports a maximum of 40 us. What does that tell you about your device's interrupt latency?
5. A colleague reports "we measured 12 us latency". What four questions do you ask before believing it applies to your product?
6. Why is a `udelay(50)` in a driver a real-time problem even though 50 us is short?
7. You measure lower latency under QEMU than the datasheet claims for real silicon. What happened?

Where to look

- `tools/qa-dev/qa-dev.c` shows exactly when the timestamp is taken, which is the one thing you must be sure of before trusting a latency measurement built on it.

Looking ahead

This is the last chapter, and with it the book's main text is complete. It ends with a driver that probes off a device tree, services interrupts, blocks a reader correctly, registers with two subsystems, and whose latency you have measured and can describe honestly. What remains is to work the exercises in §12.4 and check the ones that are not code against the solutions in Appendix A. After that, the two written papers in Appendices B to F are there to test, on paper and with nothing in front of you, how much of all twelve chapters you can reconstruct on your own.

What to do next, in rough order of usefulness: read a real driver in `drivers/` for a device you own and see how much of it you now recognise; write a driver for hardware that exists; and send a patch, because the review you get on a first patch to a subsystem mailing list is worth more than any course.

12.4 Exercises

Nine, ending with the Cross-check that closes the course. Do them in order; Exercise 12.6 onwards need both kernels built, which is `make kernel` and `make kernel RT=1`, and the second kernel's `initramfs`, which is `make rootfs RT=1`.

Where an exercise can be checked mechanically, a **Check yourself** line says how.

Solutions. The exercises that are not Code are answered in §A.12. The Code exercises have no solution: each is checked by running it, as its Check yourself line says.

Exercise 12.1 What real time claims

RECALL

- Define a real-time system without using the word “fast”.
- System A responds in 50 us on average and 30 ms at worst. System B responds in 5 ms every time, never more than 5.1 ms. Which is real time? Answer for a 6 ms deadline and for a 1 ms deadline.
- A colleague reports that a change “improved latency by 30%”. What is the single question that determines whether that is good news?
- Distinguish hard from soft real time, and say which Linux with PREEMPT_RT honestly is.

Exercise 12.2 The three sources

RECALL

- Name the three sources of latency from §12.1.2.
- For each of these, say which source it contributes to:
 - A `spin_lock_irqsave` critical section that copies 4 KB.
 - A `udelay(200)` in a probe function.
 - A low-priority task holding a mutex a high-priority task needs.
 - A hard interrupt handler that takes 300 us.
- Two of the three are somebody’s code and one is structural. Which is which, and what mechanism addresses the structural one?
- Tell the Mars Pathfinder story in four sentences, naming the three priorities involved.

Exercise 12.3 A latency budget

HAND CALCULATION

A motor controller must react to an encoder interrupt within **500 us**, every time. Measured on the target hardware:

interrupt to hard handler	max	40 us
hard handler duration	max	15 us
hard handler to threaded handler	max	120 us
threaded handler duration	max	60 us
threaded handler to userspace	max	200 us

- What is the worst-case interrupt-to-userspace latency? Does it meet the deadline?

- b) The work is moved entirely into the hard handler, removing the two thread transitions. Recompute. Does it now meet the deadline, and what have you done to every other device on the system?
- c) A driver elsewhere holds a `spinlock_t` for 300 us. Add it to your budget for both a) and b). Which arrangement survives?
- d) Under `PREEMPT_RT` that 300 us spinlock section becomes preemptible. Recompute a). What happened, and what has it cost you?
- e) The requirement is restated as “500 us, 99.9% of the time”. What changes about which numbers you need, and is that easier or harder to demonstrate?

Exercise 12.4 Reading your own driver for hostility

DESIGN

Open the driver you wrote in Chapter 11 and find:

- a) Every place it disables interrupts, and for how long.
- b) Every place it holds a lock across something that is not a handful of register accesses.
- c) Every busy-wait.
- d) Every assumption that it stays on one CPU.

For each one you find, say what it would cost under `PREEMPT_RT` and what the fix would be. If you find none of some category, say how you established that.

- e) Which of the four habits from §12.1.6 is the hardest to spot by reading, and what would you use to find it instead?

Exercise 12.5 Priorities

DESIGN

- a) You set your measurement program to `SCHED_FIFO` priority 80. Name two things that can now still preempt it.
- b) A `SCHED_FIFO` task enters an infinite loop on a single-CPU machine. What happens, and what saves you?
- c) `sched_rt_runtime_us` is 950000 out of 1000000. State what that means in one sentence, and describe the symptom of hitting it.
- d) Turning the throttle off is possible. When would you, and what do you owe the next person to work on the system?
- e) `SCHED_DEADLINE` refuses a request that `SCHED_FIFO` would have accepted. What did it do that `SCHED_FIFO` does not, and why is a refusal more useful than a missed deadline?

Exercise 12.6 Measure by subtraction

CODE

Write `lectures/L12/lab/qa_latency.c`, a platform driver that turns the work of Chapters 9 and 11 into a measurement instrument.

- In the interrupt handler, **first** take `ktime_get_ns()`, then read `TS_L0` and `TS_HI` to get the instant the device asserted its line. Reading `TS_L0` latches the pair.
- Store both, plus a sequence number and a count of records dropped, in a ring of 512 `struct qa_latency_record` (declared in `lectures/L12/lab/qa_latency.h` and printed in §12.2.3).
- Present `/dev/qa_latency`, whose read blocks until records are available and returns as many whole records as fit, with `O_NONBLOCK` and `poll` support as in Chapter 9.
- Count records dropped when the ring is full rather than overwriting.
- Use `devm_` for everything the driver core has a helper for, and `devm_add_action_or_reset` for the character device, which has none.

Check yourself: make `test L=L12` reports **PASSED**, with ten checks, and `RT=1 make test L=L12` does too after `make rootfs RT=1` and `RT=1 make build L=L12`.

- Why must `ktime_get_ns()` be the first statement rather than coming after the register reads? Estimate what you would be adding to every measurement if it came second.
- You forget to rebuild between kernels and `insmod` says `invalid module format`. Which chapter explained that, and what exactly disagreed?
- The driver drops records under load rather than overwriting the oldest. Argue for that choice, then argue against it.

Exercise 12.7 Measure from userspace

CODE

The measurement program `qa_latency_test` ships with the course. Read it before running it.

- It computes two intervals. Name them, and say which is absolute and which is only a spread. Why the difference?
- It takes `clock_gettime` once per batch of records rather than once per record. Why can it not do better, and what does a large batch tell you about the reader?
- It discards the first few reads. What would they have measured otherwise?
- The histogram buckets are logarithmic. Take the idle `PREEMPT` histogram from §12.2.4 and redraw the top three rows as linear 1 us buckets. How many rows would the full histogram need?
- Run it and read your own histogram. What fraction of samples is under 512 us, and what is the worst case? Which of those two would you put in a datasheet, and which describes the system?

Exercise 12.8 What cyclicttest is for

DESIGN

- a) Describe in two sentences what cyclicttest measures.
- b) Your cyclicttest maximum is 45 us and your device’s data still arrives late. Explain how both can be true.
- c) Name the part of the path in §12.2.1 that cyclicttest covers, and the parts it does not.
- d) qa-dev has a timestamp register. Real hardware almost never does. Given that, how do people actually measure interrupt latency on real boards? Name two techniques.
- e) Why did this course build a device with that register rather than using a real one?

Exercise 12.9 The trade, measured

CROSS-CHECK

The last Cross-check. Predict which kernel wins, measure both, and discover that the answer depends entirely on which number you look at.

a) **Predict.** Before measuring anything, write down whether you expect PREEMPT_RT to produce a **lower mean** and a **lower maximum** than PREEMPT, for the handler-to-userspace latency the lab reports. Commit to both answers separately.

b) **Measure, four ways.** Build and boot both kernels, remembering to rebuild the module for each:

```
make kernel      && make build L=L12      && make test L=L12
make kernel RT=1 && make rootfs RT=1
RT=1 make build L=L12 && RT=1 make test L=L12
```

Record the mean and the maximum for each kernel, idle and under load: eight numbers.

c) **Reconcile.** Answer each:

- Which of your two predictions was right, and which was wrong?
- The mean is **worse** under PREEMPT_RT and the maximum is **better**. Explain both, in terms of what §12.1.5 says changes.
- If a colleague measured only the mean, what would they conclude, and would they be wrong?
- State, in one sentence, why “worse average, better worst case” is the definition of the trade rather than a disappointing result.

d) **Repeat it.** Run the **same** idle measurement on the **same** kernel three more times and record the maximum each time. Compare the spread between runs against the difference you measured between the two kernels in b).

- Is your kernel-to-kernel difference larger than your run-to-run variation?
- If it is not, what have you actually established?

- How many runs would you need before you would defend the comparison in a design review?

e) **The part that matters most.** Everything you have measured was taken inside an emulator. Write a paragraph, not a sentence, answering: **what is this measurement blind to?** Cover at minimum which components other than the guest kernel are inside your numbers, why the absolute values cannot transfer to real silicon, and what specifically survives the comparison and what does not.

f) You are asked for one number for a datasheet: “interrupt-to-userspace latency”. Write the entry exactly as you would submit it, including every condition. Then write the sentence you would add to say what it does not cover.

g) Finally: your driver now probes off a device tree, services interrupts, blocks readers correctly, registers with two subsystems, and has had its latency characterised. Name the one thing you would still not deploy it without, and say which chapter told you why.

Solutions to the Exercises

Worked answers to everything that is argued or calculated on paper.

This appendix answers every exercise that is not code: each Recall, Hand calculation and Design exercise, and the parts of each Cross-check that are worked out by hand or argued. It is grouped by chapter, and each answer is headed by the exercise it answers, with the same number and kind.

The Code exercises are not answered, here or in the repository. Each of them is checked by running it: its Check yourself line names what passes when the code is right, and make test runs it on the target. An answer to read would replace the work, and the work is the point. The same goes for the half of each Cross-check that is done by machine. Where a Cross-check asks what the machine will print, the answer here is what a driver that meets its specification prints, or what the course measured on its own target; reconciling that with what your machine prints is still yours to do.

Try each exercise before you read its solution. A Recall answer is one way of saying it, not the only one, and an answer of yours that reaches the same facts by a different route is just as right. Where a Design exercise asks you to choose, the solution makes a choice and gives its reason; a different choice with a better reason is a better answer.

Every number here is either derived from the chapter that sets the exercise or was measured on the course's target and published in that chapter, and where an answer depends on a measurement it names the section that publishes it.

A.1 Chapter 1: Embedded Linux, and the Licence You Ship With It

Answers to the exercises in §1.4. Exercises 1.6 and 1.7 are Code, and are checked by running them: the first against the total its Check yourself line gives, the second on the target.

Exercise 1.1 The four pieces

RECALL

The **bootloader** brings up DRAM, finds the kernel, and hands it a device tree. The **kernel** drives the hardware and provides the system call interface. The **root filesystem** is everything that runs as a process, starting with PID 1. The **toolchain**, a compiler, a C library, a linker and a set of headers, builds the other three and never ships, and it fixes the ABI they share.

- a) The kernel. It is stored in the root filesystem, and that is the plausible wrong answer: it is built by the kernel's build from the kernel's tree, it runs in the kernel's address space, and it loads only into the kernel it was built for, which is why its path carries the version, 6.12.30. Where a file is stored does not decide which piece it belongs to.
- b) The bootloader: `u-boot.img` is U-Boot itself, step 3 of the chain in §1.1.2.
- c) The toolchain. It lives on the build machine, and nothing on the device ever runs it.
- d) The root filesystem. `/init` is PID 1, the first userspace process; on this target it is the shell script `tools/target/init`, which `ci/rootfs.sh` copies in.
- e) The kernel, although it travels beside the kernel rather than inside it. Its source lives in the kernel tree, the kernel's build compiles it, and it is written against the bindings the kernel documents, so it changes when the kernel does. The bootloader is what loads it and hands it over (§1.1.3), and on a real board U-Boot often edits it on the way, filling in the memory size and the command line, so "the bootloader" is a defensible answer if it is argued that way. On this target it is neither: QEMU writes it (§1.1.4).
- f) The root filesystem. `/bin/busybox` is the whole userland of this target.

Exercise 1.2 Why there are two bootloaders

RECALL

- a) U-Boot is around a megabyte and must run from DRAM, and DRAM does not work until its controller has been configured, while the only memory there is at power-on is a few tens of kilobytes of SRAM on the SoC. The SPL is a bootloader small enough to run from that SRAM, and its whole job is to initialise the DRAM controller and load U-Boot into the DRAM it has just brought up.
- b) The boot stops at the first handover, from the boot ROM to the next stage, before U-Boot has printed a line. The ROM loads the next stage into SRAM from wherever it is told to look; with the SPL deleted it finds either nothing it accepts or U-Boot, which does not fit in SRAM and needs the DRAM nobody has initialised. The board produces no bootloader output at all and looks dead, and the space saved is the SPL's, around 64 KB.
- c) Recoverable: a boot ROM that falls back to loading over USB or a UART, or from a removable SD card, when the flash holds nothing it can boot, or strap pins that force it to; a bad bootloader can then always be replaced from outside. Not recoverable: a ROM that

boots only from soldered flash and has no fallback. One bad write to the bootloader partition and the board is a coaster, short of a JTAG probe or a soldering iron.

Exercise 1.3 What the toolchain triplet promises

HAND CALCULATION

a) `aarch64` is the architecture: 64-bit ARM, little-endian, since the big-endian variant is `aarch64_be`. `linux` is the operating system the output runs on: it makes Linux system calls and follows Linux's conventions for an executable. `gnu` is the C library and the ABI: `glibc`, and the GNU/Linux ABI for AArch64, whose dynamically linked programs are started by `/lib/ld-linux-aarch64.so.1`. The full form has a vendor field as well, `aarch64-unknown-linux-gnu`, which the short name leaves out.

b) `arm` is 32-bit ARM, in practice a Cortex-M or Cortex-R microcontroller. `none` is the operating system, and there is none: the binary is written for bare metal. `eabi` is the ARM Embedded ABI, the calling convention and object format for code with no operating system beneath it; no C library is named, and in practice it is `newlib`. So it is meant for a microcontroller, or for a stage that runs before any operating system does. It cannot run on Linux as a program: nothing in it makes a Linux system call, its startup code and linker script assume it owns the machine from a fixed address, and there is no Linux loader or C library for it to be started by.

c) It builds, and fails at run time, when the target's dynamic loader starts it. `glibc` versions its symbols, and a program records the versions of every symbol it was linked against; since `glibc 2.34` even `__libc_start_main`, which every dynamically linked program calls, carries the version `GLIBC_2.34`. The 2.31 on the target does not have it, so the loader refuses before `main` runs, with a message of the form `version 'GLIBC_2.34' not found (required by ./prog)`. Nothing at build time can see this coming, because the build only ever saw 2.36. The fix is to build against a C library no newer than the oldest target's, or to link statically, which is what this course does for its own test programs, because its target has no C library at all.

d) It installs the package for the build machine's architecture: its headers in `/usr/include`, and an `x86-64` library in `/usr/lib/x86_64-linux-gnu`. Debian's cross-compiler searches `/usr/include` (§1.1.5), so the missing header may even be found, and that is the trap: the failure moves to the link, which cannot find `-lfoo`, because the only `libfoo` is an `x86-64` library in a directory the cross linker never searches and could not link against if it did. If the missing header is one of the few that differ by architecture, which Debian keeps in `/usr/include/x86_64-linux-gnu`, the error does not move at all. What works is to install the library *for the target*: on Debian, `dpkg --add-architecture arm64`, an `apt update`, and then `apt install libfoo-dev:arm64`, which puts the `arm64` library in `/usr/lib/aarch64-linux-gnu`, where the cross-compiler does look; or cross-compile `libfoo` and install it into the toolchain's `sysroot`. For this course's target, whose root filesystem holds no libraries, the program must also link it statically.

Exercise 1.4 What leaves the building

DESIGN

a) Row by row:

- **U-Boot:** its complete corresponding source, meaning the version you shipped with your board port in it, the configuration you built it with, and the scripts and instructions that build it. It is “or later”, so you may comply on GPL-2.0’s terms.
- **The kernel:** the same, at the exact version shipped with every patch applied, including the driver you added, which is in the tree and therefore part of the work; the `.config`; the device tree sources; and which toolchain and which commands build it (§1.2.5).
- **BusyBox:** its source at the version shipped, and the `.config` you built it with, which is one of the scripts that control its compilation.
- **libcurl:** no source. Its licence is permissive and asks only that its copyright and permission notice go with every copy, which for a binary means your documentation.
- **The logging daemon:** nothing. It is a separate process making system calls (§1.2.3).
- **Your kernel module:** that depends on the choice, and the choice has consequences. Declared GPL, it is published like the kernel. Declared proprietary, it is published nowhere, and you carry what §1.2.7 lists, a tainted kernel, no `EXPORT_SYMBOL_GPL` symbols and its maintenance against a kernel API that changes every release, and a derivative-work argument that is contested rather than settled in your favour; Exercise 1.5 works that argument through.

b) No. The obligation attaches to distributing the binary, not to modifying it: whoever receives BusyBox from you is entitled to its source whether or not you changed a line. Being unmodified makes compliance easy, since the answer is the upstream release at the version you shipped and your `.config`; it does not make it optional.

c) Neither does. `libcurl`’s licence is permissive, so linking to it, dynamically or statically, puts no obligation on the daemon’s source, only the notice from a). The `ioctl` is a system call: the daemon uses your module through the kernel’s system call interface, which is exactly the boundary the syscall note draws, and §1.2.3 names `ioctl` in its list. The module is yours in any case, so nobody else’s licence reaches the daemon through it.

d) Keep, for every release you shipped, the complete corresponding source of every GPL component exactly as it was built: the kernel with its patches, your driver, its `.config` and its device tree sources; U-Boot with the board port and its configuration; BusyBox and its `.config`; your module, if you made it GPL; and the build scripts and the toolchain, or enough to reproduce them, that turned all of that into the image you shipped. If you satisfy the GPL with a written offer, GPL-2.0 section 3(b) makes the offer valid for at least three years and open to any third party, not only to customers, and every unit you ship carries it, so in practice you keep the source until three years after the last unit ships (§1.2.5). The engineering consequence is that your build must still work then: the toolchain archived

rather than named, every source pinned, the `.config` kept as a build artefact, and a record of which source went into which shipped unit.

Exercise 1.5 Where the boundary falls

DESIGN

- a) Two works. The syscall note settles it: user programs that use kernel services by normal system calls do not fall under the heading of derived work (§1.2.3). It is the copyright holders' own statement rather than a reading of the GPL, and it is not contested.
- b) One work. Static linking puts your code and the library into one binary, and that is broadly accepted as a derivative work of both (§1.2.2). Distributing it means distributing the whole under GPL-2.0, source included.
- c) Two works, and the licence says so. The LGPL-2.1 treats a program that only links to the library as a work that uses the library, which may carry any licence you like, provided you publish the library's source and let the recipient relink your program against a modified copy of the library, which a dynamic link allows as it stands. This is the question the LGPL exists to settle.
- d) Contested. The Free Software Foundation holds that dynamically linking against a GPL library creates a derivative work; others disagree, and nothing has settled it (§1.2.2). The engineering answer is not to ship it without a lawyer's opinion, or to find a library under the LGPL or a permissive licence that does the same job.
- e) Contested, with most of the weight on one work. A module is compiled against the kernel's headers, runs in its address space and calls its functions directly, and the widely held view among kernel developers is that it is a derivative work (§1.2.4). Whatever a court would decide, the kernel enforces a version of the argument itself: a proprietary module taints the kernel and cannot use `EXPORT_SYMBOL_GPL` symbols.
- f) Two works: mere aggregation (§1.2.2). Sharing a filesystem image does not make two programs one work, and nobody seriously argues otherwise.

Exercise 1.8 An audit by hand against an audit by script

CROSS-CHECK

a) Eight of the ten carry a tag, each on its first line in a `//` comment. Two carry none, and their licence is in the header comment instead.

File	Identifier	Where
<code>rtc-ds1307.c</code>	GPL-2.0-only	Tag, line 1
<code>rtc-pl031.c</code>	GPL-2.0-or-later	Tag, line 1
<code>rtc-starfire.c</code>	None	Header comment: "License: GPL"
<code>rtc-rs5c313.c</code>	None	Header comment: the GPL, and see COPYING
<code>rtc-cmos.c</code>	GPL-2.0-or-later	Tag, line 1
<code>rtc-m41t80.c</code>	GPL-2.0-only	Tag, line 1
<code>rtc-pcf8563.c</code>	GPL-2.0-only	Tag, line 1
<code>rtc-abx80x.c</code>	GPL-2.0	Tag, line 1
<code>rtc-max77686.c</code>	GPL-2.0+	Tag, line 1
<code>rtc-s35390a.c</code>	GPL-2.0-or-later	Tag, line 1

GPL-2.0 is the old spelling of GPL-2.0-only and GPL-2.0+ the old spelling of GPL-2.0-or-later, so the ten are four GPL-2.0-only, four GPL-2.0-or-later and two untagged: four strings, and two licences.

b) From the sample alone: two licences, GPL-2.0-only and GPL-2.0-or-later, in about equal shares, and about one file in five with no tag.

c) A script that meets Exercise 1.6 reports this over the 184 files:

Identifier	Files	Of which merged
GPL-2.0-only	122	49 spelled GPL-2.0
GPL-2.0-or-later	59	28 spelled GPL-2.0+
LGPL-2.1+	1	
Untagged	2	<code>rtc-rs5c313.c</code> , <code>rtc-starfire.c</code>
Total	184	

The counts check against the total: $122 + 59 + 1 + 2 = 184$. No file in the directory carries a second tag, so taking the first loses nothing here. The single LGPL-2.1+ is `lib_test.c`, the directory's KUnit test; it is the old spelling of LGPL-2.1-or-later, which the exercise does not require the script to merge and a careful script merges anyway.

d)

- **Strings and licences.** Five distinct strings: GPL-2.0-only, GPL-2.0, GPL-2.0-or-later, GPL-2.0+ and LGPL-2.1+. Three distinct licences: GPL-2.0-only, GPL-2.0-or-later and LGPL-2.1-or-later. The strings count spellings, and the difference is the number of licences that appear under two spellings, here the two GPL ones, so $5 = 3 + 2$. The string count can

never be lower than the licence count, and a report that calls strings licences overstates what the tree is under.

- **The untagged files** are `rtc-rs5c313.c` and `rtc-starfire.c`, both of them in the sample, and the routes differ. `rtc-rs5c313.c`'s header says it is subject to the GNU GPL and sends you to `COPYING`, which is GPL-2.0-only; its `MODULE_LICENSE("GPL")` agrees, but cannot tell “only” from “or later” by itself, as `include/linux/module.h` says. `rtc-starfire.c`'s header says only “License: GPL”, with no version, and the file has no `MODULE_LICENSE` at all, since it can only be built in; its version comes from the rule in `Documentation/process/license-rules.rst` that the licence in `COPYING` applies to the kernel source as a whole. Both end at GPL-2.0-only, one because its header says so and the other because nothing says otherwise.
- **The sample did not predict the whole.** It had 40% GPL-2.0-only, 40% GPL-2.0-or-later and 20% untagged. The directory is $122/184 = 66.3\%$ GPL-2.0-only, $59/184 = 32.1\%$ GPL-2.0-or-later, $1/184 = 0.5\%$ LGPL and $2/184 = 1.1\%$ untagged. The sample over-counted untagged files eighteen times over, under-counted GPL-2.0-only, and missed a licence altogether. That is what a sample of names you recognise does: the names you know are the old, widely used drivers, and old files are the ones that predate the tags and still carry a licence in a header comment. The bulk of the directory is newer drivers tagged GPL-2.0-only, which are exactly the names nobody recognises. A random ten would have held both untagged files with probability $\frac{10 \times 9}{184 \times 183} = 90/33,672$, about 1 in 370, so this sample was chosen, not drawn.

e) There are three separate problems, and any two answer it:

- **Nobody tags a file Proprietary.** SPDX has no such identifier, and a proprietary file carries no tag at all, or a `LicenseRef-` of its own. The absence of the string proves nothing; the files that could be proprietary are the untagged ones, which have to be read, as in d).
- **One directory is not the tree.** The script looked at the `.c` and `.h` files of `drivers/rtc`. Every other directory, and every file that is not C, from assembly and device tree sources to scripts, went unexamined.
- **The tree is not the product.** What ships includes things with no source in the tree at all: firmware that drivers load at run time, much of it under licences that are not free; out-of-tree and vendor modules, whose `MODULE_LICENSE` is in a binary no scan of sources will see; and everything else in the image.

A.2 Chapter 2: The Command Line, and the System Underneath It

Answers to the exercises in §2.4. Exercises 2.6 and 2.7 are Code, and are checked on the target: the first by `make test L=L02`, as its Check yourself line says, and the second by the machine itself.

Exercise 2.1 What the shell does and what it does not

RECALL

- a) The shell. Pathname expansion happens before `exec`, so the program receives the list of names in `argv` and never sees the `*`.
- b) The program. To the shell `-l` is a word like any other; each program decides for itself what its options mean, which is why `-l` means “long” to `ls` and “lines” to `wc`.
- c) The shell. It opens `out.txt` and dup2s it onto descriptor 1 before `exec`; the program writes to descriptor 1 as it always does and cannot tell.
- d) The shell. `cd` is a builtin, because a child can change only its own directory and never its parent’s (§2.1.1).
- e) Whichever of them made the call that failed, and for a file named as an argument that is the program: `cat missing` gets `ENOENT` from its own `open` and prints the message itself. If the missing file is a redirection, `< missing`, the shell made the call and the shell reports it. The words are the C library’s text for `ENOENT` either way.
- f) The shell. Parameter expansion replaces `$HOME` with its value before the program starts, and the program receives the value.

Then: a shell leaves a pattern that matches nothing as it is, so `grep` receives three arguments: `grep`, `foo`, and the literal `*.log`. It tries to open a file called `*.log`, fails, prints `grep: *.log: No such file or directory`, and exits with status 2, which means an error, as distinct from 1, which means no match. The message reads like a missing file, and it is the shell’s expansion rule that made one. (Bash can be told otherwise, with `nullglob`; the default, and BusyBox’s `ash`, pass the pattern through.)

Exercise 2.2 Pipelines

RECALL

- a) Three processes, one for each stage, each forked by the shell and then `exec’d`, and two pipes, one between each pair of neighbours. Plumbing them takes four dup2s: `ps`’s output, `grep`’s input and output, and `wc`’s input.
- b) `B` never sees end of file. A pipe’s read end returns end of file only when every copy of the write end is closed, and the shell’s copy keeps it open after `A` has finished and exited. So `B`’s read blocks forever, and the pipeline hangs, with the shell waiting for `B`. `B` is the process that suffers; `A` runs and exits normally.
- c) The pipe’s buffer is bounded, 64 KB, sixteen pages of 4 KB. `yes` writes until it is full and then blocks, so it can never hold more memory than that. `head` reads one line, prints it and exits, and then no process holds the read end open. `yes`’s next write raises `SIGPIPE`, whose default action is to terminate it, and it dies with status $128 + 13 = 141$ (§2.1.3). Had it ignored `SIGPIPE`, the write would have failed with `EPIPE` instead, and a well-behaved `yes` would have exited on that.

d) A pipeline's exit status is its last stage's: `wc -l` ran perfectly well and printed 0, so `$?` is 0 and `grep`'s 1 is lost (§2.1.3). Add `set -o pipefail`, which makes a pipeline's status the last non-zero status among its stages; BusyBox's `ash` supports it. If the count is what you want, test it instead: `grep -c` or `grep -q` gives the answer without a pipeline at all.

Exercise 2.3 Reading `/proc/stat`

HAND CALCULATION

The interval is $6.40 - 3.90 = 2.50$ s, from the first field of `/proc/uptime`.

- a) $1901 - 1626 = 275$ context switches in 2.50 s, so $275/2.50 = 110$ per second.
- b) $71 - 63 = 8$ processes, so $8/2.50 = 3.2$ per second. processes counts forks since boot, so this is a fork rate.
- c) $699 - 252 = 447$ units of `USER_HZ`, which is $447/100 = 4.47$ s. The unit is `USER_HZ`, 1/100 s, because `/proc/stat` reports CPU time in it whatever `CONFIG_HZ` is (§2.2.2); dividing by HZ instead, 250, would give 1.79 s, 2.5 times too small. `/proc/uptime`'s second field, which the kernel computes from the same idle time, agrees: it rose by $6.99 - 2.52 = 4.47$ s.
- d) Two CPUs for 2.50 s is $2 \times 2.50 = 5.00$ s of CPU time, 500 units, and the ten fields of the `cpu` line, five of which are not zero, do rise by exactly that:

$$(19 + 383 + 699 + 81 + 27) - (16 + 341 + 252 + 75 + 25) = 1209 - 709 = 500.$$

Of it, 4.47 s was idle, $4.47/5.00 = 89.4\%$; the other 53 units are user 3, system 42, irq 6 and softirq 2. Yes, 4.47 s exceeds the 2.50 s interval, and it is not a mistake: the `cpu` line sums over both CPUs, so it can gather up to 5.00 s in 2.50 s. Per CPU, the idle time is $4.47/2 = 2.235$ s of the 2.50.

e) None of them. The CPU times are in `USER_HZ`, which is 100 whatever `CONFIG_HZ` is, so 447 units are still 4.47 s; `ctxt` and `processes` are plain counts; and the interval comes from `/proc/uptime`, in seconds. A kernel ticking at 1000 Hz behaves slightly differently, so the counts themselves could come out different on another run, but no step of the arithmetic involves HZ, and noticing that is the point of the question.

Exercise 2.4 Device numbers

HAND CALCULATION

- a) It tells you that one driver handles both: major 1, which `/proc/devices` lists as `mem`. Minors 3 and 5 are two devices of that driver's, which it tells apart by minor. It does not tell you what either device does, which is decided by the driver's code and not by the numbers; nor that the two share anything beyond the driver; nor that any driver is registered at all, which only `/proc/devices` or opening the node can tell you.
- b) The `tty` core, `drivers/tty/tty_io.c`. The excerpt lists major 5 as `/dev/tty`; the full `/proc/devices` lists it three times, as `/dev/tty`, `/dev/console` and `/dev/ptmx`, because the `tty` core registers each of minors 0, 1 and 2 separately. Minor 1 is the console, and it is an

indirection rather than a device of its own: opening it opens whichever tty the kernel's console is on, here the PL011 UART's `ttyAMA0`, named by `console=ttyAMA0` on the command line (§2.2.2).

c) The `mknod` succeeds: it writes a directory entry holding `c`, 204 and 64, and consults no driver. Opening the node succeeds as well, because major 204 is claimed by the PL011 driver, `ttyAMA`, and minor 64 is its first port: `/dev/spare` is a second name for `/dev/ttyAMA0`, the console you are typing on. They are not the same question. The first is about the filesystem, and the second is about whether a driver has registered the number (§2.2.5); here both answers happen to be yes.

d) The `mknod` succeeds, for the same reason. Opening it fails with `ENXIO`, “No such device or address”, because no driver has registered major 42 (§2.2.5). Here the answers differ, which is what makes them two questions.

e) What is gone is one directory entry in `devtmpfs`: the name. The `mem` driver and its minor 3 are still registered, any process that already had `/dev/null` open goes on using it, and `mknod /dev/null c 1 3` with a `chmod 666` gives the device its name back. Until then the loss is worse than it sounds: the next `> /dev/null` a root shell runs creates a regular file of that name, which then collects everything that was meant to be thrown away.

Exercise 2.5 Which file answers the question

DESIGN

a) `/proc/version`, whose `SMP PREEMPT` names the model the running kernel was built with (§2.2.2). The obvious alternative, the `.config` on the build machine, describes whatever you built last, which need not be what is running; `/proc/config.gz` is exact, but exists only on a kernel built with `CONFIG_IKCONFIG_PROC`, as this one happens to be.

b) `/sys/devices/system/cpu/online`, which holds one value, a list of ranges such as `0-1`, generated from the kernel's own mask of online CPUs. The obvious alternative, counting the processor lines in `/proc/cpuinfo`, depends on a format written for people and different on every architecture; it works on `arm64`, which is why the lab's test uses it, and an answer that chooses it has to argue that portability does not matter.

c) `/proc/cmdline`, exactly what the kernel was given. The alternative, the `Kernel command line`: `line` in `dmesg`, lives in a ring buffer that a machine that has been up long enough has overwritten.

d) `/proc/interrupts`: the `arch_timer` row, in the `CPU1` column, which is one count per CPU per interrupt source. The alternative, the `intr` line of `/proc/stat`, holds one total per interrupt number summed over every CPU, with no names, so it cannot answer the question at all.

e) `/proc/iomem`, read as root: the physical address map as a tree, with each region a driver has claimed nested under the device's own region with the claimant's name. Read

as anyone else, every address in it is zero. The alternative, the `reg` properties under `/sys/firmware/devicetree/base`, says where devices are, not whether any driver has claimed them: `qa-dev`'s node is there from the first boot, and no driver claims it until Chapter 6.

f) `/proc/1/comm`, the name the kernel keeps for PID 1. While `/init` runs it is `init`, and at a `make boot` prompt it is `sh`, because the script has `exec'd /bin/sh` and PID 1 is now the shell. The alternatives are worse: `/proc/1/cmdline` holds the argument vector, which for a script names the interpreter, `/bin/sh /init`, and which a program can rewrite; and `/proc/1/exe` points at `/bin/busybox`, as it does for every process on this target.

g) `/proc/56/fd/`: one symbolic link per open descriptor, named by its number and pointing at what is open, so `ls -l /proc/56/fd` answers it, and `/proc/56/fdinfo/` adds each one's offset and flags. The alternative is `lssof`, which is a program that reads exactly this directory and formats it; BusyBox's is much reduced, and on a target without it the directory is still there.

Exercise 2.8 What a two-megabyte userland costs you

DESIGN

a) `ls -l /bin/busybox` gives 2,108,608 bytes and `busybox --list | wc -l` gives 402 (§2.1.8). The average is $2,108,608/402 \approx 5,245$ bytes, about 5 KB per command. It is misleading three ways. The applets are nothing like equal: `ash`, `awk` and `vi` are large, and `true` and `yes` are a few lines. Most of the binary is shared, BusyBox's own library and the statically linked C library that every applet uses, so no applet costs 5 KB, and removing half of them would shrink the file by much less than half. And it invites a comparison with `coreutils`, about 5 MB for about 100 commands, roughly 50 KB each, that compares different amounts of function, because BusyBox's applets take fewer options.

b) `busybox --list | grep -x` and the name prints nothing for four of the five. `strace`, `ss` and `vmstat`: absent, because no BusyBox applet exists. Each is a separate program, from `strace`, `iproute2` and `procps` respectively, that could be cross-compiled and installed. `lssof`: present, a BusyBox applet. `journalctl`: absent for the other reason: it belongs to `systemd`, this system has no `systemd`, and so there is no journal for it to read.

c) Read `/proc` directly, which is what `lssof` does anyway (§2.2.3). Find the process's PID with `ps`; `ls -l /proc/PID/fd` lists one symbolic link per open descriptor, with what it points at; `/proc/PID/fdinfo/N` gives each one's position and flags; `/proc/PID/maps` shows the files it has mapped, which are not descriptors; and `cwd` and `exe` are its working directory and its binary.

d) A difference in behaviour between the BusyBox applet you ship and the GNU tool you develop against, which a script relies on without knowing, and the silent kind is the worst. The course's example is `rmmod`: BusyBox's returns success even when the kernel refused to unload the module, so a test that checks its exit status passes on a laptop and lies on the target, and Chapter 4's lab checks `lsmod` instead (§2.1.8).

Exercise 2.9 A rate measured three ways**CROSS-CHECK**

a) The rate is the difference between the two `ctxt` readings divided by the difference between the two first fields of `/proc/uptime`. The numbers are yours. Divide by the uptime difference rather than by the pause on your watch: the uptimes time the reads, and the watch times you.

d)

- **The spread** is yours to compute, and whatever it is, `ctxt-per-s` is a property of the moment. `ctxt` counts every switch on both CPUs, and on an idle target that is dominated by whatever happened to run in that second: the kernel's housekeeping, the serial console echoing what you type, and the script's own processes. Two runs a second apart measure two different seconds.
- **The digits.** The interval comes from `/proc/uptime`, which counts in 10 ms steps, so a one-second interval is known to about 1%, and a count of some tens of switches moves by one or two percent when one more lands inside it. The run-to-run spread from c) is the real uncertainty, and it is far larger than the digit after the point. An honest presentation is a whole number, or a range from several runs, stated with the interval and the conditions: "about so many per second, idle, over one second, on QEMU".
- **Hand against script.** Expect the longer interval to be steadier: it averages bursts over more time, and the fixed errors, the 10 ms steps and the switches the reading commands cause, are a smaller fraction of it. One reason it may not hold: during a hand-timed pause you type the second command, and every keystroke on the serial console is an interrupt and a wake-up of the shell, so the hand figure includes you.
- **Without the sleep.** The rate rises, and stops describing the machine. The switches the script itself causes between its two samples, forking `cut` and `awk`, their execs, their exits and the shell's waits, are about the same number whatever the interval, so dividing them by 0.14 s instead of about 1 s inflates them roughly sevenfold, and they swamp the background. The interval is also only 14 steps of `/proc/uptime`, so it is uncertain by up to 10 ms in 140, about 7%. A short interval is worse, not better, because a rate is an average over the interval, and there is no "now" to get closer to: context switches are discrete events, and a rate at an instant does not exist.

e) `vmstat` is not there: the shell says `sh: vmstat: not found`, and the status is 127. That tells you the userland is BusyBox and nothing else: there is no `procp`s, and BusyBox has no `vmstat` applet (§2.1.8). To check the answer without it, use the number BusyBox does have under another name: `nmeter %x` prints the context-switch rate once a second, computed from the same `ctxt` counter, and `nmeter` is in this build. Your hand figure from a) is a second, independent check, the same counter read by a different route; and running your script on the host, beside the `vmstat` there, checks the method rather than the number.

f) Any two of these:

- **It is a property of a moment, not of the machine.** It is one sample of an idle target, dominated by whatever ran in that second, including the measurement itself. Under any load, or a second later, it is different, and a datasheet figure has to say under what load and by what method it was measured.
- **Its precision is invented.** Three significant figures from a one-second sample whose interval is known to 10 ms and whose run-to-run spread is far larger than 0.1 per second; one or two significant figures at most.
- **It describes an emulation.** The target is QEMU on some host, and the rate depends on that host's speed and load, and on this kernel's configuration; it says nothing about an ARM board running the same software.

A.3 Chapter 3: The Kernel: Architecture, Source Tree, and Configuration

Answers to the exercises in §3.4. Exercises 3.6 and 3.7 are Code: the first is checked by `make test L=L03`, and the second is answered with a command and a number on your own target.

Exercise 3.1 What kind of program the kernel is

RECALL

a) Any three of these (§3.1.1):

- **It starts at main and is over when main returns.** The kernel has no main and never returns; after boot it runs only when something enters it: a system call, an interrupt, a fault.
- **It has an address space that nothing else can write.** The kernel is one address space shared by all of its parts, so a bug in any driver can corrupt any subsystem.
- **A crash kills it and nothing else.** A process that faults is cleaned up by the kernel; when the kernel faults, there is nothing above it to clean up.
- **It is linked against a C library.** The kernel is built `-nostdinc` and `-ffreestanding`, against its own headers.
- **Its stack grows on demand.** The kernel's is fixed: 16 KB per task on arm64.

b) It claims that the whole kernel, core, drivers, filesystems and network stack alike, runs in one privileged address space with no protection between its parts: any of them can read and write the others' memory, and a fault in one is a fault in the kernel. It does not claim that the kernel is one indivisible piece, since modules are loaded and unloaded at run time, into that same address space. And it says nothing about how the source is organised, which is intensely modular, with a directory, maintainers and internal interfaces for each subsystem. Monolithic is a claim about the address space, not about modularity (§3.1.1); the alternative it is contrasted with is a microkernel, where drivers and filesystems run as separate processes that the hardware keeps apart.

c) An oops. The fault handler finds no fixup for the address, prints Unable to handle kernel NULL pointer dereference at virtual address 0000000000000000 with a register dump and a backtrace, taints the kernel, and kills the task that was running. If there is no task to blame, because the fault was in an interrupt handler, it panics instead, and it panics anyway if `panic_on_oops` is set. The difference is structural. A process's fault is handled by a separate program that is still intact, the kernel, which owns everything the process held and can take all of it back. When the kernel faults, the code that would clean up is the code that faulted, in the address space that may now be corrupt, and nobody is above it. Killing the task does not release the locks it held or finish the update it was halfway through, so even an oops the machine survives leaves it in an unknown state. Better testing means fewer faults; it cannot give the kernel a supervisor.

d) `printf` belongs to the C library, and the C library is built on top of the kernel: `printf` formats into a buffer and hands it to `write()`, which is a system call, a request to the kernel. The kernel cannot make that request of itself, and it is built without any C library at all. What it has instead is `printk`, which formats into the kernel's own log buffer, the one `dmesg` reads, and out to the console, carries a log level, and can be called from any context, an interrupt handler included (§4.1.7).

Exercise 3.2 The two interfaces

RECALL

a) The system call interface is stable forever: a change that makes a working userspace program stop working is reverted, however correct it was. The internal API has no guarantee at all: function signatures, structures and whole subsystems may change in any release (§3.1.2).

b) **From the stable system call interface:** the userspace half of your product, the programs that talk to your driver through `open`, `read` and `ioctl` and the rest of the userland around them, keeps working across kernel upgrades and never needs rebuilding for one. The same rule applies to what your driver adds: an `ioctl` or a `sysfs` file you ship is expected to stay as it is for good. **From the unstable internal API:** every new kernel may break your driver's build, or worse, keep it building and change what a function it calls means. You rebuild and retest against every kernel you support, port the driver whenever an interface it uses moves, and hear about the change from the compiler, because nobody making it knows your driver exists.

c) A stable internal ABI would freeze the kernel's internal design, mistakes included, for good. The kernel improves, and fixes its security bugs, by changing an internal interface and every caller of it in the same patch; with binary drivers depending on the old one, nothing they touch could change again. And a binary interface is not even one thing to freeze: the layout of the same structure differs with the compiler version, the configuration (SMP, the preemption model, debugging options) and the architecture. The file

is `Documentation/process/stable-api-nonsense.rst`, which makes both arguments and ends with the answer the project prefers: put the driver in the tree.

d) Whoever changes the API. The tree must build at every commit, so a patch that changes an internal interface converts every caller in the tree in the same series, usually mechanically and often with Coccinelle, or the maintainer will not take it. Your driver is one of those callers, and you are typically asked to review the change rather than to make it. That is the maintenance cost an out-of-tree driver pays and an in-tree one does not.

Exercise 3.3 Reading Kconfig

HAND CALCULATION

Both entries sit inside the `if GPIOLIB` block of `drivers/gpio/Kconfig`, so all of this assumes `CONFIG_GPIOLIB=y`, as `kernel/qa.config` sets it.

a) No `GPIO_SYSFS` line at all, and `CONFIG_GPIO_CDEV=y`. With `EXPERT` off, `GPIO_SYSFS`'s prompt is invisible, and a value from a fragment is only ever applied to a symbol that has a visible prompt, so the `y` is ignored. The symbol has no default either, so its value is `n`, and a symbol that is neither visible nor given a default is not written to the `.config`: not even as `# CONFIG_GPIO_SYSFS` is not set. `GPIO_CDEV` keeps its default `y`. The course's `ci/kernel.sh` notices the line that vanished and prints warning: `CONFIG_GPIO_SYSFS=y` did not survive `olddefconfig` (§3.2.2).

b) `CONFIG_EXPERT=y`, `CONFIG_GPIO_SYSFS=y` and `CONFIG_GPIO_CDEV=y`. The prompt is now visible, its depends on `SYSFS` is met, so the fragment's `y` is taken; and the `select` forces `GPIO_CDEV` to `y`, which it already was by default, but now it can no longer be turned off while `GPIO_SYSFS` is on. `EXPERT` uncovers many other prompts too, and `olddefconfig` answers each of them with its default.

c) Yes, `CONFIG_GPIO_CDEV=y`. A symbol whose prompt is invisible is never asked about, but it is still evaluated: its value comes from its default, here `y`, and because it has one it is written out. Visibility decides whether you are asked; the value of a symbol nobody was asked about comes from its default, or from a `select` by a symbol that is on. That is why this course's `.config` has `CONFIG_GPIO_CDEV=y` with `EXPERT` off.

d) depends on B is a precondition: until B holds, the symbol is hidden and held at `n`. `select B` is an order: while the selecting symbol is on, B is forced on, without looking at B's own dependencies. **select** is the one that can break a depends on: if A selects B and B depends on C, then turning on A with C off forces B on anyway, and the `.config` holds a B whose depends on C is false. Kconfig prints `WARNING: unmet direct dependencies detected for B`, and the build may fail inside B's code, which was written on the assumption that C is there. That is also how an option nobody selected, with an unsatisfied depends on, ends up in a `.config`.

e) `CONFIG_GPIO_CDEV`, and it works, although the line is not what makes it work: with `EXPERT` off the symbol has no prompt, so the fragment's value is ignored exactly as in a), and it is `y` anyway because that is its default. The line survives `olddefconfig` only because

it agrees with the default; the check in `ci/kernel.sh` passes, and that is what this course's `kernel/qa.config` does. The difference shows the other way round: `# CONFIG_GPIO_CDEV` is not set would not take effect either, so with `EXPERT` off you cannot turn it off at all. (For a `/dev/gpiochipN` to appear there must also be a GPIO controller to represent; this target has one, the PL061.)

Exercise 3.4 Built in or module

HAND CALCULATION

- a) Must be `=y`: with no `initramfs` the kernel mounts the root filesystem itself, and a module lives on the root filesystem it would be needed to reach.
- b) May be `=m`: the `initramfs` is a root filesystem in RAM that the bootloader loads with the kernel, and its `init` loads the driver from there before mounting the real root.
- c) Cannot be `=m`: a `bool` has only the values `y` and `n`, so it is `=y` if anything on the system uses GPIO and `=n` otherwise.
- d) May be `=m`: it is needed only while the device is plugged in, it is loaded then through its module alias, and until then it costs nothing.
- e) Must be `=y`, for the same reason as a): the kernel has to read the root filesystem before it can load any module, so the code that reads it cannot be one.
- f) Cannot be `=m`: `PREEMPT_RT` is a `bool` in the preemption-model choice and changes how the whole kernel is compiled, turning spinlocks into sleeping locks and interrupt handlers into threads, which is not something that can be loaded later. It is `=y` only in `build/kernel-rt`, and only because `kernel/rt.config` turns `EXPERT` on as well.

Exercise 3.5 Navigating the tree

DESIGN

- a) `kernel/sched/`: `core.c` holds `__schedule()` and `pick_next_task()`, which asks each scheduling class in turn; `fair.c` is the class almost every task is in. If the name is not known: `grep -rn "pick_next_task" kernel/sched/`.
- b) `drivers/hwmon/`: a temperature sensor is a hardware monitor, and most I2C ones are there (`lm75.c`, `tmp102.c`). One used for measurement rather than monitoring may be an IIO driver in `drivers/iio/temperature/`. Search by the chip's name, which is what the datasheet gives you: `grep -rli "tmp102" drivers/`.
- c) `include/linux/`: it is in `fs.h`, at line 2062 of the pinned tree. To find the definition among thousands of uses, anchor the search on the opening line: `grep -rn "^struct file_operations {" include/`.
- d) `arch/arm64/kernel/`: `entry.S` holds the exception vector table (vectors) and the `kernel_ventry` macro, and `entry-common.c` the C handlers it calls, such as `el0t_64_`

sync_handler for a system call or a fault from userspace. Search: `grep -rn "SYM_CODE_START(vectors)" arch/arm64/`.

e) `fs/`: `open.c` defines it as `SYSCALL_DEFINE4(openat, ...)`, at line 1441, and it calls `do_sys_open()`. Search for the macro, because the name `sys_openat` is generated by it and appears in no definition: `grep -rn "SYSCALL_DEFINE.(openat," fs/`.

f) `kernel/`, because it is about process 1 exiting, which is process management. `grep -rn "Attempted to kill init" kernel/` finds `kernel/exit.c`, line 908, in a call to `panic()`. The format string goes on `exitcode=0x%08x`, so search for the fixed words and never for the number the message printed.

Exercise 3.8 A configuration for a real product

DESIGN

a) Seven components, seven decisions:

- **The SoC: =y.** Its platform symbol, interrupt controller, clocks, pin control and timer are needed before anything else can run, and most of them are `bool` anyway. Every other SoC family that `defconfig` enables is `=n`.
- **The 256 MB of RAM: nothing to choose.** Memory management is core kernel, always built in. What the size decides is how much you can afford to leave switched on, not whether anything is a module.
- **Ethernet: =y.** It is used on every boot, and the field updates of the sensor driver presumably arrive over it; as a module it would only add a step that can fail. `=m` is defensible if image size is tight, since nothing needs the network before userspace; that argument has to show the space matters.
- **The eMMC: =y,** meaning the MMC core, the SoC's host controller driver and the MMC block driver: the root filesystem is on it and there is no `initramfs` to load a module from (Exercise 3.4a)).
- **ext4: =y,** for the same reason (Exercise 3.4e)).
- **The I2C sensor: =m.** Replacing a driver without a reboot is only possible for a module: install the new `.ko`, `rmmod` the old one, load the new. The I2C core and the SoC's I2C controller driver are `=y`: they are not the part that is updated, and the sensor module needs them.
- **The watchdog: =y.** If U-Boot has already started it, the kernel has to keep it fed from early boot until the userspace daemon takes over, which a module loaded later cannot do; and the driver is small.

b) The ones on the path to the root filesystem: the eMMC driver, including anything its host controller needs from the SoC such as clocks and pin control, and `ext4`. Either one as `=m` gives the same symptom: the kernel boots, runs its initialisation, and cannot mount the root filesystem. With no eMMC driver there is no block device, so it prints `VFS: Cannot open root device`, lists the partitions it can see, which are none, and stops with `Kernel panic -`

not syncing: VFS: Unable to mount root fs on unknown-block(0,0). With the device present but no ext4, it prints No filesystem could mount root, tried: and the types it does have, and panics with the same message naming the device it found. One more decision stops the boot if U-Boot has started the watchdog: with the driver =n, or =m and loaded too late, nothing feeds it, and the board resets partway through every boot.

c) Keep one kernel for both variants. Build both sensor drivers as modules, which they are already, and let each variant's device tree say which sensor is fitted, so the right driver is loaded by its compatible string; the variants then differ in their device tree, not in their kernel. savedefconfig is how the product's configuration is stored: after any change, it writes a defconfig holding only the choices that differ from the defaults, which is small enough to review in a commit, readable in a diff, and brought up to date by olddefconfig when the kernel moves on. If a variant ever does need a different kernel option, keep it as a fragment merged over that base, the way kernel/rt.config sits over this course's configuration, rather than as a second full .config.

d) The configuration: how many modules it builds (modules.order has one line per module) and which hardware it builds them for. Expect to find a vendor or architecture defconfig building for every board anyone upstreamed: dozens of SoC families with their clock, pin control, DMA and PHY drivers, graphics, sound and wireless, thousands of modules that can never load on the product. Turning those off is what took this course's build from about an hour and roughly two thousand modules to eighteen minutes and 458 (§3.2.7). The same file turns off DWARF debugging information, which by its own comment roughly doubles the build time.

Exercise 3.9 What one config option costs

CROSS-CHECK

a) $76 + 34,048 + 10 + 4 = 34,138$ bytes, from .text, .rodata, .rodata.str1.8 and .initcall6.init (§3.2.6). The listing's Total, 34,532, also counts the metadata and the 148 bytes of __init and __exit code.

b) They should not be identical, and each difference is accounted for exactly. **configs.o's 34,048** is more than the compressed file: kernel/configs.c wraps the blob in two 8-byte markers, IKCFG_ST and IKCFG_ED, which scripts/extract-ikconfig searches for, and the same section holds the 96-byte proc_ops table behind /proc/config.gz; $33,936 + 16 + 96 = 34,048$. **The host's 33,944** includes the file name: gzip records .config and its terminating zero byte, 8 bytes, in the header, while the kernel's build compresses with gzip -n, which leaves the name out; $33,936 + 8 = 33,944$. **The target's 33,936** is the blob and nothing else, because ikconfig_init sets the file's size to the distance between the two symbols that bracket it; it is the size of build/kernel/kernel/config_data.gz.

c) The course's baseline, published in §3.1.8: Image 24,943,104 bytes, vmlinux 31,468,440 bytes, and among the sections .text at 13,021,184 and .rodata at 4,369,606.

e) The course measured two of the three and published them, drawn in Figure A.1: `vmlinux`'s section total fell by 41,208 bytes, and the Image by 65,536. It did not publish the change in the `vmlinux` file; f) says what decides it.

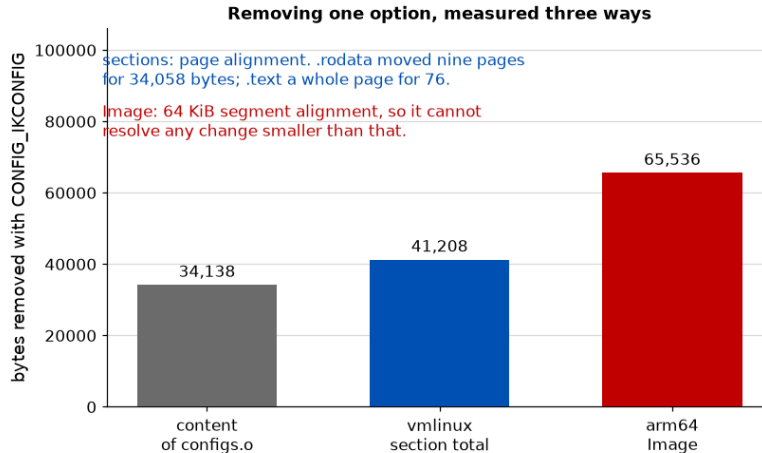


Figure A.1 The one-line change of Exercise 3.9, measured three ways on the course's target: 34,138 bytes of content in `configs.o`, 41,208 bytes of `vmlinux` sections, 65,536 bytes of Image. The first gap is page alignment and the second 64 KiB segment alignment.

f) Each of the four:

- **Closest and furthest.** The section total is closest, at $41,208/34,138 = 1.21$ times the prediction. The Image is $65,536/34,138 = 1.92$ times it, and the `vmlinux` file is further still. `readelf -l` on `build/kernel/vmlinux` shows why: the file's loadable part is the Image byte for byte, starting 64 KiB into the file, and the symbol and string tables follow it. So the file shrinks by the Image's 65,536 plus 24 bytes of `.symtab` for each symbol `configs.o` defined and the bytes of their names in `.strtab`: a little more than the Image, and the furthest of the three, by a factor of just over 1.9.
- **The per-section deltas.** `.rodata` fell by 36,864 bytes for the 34,058 that `configs.o` put there ($34,048 + 10$), and `.text` by 4,096 for its 76. Each delta is a whole number of 4,096-byte pages, the content rounded up to the page: $34,058/4,096 = 8.3$, so nine pages, 36,864; 76 bytes, one page. Both sections hold page-aligned material behind `configs.o`'s bytes (in `.text`, the hypervisor code, which starts and ends on a page), so removing anything moves it down in whole pages. Which way the rounding goes depends on how much padding was already in front of the boundary: 76 bytes moved `.text` by a page only because the code before the boundary spilled less than 76 bytes into its last page; on another build the same 76 bytes could move it by nothing. The other three deltas are small: `.init.text` fell by 112 for `configs.o`'s 108, rounded to 8-byte alignment; `.exit.text` by exactly its 40; and `.rela.dyn` by 96, four 24-byte relocation records the

linker kept for addresses stored in `configs.o`'s data, which the kernel patches when it relocates itself at boot. $36,864 + 4,096 + 112 + 96 + 40 = 41,208$.

- **The Image delta** is $65,536 = 64$ KiB exactly, one unit of `SEGMENT_ALIGN`, which is `SZ_64K` in `arch/arm64/include/asm/memory.h`. The linker script aligns the end of the text, the start of the init segment, the boundary between init text and init data, and the end of the init segment to it. So the Image is a row of segments, each starting on a 64 KiB boundary with padding in front of it, which lets each be mapped with its own permissions using the largest mappings the hardware offers. The Image can therefore only change size in 64 KiB steps, and a step happens when some segment's contents cross a boundary.
- **A second 34 KB option** would not necessarily shrink the Image by 64 KiB again. About 41 KB of sections is less than one step, so it moves the Image by 65,536 only if it takes a segment's contents back across a boundary, which depends on how much slack the first removal left in front of each one: it could be 65,536 again, or nothing. Over many options the Image shrinks by roughly the sum of what they contain; any single measurement is quantised.

g) A report that handles both worlds passes again, with seventeen checks: `run.sh` says the configuration is available as `inferred`, and the report reads `config-source: inferred` and `hz: unknown`.

h) "About 34 KB: the compressed copy of the configuration and the code that serves it as `/proc/config.gz`, which the kernel keeps in memory for as long as it runs." That is the content, 34,138 bytes, and it is the one to quote because it is the option's own cost and the same on any build. The other two measure padding as well, which belongs to wherever the boundaries happened to fall on this build: the same option could cost 64 KiB of Image here and nothing on the next build. If the colleague is fitting this particular Image into a flash partition, 64 KiB is what they pay on this build; then say that it is a number about this build, not about the option.

A.4 Chapter 4: Kernel Modules

Answers to the exercises in §4.3. Exercises 4.6 and 4.7 are Code, and are checked on the target as their Check yourself lines say.

Exercise 4.1 A module is not a program

RECALL

a) Nothing of the module's is running: its init function has returned and it has no thread of its own. What is resident is its core memory: its code, including the exit function that `rmmod` will call, its read-only data and parameter table, and its writable data, including its `struct module`; with them, its directory under `/sys/module`. Its `__init` code has already been freed. It waits to be called by something else.

b) Because `insmod` is the process that called `init_module()`, and the kernel runs your init function inside that call: `insmod` returns when `init` does. A `while (1)` never returns, so neither does `insmod`. It sits in the system call, where a signal cannot reach it, because a signal is acted on only on the way back to userspace; and the module stays in the Loading state in `/proc/modules`, where `rmmod` refuses it. A CPU spins for good, and the cure is a reboot. What you should have written is registration: a kernel thread started with `kthread_run()` that loops until `kthread_should_stop()`, or a timer or a work item if the work is periodic, started in `init` and stopped in `exit`.

c) They are section attributes, and the linker and the loader act on them. `__init` places a function in `.init.text`, and `__initdata` a variable in `.init.data`: for code built into the kernel those sections are freed at the end of boot, and in a module the loader puts them in separate init memory and frees it as soon as the init function has returned. `__exit` places a function in `.exit.text`: a module keeps it, because `rmmod` will call it, and a built-in one can never run it, so it is thrown away, at link time on most architectures and with the init code on arm64 (§4.1.1). The consequence is that calling an `__init` function after `init`, or keeping a pointer into `__initdata`, is a use after free, and `modpost` warns of a section mismatch when ordinary code refers to init code.

d) It is the kernel freeing what `__init` and `__initdata` set aside. Once the last `initcall` has run, just before it starts `/init`, the kernel releases its whole init segment, 4,672 KB on this build, the stretch from `__init_begin` to `__init_end` (§3.1.8). On arm64 most of that stretch is not `__init` code: 3.3 MB of it is the relocation table the kernel applies to itself at boot, and the `__init` code and data are about 1 MB. The loader does the same for each module, on a small scale, when its init function returns (Exercise 4.9f).

e) `MODULE_LICENSE`, `MODULE_AUTHOR`, `MODULE_DESCRIPTION` and `MODULE_VERSION`. `MODULE_LICENSE` is the one that is not optional: without it `modpost` stops the build with `ERROR: modpost: missing MODULE_LICENSE() in`, followed by the object's path, and no `.ko` is produced. A module that did load with no licence would be treated as unspecified, which is not GPL-compatible: it would taint the kernel and see no GPL-only symbol.

Exercise 4.2 Four commands

RECALL

a) `insmod` loads exactly the file it is given, with any parameters on its command line, and resolves nothing. `modprobe` takes a module name, looks it up in the dependency file under `/lib/modules/$(uname -r)`, and loads what it depends on first and then the module. `depmod` scans the modules under `/lib/modules/$(uname -r)`, reads which symbols each one needs and which it exports, and writes that dependency file. `rmmod` asks the kernel to unload a module by name, which the kernel refuses while anything holds it.

b) `insmod` resolves nothing and `modprobe` resolves dependencies, from a file `depmod` wrote. The experiment: with `qa_export` not loaded, `insmod ./qa_user.ko` fails; the kernel logs `qa_user: Unknown symbol qa_answer (err -2)` and `insmod` prints `can't insert './qa_user.ko': unknown symbol in module or invalid parameter (§4.1.4)`. Then copy both modules under `/lib/modules/$(uname -r)/`, run `depmod`, and run `modprobe qa_user`: `qa_export` is loaded first, then `qa_user`, and `lsmod` shows both, with `qa_user` using `qa_export`.

c) `modules.dep`, in `/lib/modules/$(uname -r)/`, which `depmod` generates; `make modules_install` runs `depmod` for you (§3.2.4). BusyBox's simplified `modprobe`, which is the one on this target, keeps its own version beside it, `modules.dep.bb`, and generates that itself when it is missing.

d) `/sys/module/qa_export/holders/`, which holds a link named `qa_user`. `/sys/module/qa_export/refcnt` gives only the count, 1.

Exercise 4.3 Reading vermagic

HAND CALCULATION

a) `include/linux/vermagic.h` builds the string from six parts, each an assertion about the kernel:

- 6.12.30: the kernel release it was built against.
- SMP: built with `CONFIG_SMP`, so its locking and per-CPU code assume more than one CPU.
- preempt: built for the preemptible-kernel model (`CONFIG_PREEMPT_BUILD`, which `CONFIG_PREEMPT` selects); a real-time kernel says `preempt_rt`, and a non-preemptible one says nothing.
- mod_unload: built with `CONFIG_MODULE_UNLOAD`, so modules can be unloaded and are reference-counted, which changes `struct module` and what the module's own code does.
- modversions: built with `CONFIG_MODVERSIONS`, so the module carries a checksum for every symbol it imports and the kernel checks them.
- aarch64: the architecture.

b) Each of the four:

- **Same source, no configuration change: it loads.** The vermagic string and every symbol checksum come out the same.

- **# CONFIG_SMP is not set: it loads, because nothing changed.** `arch/arm64/Kconfig` declares SMP as `def_bool y`, with no prompt, so the line cannot take effect: `olddefconfig` keeps `CONFIG_SMP=y`, and the kernel is still SMP. Not even `ci/kernel.sh` says so, because it checks only the `CONFIG_X=` lines of a fragment, not the lines that turn something off.
- **6.12.30 to 6.12.31: it loads, unless one of the interfaces it uses changed.** With `modversions` the module has a table of checksums, and in that case the loader does not compare the release at all: `same_magic()` in `kernel/module/version.c` compares the two strings from the first space onwards. The rest of the string matches, so the decision falls to the checksums: of `module_layout`, which covers `struct module` and the structures around it, and of every symbol the module imports. A stable update rarely changes one; if it did, the loader prints `qa_hello: disagrees about version of symbol` and the symbol's name, and refuses. On a kernel without `CONFIG_MODVERSIONS` the release would be compared, and the module refused.
- **PREEMPT_RT instead of PREEMPT: it is refused.** The real-time kernel's string is `6.12.30 SMP preempt_rt mod_unload modversions aarch64`, and the comparison fails at `preempt` against `preempt_rt` (§4.1.3). It must: under `PREEMPT_RT` a `spinlock_t` is a sleeping lock with a different layout, and a module compiled for the other kernel would use every lock it touches wrongly.

c) A checksum for every exported symbol, computed from the symbol's full type: its prototype and, recursively, the layout of every structure it passes. The kernel records the checksums of what it exports, and each module records, in its `__versions` section, the checksum it was built against for each symbol it imports; the loader compares them one by one. `vermagic` alone names a release and a few options, and cannot see a structure that changed shape while the string stayed the same, through a configuration option that adds a field, a vendor patch, or a rebuild with a different configuration and the same version number. The checksums check the interface the module actually uses, which is also what lets the loader stop comparing the release, so a module survives an update that did not touch anything it uses.

d) Because the failure it prevents is silent memory corruption in the one address space everything shares. A module built for other structure layouts reads and writes fields at the wrong offsets, passes the wrong arguments and takes locks of the wrong size; nothing detects any of that when it happens, and the damage surfaces later, somewhere else, as a crash or wrong data in code that has nothing to do with the module (§3.1.1). A warning would scroll out of the log and leave the corruption behind. A refusal costs a rebuild; the alternative costs a crash nobody can trace, possibly in the field.

Exercise 4.4 The licence, at two stages

HAND CALCULATION

a) The build fails at modpost, the step that runs after the objects are linked and before the .ko is finished. It sees qa_export's GPL-only export and the proprietary module's use of it, and stops with this, naming whatever your module is called:

```
ERROR: modpost: GPL-incompatible module qa_proprietary.ko uses GPL-only symbol '
↳ qa_answer'
```

No .ko is produced.

b) The build still fails at modpost, but for an unresolved symbol:

```
ERROR: modpost: "qa_answer" [.../qa_proprietary.ko] undefined!
```

The message is different because modpost can only apply the licence rule to an export it has seen. Here qa_answer is in neither the kernel's Module.symvers nor any module in the build, so as far as modpost knows the symbol does not exist, let alone that it is GPL-only. Handing it the exporter's Module.symvers through KBUILD_EXTRA_SYMBOLS brings back the message of a).

c) Whenever modpost could not see that the symbol is GPL-only. Either the module was built as in b) with KBUILD_MODPOST_WARN=1, which turns the unresolved symbol into a warning and lets the .ko be written; or it was built against a kernel, or an exporter's Module.symvers, in which the symbol was plain EXPORT_SYMBOL, and is then loaded where it has since become EXPORT_SYMBOL_GPL. The second is the one that happens on kernel upgrades.

d) The kernel logs qa_proprietary: Unknown symbol qa_answer (err -2), and BusyBox's insmod prints unknown symbol in module or invalid parameter. No licence is mentioned because the loader never refuses on grounds of licence. It checks the licence first, and marks the module as proprietary; the symbol lookup then runs with gplok false, and find_exported_symbol_in_section() skips every GPL-only table before searching it (§4.1.5). The lookup fails exactly as it would for a symbol that is not exported at all, with -ENOENT, and the loader reports, accurately, what it found: nothing.

e) It arrives last, at load time on the target, possibly in the field after an upgrade, rather than at build time on your own machine. It is word for word the message for a missing dependency or a typo, so it sends you after the wrong causes. And the symbol is plainly in /proc/kallsyms, so the message looks false. Diagnosing it requires already knowing the mechanism. The first message is early and exact, the second early and at least says what is missing; this one is late and points the wrong way.

Exercise 4.5 Choosing a log level

DESIGN

- a) `pr_info`, or `dev_info` once there is a device: one line, once per device, that says it is there and what it found is useful to whoever reads the log of a system that misbehaves. The alternative, `pr_debug`, is what many maintainers prefer, on the grounds that a working driver should be silent; that argument rests on a log that holds only what needs attention.
- b) `pr_warn`: something unexpected that the driver handled. The device misbehaved and somebody may want to know, but the driver carries on. If it can happen on every sample, use `pr_warn_ratelimited`, or it floods the log.
- c) `pr_err`: an error the driver could not handle. The probe fails and the device will not work, so say why, with the error code.
- d) `pr_debug`: only useful during development, compiled to nothing unless dynamic debug is built in, and silent until someone switches it on even then.
- e) `pr_warn`: the system works, but not as the device tree describes it, and the device tree is what somebody has to fix.
- f) **The binary:** a `pr_info` you comment out is gone from the build, and getting it back means editing and rebuilding. A `pr_debug` stays in the binary at the cost of a small descriptor per call site and a branch that is patched to a no-op until enabled, and with `CONFIG_DYNAMIC_DEBUG` off it compiles to nothing at all. **The field:** on a shipped kernel with dynamic debug, those call sites can be switched on at run time on the machine that has the problem, for one module, file, function or line, with `echo 'module qa_hello +p' > /sys/kernel/debug/dynamic_debug/control`, without rebuilding or rebooting. You get the debugging output you wrote, from the system that needs it.

Exercise 4.8 A test that cannot fail

DESIGN

- a) 0. The kernel did refuse, `delete_module()` failing with `EWOULDBLOCK`, which is `EAGAIN`, because `qa_export` has a user; and `rmmod` said so, `rmmod: remove 'qa_export': Resource temporarily unavailable`. But this target's BusyBox is built with its simplified module tools, whose `rmmod` reports the failure and then exits with success whatever happened.
- b) The obvious test is `if rmmod qa_export; then fail`. It sees 0 on both kernels, so it concludes that the module was unloaded on both: a failure reported against the kernel that correctly refused, and the same failure against the one that did not. Its verdict does not depend on the kernel at all; what it measures is BusyBox. The same exit status, checked by a test that expects an unload to succeed, passes on both, whether anything was unloaded or not, and that is the test that cannot fail.
- c) Assert on the state you care about, not on a tool's report of it; and do not trust a test you have never seen fail. The lab's check does both: it looks at `lsmod`, the state, and the

refusal it expects is one it can observe. From §2.1.8: BusyBox's `find` has no `-printf`. A check written on a laptop with GNU `find`, which decides that a module is gone because `find /sys/module -name qa_export -printf x` prints nothing, passes on the target whether the module is there or not, because BusyBox's `find` rejects the option and prints nothing on its standard output either way. It is the same class of problem: the check tests the tool, which behaves differently on the machine it was written on and the one it runs on.

Exercise 4.9 How big is a module

CROSS-CHECK

a), b) and d) are measurements of your own module, and the course publishes no figures for them; what follows is what decides each number.

c) The prediction is the sum from b): the module's code, its read-only data and strings, its parameter table, the `.note` sections, and its writable data, including the `.gnu.linkonce.this_module` section that holds its `struct module`, which on this kernel is over a kilobyte by itself.

e) Each of the six:

- **The factor** is about five by the exercise's own figure, and the rest of this part is why: the loader rounds the module up in whole pages, not in bytes.
- **A multiple of 4,096**, the page size, because `module_memory_alloc()` rounds each memory type up to whole pages with `PAGE_ALIGN` before allocating it. The count is the number of core memory types the module has anything in, one page each as long as none of them holds more than 4,096 bytes, which for a module this small is at most four.
- **What would have to differ:** one module has something in a memory type the other has nothing in, or more than a page in one type. Between `qa_hello` and `qa_export` the usual difference is `qa_hello`'s `pr_debug`. With `CONFIG_JUMP_LABEL`, which this kernel has, each `pr_debug` call site has an entry in `__jump_table`, which the loader treats as read-only after `init`, a type of its own; a `qa_export` with no `pr_debug` has nothing of that type and needs a page fewer. Exporting does not add a type: the export tables are read-only data, which both modules already have.
- **The groups:** one per memory type the module uses. Sections of a type sit together at small offsets: code (`.text`, `.exit.text`), read-only data (`.rodata`, `__param`, the `.note` sections), read-only after `init` (`__jump_table`), and writable data (`.data`, `.bss`, `.gnu.linkonce.this_module`). The `init` sections are listed too, at addresses that have already been freed. Between groups the addresses jump, because each type is a separate allocation from the module area, and `vmalloc` leaves an unmapped guard page after every allocation; where two sit next to each other, a one-page group is followed by the next two pages further on. A module is not one block but one allocation per memory type, each a whole number of pages.
- **enum `mod_mem_type`** has seven entries before `MOD_MEM_NUM_TYPES`: four core types, `MOD_TEXT`, `MOD_DATA`, `MOD_RODATA` and `MOD_RO_AFTER_INIT`, and three `init` types. So the core size

of a small module is 4,096 times the number of core types it uses, four pages at most. The allocation is `ptr = execmem_alloc(execmem_type, size);` in `module_memory_alloc()`, with `size` set by `PAGE_ALIGN` two lines earlier.

- **Why:** permissions are set per page. Code must be executable and read-only, read-only data neither writable nor executable, writable data not executable, and read-only-after-init data writable until init finishes and read-only after. A page holding two of them would need two permissions at once, which would leave code writable or data executable, exactly what `CONFIG_STRICT_MODULE_RWX` forbids; the loader sets them type by type in `kernel/module/strict_rwx.c`. The kernel's own image is laid out in separately aligned segments for the same reason (Exercise 3.9).

f) The init sections held `qa_hello_init` and anything marked `__initdata`. Once the init function has returned, the loader frees them and sets the size of each init type to zero, and `initsize` is the sum of those sizes. It is Exercise 4.1d) in miniature: the same annotation and the same freeing, done for one module by the loader after its init rather than once by the kernel at the end of boot.

g) The `coresize`, which is also the size in `/proc/modules`: “the driver occupies this many pages of kernel memory while it is loaded”. It is what the kernel actually allocated and keeps; the file includes symbol tables and metadata that are never loaded, and the sum of the sections leaves out the rounding to pages. With a 64 KB `kmalloc` in init, the answer is the `coresize` plus 65,536 bytes: 64 KiB is above the largest `kmalloc` cache, which is 8 KiB with 4 KiB pages, so it comes straight from the page allocator as sixteen pages, with no rounding. None of the three numbers changes, apart from the few bytes of code the call adds, because all three describe the module's own image: memory it allocates at run time belongs to the allocator, and appears nowhere under `/sys/module`.

A.5 Chapter 5: Character Device Drivers

Answers to the exercises in §5.4. Exercises 5.6 and 5.7 are Code, and are checked on the target as their Check yourself lines say.

Exercise 5.1 The node and the number

RECALL

- a) The number is whatever was free when the driver loaded, so the script breaks in two separate ways. **The driver moves:** a new module, a different load order or a kernel update that registers another dynamically numbered driver first, and `qa_fifo` gets a different major; the script then fails to find its device. **Something else takes 243:** after the same kind of change another driver can be handed 243, and the script opens that driver's device and talks to it without any error at all. The second is the worse of the two, because nothing fails.
- b) It creates a second name for the same device: a directory entry of type character with major 243 and minor 0, written without consulting anything (§5.1.1). Opening it opens exactly what opening `/dev/qa_fifo` opens, the same driver and the same FIFO, so bytes written through one name come out of the other.
- c) Step 5, `device_create`. `/proc/devices` lists the name as soon as the number is allocated (step 2), but the node is made by `devtmpfs` in response to the uevent that `device_create` emits (§5.1.6); no device, no uevent, no node. If `class_create` was skipped as well, there was no class to create the device in.
- d) `ENXIO` comes from the kernel's lookup of a registered `cdev` for the node's major and minor, before any driver is involved: nothing is registered for that pair. Either the driver is not loaded, or it allocated a number but never called `cdev_add`, or the node is stale and carries numbers from an earlier boot. It rules out a missing node (`ENOENT`), a permissions problem (`EACCES`), and anything in your own `open`, which was never called.

Exercise 5.2 The table

RECALL

- a) `.owner` is what makes `open` take a reference on the module, and the reference is what makes `rmmod` refuse with "Module is in use". Without it the module's use count stays at zero while a process holds an open file whose `f_op` points at the module's `file_operations` table. `rmmod` succeeds and frees the module's code and data, table included. The process's next `read`, `write` or `close` calls through a pointer into freed memory: an oops if you are lucky, and whatever has been loaded there since if you are not.
- b) Not a bug, as long as there is no per-open state to free, which is the case here (the table in §5.2.2 marks `.release` as not required). Nothing is called instead: when the last reference to the file goes, the VFS calls `.release` only if it is set, and otherwise simply drops its own reference on the module and frees the `struct file`.
- c) Yes. A missing `.open` is not a refusal: the character device layer calls the driver's `open` only if there is one, and otherwise the `open` succeeds and returns a descriptor.
- d) The device is live. Any process that opens a node with your numbers, and a node can exist before `device_create` makes one, can call your `open`, `read` and `write` while the rest of your `init` function is still running, and on another CPU at the same time. The rule: **everything**

a callback touches must be fully set up before the call that publishes it, so the FIFO is created before `cdev_add`, never after (§5.2.2).

e) It is called when the last reference to the open file goes, which is when the last descriptor referring to it is closed. The simplest case: a process opens the device and then calls `fork`. Parent and child now hold descriptors for the same open file. The parent's `close` drops one reference and calls nothing; the child's `close` drops the last and calls `release`. `dup` followed by two `closes` in one process does the same.

Exercise 5.3 Return values

HAND CALCULATION

- a) 40. `read` returns 40, the buffer holds the 40 bytes, and the FIFO is now empty. A short count is a success.
- b) `-EAGAIN`. `read` returns `-1` with `errno` set to `EAGAIN`, “Resource temporarily unavailable”.
- c) 0. A count of 0 returns 0 and consumes nothing, so the level stays 40. The program sees the same 0 that means end of file everywhere else, which is harmless only because it asked for nothing.
- d) 60. The first 60 bytes are accepted and the FIFO is full; the program sees 60 and must write the other 40 later.
- e) `-ENOSPC`. `write` returns `-1` with `errno` set to `ENOSPC`, “No space left on device”.
- f) `-EFAULT`, provided there are bytes to deliver: the driver takes them from the FIFO into its bounce buffer, `copy_to_user` fails on the unmapped page, and `read` returns `-1` with `errno` set to `EFAULT`, “Bad address”. Note what the specification in §5.2.2 does with the bytes it had already taken: they are gone, consumed and never delivered. (With the FIFO empty the driver returns `-EAGAIN` before it touches the pointer.)
- g) `cat` takes the 0 as end of file and exits with status 0, having printed nothing. The loop ends at once, exactly as at the end of a file, and the program carries on as if the device had finished. The `per`ror check never fires, because 0 is not less than 0. **None of the three reports a problem**, which is why returning 0 for “nothing yet” is a silent bug.

Exercise 5.4 A sequence of reads

HAND CALCULATION

a)

Step	Call	Returns	Level after
1	write(fd, "abcdefghij", 10)	8	8
2	read(fd, buf, 3)	3	5
3	write(fd, "klm", 3)	3	8
4	read(fd, buf, 100)	8	0
5	read(fd, buf, 100)	-EAGAIN	0

Step 1 is a short write: abcdefgh goes in and ij is refused. Step 5 is -EAGAIN and not 0, because the FIFO is empty rather than finished.

b) defghklm: the five bytes step 2 left behind, then the three step 3 added, oldest first. Step 4 overwrites the first eight bytes of buf, so the abc step 2 put there is gone.

c) Room is decided by the level, not by how much has ever been written. Step 2 took three bytes out, leaving three slots free, and step 3 asks for exactly three; in a ring those are the slots the first bytes occupied, reused by a write that wraps around the end of the storage.

d) A FIFO of capacity 8 that wastes a slot holds 7, and two steps change. **Step 1** returns 7 instead of 8, because only seven bytes fit. **Step 4** then returns 7 instead of 8, because there are only seven to take, and buf holds defghklm: the h was never stored. Steps 2, 3 and 5 return what they did before, 3, 3 and -EAGAIN, although the levels after steps 1 to 3 are each one lower (7, 4, 7). This is the implementation that `qa_fifo_uses_its_whole_capacity` exists to reject.

Exercise 5.5 Where does this belong

DESIGN

a) `read()` on the device node: it is the device's data, a stream consumed in order, which is exactly what the file interface is for.

b) A read-only sysfs attribute: one value per file, readable by any tool. It already exists without further work, since the capacity module parameter appears as `/sys/module/qa_chardev/parameters/capacity`.

c) A read-only sysfs attribute. A field engineer's tools will read it on production systems, where debugfs is usually not mounted and often not built in, so it has to live somewhere that is guaranteed to be there.

d) debugfs. It is only useful while you are developing the driver, and debugfs is the one place that is explicitly not an interface: you may change it or delete it whenever you like.

e) An `ioctl` on the open device. It is an operation rather than a value, and it acts on the stream that the caller has open; terminals do the same thing through an `ioctl`, which is

what `tcflush` is. A write-only sysfs attribute is defensible, but sysfs is for attributes, and an action dressed as one is harder to discover and to document.

f) A read-only sysfs attribute: a single fact about the device that never changes, and one that tools and udev rules can match on.

g) The sysfs attributes and the `ioctl` are ABI: the moment a program or a script uses them, their names, formats, numbers and meaning are fixed, and sysfs attributes are documented as such under `Documentation/ABI/`. `debugfs` is explicitly not. What follows for a product is that everything in sysfs or behind an `ioctl` has to be designed as if it will never change, because it will not be allowed to, and that nothing a shipped tool or a field engineer depends on may live in `debugfs`, which the product's kernel may not even contain. (The byte stream behind `read()` is an interface too, but its contract, bytes in order, is the file interface's own rather than one you invented.)

Exercise 5.8 Reading the tests for what they miss

DESIGN

a) Take the design §5.2.1 suggests, a head and a level, with `qa_fifo_put` copying to `(head + level) % capacity` and then storing `level = level + n`. The FIFO has capacity 8 and is empty, and two writers call `qa_fifo_put` with four bytes each, on two CPUs:

1. Writer A reads `level`, 0, and copies AAAA into slots 0 to 3.
2. Writer B reads `level`, still 0, and copies BBBB into the same slots, overwriting A's bytes.
3. A stores `level = 0 + 4`; B stores `level = 0 + 4`.

Both calls return 4, so both writers believe their bytes are in, but the FIFO holds four bytes, all B's; A's data is lost without a trace. Change the timing and the level can end at 8 with slots 4 to 7 never written, so a reader is handed four bytes of garbage. **None of the ten tests notices**, because every one of them runs on a single thread and never calls `qa_fifo_put` twice at once.

b) Any three of these, each worth testing:

- **The -EFAULT path.** Nothing passes a bad pointer. Cheap to test: a program that calls `read` and `write` with an address in an unmapped page, expecting `EFAULT` and no oops.
- **Unloading while the device is open.** Nothing checks `.owner`. Cheap: hold the node open in one process and expect `rmmod` to refuse with "Module is in use".
- **A failing init.** Nothing makes any `init` step fail, so no error path is ever run. Test with `insmod qa_chardev.ko capacity=0`, which must fail cleanly because `qa_fifo_create(0)` returns `NULL`, leaving no node, no class and no number behind; later steps need fault injection.
- **Huge requests.** The tester writes 4096 bytes at a time. A read asking for a gigabyte must return a short count, not fail its bounce-buffer allocation.

- **Memory pressure.** The bounce buffer's allocation failing must give `-ENOMEM` rather than a crash; this needs the kernel's fault injection (`CONFIG_FAILSLAB`).
- c) Fewer than it seems, because a shell sees a command's exit status and its error message, and those carry more than nothing: `cat` exits with status 0 on end of file and with 1 and "Resource temporarily unavailable" on `-EAGAIN`. Two checks it genuinely cannot make:
- **lseek refused with `-ESPIPE`.** No command exposes the result of an `lseek`; `dd skip=` on a file it cannot seek quietly reads and discards instead.
 - **Which error a call failed with.** The `-EAGAIN` and `-ENOSPC` checks compare `errno` with a constant. A script sees only that a tool failed and a message whose wording belongs to the tool and the C library, BusyBox's differing from coreutils', from tools that may retry a short write or treat one error like another before you ever see it.
- d) Neither. The KUnit suite never loads `qa_chardev`, and the tester and `run.sh` only ever run the path on which every init step succeeds. What would catch it is a test that makes `class_create` fail, for example by registering a class called `qa_fifo` first so that it returns `-EEXIST`, run on a kernel with `CONFIG_DEBUG_KMEMLEAK`: after the failed load, a `kmemleak` scan reports the FIFO's allocation as unreferenced.

Exercise 5.9 A byte count worked out twice

CROSS-CHECK

- a) With a capacity of 256:

Step	Call	Returns	Level after
1	write of 200 bytes	200	200
2	write of 100 bytes	56	256
3	read of 512 bytes	256	0
4	read of 512 bytes	<code>-EAGAIN</code>	0
5	write of 300 bytes	256	256

The device accepts $200 + 56 + 256 = 512$ bytes in all and delivers 256 of them, so 256 have passed all the way through and 256 are still inside when the sequence ends. A total of 600 counted the requests rather than the returns, and 500 counted step 2 as a failure.

- c) A driver that meets the specification gives exactly the table above, so any difference is in your prediction or your driver. **Step 2** is the one people get wrong by predicting `-ENOSPC`. The contract for `write` in §5.2.2 decides it: `-ENOSPC` only when the FIFO is *full*, and otherwise as many bytes as fit, which here is $256 - 200 = 56$. Predicting `-ENOSPC` there classes a value as an error; predicting 0 for step 4 classes an error as end of file. The total counts step 2 as 56, as the machine does.
- d) With `capacity=128` the program sees 128, `-ENOSPC`, 128, `-EAGAIN` and 128. Steps 1, 3 and 5 each return 128 instead of 200, 256 and 256, and step 2 changes kind as well as value: after

step 1 the FIFO is full, so it is refused rather than short. Only step 4 is unchanged, because it depends only on the FIFO being empty, and the 512-byte read in step 3 empties a FIFO of any capacity up to 512.

e) The shipped tester finds the capacity by writing until it is refused, so on the `capacity=128` device it reports “capacity measured from userspace: 128 bytes” and passes every check. Your program compares against 200, 56, 256, -EAGAIN and 256, and reports four failures against a driver that is correct. The lesson: a test that encodes a configuration tests the configuration. Derive the expectation from the system under test, by measuring it or by reading the parameter back from sysfs, or a legitimate change of configuration turns into failures, and a team that learns to shrug at failing tests stops seeing the real ones.

f) Nothing is broken. The FIFO already held 200 of its 256 bytes, as it does after step 1, so 56 bytes were free: the driver accepted 56 and said so. That is a short write, and their program should have looked at the return value and written the remaining 244 bytes later, once a reader had made room, in a loop, exactly as for a pipe or a socket. The line they had not read is the first row of the table in §5.1.4: this many bytes were transferred, and **a short count is normal, not an error**.

A.6 Chapter 6: Memory, MMIO, and Resource Management

Answers to the exercises in §6.4. Exercises 6.7 and 6.8 are Code, and are checked on the target as their Check yourself lines say.

Exercise 6.1 Two kinds of address

RECALL

a) `void *`: yes; it is a kernel virtual address, such as `kmalloc` returns. `phys_addr_t`: no; it is a number, where something sits on the bus, and the CPU can reach it only through a mapping, which for a device’s registers is `ioremap`. `void __iomem *`: not directly; only through the accessors, `readl` and `writel` and their family. `char __user *`: no; only through `copy_to_user`, `copy_from_user`, `get_user` and `put_user`, which check the address and survive a fault on it.

b) They put a pointer in an address space of its own. When `sparse` runs, `__iomem` expands to `__attribute__((noderef, address_space(__iomem)))`, and `__user` to the same with its own space (`include/linux/compiler_types.h`); for the compiler they expand to nothing. `sparse`, run with `make C=1`, then warns wherever such a pointer is dereferenced or passed where a pointer to another address space is expected: a user pointer handed to `memcpy`, or an `__iomem` pointer stored in a `void *`. The annotation records in the type which side owns the address, and the checker enforces it.

c) No to both. An entry means that somebody registered the range in the kernel’s resource tree: a driver that called `request_mem_region`, or the kernel itself for System RAM and its

own image. It is bookkeeping, and says nothing about whether a virtual mapping exists; mapping is `ioremap`, a separate call, and a driver can map a range without claiming it (§6.2.2). Absence means only that nobody has claimed the range. `qa-dev` was present and answering at `0x0C000000` from the first boot, and Exercise 1.7 found nothing there because no driver for it existed until this chapter's.

d) The check. The code works because it still reaches the registers through `readl`, and C converts a `void *` to the accessor's `const volatile void __iomem *` argument without complaint. What is gone is the annotation that made a mistake visible: the next change that dereferences `regs` directly, or hands it to `memcpy`, compiles silently and passes sparse, and on an architecture where registers need their own instructions or byte handling that code is simply wrong. You have also lost the documentation: a reader can no longer tell from the type that the pointer is not memory.

Exercise 6.2 The allocators

RECALL

a)

- **48-byte state structure:** `kzalloc(sizeof(*priv), GFP_KERNEL)`. It is small, comes from a slab cache, and is zeroed, so no field starts as garbage.
- **2 MB software buffer:** `vmalloc`, or `kvmalloc`. Nothing needs it physically contiguous, and as one contiguous block it would be $2\text{ MB}/4\text{ KB} = 512$ pages, order 9, which a system that has been up for a while often cannot supply.
- **16 KB DMA buffer:** `kmallocc(..., GFP_KERNEL)`, which is physically contiguous and at four pages, order 2, small enough to succeed on a fragmented system, then handed to the device through the DMA API with `dma_map_single`; or `dma_alloc_coherent`, which allocates and maps in one call. Never `vmalloc` (§6.1.6).
- **1 MB buffer hardware never sees:** `kvmalloc`. Contiguity is not needed, so it can try `kmallocc` for the 256 pages, order 8, and fall back to `vmalloc` when that fails. Free it with `kvfree`.

b) 131072 bytes is 128 KB, $131072/4096 = 32$ pages, order 5. `kmallocc` serves anything above 8 KB, its largest slab cache on 4 KB pages, straight from the page allocator, so the request needs one free, physically contiguous block of 32 pages. On a freshly booted machine free memory is in large blocks; after three weeks it is scattered in small ones, and there can be plenty of it free with no block of order 5 anywhere. Order 5 is also above `PAGE_ALLOC_COSTLY_ORDER`, which is 3 (`include/linux/mmzone.h`): the allocator gives an allocation that size a limited attempt at reclaim and compaction and then returns `NULL`, rather than retrying as it would for a small one. The file is `/proc/buddyinfo`, which lists the free blocks of each order in each zone; on the customer's machine the columns from order 5 up are zero or close to it, while `/proc/meminfo` still shows memory free.

c) Because you do not know which you got, `kvmalloc` returns `kmalloc` memory when the `kmalloc` attempt succeeds and `vmalloc` memory when it does not, and nothing tells the caller which. In testing the first attempt always succeeds, so the driver works; on the day fragmentation defeats it, the buffer is `vmalloc` memory, which is not physically contiguous, and `dma_map_single` refuses it with a warning and `DMA_MAPPING_ERROR` (`dma_map_single_attrs` checks `is_vmalloc_addr`, `include/linux/dma-mapping.h`). It fails exactly where fragmentation made the difference, which is in the field.

d) `kmalloc` and `kzalloc` go with `kfree`, `vmalloc` with `vfree`, `kvmalloc` with `kvfree`, and `alloc_pages` with `__free_pages`, which takes the struct `page *` and the order; `free_pages` takes an address and goes with `__get_free_pages`. Mismatched, `vfree` of `kmalloc` memory finds no `vmalloc` area, warns “Trying to `vfree()` nonexistent vm area” and frees nothing, which is a leak; `kfree` of `vmalloc` memory takes an address outside the linear map for a slab object, which is undefined and at best an immediate oops. The mismatch that survives testing is `kfree` of `kvmalloc` memory: correct for as long as `kvmalloc` returns `kmalloc` memory, which in testing is always, and a crash the first time it falls back. `kvfree` checks which it was given, and is correct for both.

Exercise 6.3 GFP flags

HAND CALCULATION

a) `GFP_KERNEL`. `read()` runs in process context on behalf of the caller, where sleeping is allowed, so nothing goes wrong, provided no spinlock is held at the allocation.

b) `GFP_ATOMIC`. `GFP_KERNEL` may enter direct reclaim and sleep, and a handler may not sleep. With `CONFIG_DEBUG_ATOMIC_SLEEP` the kernel prints “BUG: sleeping function called from invalid context” on the first call, before it has needed to sleep; without it the code works until the day the allocator has to reclaim, and then sleeps in interrupt context: “BUG: scheduling while atomic” and quite possibly a hung CPU.

c) `GFP_ATOMIC`; or better, allocate with `GFP_KERNEL` before taking the lock and do only the linking-in under it. With `GFP_KERNEL` the holder may sleep with the lock held and preemption disabled: “scheduling while atomic”, every other CPU that wants the lock spinning while it sleeps, and a deadlock if the task scheduled in its place on the same CPU wants the lock too.

d) `GFP_KERNEL`. A module’s init function runs in process context, in the `finit_module` system call made by `insmod`, and may sleep. Nothing goes wrong.

e) The only sense in which it is high priority is that it may dip into a reserve kept below the normal watermark: `GFP_ATOMIC` is `__GFP_HIGH | __GFP_KSWAPD_RECLAIM` (`include/linux/gfp_types.h`). What it cannot do is what makes an allocation succeed under pressure: reclaim, write back, compact, and wait for any of them; it may only wake `kswapd`. So `GFP_ATOMIC` is the one more likely to return `NULL`. When the free pages and the reserve are gone it fails at once, where `GFP_KERNEL` would have reclaimed memory and succeeded. That is why code using it must handle failure.

f) No. It is a bug that has not fired yet. It works because every allocation so far found memory without needing to reclaim, so it never slept. It appears when memory pressure forces the allocator into direct reclaim while the spinlock is held: the task sleeps in atomic context, which is “scheduling while atomic”, a stall on every CPU waiting for the lock, or a deadlock. `CONFIG_DEBUG_ATOMIC_SLEEP` would have reported it on the first run: every `kmalloc` calls `might_alloc()`, which calls `might_sleep_if()` for any flag that allows blocking (`include/linux/sched/mm.h`), so the check fires whenever the line runs, whether or not it needed to sleep.

Exercise 6.4 Translating a device tree address

HAND CALCULATION

a) `0x40002000`, for `0x100` bytes, which is 256. The reg is read in the bus’s cells, one address and one size: child address `0x2000`, length `0x100`. The bus’s `ranges` maps child `0x0` to parent `0x0_40000000` for `0x1000000` bytes, and `0x2000` lies inside that child window, so

$$0x40000000 + (0x2000 - 0x0) = 0x40002000.$$

The parent is the root, whose address space is the CPU’s physical address space, so the translation ends there.

b) Two: one address cell and one size cell. You know from the *parent*: reg is read in the `#address-cells` and `#size-cells` of the node’s parent, and `bus@40000000` declares one of each. The widget’s own properties say nothing about it.

c) Four. An entry in `ranges` is a child address, a parent address and a length. The child address is in the bus’s own `#address-cells`, one cell: `0x0`. The parent address is in the parent’s, and the root declares two: `0x0 0x40000000`, the high cell and then the low, so `0x0_40000000`. The length is in the bus’s `#size-cells`, one cell: `0x1000000`. $1 + 2 + 1 = 4$.

d) They map physical `0x2000`, which is not the widget but whatever this SoC decodes at that address: on many parts boot ROM or on-chip SRAM, on some nothing at all. Most likely they read something plausible, the contents of a ROM or zeros, with no error of any kind, as `base=0xC` does on `qa-dev` in §6.2.6; if nothing decodes there, the first read raises a synchronous external abort. Either way it is the identity check that tells them.

e) Child address `0x0` on this bus is address `0x40000000` on the parent bus, for `0x1000000` bytes, which is 16 MB: the bus’s addresses `0x0` to `0xFFFFF` are the parent’s `0x40000000` to `0x40FFFFFF`.

Exercise 6.5 Ordering

DESIGN

a) `writel` for CTRL and `readl` for STATUS, the ordered defaults. With `_relaxed` for both, their order does not change: both go to one device through the mapping `ioremap` made, and accesses from one CPU to one device arrive in program order whether relaxed or not (§6.2.5; Documentation/memory-barriers.txt). What goes wrong is something else, and it goes wrong with the ordered forms too. A conversion takes time, and a status read issued straight after the command will usually find it not finished. The driver must poll STATUS until it says done, with a timeout, which is what `readl_poll_timeout` does, or wait for the device's interrupt.

b) Nowhere among the register writes themselves. All seventeen go to one device, and relaxed accesses to one device stay in program order, so the sixteen configuration writes land before G0 without any barrier. A barrier is needed in one place: before the G0 write, and only if the device will read something from RAM that the driver wrote first, a buffer or a descriptor. So the sixteen are `writel_relaxed` and G0 is a `writel`. That buys sixteen barriers fewer on every start; on arm64 each `writel` puts a `dmb_oshst` in front of its store (`arch/arm64/include/asm/io.h` and `barrier.h`). It is a small saving, and worth having only on a path where it has been measured to matter.

c) Because what you observe points away from the cause. The failure is intermittent and depends on timing, so it appears on one machine and not another, and under load rather than on the bench. Adding a `printk` or attaching a debugger changes the timing and makes it vanish. It shows up later and somewhere else, as data that is wrong, never at the line that is missing a barrier. And the code, read line by line, is correct in program order, which is the order everyone reads it in. A strongly ordered machine, such as an x86 desktop, never reorders a store to memory past a later write to a register, so the code works there every time.

d) Because real silicon is not lenient in any predictable way. On a real device a byte or halfword write to a 32-bit register may be widened by the interconnect with the other bytes written as zeros, or dropped, or answered with a bus error; a narrow read of a register with side effects, such as `SAMPLE`, pops a whole sample and delivers a quarter of it. None of those is what the driver meant, and which one you get depends on the chip. A model that widened quietly would let a wrong driver pass every test here and fail on the first board; one that rejects the access makes the bug fail here, where a failure costs a reboot. The documented access width is part of a register's contract, and the model enforces it.

Exercise 6.6 Coherent or streaming

DESIGN

a) `dma_alloc_coherent`. Both sides touch small entries constantly: the driver writes an entry, the device reads it and writes a status back, thousands of times a second. With a streaming mapping every exchange would need a sync in each direction; coherent memory makes each side's writes visible to the other without one, and the ring is small enough that slower uncached CPU access costs nothing that matters. It still needs the `writel` barrier before the doorbell, because coherent does not mean ordered (§6.1.6).

b) `dma_map_single` with `DMA_FROM_DEVICE`, on a buffer from `kmalloc` (64 KB is 16 pages, order 4) or `alloc_pages`. The device writes it once, and then the CPU reads all 64 KB in bulk to copy them to userspace. One mapping and one cache invalidate at the unmap, and the CPU reads cached memory at full speed; in coherent memory, which is usually uncached, the 64 KB copy would be slow.

c) `dma_alloc_coherent`. Four bytes that the driver writes and the device reads over and over: there is no bulk access to make faster, and a streaming mapping would need a `dma_sync_single_for_device` after every write. It is exactly the kind of status word §6.1.6 names.

d) It broke the ownership rule: between the map and the unmap the buffer belongs to the device, and the CPU read it without borrowing it back. On a machine whose DMA is not coherent with the CPU's caches, the read pulls a line of the buffer into the cache holding whatever was in memory before the device wrote it, so the peek sees stale data; depending on what the architecture does at the unmap, that stale line can also outlive the transfer, and the driver then reads old data where the device's should be. Where the buffer is bounced, because the device cannot reach the memory it is in, the device writes a bounce buffer and the peek reads the original, which the kernel fills only at the unmap. Testing did not catch it because the machines it ran on were coherent, an x86 desktop, an arm64 system with coherent DMA, or an emulator with no caches to go stale, and nothing was bounced; there the read is harmless. It breaks on an SoC whose DMA master is not cache-coherent, which is common in embedded parts, and on any system that bounces the buffer. The fix is `dma_sync_single_for_cpu` before the peek and `dma_sync_single_for_device` after it.

e) `kvmalloc` is exactly wrong here. On the fragmented system they are worried about, the `kmalloc` attempt fails and `kvmalloc` falls back to `vmalloc`, whose memory is not physically contiguous and cannot be handed to a device. The kernel refuses it: `dma_map_single_attrs` checks `is_vmalloc_addr` and returns `DMA_MAPPING_ERROR` with the warning "rejecting DMA map of vmalloc memory" (`include/linux/dma-mapping.h`). So the driver works in testing and fails in the field, on precisely the system the change was meant to help. Use `kmalloc`, allocated once when the device is opened or probed and kept rather than per transfer, with its failure handled; or `alloc_pages` or `dma_alloc_coherent`; or, if the device can do scatter-gather, a list of single pages mapped with `dma_map_sg`, which needs no contiguity at all.

Exercise 6.9 What a wrong address does

DESIGN

a) Zeros: the course measured `id=0x00000000` `version=0x0000` (§6.2.6), and nothing faults, because `0xC` lies in the virt machine's flash window, which starts at physical 0 and is `0x08000000` bytes long (`hw/arm/virt.c` in QEMU), and reads as zeros here. The identity check compares `0x00000000` with `0x51414456`, fails, unwinds, `iounmap` and then `release_mem_region`, and returns `-ENODEV`; `insmod` reports "No such device" and nothing is left behind. A driver without the check would have carried on as if it had the device: written `0xDEADBEEF` to `SCRATCH`, which here is physical `0xC + 0x30 = 0x3C`, into whatever lives there, and then gone on to configure and read a device that is not there, failing later and elsewhere with nothing in the log pointing at the address.

b) What the course recorded (§6.2.6): the read of `QA_DEV_ID` at `0xC001000` raises "Internal error: synchronous external abort: 0000000096000010", with the program counter in `qa_mmio_init+0x98`, called from `do_one_initcall`. The syndrome decodes as a data abort taken in the kernel (the top six bits, `0x25`) on a read, with fault status `0x10`, a synchronous external abort. Nothing decodes that address: the platform bus window is `0x0C000000` to `0x0DFFFFFF`, and `qa-dev` occupies only its first page, `0x0C000000` to `0x0C000FFF`. `insmod` is killed. The module is left half-loaded, its `init` never having returned, so it can be neither used nor removed, and it still holds the region it claimed. The only way out is a reboot.

c) "It did not crash" is not evidence that a mapping is right. A wrong address can read plausible values with no error, or abort the machine, depending only on what the machine decodes there, and the quiet outcome is the more dangerous of the two. The evidence has to be positive: an identity register that reads the value the datasheet promises, and a scratch register that reads back what was written.

d) On a board on your bench, the wrong-but-mapped address may belong to another peripheral, so the scratch write can reprogram something real, and the abort costs a power cycle and a walk across the room, or a JTAG probe if it happened early in boot; on a product in the field, either one is a device that fails or hangs at a customer's, and bringing it back costs a truck or a recall. §1.1.4 named what the course gives up by booting in QEMU, the experience of real boards and of recovering one, and this is the other side of that trade: here both experiments are free, repeatable and two seconds long, which is why the course asks you to run them.

Exercise 6.10 An address worked out by hand

CROSS-CHECK

a) `reg` is eight bytes, `00 00 00 00 00 00 10 00`, which is two cells, `0x00000000` and `0x00001000`. `ranges` is sixteen bytes, `00 00 00 00 00 00 00 00 0c 00 00 00 02 00 00 00`, which is four cells: `0x00000000`, `0x00000000`, `0x0C000000` and `0x02000000`. `reg` has two cells because the platform bus declares one address cell and one size cell; `ranges` has four because its parent address is in the root's two cells. QEMU writes exactly these: `ranges` as 0, the high and low halves of the bus base, and the bus size (`hw/core/sysbus-fdt.c`), and `reg` as the device's offset on the bus and `QA_DEV_MMIO_SIZE` (`tools/qa-dev/integrate.py`).

b) The two numbers to add are `reg`'s address cell and the parent address in `ranges`, not either size:

$$0x0C000000 + (0x00000000 - 0x00000000) = 0x0C000000,$$

the child's address less the child base of the range, added to the parent base. The high parent cell is 0, so the full 64-bit address is `0x00000000_0c000000`. The window is `0x1000` bytes, 4096, one page, which is `QA_DEV_MMIO_SIZE`; it ends at `0x0C000FFF`, well inside the bus window, which runs to `0x0C000000 + 0x02000000 - 1 = 0x0DFFFFFF`.

c) Offset 0 is `QA_DEV_ID`, and the header gives its value: `QA_DEV_ID_MAGIC`, `0x51414456`, which is "QADV" in ASCII, big end first (`0x51 Q`, `0x41 A`, `0x44 D`, `0x56 V`). Offset 4 is `QA_DEV_VERSION`, for which the header gives only the layout, major in bits 15:8 and minor in bits 7:0; but the compatible string it defines, `qacademy,qa-dev-1.0`, says version 1.0, so predict `0x00000100`.

d) A correct module at that address prints `id=0x51414456 version=0x0100`; those are the two strings `make test L=L06` looks for in the kernel log.

e)

- It should match. `run.sh` computes the same address the same way, prints "so the registers are at `0xC000000`, 4096 bytes", and loads the module with `base=201326592`, which is `0x0C000000` in decimal. If yours differed, you took `reg` at face value (`0x0`), added the wrong cell of `ranges`, the high half of the parent address or the size, or read the bytes with `-tx4`. `run.sh` reads raw bytes with `od -tx1`, reassembles them four at a time, most significant first, and adds `reg`'s first cell to the third cell of `ranges`.
- `-tx1` is right. `-tx4` decodes each group of four bytes as a native-endian integer, and the target is little-endian, so every big-endian cell comes back byte-swapped: `reg` reads `00000000 00100000` and `ranges` reads `00000000 00000000 0000000c 00000002`. The same arithmetic then gives an address of `0xC` and a window of `0x100000`, a round megabyte. It is plausible because it has the right shape: small hex numbers in the right places, and an address that is a real address, which reads zeros without complaint (§6.2.6).
- The constant is `0x0C000000`. It is the parent base in `ranges`, declared by the parent node, `platform-bus@c000000`; it comes from QEMU's virt machine, which places its platform

bus at `0x0C000000` with `0x02000000` bytes (`hw/arm/virt.c`) and writes that into ranges when it generates the device tree.

- It is in the device tree QEMU generated and handed to the kernel, readable under `/sys/firmware/devicetree/base`, and today on `insmod`'s command line, where `run.sh` put it. For the driver to find out, it has to become a platform driver matching the compatible `qacademy,qa-dev-1.0`: the kernel then creates the device from the node, translates its `reg` through every ranges above it into a `struct resource`, and hands it to `probe`, where `platform_get_resource`, or `devm_platform_ioremap_resource`, gives the driver its registers. That is Chapter 10.

f) $0x0C000000 + (0x1000 - 0x0) = 0x0C001000$, and `QA_DEV_ID` still reads `0x51414456`. The node is not a separate claim about the device; it is QEMU's description of where it put it, generated from the address it assigned (`tools/qa-dev/integrate.py`). A node that says `0x1000` means the device sits one page further up, and its identity register is at offset 0 of its window wherever that is: the identity belongs to the device, not to the address. Only if the node were edited by hand while the device stayed at `0x0C000000` would `0x0C001000` be the unbacked page of §6.2.6, and the read abort.

A.7 Chapter 7: Concurrency and Locking

Answers to the exercises in §7.4. Exercises 7.6 and 7.7 are Code, and are checked on the target as their Check yourself lines say.

Exercise 7.1 Where concurrency comes from

RECALL

a) SMP, preemption, interrupts, timers and workqueues (§7.1.1). On a single-CPU kernel built with `PREEMPT_NONE`, two remain: interrupts, and timers, because a `timer_list` callback runs in softirq context on the way out of an interrupt and so can run in the middle of your code just as a handler can. SMP goes with the second CPU and preemption with `PREEMPT_NONE`. A workqueue item runs in a kernel thread, which on such a kernel gets the CPU only when your code gives it up by sleeping or returning, so it can overlap you only across a sleep.

b) That nothing else runs this code at the same time, and that it cannot be stopped mid-function. So it protects shared data, if at all, by disabling interrupts on its own CPU, which does nothing about another CPU running the same code at the same moment, and its read-modify-write sequences can be preempted halfway with another task running the same code in between. What breaks is data shared between callers, silently and intermittently: lost updates, a ring buffer whose level no longer matches its contents. The fix is a lock, which the old code has no notion of.

c) Interrupts. The answer does not change on a single CPU: the handler can run on top of `read()` on the same CPU at any point where interrupts are enabled, which is the case

`spin_lock_irqsave` exists for (§7.1.5). A second CPU adds SMP as well, since the handler can then also run on the other CPU while `read()` runs, and that is what the lock itself excludes. `spin_lock_irqsave` covers both.

d) Three: a load of the counter into a register, an add, and a store back. The add happens in a register, where no other CPU can see it. What matters is the other CPU's load and store falling between this CPU's load and its store: if both load n before either stores, both store $n + 1$, and one of the two increments is lost (§7.1.2).

Exercise 7.2 Atomics and their limits

RECALL

a) The ring buffer of Chapter 5. Its invariant: `level` is between 0 and the capacity, and the `level` bytes starting at `head`, wrapping at the capacity, are exactly the bytes written and not yet read, in the order they were written. It mentions the `head`, the `level` and the array at once, so making each of them atomic leaves it breakable between any two of their updates (§7.1.3); Exercise 5.8 constructs an interleaving that does it.

b) Make `head` and `level` plain integers and put the whole test, store and increment inside one critical section: take a spinlock; if `level < capacity`, store the byte at `(head + level) % capacity` and increment `level`; release the lock. A spinlock, because the section is a handful of instructions that never sleep and so can be held briefly from any context; `spin_lock_irqsave` if an interrupt handler also reads or writes the buffer. A mutex would be correct too if every caller is in process context, but buys nothing for a section this short. The point is that the test and the update become one indivisible step, so no second writer can pass the same test and pick the same slot in between.

c) Atomics protect one variable; locks protect an invariant: if what must be true before and after mentions more than one variable, you need a lock.

d) The compiler may load `flag` once, before the loop. Nothing in the body can change `flag` as far as the compiler can see, no store, no call and no barrier, so reading it again would be redundant, and the loop spins on a register forever. The fix is `while (!READ_ONCE(flag)) cpu_relax();`, with the thread that sets it using `WRITE_ONCE(flag, 1)`. It is not a memory barrier: `READ_ONCE` forces one real load on every iteration and orders nothing on the CPU. If the waiter must then read data the other thread wrote before setting the flag, that needs ordering as well: `smp_store_release` to set it and `smp_load_acquire` to read it, or a lock.

Exercise 7.3 Choosing a primitive

HAND CALCULATION

- a) A spinlock: two kernel threads run in process context, no handler touches the counter, and the section is one increment, far too short to be worth sleeping over. (A mutex would also be correct, and an `atomic_t` better still, but that is not one of the choices.)
- b) A spinlock with `_irqsave`: the handler takes the lock in interrupt context, so `read()` must keep interrupts off its own CPU while it holds it, and `irqsave` does so without assuming the state it was called in.
- c) A mutex: `copy_to_user` may fault and sleep, so the section may sleep, and only a mutex may be held across a sleep.
- d) A spinlock: 2 microseconds is short, nothing in the sequence sleeps, and one sequence must not be interleaved with another CPU's. If the interrupt handler also touches those registers, it is `_irqsave`, as in b).
- e) A spinlock with `_irqsave`: the timer callback runs in softirq context, so `write()` must keep it off its own CPU while it holds the lock, and with interrupts off no softirq runs there either. (The exact minimum is `spin_lock_bh` in `write()` and a plain `spin_lock` in the callback.)
- f) A mutex: an allocation that may have to reclaim is `GFP_KERNEL` and may sleep, so the lock held across it must be one a sleeper may hold. The alternative is to allocate before taking a spinlock.
- g) b) and e). In b), `read()` holds the plain spinlock, the device interrupts that CPU, and the handler spins on the lock; the holder cannot run again until the handler returns, and the handler never returns, so that CPU is gone, and so is every other CPU that then wants the lock. In e) it is the same with the timer: the timer interrupt arrives while `write()` holds the lock, the timer softirq runs the callback on the way out of it, on the same CPU, and the callback spins on a lock whose holder is underneath it. (A plain spinlock in c) or f) is a different bug, sleeping while holding one, which `CONFIG_DEBUG_ATOMIC_SLEEP` reports.)

Exercise 7.4 What the race costs

HAND CALCULATION

- a) $4 \times 20,000 = 80,000$.
- b) $80,000 - 63,562 = 16,438$ lost, and $16,438/80,000 = 20.5\%$.
- c) The range is $30,047 - 1,564 = 28,483$ increments, and $28,483/80,000 = 35.6\%$ of the expected total: the runs lost from $1,564/80,000 = 2.0\%$ to $30,047/80,000 = 37.6\%$. The spread says that the loss is not a property of the program but of the timing of one run. A single run is one draw from a very wide distribution and predicts nothing about the next: the same code on the same machine lost 2% once and 38% another time, so one run can neither size the bug nor show whether a change made it better.

d) Zero: yes. A lost update needs one thread's store to land between another thread's load and store; if the threads never run their loops at the same time, that never happens and the count is exact. It has been seen: the first version of the module, without the barrier, lost nothing three runs in a row, and at 200 and 2,000 increments per thread every run was exact (§7.1.2).

60,000: yes, in principle, although nothing close to it was observed. A lost update is a stale store: a thread writes back the value it loaded plus one, wiping out every increment made since its load, not just one. A thread preempted between its load and its store, on a CONFIG_PREEMPT target running four threads on two CPUs, can come back after the others have added thousands and erase all of them with a single store. The final value can in fact be as low as 2: thread A loads 0; the others run everything except B's last increment, reaching 59,999; A stores 1; B loads 1; A runs its remaining 19,999, reaching 20,000; B stores 2. So a loss of 60,000, a final count of 20,000, is possible. The largest loss measured was 30,047.

e) First, one sample of a nondeterministic result. A single exact run is what the broken program produces some of the time, and at 2,000 per thread it produced it every time, so it says nothing about the next run. Second, taking a run as the argument for correctness. counter++ on a shared variable from several threads is three unprotected steps, and that is a race by construction whatever any run returns: whether the lock is needed follows from the code, and an observation can show a bug but never its absence.

Exercise 7.5 Atomic context

DESIGN

- a) Not atomic: read() runs in process context for the calling task. Legal.
- b) Atomic, in an interrupt handler. Illegal: GFP_KERNEL may sleep to reclaim.
- c) Atomic. Legal: GFP_ATOMIC never sleeps, and the handler must cope with NULL.
- d) Atomic, since a spinlock is held. Illegal: mutex_lock sleeps when the mutex is contended.
- e) Not atomic: holding a mutex does not make you atomic, and you are still in process context. Legal; the section inside the spinlock is then atomic, and nothing in it may sleep.
- f) Atomic. Illegal: copy_to_user may fault, and bringing the page in may sleep.
- g) Atomic: a timer callback runs in softirq context. Illegal: msleep sleeps, every time.
- h) b), d) and f). g) never works: msleep always calls into the scheduler, and from softirq context that prints "BUG: scheduling while atomic" on the first run. The three that usually work:
 - b) works because the allocation is served from a slab cache without reclaim. It fails the day memory is short and the allocator has to reclaim, which means sleeping inside a handler.

- d) works because an uncontended `mutex_lock` is a single atomic compare-and-exchange and never sleeps. It fails the first time another task holds the mutex, and the spinlock holder goes to sleep.
 - f) works because the user's buffer is resident, so the copy never faults. It fails when the page is not there, never touched or since reclaimed, and the copy has to fault it in, which may sleep. On a kernel that counts preemption, as this course's does, the fault handler sees that it is in atomic context and declines the fault (`fault_handler_disabled()`, `include/linux/uaccess.h`), so the copy comes up short and the driver returns `-EFAULT` instead. Either way it happens only under memory pressure.
- i) `CONFIG_DEBUG_ATOMIC_SLEEP`. It puts a check at the top of every call that might sleep, before the call decides whether it has to: `might_alloc()` in `kmalloc`, `might_sleep()` in `mutex_lock`, and `might_fault()` in `copy_to_user`. All three print "BUG: sleeping function called from invalid context" the first time they run in atomic context, on a machine with plenty of memory, an uncontended mutex and a resident page.

Exercise 7.8 What the tools do not see

DESIGN

- a) Lockdep watches lock acquisitions. At each one it records which locks were already held, adds the resulting edges to its graph and checks them, and checks the context the lock is taken in (§7.2.3). `qa_race` with `locked=0` takes no lock at all, so as far as lockdep is concerned nothing happens: no acquisition, no edge, no context to check. The lost updates are plain loads and stores to a shared variable, of which lockdep has no model; it cannot know that the variable needed a lock.
- b) Lockdep checks the locks you take, never the locks you did not take. A codebase that "passes with lockdep enabled" has shown that on the paths its tests ran, the locks it took were taken in a consistent order and in legal contexts. That says nothing about data that should have had a lock and had none, and nothing about paths the tests never ran.
- c)
- **Waiting for an event rather than a lock:** two threads each wait for a completion, or a wait queue condition, that only the other will signal. The graph is built from lock acquisitions, and waiting for an event is not one, so the cycle has no edges.
 - **Waiting on hardware:** a driver holds a lock while it polls for a status bit, or waits for an interrupt, that never comes. The other party in the cycle is a device, which lockdep cannot see at all.
- d) The error paths: an allocation that fails, a device that times out or answers wrongly or disappears, a probe that fails halfway and unwinds, a signal that interrupts a sleep, a remove while a file is still open. A test suite drives the normal path because that is what it can cause; failures have to be injected, and mostly are not. It matters more for lock ordering because an inversion takes two orders and lockdep reports only once it has seen both, and the unusual

order typically lives on exactly those paths: cleanup that takes locks in the reverse of the order set-up took them. The failure is also not confined to the rare path. Most bugs on an unexercised path hurt only the call that takes it; an ordering bug deadlocks the machine when the rare path meets the common one.

e) The module's licence is not GPL-compatible, so loading it tainted the kernel, and that taint is one that switches lock debugging off for the rest of the boot: the module loader adds `TAINT_PROPRIETARY_MODULE` with `LOCKDEP_NOW_UNRELIABLE`, and `add_taint` then turns lockdep off (`kernel/module/main.c`, `kernel/panic.c`). The line is Disabling lock debugging due to kernel taint. You first met it in Chapter 4, in Exercise 4.7.

Exercise 7.9 A number computed twice

CROSS-CHECK

- a) $4 \times 20,000 = 80,000$.
- b) A correct module prints `actual=80000 lost=0` on every run; with the spinlock switched on, the course's runs all returned exactly 80,000 (§7.1.2).
- d) Worked here with the course's ten unlocked runs from §7.1.2 in place of your five.
 - With the lock, the result is determined by the program rather than sampled from it: each of the 80,000 increments is a whole load, add and store that no other increment can interleave with, so the counter must end at $4 \times 20,000$, and 79,999 would be a bug, not noise. Nothing is measured on a scale that has an error; the program counts. Chapter 3's Image size is an exact count too, but not one you can predict: it depends on the configuration, the compiler and the source, so you measure it, and a different build legitimately gives a different number. Chapter 2's context-switch rate is a rate over a sampling interval, which varies with the load and with the interval you happen to catch, so two correct measurements differ. Here the prediction is arithmetic, and the measurement must agree with it to the last digit.
 - The ten losses sum to 154,616, so the mean loss is 15,461.6, which is 19.3%, and the mean actual is 64,538.4. The range is 1,564 to 30,047, a spread of 28,483, or 35.6% of the expected total. The mean on its own is not useful: the spread is nearly twice the mean ($28,483/15,461.6 = 1.84$), and both depend on the conditions, the number of threads and CPUs, the increments per thread, the barrier and the load. To mean anything it needs the spread, the number of runs and the conditions beside it; and what matters for the bug is not the average loss but that the loss is not zero.
 - If every collision lost exactly one increment, the worst run, 30,047 lost, took 30,047 collisions. But a thread preempted between its load and its store erases every increment made while it was off the CPU, so a single collision can lose thousands. Each collision loses at least one, so 30,047 is an upper bound on the number of collisions, not a count. Even as a count it is plausible: 30,047 in a few hundred milliseconds is of the order of 100,000 a second ($30,047/0.3 \text{ s} \approx 100,000$), for two CPUs doing nothing but load, add and

store on one shared variable. The collision rate is limited by the rate of increments, not by the clock.

e) The course's twelve runs (§7.1.2): at 200 and at 2,000 per thread, 0%, 0% and 0%; at 20,000, 23.3%, 20.8% and 6.9%; at 200,000, 31.7%, 34.9% and 26.5%.

- The loss appears at 20,000 and at 200,000. At 200 and 2,000 every run is exact.
- The bug is not absent at that size; the code is identically broken. What differs is that at 2,000 per thread no two threads are in their loops at the same time for long enough to collide: each thread's work is over before the others have got going, so the race has no window to happen in.
- Released together means released from the barrier together, not running together. After the release each thread still has to be woken and scheduled onto a CPU, and with four threads on two CPUs two of them wait for a CPU in any case. The overlap has to last long enough for two threads to be executing their loops on the two CPUs at once, or for one to be preempted between its load and its store, and it is competing against the wake-up and scheduling latency after the release: at 200 or 2,000 increments the whole loop is shorter than that stagger. The measurements put the threshold between 2,000 and 20,000 increments per thread, and §7.1.2 estimates it at about ten thousand. How long that is in time depends on this machine's increment rate and its scheduling latency, which the chapter does not publish; time a run near the threshold to find out.
- They would conclude that the code is correct, and they would be very confident, every run exact, run after run. They would be wrong, and the confidence would come from repeating a test that could not exercise the race, which is no evidence at all.

f) The bug report gets the unpredictable one: the unlocked losses, with their spread and the conditions they came from (four threads, 20,000 increments each, two CPUs, the barrier), because a report's job is to show that the bug exists, how bad it is, and how to reproduce it. The test gets the predictable one: `locked=1` must give exactly 80,000, because a test has to pass or fail the same way on the same code every time, and only an exact prediction can be asserted exactly. An assertion on the unlocked run can only be "lost more than nothing", and even that is safe only because the barrier and 20,000 increments made a zero loss something never measured, which is why `run.sh` runs it three times and asks only that at least one run lose something.

g) For this program, yes. The only shared state is one counter, and `atomic_inc` makes its load, add and store indivisible, so the count comes out at exactly 80,000. With a running maximum alongside, no. The counter and a maximum that must agree with it form an invariant over two variables, and updating the maximum is a read, compare and write separate from the increment: two threads can each read, compare and store in the wrong order, and a reader can see one variable updated and not the other. Per-variable atomicity does not make the sequence atomic. §7.1.3 answers it: atomics protect one variable, locks protect an invariant.

A.8 Chapter 8: Interrupts and Deferred Work

Answers to the exercises in §8.4. Exercises 8.6 and 8.7 are Code, and are checked on the target as their Check yourself lines say.

Exercise 8.1 The three numbers

RECALL

- a) The property has three cells. 0 says the interrupt is an SPI, a shared peripheral interrupt; 112 is its number among the SPIs; 4 is the trigger. On a real board whoever wrote the device tree copied 112 out of the SoC's manual. Here QEMU writes it: it places qa-dev on the virt machine's platform bus, whose interrupts start at SPI 112, and generates the node with the number it assigned. 144 is the same interrupt as the GIC numbers it, the identity the controller signals to the CPU, and the hardware column of /proc/interrupts. 17 is the Linux IRQ number, handed out when the GIC's IRQ domain mapped hardware interrupt 144 for the driver: an index into the kernel's own table, derived from neither of the others (§8.1.1).
- b) Where the SPIs begin in the GIC's own numbering. Interrupts 0 to 15 are SGIs, the software-generated interrupts CPUs send one another, and 16 to 31 are PPIs, private to each CPU, the per-CPU timer among them. The device tree counts within the SPIs and the GIC counts all of them, so the GIC driver adds 32 when it translates an SPI's cell, and 16 for a PPI's (drivers/irqchip/irq-gic.c).
- c) **The Linux number.** It is handed out in the order mappings are created, so an extra device whose interrupt is mapped earlier in the boot takes a number and moves everything after it. The DT cell and the GIC number describe the wiring, which an extra device does not change. One qualification for this machine: QEMU hands out the platform bus's interrupts to dynamic devices in the order they are created, first free first, so an extra device of that kind created before qa-dev on the command line would move 112 and 144 as well.
- d) Two mistakes. 112 is the device tree's number and appears nowhere in /proc/interrupts, which shows the Linux number, 17, first and the GIC's, 144, in its hardware column; the thing to grep for is the name passed to request_irq, qa_irq. And a missing line does not mean a missing driver: a line appears only while a handler is registered for the interrupt, so a loaded module that has not requested it, or has already freed it, is absent too. Whether the module is loaded is a question for lsmod.
- e) 4 is IRQ_TYPE_LEVEL_HIGH: level-sensitive, active high, so the interrupt is pending for as long as the line is high. 1 would have been IRQ_TYPE_EDGE_RISING: the controller notices the low-to-high transition and nothing else. For a GIC SPI these are the only two allowed. Configured as an edge, qa-dev, which holds its line high until its status is cleared, would lose any event that arrived while the line was still high, because no new edge happens (§8.1.4).

Exercise 8.2 Reading /proc/interrupts

RECALL

- a) 17, the Linux IRQ number. 1789, how many times this interrupt was taken on CPU0, and 0, on CPU1. GIC-0, the interrupt controller, the first GIC. 144, the hardware number, the GIC's own. Level, the trigger type. qa_irq, the name passed to request_irq; there is one per registered handler, separated by commas on a shared line (§8.1.2).
- b) **Nothing is wrong.** The GIC delivers an SPI to one CPU at a time, and Linux points it at the first online CPU in the interrupt's affinity mask. The mask allows both CPUs by default, so CPU0 gets every one, and nothing on this minimal target, no irqbalance, ever moves it. **Or something has taken CPU1 out of it:** the mask was narrowed to CPU0, by hand or with the irqaffinity= boot parameter, or CPU1 is not taking interrupts at all. To tell them apart, read /proc/irq/17/smp_affinity_list, which CPUs are allowed, and /proc/irq/17/effective_affinity_list, where the interrupt actually goes: 0-1 and 0 is the first explanation. CPU1's column for the other interrupts, the timer and the IPIs, shows whether it takes interrupts at all. Then write 1 to smp_affinity_list and watch CPU1's count for interrupt 17 start to climb.
- c) Ruled out: anything in your handler, which is not running, and a missing acknowledgement, which would make the count race rather than stand still. Still possible: everything between the device and the CPU. The driver asked for the wrong interrupt, from the wrong node or the wrong index in it; the trigger type is wrong; the line is masked in the controller; or the device is not asserting at all, because ENABLE or IRQ_EN is not set, or because PERIOD_NS is below QA_DEV_MIN_PERIOD_NS, 1,000 ns, at which the device refuses to start. And a count that stands still need not be zero: it may be left over from an earlier load.
- d) The source of the interrupt is never cleared. On a level-triggered line the handler returns, the line is still asserted, and the handler is entered again at once, as fast as the CPU can take it: the missing acknowledgement of §8.1.6. On qa-dev that is a handler that neither writes IRQ_STATUS nor empties the FIFO.
- e) No. The counts belong to the interrupt descriptor, not to your registration, and free_irq does not reset them: the run in §8.1.2 read 919 after the first load and 993 immediately after the second. So any rate has to be a difference between two readings, divided by the time measured between them; a single reading counts everything since the descriptor was created, across every load.

Exercise 8.3 The acknowledgement

HAND CALCULATION

a) Every 1 written clears its bit, so SAMPLE is cleared, which was wanted. The OVERRUN that arrived a microsecond after your read is cleared too, and the same write resets the sticky OVERRUN in STATUS: the device dropped a sample and nothing in the driver will ever know. The other thirty bits do not exist, and the ones written to them do nothing.

b) SAMPLE is cleared exactly as before. The late OVERRUN is not: its bit was 0 in the value written, and writing 0 leaves a bit alone. So it stays set, the line stays asserted, since the device raises it while either bit is set, and the handler is entered again at once, reads 0b10, and deals with the overrun. Nothing is lost.

c) With a bit that only the write clears:

1. The handler reads 0b01.
2. It writes 0b01 back. The bit clears and the line drops.
3. A new sample arrives: it is queued, its bit is set again, and the line rises.
4. The handler drains the FIFO, taking the new sample with the others.
5. It returns IRQ_HANDLED, with the bit still set and the FIFO empty.
6. The line is still asserted, so the handler runs again: it reads 0b01, acknowledges, finds nothing to drain, and returns.

One interrupt with nothing to do, and **no data lost**: the sample was taken in step 4. On qa-dev itself even that interrupt does not happen. The read that empties the FIFO clears SAMPLE (part f)), so step 5 returns with the bit clear and the line low, and the controller counts a level-triggered interrupt as pending only while its line is high, so the rise in step 3 leaves nothing behind.

d)

1. The handler reads 0b01.
2. It drains the FIFO.
3. A new sample arrives after the last read of the FIFO: it is queued, its bit is set, and the line rises.
4. The handler writes the 0b01 it read in step 1, which clears the bit the new sample set, and the line drops.
5. It returns IRQ_HANDLED, and nothing is pending.

The sample is in the FIFO and nothing will interrupt for it. It is collected only when some later event sets the bit again: on qa-dev, the next sample, one period late; on a device whose events are sporadic, whenever the next one comes, if one does.

e) **c) is survivable and d) is not.** c) costs work: one extra entry to the handler that finds nothing, which is visible, limits itself, and loses no sample. d) destroys information: the only notice of an event is cleared, its data waits on some unrelated later event, and nothing

anywhere reports it. That is why §8.1.5 acknowledges exactly what it read, and does it before the drain.

f) **Empty the FIFO.** Reading `QA_DEV_SAMPLE` pops one sample, and the pop that takes the level to zero clears `IRQ_SAMPLE` and lowers the line, without the driver writing `IRQ_STATUS` at all. To find it, search `tools/qa-dev/qa-dev.c` for every assignment to `irq_status`. It is set in `qa_dev_timer_expired`; cleared by a write to `IRQ_STATUS` in `qa_dev_write`; zeroed by `CTRL_RESET`, which throws the samples away with it; and cleared in `qa_dev_fifo_pop` when `fifo_level` reaches zero, which is the answer. It covers `SAMPLE` only: `OVERRUN` is cleared by a write to `IRQ_STATUS`, or a reset, and by nothing else.

Exercise 8.4 An interrupt budget

HAND CALCULATION

- a) One every 250 us is $1,000,000/250 = 4,000$ interrupts a second. At 8 us each, $4,000 \times 8 = 32,000$ us, 32 ms of every second: **3.2%** of one CPU.
- b) The rate is unchanged, 4,000 a second, and $4,000 \times 40 = 160,000$ us, 160 ms of every second: **16%**.
- c) **250 us**, where $4,000 \times 250$ us is the whole second. In practice sooner, because taking and leaving an interrupt costs time the handler's own figure leaves out, and nothing else runs on that CPU meanwhile. Past 250 us the handler cannot finish one interrupt before the next is due, and the backlog grows until the device's FIFO overflows.
- d) Sixteen samples at one per 250 us fill the FIFO in $16 \times 250 = 4,000$ us: **4 ms**. Counting from the last drain, the worst case has the first new sample arrive at once, the sixteenth fill the FIFO 3.75 ms later, and the seventeenth, at 4 ms, dropped. On this course's kernel that is exactly one jiffy.
- e) The hard handler is now 10 us: $4,000 \times 10 = 40$ ms a second, **4%** of a CPU with interrupts disabled. The threaded half is $4,000 \times 30 = 120$ ms a second, 12%, in a thread the scheduler can place, preempt and prioritise. What does not change is the rate, 4,000 a second; the point at which the CPU saturates, because each interrupt still costs 40 us of a 250 us period; and the 4 ms of part d), which belongs to the device's FIFO and rate, although if the thread does the draining it is now the thread's scheduling delay that must stay under it. **The system's total work does not fall.** It rises a little: still 40 us an interrupt, 16% of a CPU, plus waking the thread and switching to it and back, on every interrupt. What moved is who waits: 12% of a CPU went from time nothing can preempt to time the scheduler decides about (§8.2.6).

Exercise 8.5 Where does the work go

DESIGN

- a) **Hard handler.** It is a couple of register accesses, only the handler can do it, and until it is done the line stays asserted.
- b) **Threaded handler**, with `IRQF_ONESHOT` so that the line stays masked while it runs. 4 KB through a register is 1,024 reads of 32 bits, long enough to matter (§8.2.5), and a FIFO that deep is itself the buffer that lets the copy wait for the thread. The case for the hard handler is the one in §8.2.1, work that would be lost if it waited, and it has to show that the thread's scheduling delay could outlast the FIFO.
- c) **Whichever half puts the data in the buffer, straight after putting it there.** `wake_up_interruptible` does not sleep and is safe in a hard handler (§9.1.4), so in the lab's design, where the hard handler drains, the hard handler. Waking before the data is there wakes a reader to an empty buffer.
- d) **Threaded handler**, or a workqueue if the allocation is not tied to one interrupt: `GFP_KERNEL` may sleep while the kernel reclaims memory. Better still, allocate in advance.
- e) **Workqueue**, as delayed work queued 10 ms ahead. The retry is not one per interrupt, and a threaded handler that slept for 10 ms would keep the line masked, and the next interrupt waiting, for all of it.
- f) **Hard handler:** an `atomic_inc`, or an increment under the lock the handler already holds, costs nanoseconds.
- g) **Threaded handler.** `dev_err` is legal in a hard handler, because `printk` never sleeps, but it is not cheap: the message may be written out to a serial console before the call returns, at 115,200 baud about 87 us a character. A hard handler that can log on every interrupt should at least use `dev_err_ratelimited`.
- h) d) and e). Both reach `schedule()` from a context that has no task of its own to put to sleep. A `GFP_KERNEL` allocation is marked as a place that may sleep, and under memory pressure it does, entering `reclaim;msleep` always does. In a hard handler the scheduler finds itself called with interrupts disabled in the middle of an interrupt, reports `BUG: scheduling while atomic`, and the machine is left with an interrupt that never completes. The allocation is the worse of the two, because it sleeps only when memory is short, so the bug waits for a machine under load.
- i) `CONFIG_DEBUG_ATOMIC_SLEEP`, which `kernel/qa.config` enables. It makes every call that may sleep check its context, and prints `BUG: sleeping function called from invalid context with a backtrace at the call`, every time, including the allocations that would not have slept that time.

Exercise 8.8 The interrupt that never stops

DESIGN

a) `insmod` never returns. The storm starts inside the module's `init`, the moment it sets `IRQ_EN`, on CPU0, where the interrupt is routed, and the task that was interrupted there, `insmod` in the course's run, never runs again. The console says nothing for over twenty seconds; in the course's run the first line after the module's own two was the RCU stall report, at 29.8 s of uptime (§8.1.6). The wait is the stall detector's timeout, `CONFIG_RCU_CPU_STALL_TIMEOUT=21` on this kernel, counted from the start of the grace period that CPU0 is holding up.

b) `rcu: INFO: rcu_preempt detected stalls on CPUs/tasks:, then 0-....: (0 ticks this GP) and Sending NMI from CPU 1 to CPUs 0:.` A grace period ends only when every CPU has passed through a quiescent state: a context switch, the idle loop, or user mode. CPU0 is in hard interrupt context, re-entering the handler for ever, and passes through none of them; "0 ticks this GP" says it has not even taken a timer tick. After 21 seconds RCU reports the CPU that is holding it up, and CPU1, which is still running, asks CPU0 for a backtrace, which is what would name the handler. RCU is not the cause but the first victim with a timeout; this kernel builds neither the soft- nor the hard-lockup detector, which would otherwise have spoken first.

c) **TIMEOUT.** `ci/test.sh` kills QEMU after 120 seconds and reports "L08: TIMEOUT" with the path to the serial log. A failure means the script ran to the end, printed QA-TEST-END with its status, and some checks failed. Here `run.sh` never got past `insmod`: there is no status, and nothing is known about any check after it. Calling that a failure would claim the checks ran; the distinction sends you to the serial log, where the evidence is, rather than to a list of check results that does not exist.

d) `grep qa_irq /proc/interrupts`, twice, a second apart, on CPU1, the CPU still running. It would show interrupt 17's count on CPU0 growing by far more than the thousand a second the period allows, while CPU1's stays at 0: the racing count of §8.1.2, and the diagnosis in one line.

e) **No.** An edge-triggered line interrupts on a transition, not a level. `qa-dev` raises its line and holds it until the bit is cleared, so with nothing clearing it there is one rising edge, one interrupt, and then silence: the line stays high and no new edge ever comes. The machine is fine and the device looks dead. The FIFO fills in sixteen periods, `OVERRUN` sets, and every later sample is dropped, silently.

Exercise 8.9 Interrupts counted two ways

CROSS-CHECK

a) $2\text{ s}/1\text{ ms} = 2,000$.

c)

- **A delta**, because the count belongs to the descriptor and survives `free_irq`: across a reload it carried on from 919 to 993 (§8.1.2). A single reading counts everything since boot, and has no interval to divide by.
- **Using the measured time makes the shortfall larger, not smaller.** Take the course's hard-handler run, 1,789 interrupts in 2.11 s (§8.2.4). Against 2.00 s the ideal is 2,000 and the shortfall $211/2,000 = 10.6\%$; against 2.11 s the ideal is 2,110 and the shortfall $321/2,110 = 15.2\%$. It moves up by 4.7 points, because the interval was longer than two seconds, so more interrupts were due.
- The hypothesis is yours to write down, and part d) is where it meets the source.

d)

- **Relative.** The line is `timer_mod(s->timer, qemu_clock_get_ns(QEMU_CLOCK_VIRTUAL) + s->period_ns)`, run from the timer's own callback after the sample is queued: the next expiry is "now plus a period", where now is when QEMU got round to running the callback, not the moment it was due.
- So each interval is the period plus the callback's lateness, plus the few microseconds the callback spends before it reads the clock again. Every late callback pushes all the later ones back, so lateness accumulates rather than averaging out. Note what is not in it: your driver. The device re-arms inside its own callback, whether or not the handler has run.
- A shortfall s against a period P means each interval is $P/(1 - s)$, so the missing term is $o = Ps/(1 - s)$. At 15% and 1,000 us that is $1,000 \times 0.15/0.85 = 176\text{ us}$; the run above, at 15.2%, gives 179 us, and the Cross-check's own 1 ms row, at 17.6%, gives 213 us. A couple of hundred microseconds is plausible for an emulator whose timers are serviced by threads on a busy host, and would be absurd for a hardware timer. Nothing inside the guest can confirm it.

e)

- **The implied overhead would be the same in every row**, and the shortfall would fall as the period grows, as $o/(P + o)$. With the 1 ms row's 213 us: at 10 ms, $213/10,213 = 2.1\%$, about 206 interrupts in 2.1 s, where 190 were measured; at 0.1 ms, $213/313 = 68\%$, about 6,700 in 2.1 s, where 13,085 were measured. Too few at the slow end, twice too many at the fast end.
- `/proc/uptime` is the guest kernel's clock, counted from the generic timer that QEMU emulates from the same virtual clock the device's timer runs on. So it contains the kernel, whose `sleep 2` overran to about 2.1 s, and QEMU. It does not contain your driver, which decides neither when interrupts are raised nor how time is kept.

- A candidate, not a finding. At 10 ms the guest's CPUs are idle almost all the time and QEMU's threads sleep in the host, so each timer waits on a host wake-up, whose lateness is up to the host's scheduler. At 0.1 ms the guest is busy, QEMU is awake, and an expired timer is noticed sooner. The table's pattern, 947 us of lateness at 10 ms and 63 us at 0.1 ms, fits that, and does not prove it.
- One to run: read the device's timestamp registers, QA_DEV_TS_L0 and TS_HI, in the handler and difference successive stamps. That is the device's own interval between assertions, in the emulator's clock, with the guest's handling taken out; repeat it with the other CPU kept busy, and see whether the lateness at 10 ms falls. One you cannot run here: the same driver on real hardware with an oscilloscope on the interrupt line, which has no emulator in the measurement at all.

f)

- The line is a level. Once SAMPLE is set it stays high until the handler clears it, and a sample that arrives while it is already high changes nothing on the line, so no second interrupt is raised; the sample joins the FIFO, and the handler's drain takes everything that has accumulated. At 0.1 ms, whenever the time from assertion to drain exceeds 100 us, a second sample is waiting: $14,389 - 13,708 = 681$ extra samples, 1.05 per interrupt.
- At 1 ms the handler always drains well inside a period, so every assertion finds exactly one sample. The period changed against the time from assertion to drain; the driver did not.
- OVERRUN: bit 1 of IRQ_STATUS, and the sticky bit 3 of STATUS. The device sets it when a sample arrives to find the FIFO full, which would take sixteen undrained samples, 1.6 ms without service at this period. In the course's run it did not happen: the interrupts coalesced and no data was lost (§8.2.6). To check a run of your own, record the bit in the handler, or run the device in counter mode, where sample n is n , and look for a gap: a dropped sample still advances the device's count.

g) Wrong: **"dropping"**, when nothing is dropped: the device raises fewer interrupts, because each interval is the period plus the emulator's lateness, and every interrupt it raises is serviced. **"15%"**, when the shortfall depends on the period, 8.7%, 17.6% and 38.6% at 10, 1 and 0.1 ms, and even two runs at 1 ms differ, 15.2% and 17.6%. **"the driver is broken"**, when the driver's two counters agree with the device and the cause is in the device model's re-arm and the emulator. Supported by the course's measurements: "At PERIOD_NS = 1,000,000, qa-dev raised 1,789 interrupts in 2.11 s of guest uptime, 15.2% fewer than the 2,110 the period implies, and the driver serviced every one of them, one sample each. The device re-arms its timer from when its callback ran, so each interval is the period plus the emulator's lateness, about 180 us here, and that lateness is not constant across periods." Still unknown: why the lateness depends on the period; how much of it is QEMU, the host, or the guest's load; whether real hardware shows any of it; and anything about latency at all, since every figure here is a count or an average.

h) Not a number from any of this: counts say how many interrupts were raised, not how long any of them waited. What belongs on a datasheet is a distribution of the interval from the line's assertion to the handler, over many thousands of interrupts, with its mean, high percentiles and the worst value observed, and the conditions beside it: kernel version and configuration (6.12.30, PREEMPT, HZ=250), hard or threaded handler, interrupt rate, system load, number of samples and duration, and the platform, which here is an emulator. Refuse to claim a guaranteed worst case, any number for real hardware, and anything inferred from counts or averages. The register is QA_DEV_TS_L0/TS_HI, which records the instant the device asserted its line, in the emulator's clock that the guest's own clock also runs on. Latency is the interval from that instant to the handler, and software can read when its handler ran but not when the line went up; on real hardware only an instrument on the wire sees that. The register hands software the start of the interval, so it can subtract instead of infer.

A.9 Chapter 9: Sleeping, Waiting, and Time

Answers to the exercises in §9.4. Exercises 9.6 and 9.7 are Code, and are checked on the target as their Check yourself lines say.

Exercise 9.1 Sleeping and spinning

RECALL

a) The sleeping task is in TASK_INTERRUPTIBLE, S in ps, or TASK_UNINTERRUPTIBLE, D; it is not on the run queue, and it costs no CPU time at all, only its memory and the wake-up when it comes. The spinning task is in TASK_RUNNING, R; it is on the run queue, and it costs a whole CPU while it has one, every time slice taken from a task with work to do (§9.1.1).

b) A signal, even SIGKILL, is acted on only when the task is on its way back to userspace, and a task in TASK_UNINTERRUPTIBLE is not woken by signals at all, only by an explicit wake. So the signal waits, pending, until whatever the task is waiting for finishes. That is usually I/O: a disk or network-filesystem request, a page being read in, a device operation. A D in ps says something the task depends on is not completing, a stuck device, a hung driver, a server that has gone away; the process is the symptom, and the cause is below it. On Linux such a task also counts in the load average.

c) TASK_INTERRUPTIBLE. A user can always interrupt or kill the reader, and your driver answers with -ERESTARTSYS. Put a reader in TASK_UNINTERRUPTIBLE to wait for a device that never produces data, and you have made the unkillable D process of part b) yourself. The uninterruptible state is for short waits that cannot be abandoned half way. (For a wait that only a fatal signal should end, there is a third, TASK_KILLABLE, which wait_event_killable uses.)

d) The task's state must be set to something other than TASK_RUNNING before the call. With the state still TASK_RUNNING, schedule() leaves the task on the run queue and may pick it

again at once: a yield. And for the sleep ever to end, the task must be somewhere a wake-up can find it, on the wait queue the waker uses.

Exercise 9.2 The lost wakeup

RECALL

a)

1. The reader calls `fifo_empty()`, which finds the FIFO empty and returns true.
2. **The interrupt lands here**, after the test and before `set_current_state`. The handler pushes a sample and calls `wake_up_interruptible`. The reader is on the queue but still `TASK_RUNNING`, so there is nothing to wake, and the wake does nothing.
3. The reader sets `TASK_INTERRUPTIBLE` and calls `schedule()`, and leaves the run queue with a sample waiting for it.
4. Nothing wakes it until the next `wake_up`: on a device that interrupts again, a period later; if that was the last event, never.

b) Two instructions: the load inside `fifo_empty()` that finds the FIFO empty, and the store in `set_current_state`. That is nanoseconds, and how soon it is hit is the chance of one attempt times the attempts per second. An interrupt at an unrelated moment lands in a window w wide, in a period of 1 ms, with probability $w/1$ ms, of the order of a few in a million for a few nanoseconds; and a reader that blocks for each sample offers the window up to a thousand times a second. That is a few chances in a thousand every second: on paper, minutes of continuous running. On a real machine it depends on how long the window really lasts and on when the reader's sleeps fall relative to the interrupts, and it is found by running it. Either way it is found in a test run, not once in a product's lifetime.

c) `long __int = prepare_to_wait_event(&wq_head, &__wq_entry, state);`. It puts the task on the queue and sets its state, under the queue's lock, before the condition is tested; a `wake_up` that comes after it finds a task to wake (§9.1.3).

d) **The lost wakeup**: the test has to happen after the task is queued, so the macro must be able to evaluate it there, inside the loop; a variable's value passed in would be read once, at the call, before queueing, which is exactly the bug. **After a wake**: a wake does not mean the condition is true, because another reader may have taken the data or the wake may be spurious, so the macro evaluates the expression again every time round the loop.

e) It must be **cheap**, and **free of side effects**, because it is evaluated once before the task is queued and again after every wake, as many times as that takes. It is also evaluated with the task already in `TASK_INTERRUPTIBLE`, so it must not sleep: a sleeping call there resets the state, and `CONFIG_DEBUG_ATOMIC_SLEEP` warns do not call blocking ops when `!TASK_RUNNING`.

f) It measures the wrong thing. What decides how often the bug fires is not the width of the window but the chance of one attempt times the number of attempts: the window is offered every time a reader sleeps, a thousand times a second, on every unit shipped, for

years, and a chance that is rare per attempt is certain in aggregate. Rare is also the worse outcome: a bug that fires once a week passes every test and cannot be reproduced when it is reported. And a window you can point at is a bug whatever its width, when the fix, queueing before testing, costs nothing.

Exercise 9.3 Jiffies

HAND CALCULATION

a) $1,000,000/250 = 4,000$ us, 4 ms.

b) With HZ dividing 1,000 exactly, `msecs_to_jiffies(m)` is $(m + 3)/4$ in integer arithmetic: $(1 + 3)/4 = 1$, $(4 + 3)/4 = 1$ and $(10 + 3)/4 = 3$. It rounds **up**, and that matters because a timeout must never be shorter than it was asked to be. Rounded down, `msecs_to_jiffies(1)` would be 0, and `msleep(1)`, which sleeps while its count of jiffies is not zero, would not sleep at all; and 10 ms would become 2 jiffies, 8 ms.

c) One jiffy more than `msecs_to_jiffies(n)`: `msleep(1)` is $(1 + 1) \times 4 = 8$ ms, `msleep(4)` is also $(1 + 1) \times 4 = 8$ ms, and `msleep(10)` is $(3 + 1) \times 4 = 16$ ms. The course measured 8,000, 8,000 and 15,998 us (§9.2.4). The extra jiffy is not in `msleep`, which passes `msecs_to_jiffies(n)` to `schedule_timeout` unchanged; it is the timer wheel's, which rounds every expiry up by a jiffy so that a timer armed part of the way through a tick never fires early (`calc_index` in `kernel/time/timer.c`). Each call in a loop starts just after the tick that woke the one before, so it sleeps $n + 1$ whole jiffies.

d) `msleep(1)` and `msleep(4)`, 8 ms each. The loop runs eight times slower than its author believes: 1,000 iterations take 8 s, not 1 s. Asking for less than a jiffy buys nothing.

e) At HZ=1000 a jiffy is 1 ms and `msecs_to_jiffies(n)` is n : `msleep(1)` is $1 + 1 = 2$ ms, `msleep(4)` is $4 + 1 = 5$ ms, and `msleep(10)` is $10 + 1 = 11$ ms.

f) `>` breaks when the counter wraps between computing the deadline and comparing against it. Computed 50 ticks before the wrap, `jiffies + 125` wraps round to 75, and `jiffies > timeout` is true at once: the timeout expires immediately. Computed so that `jiffies` wraps first, the comparison is false until the counter comes all the way round, and the timeout never expires. `time_after` compares the signed difference, which is right as long as the two are less than half the range apart. The wrap comes every $2^{32}/250 = 17,179,869$ s, **198.8 days**; but the first one comes **five minutes after boot**, because the kernel starts the count at `INITIAL_JIFFIES`, $-300 \times \text{HZ}$, on purpose, so that code with this bug fails in its first five minutes rather than after half a year. (On the course's 64-bit target `jiffies` itself never wraps; its low 32 bits still do, five minutes after boot, which catches code that keeps it in a `u32`.)

Exercise 9.4 Choosing a delay

HAND CALCULATION

- a) **udelay(2)**. Inside a spinlock nothing may sleep, and 2 us of busy-waiting costs less than any sleep would to arrange.
- b) **usleep_range** between reads of the bit, for example `usleep_range(100, 200)`, with an overall timeout; `readl_poll_timeout` is that loop already written. probe may sleep, `udelay` would burn the whole 500 us, and `msleep(1)` would cost 8 ms a poll on this kernel.
- c) **msleep(100)**, in process context: `msecs_to_jiffies(100)` is 25, so the call takes 26 jiffies, 104 ms, and a quantum of 4 ms is noise at that length. If the retry must not block its caller, a timer.
- d) **None of them** (this is the trick). A hard interrupt handler may not sleep, and a busy-wait of 5 ms, `mdelay(5)`, is legal but spends 5 ms with interrupts disabled on that CPU. Do not wait in the handler: acknowledge, and do the waiting in the threaded half, or arm a timer to do the next step 5 ms later.
- e) **A timer**, a `timer_list` armed 2 s ahead and cancelled when the device answers: a timeout that is usually cancelled and whose precision is irrelevant is the case the wheel exists for (§9.2.5).
- f) **usleep_range(50, 50)**: process context may sleep, and with no slack an `hrtimer` is the most accurate way to. The alternative is `udelay(50)`, and the course's measurements are its argument: `usleep_range(100, 100)` took 307 us and `udelay(100)` 112 (§9.2.4), so on this target a sleep carries about 200 us of wake-up overhead. If the requirement is 50 us and not 250, `udelay` is the one that delivers it, at the price of 50 us of a CPU every time.
- g) **d)**: of the four ways to wait it offers, none is right in a hard interrupt handler, and the right answer is not to wait there.

Exercise 9.5 The four cases of read

DESIGN

- a) **160.** The count is in bytes and a sample is 32 bits, so 100 samples is 400 bytes, and `read` returns the 40 samples there are, 160 bytes. The program sees a short read at once, and calls `read` again if it wants more. (Had the 100 been bytes, 25 whole samples: 100.)
- b) **Nothing yet:** the driver sleeps in `wait_event_interruptible` until the handler adds a sample and wakes it, then returns the whole samples available, at least 4 bytes. The program sees `read` block for up to a period, about a millisecond on a running `qa-dev`, and then return a positive count.
- c) **-EAGAIN**, at once. The program sees `-1` with `errno` set to `EAGAIN`, “Resource temporarily unavailable”.
- d) **-ERESTARTSYS:** the wait returns it because a signal is pending, and the driver passes it on. The program never sees it. `Ctrl-C` sends `SIGINT`, whose default action ends the process, so `read` never returns and the program dies, which is what `Ctrl-C` is for. A program with a handler installed with `SA_RESTART` runs the handler and has its `read` restarted without noticing; one whose handler has no `SA_RESTART` sees `-1` with `errno` set to `EINTR`.
- e) The driver now has something that can wake a reader, the interrupt handler of Chapter 8, and a way to sleep without losing the wake. Chapter 5 had neither: it could not wait correctly, so the only honest answers were the data or “nothing now”, and 0 would have said “nothing ever”.
- f) **-EINTR** takes the decision away from the kernel. The program sees `EINTR` whenever a handled signal arrives, even one installed with `SA_RESTART`, which asked for the call to be restarted; a program that does not retry on `EINTR` then fails for no reason it can see, whenever a `SIGCHLD` or a timer signal happens by. **Ignoring the return value** depends on what comes next. If the code goes on to take data from a FIFO that is still empty, it returns 0, end of file, and the program stops as if the device had finished. If it loops back to wait, the signal is still pending, so every wait returns at once and the `read` spins in the kernel at full speed without ever returning, and a signal is delivered only on the way back: a reader that cannot be killed (§9.1.5).
- g) End of file: no more data, ever. `cat /dev/qa_wait` exits at once with status 0 having printed nothing, as if the device had finished, and so does any loop that reads until `read` returns 0.

Exercise 9.8 A timer instead of an interrupt

DESIGN

- a) Load the module with `ENABLE` set and `IRQ_EN` clear, and arm a `timer_list`. Its callback reads the device's FIFO, puts the samples into the driver's FIFO under the spinlock, wakes the readers, and re-arms itself. The callback runs in softirq context (§9.2.5): it may touch registers, take a spinlock and wake a queue, and may not sleep, take a mutex, allocate with `GFP_KERNEL` or call `copy_to_user`. What moves: the lock's process-context side must now disable softirqs as well, `spin_lock_bh` or the `irqsave` it already has, and anything that sleeps stays in read. Two details. 4 ms is one jiffy at `HZ=250`, the finest a `timer_list` does, and it must be re-armed from its previous expiry, `mod_timer(&t, t.expires + 1)`: re-armed from `jiffies + 1` it polls every 8 ms, for the same reason `msleep(1)` takes two jiffies. And on unload it re-arms itself, so it is stopped with `timer_shutdown_sync`, which also refuses any later re-arm.
- b) The resolution is no longer the jiffy: any period, 4 ms or 1 ms, exactly, and periodic from its previous expiry with `hrtimer_forward_now` and `HRTIMER_RESTART`. The callback runs in hard interrupt context, which is the default on this kernel, so it is more restricted than the `timer_list`'s: the lock shared with read must be taken `irqsave`, and its time is interrupt time again. (A `_SOFT` mode moves it to softirq, and under `PREEMPT_RT` any hrtimer not marked `_HARD` is moved there.)
- c) With the `timer_list` the sample waits in the device's FIFO until the next poll, up to 4 ms, more if the timer is late. With the hrtimer, up to its period, which can be as short as the wake-ups you are willing to pay for. With the interrupt it is taken when it arrives. Polling adds, on average, half its period to the latency of every sample.
- d) Sixteen samples at one a millisecond fill the FIFO in 16 ms, and the seventeenth, 16 ms after the first undrained one, is dropped: two polls must never be more than **16 ms** apart, and the nominal period has to leave room for how late the timer can be. With a FIFO of 4 it is **4 ms**, exactly one jiffy, which a `timer_list` cannot promise, since it may fire a jiffy late; that needs an hrtimer, or the interrupt.
- e) Poll when you know when the data will be there, or when it is always there: a steady stream in which every poll finds work, which is why network drivers switch to polling under load. Poll too when the device cannot interrupt. Use the interrupt when events are sporadic, or when latency matters: a poll that usually finds nothing is pure cost, and its period is added latency. A timer is what you use when nothing tells you (§9.2.5).

Exercise 9.9 A millisecond that is not a millisecond**CROSS-CHECK**

a) One jiffy is 4 ms, and each `msleep` is one jiffy more than `msecs_to_jiffies` gives:

Call	Jiffies	Predicted
<code>msleep(1)</code>	$1 + 1 = 2$	8,000 us
<code>msleep(4)</code>	$1 + 1 = 2$	8,000 us
<code>msleep(10)</code>	$3 + 1 = 4$	16,000 us
<code>usleep_range(1000, 1000)</code>	not quantised	a little over 1,000 us

`usleep_range` uses an `hrtimer`, so it costs the request plus the time to take the timer interrupt and switch the task back in; the model gives no figure for that.

c)

- `msleep(1)`: 8,000 us for 1,000 asked, **eight times**. The predictions of part a) expect it exactly; a prediction from “at least one jiffy” alone says 4 ms, out by two. Followed as written, the prediction furthest from its measurement is `usleep_range`’s: 1,000 us against 1,443, a factor of 1.44 (§9.2.4).
- `msleep(1)` and `msleep(4)`, 8,000 us each. Not a bug in the measurement or in `msleep`, but in the expectation: `msleep` promises at least what it was asked, in jiffies, and 1 ms and 4 ms are both one jiffy.
- `msleep` passes `msecs_to_jiffies(n)` to `schedule_timeout` with nothing added. The sentence is “at least @timeout jiffies are guaranteed to pass before the routine returns”, from its comment in `kernel/time/timer.c`, for the uninterruptible sleep `msleep` uses. The call arrives part of the way through a jiffy, and counting that partial jiffy as one would return early, so the timer must fire at the $(n + 1)$ th tick after the call; the timer wheel does it by rounding every expiry up by one jiffy. In a loop each call starts just after a tick, and so waits $n + 1$ whole jiffies.
- A high-resolution timer, programmed on the hardware timer rather than on the jiffy wheel. What it gave up is the range: `usleep_range(1000, 1000)` leaves the kernel no slack, so it must program a wake-up for this sleep alone rather than fold it into one already due, which is the saving the range exists for.

d) `msecs_to_jiffies(5)` is $(5 + 3)/4 = 2$, so `msleep(5)` takes $2 + 1 = 3$ jiffies, **12,000 us**. That is the test that separates the accounts: “one jiffy more than `msecs_to_jiffies`” predicts 12 ms, where “twice the rounded-up request” would predict 16. The course does not publish this measurement; the prediction is 12,000 us.

e) At `HZ=1000`: `msleep(1)` 2 ms, `msleep(4)` 5 ms, `msleep(10)` 11 ms, and `usleep_range(1000, 1000)` a little over 1 ms, as before. The three `msleep`s change and `usleep_range` does not. `msleep` is quantised: it costs its request rounded up to jiffies, plus one, and the jiffy is `HZ`’s. `usleep_range` is not: it costs its request plus a wake-up overhead that has nothing to do with `HZ`.

f) On this kernel $1,000 \times 8$ ms, 8 s; at HZ=1000, $1,000 \times 2$ ms, 2 s. Written as `msleep(1000)`; it takes one second on both, to within the timer wheel's rounding. At that length the wheel places the timer in a coarser level and rounds its expiry up to that level's step: 8 jiffies at HZ=250, so between 1,000 and 1,032 ms, and 64 jiffies at HZ=1000, so between 1,000 and 1,064 ms. Either is a second, for a comment.

g) Put what was measured and how: average durations over 200 repetitions on an idle QEMU guest, kernel 6.12.30 with PREEMPT and HZ=250, and the structure that is not a measurement, that `msleep(n)` waits at least n ms and in a loop costs `msecs_to_jiffies(n)` plus one jiffies. Refuse to claim any maximum or worst case, and any figure under load or on other hardware: an average over an idle machine says nothing about the tail, and these numbers contain the emulator. Measure instead the whole response, from the event to the driver's action, as a distribution over a long run under realistic load, on the product's own hardware and configuration, and report its high percentiles and the worst value seen, with a trace of what caused the outliers; and even then a measured maximum is an observation, not a bound. Chapter 12 is that measurement.

A.10 Chapter 10: The Device Model and the Device Tree

Answers to the exercises in §10.4. Exercises 10.6 and 10.7 are Code, and are checked on the target: the first as its Check yourself line says, the second by the `unbind` and `rebind` that `run.sh` performs.

Exercise 10.1 Bus, device, driver

RECALL

a) A **bus** is a matching policy: the rule by which the kernel decides which of the drivers registered with it can handle which of the devices registered with it, and the two lists it applies the rule to. A **device** is the kernel's record that something exists, with whatever describes it: its identity, its registers, its interrupts. A **driver** is code that says which devices it can handle and supplies the `probe` and `remove` that take one on and give it up.

b) It is for devices that cannot be discovered: the blocks inside an SoC, which sit at fixed addresses on the processor's own interconnect and announce nothing. That is also what its devices share: something other than the hardware has to declare that each one exists, and on this target that something is the device tree. The bus then matches them against drivers by `compatible`, by ACPI identity or by name (`platform_match`, `drivers/base/platform.c`).

c) The kernel created it, from the `qa-dev@0` node in the device tree QEMU generated, during boot and before any module could be loaded. `of_platform_default_populate_init`, an `arch_initcall_sync` (`drivers/of/platform.c:600`), walks the tree from the root. It descends into `platform-bus@c0000000` because that node's `compatible` includes `simple-bus`, and creates a platform device for each child. `of_device_alloc` translates the node's `reg` into

a resource there and then, and the device is named `c000000.qa-dev` from that translated address and the node's name (`of_device_make_bus_id`, `drivers/of/device.c:302–309`).

d) No; either order works, and that is the point of the model. **Device first**, as `qa-dev` is: registering the driver makes the bus try it against every device not yet bound, and `probe` runs inside `insmod`. **Driver first**: the driver waits, and when the device is added later, because it was plugged in, or an overlay added its node, or the driver of its parent probed, the bus tries the new device against every registered driver and `probe` runs then. The author does nothing differently for either: `probe` is written the same way. What the author must not do is assume an order between devices, which is what `-EPROBE_DEFER` is for.

e) `/sys/bus/<bus>/devices` groups devices by how they are attached, which is the matching policy that found them; `/sys/class/<class>` groups them by what they are to userspace, whatever they are attached by. A program looking for all the serial ports reads `/sys/class/tty`, where a PL011, a USB serial adapter and a virtual console all appear, although the first is on the `amba` bus, the second on `usb`, and the third on no bus at all.

Exercise 10.2 Probe and remove

RECALL

a) Once for each device that matches: two `qa-dev` nodes, or two of the same chip on a board, mean two calls to the same function in the same loaded module. It also runs again for a device that is unbound and bound again, and after a deferral. That rules out keeping anything that belongs to a device in a file-scope variable. Everything a device needs hangs off its own `struct device`: allocated with `devm_kzalloc` against it, attached with `platform_set_drvdata`, and found again from whatever pointer each callback is handed.

b) Any machine with two of the device, as in part e) of Exercise 10.9. The second `probe` overwrites `regs`, so from then on everything done for the first device goes to the second device's registers. The first device's handler reads and acknowledges the second device's `IRQ_STATUS`, never its own, so the first device's level-triggered line is never cleared: an interrupt storm (§8.1.6), or, if the handler correctly returns `IRQ_NONE` when the status it read shows nothing pending, the kernel disabling the line with `nobody_cared` (§8.1.3). And unbinding the second device unmaps the registers the first is still using.

c) Because by the time `remove` is called, the decision has been made somewhere the driver has no say: `rmmod`, a write to `unbind`, or the device itself going away, perhaps already physically gone. The core has nothing to offer in return for an error. It cannot keep a driver bound to a device that is being removed, or unload half a module, and a `remove` that stops partway leaves state that nobody can finish tearing down. So a failure from `remove` would mean nothing, and the void return type (`include/linux/platform_device.h:245`) says so. What the driver must do instead is make every step of `remove` succeed, which with `devm_` is little more than stopping the device.

d) It says “something I need is not available yet; call me again later”: a clock, a regulator, a GPIO or an interrupt controller whose driver has not probed. The core puts the device on its list of deferred devices without reporting a failure (`driver_deferred_probe_add`, `drivers/base/dd.c:132`), tries every device on that list again each time another driver binds successfully, and once more when the initcalls are done; anything still waiting is listed in `/sys/kernel/debug/devices_deferred`. The functions that look up a dependency, such as `devm_clk_get` and `platform_get_irq`, return `-EPROBE_DEFER` themselves, so a driver usually only passes it on. The alternative is to return a real error such as `-ENODEV`: the probe fails for good, and the driver works only on a kernel where the dependency happened to probe first, which is decided by link and initcall order and not by anything the driver controls.

e) Whatever makes the device reachable from userspace: registering a character device, a class device, or the device with a subsystem such as IIO or GPIO. The reason is the one §5.1.3 gave: after `cdev_add` the device is live, and a call can arrive in your code, on another CPU, before probe has returned. So everything that call touches, the mapping, the interrupt, the private structure, must be set up before the call that publishes it. The driver core keeps the same rule for the attributes in `.dev_groups`: it adds them only after probe has returned successfully (`drivers/base/dd.c:676`).

Exercise 10.3 Reading a device tree

HAND CALCULATION

a) Two: one address cell and one size cell. You know because thing’s parent, `bus@10000000`, declares `#address-cells = <1>` and `#size-cells = <1>`; the node itself does not say. So `reg = <0x2000 0x40>` is one region, at address `0x2000` and `0x40` bytes long.

b) Four. The child address takes the bus’s own `#address-cells`, 1: `0x1000`. The parent address takes the root’s `#address-cells`, 2: `0x0 0x10000000`, which is `0x0_10000000`. The size takes the bus’s `#size-cells`, 1: `0x100000`. So child addresses `0x1000` to `0x100FFF` map to parent addresses `0x10000000` to `0x100FFFFF`.

c) `0x2000` lies inside the child range, which starts at `0x1000`, so the offset into the range is `0x2000 - 0x1000 = 0x1000`, and the registers start at `0x10000000 + 0x1000 = 0x10001000`. They end at `0x10001000 + 0x40 - 1 = 0x1000103F`. Adding the child address to the parent base without subtracting the child base, which is the mistake a range that does not start at zero exists to catch, gives `0x10002000`, one page too high.

d) `acme, thing-v2`. A driver binds if its table contains any of the node’s strings, and where the table contains several, the entry for the earlier string scores higher (`__of_device_is_compatible`, `drivers/of/base.c:334`), so a driver that knows both uses its `thing-v2` entry and whatever match data goes with it. The second string is the fallback: it says the chip is compatible with the original thing, so a kernel whose only driver is for that, an older one for instance, still binds the device and drives what the two have in common.

e) Cell 0 is 0, an SPI; cell 1 is 45, 0x2D, the number among the SPIs; cell 2 is 1, rising edge. The GIC's own numbering adds the SPI base of 32, so this is hardware interrupt $45 + 32 = 77$, 0x4D, edge-triggered on the rising edge.

f) Yes: `acme,thing` is the node's second string, and one match is all that binding needs. With only `acme,thing-v3` it does not bind, because that string is not in the node's list, and `probe` is never called. A driver for a newer chip cannot claim an older one; compatibility is declared by the device, not by the driver.

Exercise 10.4 What the helpers do

HAND CALCULATION

a) At the moment of the call it reads nothing from the tree. It takes the device's memory resource 0, which the kernel built from `reg` entry 0, translated through every ranges above it, when it created the device (`drivers/of/platform.c:133`); then it claims that region and maps it. On success it returns the mapping, a `void __iomem *`. On failure it returns an error packed into the pointer: `-EINVAL` if there is no such memory resource, `-EBUSY` if the region is already claimed, `-ENOMEM` if the mapping fails (`__devm_ioremap_resource`, `lib/devres.c:135–165`). It never returns `NULL`.

b) Entry 0 of interrupts, with `interrupt-parent` to find the controller, or `interrupts-extended`, which names both. It has that controller's IRQ domain translate the specifier and create the mapping, and returns the Linux IRQ number, which is positive. On failure it returns a negative error: `-EPROBE_DEFER` if the controller has not registered its domain yet (`drivers/of/irq.c:470`), `-ENXIO` if there is no such interrupt, and `-EINVAL` rather than 0 if the mapping produced 0 (`drivers/base/platform.c:234–238`). It never returns 0.

c) The property `fifo-depth`, as one big-endian cell, which it stores in `v` in the processor's own byte order. It returns 0 on success. On failure it returns `-EINVAL` if the property does not exist, `-ENODATA` if it has no value, and `-EOVERFLOW` if it is too short to hold a cell (`include/linux/of.h:1350`), and leaves `v` untouched, which is what makes the default idiom of §10.2.6 work.

d) `platform_get_irq` returns a negative error code as an `int`, and a positive number on success: test `irq < 0`, and return `irq` if it is. `devm_platform_ioremap_resource` packs its error into the pointer: test `IS_ERR(regs)`, and return `PTR_ERR(regs)` if it is. `of_property_read_u32` returns zero for success and delivers the value through its third argument: any non-zero return is the failure, which for an optional property means “use the default” rather than a reason to give up.

e) It works on every board where the call succeeds, because the number is positive and the check passes, and that includes every board the driver was tested on. It silently does the wrong thing on every board where the call fails, because `platform_get_irq` never returns 0 and the check never fires. The negative error goes on to `devm_request_irq` as an unsigned `int`, a huge number with no descriptor behind it, and the request fails with `-EINVAL` (`kernel/`

irq/manage.c:2178–2180). The worst case is a board whose interrupt controller probes after this driver: `platform_get_irq` returns `-EPROBE_DEFER`, which it does not print, the driver turns it into `-EINVAL`, the core reports that the probe failed with error -22, and the device is never tried again.

f) From §6.2.7: `request_mem_region` and `ioremap`, the two acquisitions, and `iounmap` and `release_mem_region`, the two releases. With `devm_` the releases happen at unbind, so they disappear from the error path and from `remove` alike.

Exercise 10.5 What belongs in a device tree

DESIGN

a) The device tree, as `reg`. Where the block sits is a fact about the SoC and the board, different on every design that uses the chip, so the driver must not contain it.

b) The device tree, as `interrupts` and `interrupt-parent`. Which line of which controller the device is wired to is again a fact about the board.

c) A module parameter, or nothing at all. The depth of a software FIFO trades memory against the risk of dropping data: a policy of this system’s software, not a fact about the hardware, and the same board may want different depths for different uses. A constant in the driver is right until somebody needs to tune it, and a module parameter is right then. The `fifo-depth` property that §10.2.6 reads is legitimate only if it describes the device’s own hardware FIFO, as `QA_DEV_FIFO_DEPTH`, 16, does for `qa-dev`: that is hardware description.

d) Neither, normally. How many channels a chip has follows from which chip it is, and `compatible` already says that; the driver looks the number up from the match, typically in the `.data` of its `of_device_id` entry. A property is defensible only when the number depends on the board rather than on the chip, for instance on which inputs are wired to anything, and bindings usually describe that with a child node for each connected channel. That is what an answer of “the device tree” has to argue.

e) Not the device tree, which describes hardware. A module parameter at most, and on this kernel not even that: `dev_dbg` with dynamic debug, which `kernel/qa.config` enables, switches a driver’s messages on and off at run time, call site by call site, with no parameter at all (§4.1.7).

f) The device tree: which crystal is fitted is a fact about the board. It is best described as a clock rather than as a bare number: a `fixed-clock` node carrying `clock-frequency`, referenced from the device’s `clocks` property, so that the driver asks the clock framework for the rate (Chapter 11).

g) Neither. What a node in `/dev` is called is userspace’s decision: the kernel’s subsystem gives it a default name, and `udev` or `mdev` rules rename it or add links to suit the system. It is not a fact about the hardware, so it cannot be in the tree, and a parameter would move a

naming policy into the driver. The nearest thing a device tree legitimately carries is an alias such as `serial0`, which numbers an instance and names no file.

h) The device tree describes the hardware, what would be true of the board under any operating system and any driver; everything that is a choice about how software uses that hardware belongs somewhere else.

i) Anything put in a device tree has to be supported for as long as a DTB containing it exists. The DTB may come from firmware that is never updated, so every later kernel must accept it: a property cannot be renamed, removed or given a new meaning, and a mistake is kept. The binding is also shared with every other consumer of the tree, the bootloader and other operating systems, which is why it is reviewed as hardware description. A module parameter is set on a command line or in a configuration file that ships in the same system image as the module that reads it, and changes with it: it is still an interface, visible under `/sys/module`, but the people who can change it are the people who ship the kernel. The device tree often belongs to somebody else entirely.

Exercise 10.8 Autoloading

DESIGN

a) The four steps:

1. `MODULE_DEVICE_TABLE` has `modpost` turn the match table into alias strings in the `.ko's .modinfo`.
2. `depmod` collects the aliases of every installed module into `modules.alias`.
3. The device's `uevent` carries a `MODALIAS=` string built from its node.
4. `udev` receives the `uevent` and runs `modprobe` with that string, which finds the module through `modules.alias`.

Step 4 is the one missing: there is no `udev`, the `init` script starts no `mdev`, and the kernel is built without a `uevent` helper (`CONFIG_UEVENT_HELPER` is not set). For your own module step 2 is missing as well. It is loaded from `/lab` rather than installed under `/lib/modules`, so the `modules.alias` on the target, which `ci/rootfs.sh` copies across with the kernel's own `modules`, has never heard of it.

b) Step 1, and through it step 2. Without it the `.ko` advertises no alias, `depmod` has nothing to collect, and `modprobe` given the device's `MODALIAS` finds no module, so the driver is never loaded automatically. Loaded by hand it works perfectly, because binding uses the `of_match_table` itself (`platform_match`, `drivers/base/platform.c:1345`) whether or not it was exported, which is why the omission goes unnoticed on a target that loads everything by hand.

c) `OF_COMPATIBLE_0=qacademy,qa-dev-1.0` is the node's compatible property, copied into the `uevent` by `of_device_uevent` (`drivers/of/device.c:238–239`). QEMU wrote that property into the tree it generated, from `QA_DEV_DT_COMPATIBLE` in `qa-dev.h`, in `add_qa_dev_fdt_node` in `tools/qa-dev/integrate.py`. The `MODALIAS` line, `of:Nqa-devT(null)Cqacademy,qa-`

dev-1.0, is built from the same node by `of_modalias` (`drivers/of/module.c:29–50`): `N` and the node's name, `T` and its `device_type`, which this node does not have, and `C` before each compatible string.

d) Two aliases, which `modpost` generates from the one entry in the table (`scripts/mod/file2alias.c:366–381`): `of:N*T*Cqacademy,qa-dev-1.0` and `of:N*T*Cqacademy,qa-dev-1.0C*`, the second of which also matches a node that lists further compatible strings after this one. `depmod` would collect them into `/lib/modules/6.12.30/modules.alias`, with an indexed copy in `modules.alias.bin`; here it has not, because the module is not installed there.

e) The steps are the same; what differs is where the identity comes from. A USB device describes itself: when it is plugged in, the USB core reads its descriptors, vendor, product and class, and the `uevent` carries a `MODALIAS=usb:v...p...` built from them, which the aliases of drivers declaring `MODULE_DEVICE_TABLE(usb, ...)` match. A device soldered to the board describes nothing: a block of registers at an address has no descriptors to read and no bus that could ask for them. Its identity has to come from outside, and the device tree is where it comes from: the node's `compatible` is what the kernel creates the device from and builds its `MODALIAS` out of. Without the node there is no device, no `uevent` and nothing to match.

Exercise 10.9 An address derived three ways

CROSS-CHECK

a) The bytes, which §10.2.9 prints, grouped four to a cell, most significant first, with the parent's ranges alongside:

reg	00 00 00 00	00 00 10 00		
	0x00000000	0x00001000		
ranges	00 00 00 00	00 00 00 00	0c 00 00 00	02 00 00 00
	0x00000000	0x00000000	0x0c000000	0x02000000
interrupts	00 00 00 00	00 00 00 70	00 00 00 04	
	0x00000000	0x00000070	0x00000004	

The parent, `platform-bus@c000000`, declares one address cell and one size cell, so `reg` is child address `0x0` and size `0x1000`. In `ranges` the child address is one cell, `0x0`; the parent address is two, because the root declares `#address-cells = <2>`, so it is `0x00000000_0c000000`; and the size is one cell, `0x02000000`. The device's window lies inside that range, so the physical address is `0x0C000000 + (0x0 - 0x0) = 0x0C000000`, the size `0x1000`, and the window runs from `0x0C000000` to `0x0C000FFF`. The interrupt is type 0, an SPI; number `0x70`, which is 112; flags 4, level high. The GIC hardware interrupt is `112 + 32 = 144`, `0x90`.

b) The name is `c000000.qa-dev`, and the number in it is the physical address, `0xC000000`, not the `0x0` in `reg`. So the translation happened before any driver was involved: when the kernel created the device at boot, which translated `reg` through `ranges` and named the device from the result (`of_device_make_bus_id`, `drivers/of/device.c:302`).

c) A correct driver prints start as `0xC000000` and end as `0xC000FFF`, which is the last byte of the window rather than one past it (`0xC000000 + 0x1000 - 1`), and the number `platform_get_irq` returned, which was 17 on the course's target (§10.1.6).

d) **They agree**, on `0x0C000000`. A hand figure of `0x0` took `reg` at face value, or took the first parent cell, `0x00000000`, as the base. That cell is only the high half of a two-cell address; the base is that cell and the next together, `0x00000000_0c000000`.

17 and 144 are both right because they are numbers in two different spaces (§8.1.1). 144 is the GIC's hardware number, fixed by the wiring. 17 is the Linux IRQ number, an index the kernel handed out when the GIC's IRQ domain first mapped hardware interrupt 144, and it depends on how many mappings were made before it. In `/proc/interrupts` the first column, `17:`, is the Linux number; after the per-CPU counts come the controller, GIC-0, its hardware number, 144, and the trigger, Level.

-tx4 reads each group of four bytes as one integer in the host's own byte order, which on this target is little-endian, so it byte-swaps every cell:

```
reg      00000000 00100000
ranges   00000000 00000000 0000000c 00000002
interrupts 00000000 70000000 04000000
```

-tx1, reassembled most significant byte first, is correct, because device tree cells are big-endian on every machine (§10.2.4). The wrong answer is plausible because four of the nine cells are zero and read the same either way, and the rest still look like numbers: `0x0000000C` is as good an address as any to someone who has not seen the right one, and `0x00100000` is a perfectly ordinary 1 MiB window. Only `0x70000000` as an interrupt looks absurd, and a reader who stops at the address never gets that far.

e) **The address** is in the device tree QEMU generated when it started, the file `dumpdtb` would have written (§10.2.8), which the kernel shows under `/sys/firmware/devicetree/base:node /platform-bus@c000000/qa-dev@0`, property `reg`, translated by the parent's `ranges`. The number itself is QEMU's choice: its virt machine puts the platform bus at `0x0C000000` (`hw/arm/virt.c:187`), and the first device placed on it goes at offset 0.

The second device. With a private structure allocated for each device, each probe has its own registers, interrupt and counters, so the two `interrupts` attributes count two different devices, as the two lines of `/proc/interrupts` in the exercise do, 426 and 420. With a file-scope `static`, the second probe overwrites the first one's state: both attributes read the same numbers, the handler for interrupt 17 reads and acknowledges the second device's registers, so the first device's level-triggered line is never cleared, and unbinding either device unmaps registers the other still uses. The Chapter 6 driver could not have coped at all: it maps the one address its `base=` parameter names, a module can be loaded only once, and so it drives exactly one device, at an address somebody had to type.

0xC001000. In Exercise 6.9 nothing was there: it is one page past the only device, inside the platform bus's window but backed by nothing, and reading it took the machine down. Now

QEMU has placed a second `qa-dev` there, in the first free page-aligned slot on its platform bus (`hw/core/platform-bus.c`), with the next free interrupt, SPI 113, which is GIC 145; and the same address is a working device. An address alone tells a driver nothing about what is at it, or whether anything is; only the description can.

A different SoC needs nothing changed in the driver. That board's device tree supplies a node with the same `compatible` and its own `reg`, `ranges` and `interrupts`, and the same source is rebuilt for that board's kernel. The driver changes only if the block itself changes, and then by gaining an entry in its match table.

f) The device tree is the board's description, written by whoever designed the board and supplied with it; the driver is shared by every board that carries the chip. An address in the driver is right on one board and wrong on all the others, and on those it claims and reads whatever lives there, which Exercise 6.9 showed can take the machine down. It also hides the fault it was meant to cover: a wrong tree should fail loudly, where its owner will see it and fix it, not be papered over by a second copy of the same fact that will drift from the first. And the fallback cannot do what it promises anyway. With no node there is no platform device and `probe` is never called, so the only case in which the fallback could run is one in which the tree did supply an address, and then the tree's address is the one to believe.

A.11 Chapter 11: Kernel Frameworks

Answers to the exercises in §11.3. Exercises 11.6 and 11.7 are Code, and are checked on the target: the first as its Check yourself line says, the second with the same tools that run `sh` uses.

Exercise 11.1 What a framework gives you

RECALL

a) The five of §11.1.1: no way for a program that does not already know about it to find it; no documented interface, only the one in your head and in the source; no existing tools, so every user writes their own reader; no power management, so `suspend` and `resume` do nothing; and no way to say what the device *is*, only what it does. IIO would also have given it a scale and units, and a buffered stream to userspace (§11.1.8).

b) You implement an interface somebody else designed, and in exchange you stop having to design, document, version and support one of your own, and every tool that already speaks it works with your device.

c) First, it is an ABI as soon as a second program uses it: its numbers and structure layouts are frozen, and you may add to them but never change or remove anything. Second, it is a portability trap: a 32-bit process on a 64-bit kernel lays the argument structure out differently, so the `ioctl` needs a `compat_ioctl` handler, or a structure with fixed-width types and no implicit padding. Third, it is undiscoverable: nothing lists the devices that implement it, and no tool speaks it. The second is the one that appears only on a particular kind of system:

a 64-bit kernel running 32-bit userspace, which is common in embedded work, an arm64 kernel under an older 32-bit Arm root filesystem for instance. It is invisible until somebody runs a 32-bit binary.

d) There is none, which is the point. Nothing records which drivers implement which `ioctl` numbers: a number means something only to the driver that defined it, and the only way to find the devices is to know their names already. Trying a device with an `ioctl` it does not implement fails with `ENOTTY` at best, and does something else entirely if its driver happened to pick the same number.

Exercise 11.2 The tour

RECALL

a) **i2c** and **iio**: an I2C client, because that is how it is attached, and an IIO device, because it measures. A temperature sensor that watches the health of the board itself is often a `hwmon` device instead, a subsystem the tour in §11.1.3 leaves out.

b) **gpio**. On a real board the LED is usually handed on to the `leds` subsystem through the `gpio-leds` driver, which consumes the line; the pin is still a GPIO.

c) **watchdog**.

d) **input**: it is a source of human input. Its driver usually reads the encoder's two lines through `gpio`, but what the device is, is input.

e) **mtdev**: raw flash, with no controller in front of it doing wear levelling or bad-block handling.

f) **spi** and **iio**.

g) **clk**: a clock source, described once in the device tree and shared by its three consumers, whose enables the clock framework counts.

h) a) and f), each an **i2c** or **spi** device and an **iio** device. That is not a contradiction, because the two answer different questions, the same two that `/sys/bus` and `/sys/class` answer (§10.1.2): the bus is how the device is attached and how a driver is matched to it, and the class is what the device is to userspace. One driver does both. It registers with the bus as an I2C or SPI driver, and its probe registers an IIO device.

i) **regmap**. It is a driver-side library rather than an interface: it gives a driver register access over I2C, SPI, MMIO and other buses from a description, the register width and which registers are readable, writable, volatile or cacheable, and does the transfers, the byte order, the read-modify-write and the caching itself. The only view of it from outside is in debugfs, which is not an interface.

Exercise 11.3 Reading an IIO device

HAND CALCULATION

a) With no offset the value is the raw count times the scale: $2048 \times 0.805664 = 1649.999872$, so **1650.0 mV**, or 1.650 V. The units of the scale are millivolts per count. The IIO ABI says so: the documentation of `in_voltageY_raw` states that its units after the scale and offset are applied are millivolts (Documentation/ABI/testing/sysfs-bus-iio:141–153), so the scale is whatever turns one count into millivolts. The number gives the device away, too: $3300/4096 = 0.8056640625$, a 12-bit converter with a 3.3 V reference, and 2048 is half of full scale.

b) The kernel does not use floating point, so a driver cannot cheaply produce a fractional voltage, while the raw count is exactly what the hardware delivers and the scale can be stated exactly, as a fraction if need be. In buffered mode the samples go to userspace as the hardware produced them, at full rate, with no conversion per sample in the kernel, and userspace applies one scale to all of them. And the scale belongs to the channel's configuration, which can change with its gain or range, so it is published once rather than folded into every value.

c) The offset is added to the raw value before scaling (Documentation/ABI/testing/sysfs-bus-iio:455–457): $(2048 + (-100)) \times 0.805664 = 1948 \times 0.805664 = 1569.433472$, so **1569.4 mV**, about 1.569 V. The order is raw, plus offset, times scale.

d) Only raw. A counter has no unit to convert to, so a scale would claim one and an offset would claim a zero point; without them userspace takes the value as a plain count. type should say what the value is, and `IIO_VOLTAGE` tells every tool that reads it that it is a voltage, which is false. IIO does have `IIO_COUNT`, which makes `in_count0_raw` (include/uapi/linux/iio/types.h:44), but the ABI marks that attribute as deprecated in favour of the counter subsystem (Documentation/ABI/testing/sysfs-bus-iio:2005–2009), so a device that really is a counter belongs there, not in IIO. The lab uses `IIO_VOLTAGE` because qa-dev stands in for a converter, and its counter mode is a test pattern rather than what the device is.

Exercise 11.4 The pin that is not yet a GPIO

DESIGN

a) `pinctrl`, and the configuration is written in the device tree. The pin controller's node holds groups naming the pins and the function each is to have, with their pull and drive settings, and the node of the device that uses them selects one by phandle:

```
pinctrl-names = "default";
pinctrl-0 = <&uart0_pins>;
```

b) No. The driver core has the pin controller apply the device's default state before probe is called (`pinctrl_bind_pins` in `really_probe`, `drivers/base/dd.c:635`), so by the time your code runs the pins are already set. Which function a pin has depends on the board, the same

pin of the same SoC being a UART line on one board and a GPIO on the next, so it belongs in that board's tree. And the multiplexing registers belong to the pin controller, shared by every device on the SoC: a driver that writes them hardcodes one SoC's register layout and one board's wiring, and races every other user of the same register.

c) `devm_clk_get(&pdev->dev, NULL)` to get the clock, checked with `IS_ERR`, and `clk_prepare_enable(clk)` to turn it on, checked for a negative return; `include/linux/clk.h` also has `devm_clk_get_enabled`, which does both and turns the clock off again at unbind. Writing the gate register directly instead goes wrong because the clock framework counts enables, turns a clock off when the count reaches zero, and does not know about you. When the other driver disables the clock, in its `suspend` or its `remove`, the count reaches zero and the framework gates it, and your device stops responding underneath you. The other way round, if your `remove` writes the gate off, the other driver's device dies. Even with no second driver, the framework switches off every clock nobody has claimed at the end of boot (`clk_disable_unused`, `drivers/clk/clk.c:1531–1570`), yours included; and it enables a clock's parents before the clock, which a write to one gate does not.

d) Get it with `devm_regulator_get` and turn it on with `regulator_enable`. The framework counts its consumers and switches the supply off only when the last one lets go; it enables the supply's own parent supplies first; it holds every consumer to the voltage limits and permissions the board declares in its device tree; it waits for the supply to settle before returning; and once boot has finished it switches off every regulator no consumer has claimed (`regulator_late_cleanup`, `drivers/regulator/core.c:6336–6361`), which would include yours if you had only written the register. The write itself would also mean talking to the power management chip over its bus, which belongs to that chip's driver.

e) `-EPROBE_DEFER`. The lookups return it themselves when the provider has not registered yet: `devm_clk_get`, `devm_regulator_get`, and the pin controller lookup, which the core does before probe even runs. probe passes it on, ideally through `dev_err_probe`, which records why. The core puts the device on its list of deferred devices and tries it again whenever another driver binds successfully (`drivers/base/dd.c:36–50`), so it probes once its providers have; anything still waiting at the end of boot is listed in `/sys/kernel/debug/devices_deferred`.

Exercise 11.5 When a character device is right**DESIGN**

a) A block in an FPGA on a company's own board that runs a protocol of the product's own: say, a controller that takes a list of timed commands for a production machine's actuators and reports as each one completes. It fits no subsystem because it is not a member of any class of hardware that other vendors make. It does not measure a value, it is not a set of pins, a network interface or a store, and its operations are defined by the product, so no subsystem's model has a place for them. An answer that names a subsystem instead has to argue that the device's operations are that subsystem's: if the block turned out to be a set of PWM outputs, it would belong in `pwm`.

b) `misc_register`. One device needs one node, and the `misc` device gives it one: a dynamic minor under major 10 (`include/linux/miscdevice.h:74`, `include/uapi/linux/major.h:26`), with the class device and so the node created for it, in one call undone by one call. Chapter 5's long way, a number, a `cdev`, a class and a device, each with its unwind, is for a driver that needs a range of minors or a major of its own, such as many nodes for one device.

c) A specification of each: its number, built with `_IOR`, `_IOW` or `_IOWR` with the right direction and size; the argument's layout and what every field means; which errors it returns and when; and the header, in the kernel's userspace API form. A layout that is the same for a 32-bit and a 64-bit caller, with fixed-width types, explicit padding and any pointer carried as a `__u64`, or else a `compat_ioctl` that translates. Room to grow, a flags field that must be zero today or a size field, so that the next change does not need a new number. And the promise never to change or remove any of it. For how long: for as long as any program that uses it exists, which in practice means forever, because there is no deprecation period for a customer's binary (§11.1.2).

d) **The long** is 4 bytes in a 32-bit process and 8 in a 64-bit kernel, so the structure has a different size and layout for the two; and because `_IOR` encodes the size in the command number, a 32-bit caller's number does not even match the 64-bit kernel's. It needs a `compat_ioctl` that knows both layouts. **The pointer** has the same size problem, and a worse one: a pointer means nothing on the other side of the boundary. A kernel pointer handed to userspace leaks a kernel address, which defeats address-space randomisation, and is of no use to the program; a userspace pointer has to be carried as a `__u64` and turned back with `u64_to_user_ptr` (`include/linux/kernel.h:50`). The fix for both is fixed-width types.

e) "No subsystem fits" is a statement about the device, and it is a reason: there is no existing interface, so you write one and accept that you own it. "Implementing a subsystem interface looked like more work" is a statement about the author's afternoon, and it is an excuse: the interface is a cost paid once, by you, and the invented ABI a cost paid for as long as the device is in use, by you and by everyone who has to write a tool for it.

Exercise 11.8 What you gave up

DESIGN

a) No. A read of `in_voltage0_raw` returns at once with the most recent sample, whether or not a new one has arrived since the last read; nothing makes it wait, and `poll` on the attribute is not woken by new samples, because the driver never tells `sysfs` about them. What has been lost is waiting for data without spinning, and with it every sample but the latest: whatever arrives between two reads is gone, and so are the order and the timing of the samples.

b) IIO's buffers, described in `Documentation/driver-api/iio/buffers.rst` and `triggered-buffers.rst`: the driver pushes each sample into a buffer in the kernel as it arrives, and userspace enables the channels it wants and reads whole batches of samples from `/dev/iio:deviceN`, blocking or polling until they are there. A driver must add a `scan_index` and a `scan_type` to its channel and set up a buffer, which for a device that raises its own interrupt per sample, as `qa-dev` does, is `devm_iio_kfifo_buffer_setup` with the handler calling `iio_push_to_buffers` (a device that has to be told when to sample uses `devm_iio_triggered_buffer_setup` and a trigger instead), and it needs `CONFIG_IIO_BUFFER` and `CONFIG_IIO_KFIFO_BUF`, which `kernel/qa.config` already enables.

c) Anything that is not a channel value. The interrupt timestamp in `TS_L0` and `TS_HI`, which Chapter 12 is built on, or the overrun flag in `STATUS`: with a character device of your own you added a field to what read returned, or an `ioctl`. In IIO you have to find the framework's place for it, a timestamp channel or an event, or add an attribute of your own, which is new ABI again. A smaller one on this target: the module now depends on `industrialio`, which has to be loaded first, as `run.sh` does with `modprobe`.

d) A device that exchanges messages rather than producing values: an FPGA block that takes a job description and returns a result of varying length, for instance. IIO's model is a set of channels, each a value of fixed size, sampled together into scans of fixed size. Commands, replies and records of varying length have no channel to be, so the driver would invent channels, spread each message across them and leave userspace to put it back together, where a misc device that reads and writes whole messages says what it does.

e) That the claim is false. The GPIO character device needed a second version because the first got things wrong, and the first is still built and supported on this kernel, marked deprecated (`CONFIG_GPIO_CDEV_V1=y`; `drivers/gpio/Kconfig:86–94`), because programs use it. Accurately: **a framework does not remove the ABI problem, it moves it to the subsystem's maintainers**, who design, document and support the interface and its migrations once for every driver in it, instead of every driver author doing it alone (§11.1.8).

Exercise 11.9 A sample rate measured through an interface that knows nothing about it

CROSS-CHECK

a) A `PERIOD_NS` of 1 000 000 is 1 ms, so 1,000 samples a second, the first of them one period after the device is enabled (`qa_dev_arm_timer`, `tools/qa-dev/qa-dev.c:154`). In counter mode sample n has the value n and the count starts at zero (`qa-dev.c:139–145`), so the thousandth sample has the value 999:

- one second after the driver loads and enables the device, `in_voltage0_raw` reads **999**, or 998 if you read just before the thousandth sample is due;
- two reads a second apart differ by **1000**;
- at `period_ns=500000`, 2,000 samples a second: **1999** and **2000**.

The first prediction assumes the count is at zero when the driver loads: after a boot in which the device has not run, or with a driver that writes `CTRL_RESET`, which clears it (`qa-dev.c:268–272`). Disabling the device stops its timer but keeps the count, so on a second load in the same boot the value carries on from where it stopped, and only the difference can be predicted.

c) Reconcile:

- About 880: §11.1.4 measured 881, from 195 to 1076 in a second, 11.9 per cent short of 1,000. You have seen it in Chapter 8's Cross-check, Exercise 8.9, as an interrupt count short of the period's prediction by a similar margin: 1,731 in about 2.1 s at 1 ms, which is about 824 a second. The effect is the device's timer. `qa-dev` arms the next sample one period after the moment its timer callback runs, not on an absolute schedule (`qa-dev.c:154`), so every interval is the period plus however late QEMU ran the callback. Exercise 8.9 also showed that this relative re-arm is part of the explanation and not all of it, because the lateness it implies is not constant.
- The same shortfall. The value is generated inside the device: the count goes up for every sample the device takes, even one it then drops because its FIFO is full (`qa-dev.c:145` and 173). So the difference between two values counts the samples the device produced, and nothing the driver or IIO does can add one or lose one; IIO decides only when you look. To tell, count something else over the same interval and compare: the change in `qa-dev`'s line of `/proc/interrupts`, which at 1 ms is one interrupt per sample (Exercise 8.9), should equal the change in the value. The one thing the reading path does change is the interval. From a shell, the time between two reads is the `sleep` plus the cost of starting the programs in between, which §11.1.8 puts at tens of milliseconds a command. That makes a shell's figure a few per cent too high, never too low, so it cannot be the shortfall.
- Counter mode made it possible: each value is the device's own sample number, so the value is a running total, and you supplied the time yourself by reading a second apart. In `QA_DEV_MODE_NOISE` the values are a pseudo-random 12-bit sequence (`qa-dev.c:133–136`), and a difference means nothing. You would have had to count samples some other way: the interrupt count in `/proc/interrupts`, a count the driver publishes as Chapter 10's

samples attribute did, or IIO's buffered mode with timestamps. Or put the device in sawtooth mode, where the value is n modulo 4096 and a difference taken modulo 4096 still counts samples, as long as fewer than 4,096 arrive between two reads; 881 do.

d) Measured on the course's target (§11.1.8): 571 microseconds a read from a C program, 1,748 reads a second, against 29,700 microseconds from a shell loop of `cat`, 34 a second. The C program is faster by $29\,700/571 \approx 52$ times, the chapter's "fifty". The shell pays for a fork and an `exec` on every value, and even the C program pays for at least one system call and for turning the number into text, for one value. The `sysfs` path is a way to inspect a device, not a way to move samples.

e) To see every sample, the reader must read at least as often as the device produces one, so the limit is the rate of reads. From a shell that is about 34 a second: above about 30 Hz a shell loop misses samples (§11.1.8). From C it is 1,748 a second at best, before the program does anything with the data; `qa-dev`'s 881 a second already takes half of that, so around 1 kHz a loop is marginal, and above about 1.7 kHz it cannot keep up at all. What IIO offers instead is its buffered mode, triggered or not: the driver pushes each sample into a buffer in the kernel as it arrives, and userspace reads them in batches from `/dev/iio:deviceN`, blocking until they are there, so one system call carries many samples and none is lost while the buffer has room. It exists because a value-at-a-time interface cannot keep up with a device that samples.

f) "Programmed for a 1 ms period, 1,000 samples a second, `qa-dev` produced about 881 samples in a second, which is the difference between two counter values read through `in_voltage0_raw` a second apart. The shortfall is in the device model's timing, which arms each sample one period after its timer actually fired and is serviced late by QEMU, and not in the driver or in IIO; in direct mode the driver delivers no stream at all, only the latest value when asked." Measured: the two values, their difference, and the interrupt count over the same interval if you took it. Inferred: that the shortfall belongs to the device's timer rather than to the driver, which rests on reading `qa-dev.c` and on Chapter 8's measurements; and that the interval between the reads was one second, when from a shell it was a little longer.

A.12 Chapter 12: Real-Time Linux

Answers to the exercises in §12.4. Exercises 12.6 and 12.7 are Code, and are checked by running them on the target.

Exercise 12.1 What real time claims

RECALL

- a) A real-time system is one whose answer is correct only if it arrives within a stated deadline, every time: a late answer is a wrong answer, however right its content.
- b) **For 6 ms, B is real time and A is not.** B's worst observed response, 5.1 ms, is inside the deadline; A's is 30 ms, five times the deadline, and its 50 us average does not change that. **For 1 ms, neither is.** B misses every time, since it never responds in less than 5 ms. A meets the deadline on most responses but not on all of them, and a deadline met most of the time is a deadline missed. Being real time is a property of a system against a deadline, not of the system alone.
- c) "Which number improved: the average or the worst case?" A mean 30% lower says nothing about the maximum, and a real-time requirement is a statement about the maximum (§12.1.1). The next questions are under what load and for how long (§12.2.5).
- d) In **hard** real time a missed deadline is a system failure: an airbag, a motor commutation loop. In **soft** real time it degrades quality: audio, video. Linux with PREEMPT_RT is honestly good soft real time. It is used for hard real time, but only by people who have measured their own system very carefully, because nobody can prove a worst case for a kernel of that size; it can only be measured. It is not a hard real-time operating system in the sense that a small RTOS with a provable worst case is.

Exercise 12.2 The three sources

RECALL

- a) Interrupts disabled, preemption disabled, and priority inversion (§12.1.2).
- b)
- **A `spin_lock_irqsave` section copying 4 KB:** interrupts disabled, on that CPU, for the length of the copy, with preemption disabled along with them. (On PREEMPT_RT a `spinlock_t` taken with `_irqsave` does not disable interrupts at all, Documentation/locking/locktypes.rst:238. It is a sleeping lock there, and the cost of the section moves to whoever waits for the lock.)
 - **A `udelay(200)` in a probe function:** on this course's PREEMPT kernel, none of the three by itself. `probe` runs in process context and is preemptible, so a higher-priority task simply preempts it, and the 200 us is lost only to the probe. It becomes preemption-disabled time on a PREEMPT_NONE or PREEMPT_VOLUNTARY kernel, where there is no preemption point inside it, or on any kernel if a spinlock is held around it, and interrupts-disabled time under `spin_lock_irqsave`. That is why busy-waiting is a habit of its own in §12.1.6: what it costs depends on the context around it.
 - **A low-priority task holding a mutex a high-priority task needs:** priority inversion, as soon as a medium-priority task keeps the holder off the CPU. Without the medium task it is ordinary blocking, bounded by the holder's critical section.

- **A hard interrupt handler that takes 300 us:** interrupts disabled. A hard handler runs with interrupts masked on its CPU, so for 300 us no other interrupt is taken there and nothing is scheduled there either.

c) **Interrupts disabled and preemption disabled are somebody's code:** a section someone wrote, fixed by making it shorter. **Priority inversion is structural:** it needs no long section at all, only three tasks, one lock and an unlucky interleaving. The mechanism that addresses it is **priority inheritance:** while a low-priority task holds a lock a higher-priority task waits for, it runs at the waiter's priority, so no medium-priority task can preempt it (§12.1.3). In Linux that is `rt_mutex`, which `PREEMPT_RT` puts under `mutex` and `spinlock_t`.

d) Mars Pathfinder landed in July 1997, and a few days later began suffering total system resets, each of which lost data. The high-priority bus management task needed a mutex held by the low-priority meteorological task, and the medium-priority communications task, which needed neither, preempted the meteorological task and kept it from releasing the mutex. The bus management task therefore did not run in time, and a watchdog concluded that something had gone badly wrong and reset the computer. The VxWorks mutex had been created with priority inheritance switched off, and the fix, uploaded from Earth, switched it on.

Exercise 12.3 A latency budget

HAND CALCULATION

a) Add the stages:

$$40 + 15 + 120 + 60 + 200 = 435 \text{ us} \leq 500 \text{ us}.$$

It meets the deadline, with 65 us to spare. The sum of the per-stage maxima is a safe bound on the chain only because each figure is a maximum; the stages' worst cases need not happen together, so the true end-to-end worst case is at most 435 us.

b) With the work in the hard handler, the reaction happens there, and nothing on the path waits for the thread or for userspace. The handler now does its own 15 us and the thread's 60 us:

$$40 + (15 + 60) = 115 \text{ us}.$$

It meets the deadline with 385 us to spare. What it has done to every other device: the hard handler now runs for 75 us with interrupts masked on its CPU, so every other interrupt routed there can wait up to 75 us longer, and no task of any priority runs there meanwhile. The deadline was met by moving 75 us into everybody else's worst case, the first of the three sources in §12.1.2. (On `PREEMPT_RT` the kernel would force this handler into a thread unless it asked for `IRQF_NO_THREAD`, which it would then need, and which makes the imposition explicit.)

c) The other driver's `spin_lock` disables preemption, not interrupts. It therefore delays the points at which a task is scheduled, the threaded handler and the reader, and not the hard interrupt.

- a) $435 + 300 = 735$ us. It fails, by 235 us.
- b) 115 us, unchanged: a preemption-disabled section cannot delay a hard interrupt, because interrupts are still enabled. It survives.

The arrangement in b) survives. Even if the other driver had taken its lock with `spin_lock_irqsave`, b) would be $115 + 300 = 415$ us and would still survive, while a) stays at 735 us.

d) Back to $40 + 15 + 120 + 60 + 200 = 435$ us, meeting the deadline with 65 us to spare.

What happened: the other driver's `spinlock_t` is an `rt_mutex` under `PREEMPT_RT`, so its holder is preemptible. Your threaded handler runs at `SCHED_FIFO` priority 50 by default (`sched_set_fifo`, `kernel/sched/syscalls.c:858`), preempts a holder that is an ordinary task, and your chain stops paying for somebody else's critical section. **What it has cost you:**

- Every stage figure was measured on `PREEMPT` and no longer holds. Under `PREEMPT_RT` the averages rise: in this chapter's measurement the mean rose by 23% idle and 42% loaded (§12.1.8). Your own hard handler, requested without `IRQF_NO_THREAD`, is itself forced into a thread, so the 15 us and 120 us stages change shape. The 435 us means nothing until it has been measured again on the real-time kernel.
- The other driver's section now takes longer in wall-clock time, because it is preempted.
- If your path needs the same lock, you still wait for its holder, boosted by inheritance, for up to its 300 us. If the holder is another interrupt thread at the same priority 50, `SCHED_FIFO` does not preempt an equal, and the 300 us stays in your budget until you raise your thread's priority. Priorities are now something you have to design.

e) **Which numbers:** no longer the maximum of each stage but the distribution of the end-to-end latency, and in particular its 99.9th percentile, measured end to end. Percentiles do not add: the 99.9th percentile of a sum is not the sum of the stages' 99.9th percentiles, and can be larger or smaller, so a budget of per-stage figures no longer works. The requirement must also say what the other 0.1% may be. At one interrupt a millisecond, one in a thousand is one late reaction every second, so it needs to say how late one may be and whether two may come in a row.

Easier to demonstrate, because a percentile can be established by a finite measurement with a stated confidence and a maximum never can. If n independent samples show no miss, the miss rate is below 0.1% with 95% confidence once $(1 - 0.001)^n \leq 0.05$, which is $n \geq \ln 0.05 / \ln 0.999 \approx 2,995$: about three thousand samples, and more for every miss you do see. Latency spikes are not independent, since one blocking event delays a run of consecutive interrupts, so in practice you need many times that, under the product's load. It is still a question a measurement can answer. Whether the requirement is maximum or percentile, nothing can be demonstrated from per-stage figures alone.

Exercise 12.4 Reading your own driver for hostility

DESIGN

The answer below is for a driver written to the specifications of Exercises 8.6, 9.6, 10.6 and 11.6. Its handler is registered with `devm_request_irq` and no flags. It reads `IRQ_STATUS`, drains the device's FIFO, keeps the latest sample in an `atomic_t` for `read_raw`, and acknowledges. It also has a `gpio_chip` over `GPIO_OUT` and `GPIO_IN`. Yours may differ, and the method is the point.

a) **The hard handler, for its whole length.** It runs with interrupts masked on its CPU while it reads `IRQ_STATUS`, drains up to `QA_DEV_FIFO_DEPTH`, 16 samples, one MMIO read each, and writes the acknowledgement: bounded by about twenty register accesses, each a trap into QEMU's device model on this target. **Any `spin_lock_irqsave` in process context.** Chapter 9's read took the FIFO lock that way (§8.1.8). Chapter 11 deleted read, so with `read_raw` using `atomic_read` as §11.1.4's does, none remains, unless your `gpiochip`'s set does its read-modify-write of `GPIO_OUT` under `spin_lock_irqsave`, which is a few register accesses long. *Under `PREEMPT_RT`:* the handler is forced into a thread, because it was requested with neither `IRQF_NO_THREAD` nor `IRQF_ONESHOT` (`kernel/irq/manage.c:1372`). It runs as `irq/N...` with interrupts enabled, and the interrupts-off time shrinks to the kernel's generic primary handler. `spin_lock_irqsave` on a `spinlock_t` no longer disables interrupts at all. The cost is nothing and the fix is to leave it alone. What must not be added is `IRQF_NO_THREAD` to "keep it fast".

b) **The FIFO lock in the handler,** if it is held across the drain loop: up to 16 MMIO reads under a lock. Chapter 9's read also copied out of the driver's FIFO under it; that went with read in Chapter 11, unless you kept the FIFO. The fix is to read the samples into a local array first and take the lock only to copy them in. *Under `PREEMPT_RT`:* the lock is an `rt_mutex`, so the section stops being preemption-disabled time. Instead it is time for which a waiter of any priority can be blocked, bounded by inheritance, and the handler thread and a process-context holder now sleep on each other rather than spin. Shorten the section; do not change its type. A `raw_spinlock_t` here would put back exactly the latency `PREEMPT_RT` removed (§12.1.5).

c) **None.** Nothing in Chapters 8 to 11 asks for `udelay`, `mdelay`, `ndelay` or a loop that polls a register until it changes. The drain loop is not a busy-wait: it reads until the FIFO is empty, and the device's depth of 16 bounds it. *Established by* searching the source for `udelay`, `mdelay`, `ndelay`, `cpu_relax` and the `readl_poll_timeout` family, and by reading every loop whose exit condition is a register value to check that it has a bound other than the device's cooperation. Had there been one, the fix is `usleep_range` in process context (§9.2.3). Under `PREEMPT_RT` a `udelay` in the threaded handler burns its time preemptibly, but it still holds up the thread and everything waiting for it.

d) **None.** The counters are `atomic_t` (Exercise 8.6), the sample is an `atomic_t`, and there is no `DEFINE_PER_CPU`, `this_cpu`, `smp_processor_id`, `get_cpu` or `preempt_disable`. *Established by* searching for exactly those. The nearest thing is the handler's plain `spin_lock`, which

§8.1.8 justifies by “interrupts are already masked there”. Under `PREEMPT_RT` that reason no longer holds, since the handler is a thread with interrupts enabled, but the code is still correct. The process-context side takes the same `spinlock_t`, both are now the same sleeping lock, and forced threading adds `IRQF_ONESHOT` (`kernel/irq/manage.c:1382`), so the line stays masked until the thread finishes and the handler cannot run twice at once.

e) **Long critical sections**, and interrupts-off time generally. Reading shows where a section starts and ends, but not how long it takes: a loop’s bound comes from the device at run time, and a call into unfamiliar code has a cost the call site does not show. Use the kernel’s latency tracers, which `kernel/qa.config` enables for this chapter and both kernels have. `irqsoff` and `preemptoff` (`CONFIG_IRQSOFF_TRACER`, `CONFIG_PREEMPT_TRACER`) record the longest interrupts-off or preemption-off section since they were reset, with the stack that caused it. `wakeup_rt` (`CONFIG_SCHED_TRACER`) records the longest wait of the highest-priority task to get the CPU. An answer that picks per-CPU assumptions has a case too: the reliance is an absence, and nothing in the code names it. It must then say what finds it, and no tool does. You find it by reading every per-CPU access and asking what keeps the other tasks on that CPU away from it.

Exercise 12.5 Priorities

DESIGN

a) Any two of these:

- **Hardware interrupts.** A hard handler runs on top of whatever task is on its CPU, whatever its priority, and on `PREEMPT` so do the softirqs processed on the way out of it.
- **Anything of higher priority:** `SCHED_FIFO` and `SCHED_RR` tasks at 81 to 99, every `SCHED_DEADLINE` task, since the deadline class ranks above the real-time one, and the kernel’s stop-class threads, migration/N.
- **The throttle**, once real-time tasks on the CPU have used their 950 ms of the second (part c).

Note what does *not*: on `PREEMPT_RT` interrupt threads run at `SCHED_FIFO` 50 (`kernel/sched/syscalls.c:858`), below 80. That includes the thread that services the device the program is measuring, which is harmless only because the program blocks in read.

b) Nothing of lower priority runs on that CPU again. A `SCHED_FIFO` task that never blocks is never preempted by anything below it, so every ordinary task, including the shell you would use to kill it, is starved, and the machine is hung for everything but interrupts and higher-priority tasks. **What saves you is RT throttling.** By default real-time tasks may use 950,000 us of every 1,000,000 (`kernel/sched/rt.c:15` and `21`), so every second the CPU has 50 ms for everything else. The shell runs, slowly, and you can kill the task. 6.12 also has a second safeguard with the same effect, the fair server, which reserves 50 ms in every 1,000 ms for ordinary tasks (`kernel/sched/deadline.c:1637`).

c) In every `sched_rt_period_us`, one second by default, the real-time tasks on a CPU may run for at most 950,000 us between them, so 50 ms of every second is left for everything else. The symptom: a real-time task running flat out stops dead for up to 50 ms once a second, a periodic gap that nothing in the task explains, and the kernel prints “`sched: RT throttling activated`” once (`kernel/sched/rt.c:900`).

d) **When:** the real-time tasks’ CPU use is bounded by design and shown to be, because each of them blocks within its period by analysis and by measurement, and a 50 ms stall once a second would itself break a deadline. A control loop on a core given over to it is the usual case. It is done by writing `-1` to `sched_rt_runtime_us`, which means “no limit” (`Documentation/scheduler/sched-rt-group.rst:104`). **What you owe the next person:** a written record of where the setting is made and why; what bounds each real-time task’s CPU use, and the evidence; what now protects the system from a real-time task that loops, a hardware watchdog or a higher-priority monitor; how to recover a board that hangs; and a test that fails if the setting is lost or a real-time task spins. The safeguard is gone, and whoever changes the real-time code next has to know that.

e) It performed an **admission test**. It added the new task’s runtime divided by its period to that of every deadline task already admitted and compared the total with what the system allows deadline tasks. By default that is the same 95% of each CPU, since the `-rt` knobs are used for deadline admission control (`Documentation/scheduler/sched-deadline.rst:594`). When the total would be exceeded, `sched_setattr` fails with `EBUSY` (`kernel/sched/syscalls.c:687–689`). `SCHED_FIFO` has no admission control at all: it accepts any number of tasks at any priorities and finds out at run time that they do not fit. **A refusal is more useful** because it happens once, at configuration time, deterministically, where the program can report it and a test can catch it. A missed deadline happens later, intermittently, under a load nobody reproduced, in the field. On more than one CPU the test bounds how late a task can be rather than promising it never is (`Documentation/scheduler/sched-deadline.rst:453–463`), and that is still a statement a priority cannot make.

Exercise 12.8 What cyclicttest is for

DESIGN

a) `cyclicttest` runs one or more threads, normally at a real-time priority. Each sleeps until an absolute time, notes when it actually woke, and sleeps again one interval later, and the program reports the minimum, latest, average and maximum lateness of those wakeups for each thread. It therefore measures how late the timer interrupt and the scheduler deliver a waiting task to its CPU, under whatever load runs alongside it.

b) The 45 us describes a timer’s interrupt and the wakeup of a high-priority thread. Your data travels a different path, none of which `cyclicttest` ran: your device’s line and the interrupt controller, your handler, the step to its thread, your driver’s locks, the wakeup of your reader, and your reader’s own priority. `cyclicttest` usually runs at `SCHED_FIFO` 80 or above, while a reader left at `SCHED_OTHER` waits for the fair scheduler, and on `PREEMPT_RT`

your interrupt thread at 50 waits behind any busy real-time task above it. On `PREEMPT_RT` the timer that wakes a real-time `cyclictest` thread even expires in hard interrupt context (`kernel/time/hrtimer.c:2043–2044`), so its path has no thread step at all. Any stage of yours can be late while the timer path is fast.

c) It covers the nearest thing to (1) and (3), *for a timer*: the timer interrupt being taken, and the woken thread getting the CPU, with no (2) for a real-time thread, as in b). It covers none of the path for your device: not (1) for your line, not (2) from your handler to its thread, not (3) from your driver's wakeup to your reader, and so not the sum.

d) Any two of these:

- **A GPIO and an instrument.** The handler, or the program in userspace, toggles a GPIO. An oscilloscope or logic analyser watches it against the signal that caused the interrupt, a signal generator or the device's own line, and the delay between the two edges is measured by something outside the system (§12.2.6).
- **A loopback on one clock.** The CPU drives a GPIO output wired to an interrupt input, reading its own counter as it does, and the handler reads the same counter. Both ends are then in one clock domain, which is what `qa-dev`'s register gives the course.
- **The kernel's tracers.** `irq_handler_entry` events and the `irqsoff`, `preemptoff` and `wakeup_rt` tracers show where the time between handler and task goes. They cannot see the delay before the CPU takes the interrupt, which only an external reference can.

e) Because the course runs on an emulator, and an emulated board has no pin to put a probe on, so the techniques in d) are not available. A device that stamps the moment it raised its line is the only reference point such a target can have, and it lets the technique, subtracting a device's timestamp from the handler's, be taught to every reader on the same machine and checked automatically by `make test`. A real device would bring whatever it happens to do and no timestamp at all, since real hardware almost never has one (§12.2.2). The price is the caveat in §12.2.3: the device's clock has its own origin, so the interval it gives is only a spread.

Exercise 12.9 The trade, measured

CROSS-CHECK

a) The prediction is yours. The usual one is that PREEMPT_RT lowers both, and half of it is wrong: what the course measured is in b).

b) What the course measured and published in §12.1.8, one run of 2,000 samples each, handler-to-userspace:

Kernel	Load	Mean	Worst
PREEMPT	idle	449 us	11,321 us
PREEMPT_RT	idle	551 us	4,964 us
PREEMPT	busy	1,685 us	29,729 us
PREEMPT_RT	busy	2,393 us	18,559 us

Yours will differ, and §12.2.6 says why that is expected.

c)

- **Which prediction:** with the table above, “lower maximum” is right both idle and busy, and “lower mean” is wrong both idle and busy.
- **The mean is worse** because PREEMPT_RT makes every path do more. Every interrupt costs a switch into its thread and out again, and softirqs run in threads too. Every spinlock_t acquisition is an rt_mutex operation that also does rcu_read_lock and migrate_disable (kernel/locking/spinlock_rt.c:50–51), and when contended it sleeps and does inheritance bookkeeping instead of spinning briefly. All of that is paid on every sample. **The maximum is better** because the long stretches during which nothing could preempt, spinlock_t sections anywhere in the kernel and softirq processing, are now preemptible, and inversion is bounded by inheritance. The reader’s worst wait is no longer the longest non-preemptible section in the kernel. (In this lab, a handler requested with devm_request_irq is itself forced into a thread on PREEMPT_RT, so its first statement, the ktime_get_ns(), runs after the hard-interrupt-to-thread step; part of the thread cost the mean pays is outside the measured interval, which makes the rise in the mean more notable, not less.)
- **A colleague who measured only the mean** would conclude that PREEMPT_RT is worse, by 23% idle ($551/449 = 1.23$) and 42% loaded ($2,393/1,685 = 1.42$), and reject it. For a throughput system they would be right. For anything with a deadline they would be wrong, because a deadline is a statement about the maximum, which in the same runs fell by a factor of 2.3 idle ($11,321/4,964 = 2.28$) and 1.6 loaded ($29,729/18,559 = 1.60$).
- **In one sentence:** real time is defined by the worst case, and PREEMPT_RT buys a lower worst case by making the kernel preemptible almost everywhere, which costs something on every path, so a worse average with a better maximum is exactly what was paid for.

d) The course published one repeat, of the idle PREEMPT run (§12.2.5): 11,321 us and then 2,502 us, a spread of 8,819 us, or a factor of $11,321/2,502 = 4.5$, on one kernel with nothing

changed. The difference between the kernels' idle maxima was $11,321 - 4,964 = 6,357$ us, a factor of 2.3.

- **No.** The run-to-run spread, 8,819 us, is larger than the kernel-to-kernel difference, 6,357 us, and the repeat, 2,502 us, is below the PREEMPT_RT maximum of 4,964 us. One pair of runs could have been read either way.
- **Then you have established nothing about which kernel has the lower maximum.** What you have established is that the maximum of 2,000 samples is not a stable statistic on this target, which is itself the result §12.2.5 states. The means point the same way idle and busy, but one run each supports no more than that.
- **How many:** enough that the two kernels' per-run maxima stop overlapping. If both came from the same distribution, the chance that all n of one kernel's runs fall below all n of the other's is $n!n!/(2n)! = 1/\binom{2n}{n}$. That is $1/20 = 5\%$ for three runs each, $1/70 \approx 1.4\%$ for four, and $1/252 \approx 0.4\%$ for five. Four or five runs of each, with no overlap, is what I would defend for the *direction*. For a *bound* no number of two-second runs is enough: that needs hours per run, under the product's load (§12.2.5).

e) Everything in these numbers passed through far more than the guest kernel. QEMU translates every guest instruction, so every path costs many times what it would on silicon. The interrupt controller, the generic timer and qa-dev itself are device models, and qa-dev's sample timer is a QEMU timer that fires when QEMU gets round to it. QEMU's vCPUs are threads of an ordinary process on the host, scheduled by a host kernel that is not real time and shares its machine with everything else running there, so a vCPU descheduled on the host is a stall the guest can neither see nor tell apart from its own latency; §12.2.6 names that as frequently the largest term. Inside the guest, both kernels carry CONFIG_PROVE_LOCKING and CONFIG_DEBUG_ATOMIC_SLEEP (kernel/qa.config, kernel/rt.config), so every lock acquisition does the validator's bookkeeping. qa_latency_test sets no scheduling policy, so the reader is an ordinary SCHED_OTHER task, and under load it competes as an equal with four processes that never block. The loaded figures are to a large extent the fair scheduler's time slices, not interrupt latency. The measurement is also blind to parts of the path by construction. Its absolute figure starts at the handler's ktime_get_ns(), so the time from the device raising its line to the handler running appears only as a spread above an unknown minimum. On PREEMPT_RT a handler requested with neither IRQF_NO_THREAD nor IRQF_ONESHOT is forced into a thread (kernel/irq/manage.c:1368–1404), so its first statement runs in the thread, and the step from hard interrupt to thread moves out of the absolute interval into the spread: the two kernels' columns do not cover quite the same stretch of path. The device rewrites its timestamp at every sample (tools/qa-dev/qa-dev.c:171), so a handler that runs more than one period late reads a later sample's stamp, and the spread cannot show a delay longer than about one period, which is exactly the kind that matters. And 2,000 samples are two seconds, too short to have met the rare events a worst case is made of; one repeat moved the maximum by a factor of 4.5. **So the absolute values cannot transfer.** Their dominant terms belong to the host and the emulator, and none of those

exists on the product; on real silicon the same kernel's figure would typically be tens of microseconds (§12.2.6). **What survives** is what is common to both columns and therefore cancels: the same emulator, host, load, reader and method on both kernels, so the direction of the trade, a mean that rises under PREEMPT_RT and a maximum that falls, is a property of the mechanism, and it held idle and busy alike. **What does not survive** is every number, the factors of 2.3 and 1.6, the percentages, and even the direction of the idle maximum until repeated runs separate the two kernels, since one repeat of a single kernel moved further than the gap between them.

f) The entry, for the kernel a deadline would choose:

Handler-to-userspace latency, from the `qa_latency` interrupt handler to a blocking `read()` returning in a `SCHED_OTHER` process: mean 551 us, maximum 4,964 us, idle; mean 2,393 us, maximum 18,559 us, under four CPU-bound processes. Linux 6.12.30 PREEMPT_RT, arm64 defconfig with `kernel/qa.config` and `kernel/rt.config`, including `CONFIG_PROVE_LOCKING` and `CONFIG_DEBUG_ATOMIC_SLEEP`. `qemu-system-aarch64 9.2.0, -M virt -cpu cortex-a72 -smp 2 -m 512`, software emulation, on a named host with its load stated. Device period 1 ms; 2,000 samples per condition, about two seconds; one run each, not repeated.

The sentence: *“This is not interrupt-to-userspace latency: it omits the time from the device asserting its interrupt to the handler running, which this target can report only as a spread above an unknown minimum; and because it was measured under an emulator whose host scheduling dominates it, it predicts nothing about this kernel on real hardware.”* Written out with every condition, the entry shows that a figure from this target belongs in a lab report and never in a product datasheet. For a datasheet you measure the product, as in §12.2.6.

g) One defensible answer: **a latency measurement on the product's own hardware**, with debug options off, for hours, under the product's load, against an external reference. Its latency has been characterised only inside an emulator, and §12.2.6 is where this book says why that characterisation predicts nothing about silicon. The first place the course made the same point about a number and its conditions was Chapter 2's Cross-check, Exercise 2.9. An answer that names concurrency testing instead is as good if it argues why that comes first. Every test in the course runs one thing at a time, and `lockdep` finds only the orderings your tests reach (§7.2.5), so a driver whose locking has never been exercised under contention is unverified in a way no latency figure touches.

The Self-Assessment Papers

This appendix and the four after it hold two complete four-hour papers for *Embedded Linux and Kernel Drivers*, with model answers. Appendices C and D are the papers, questions only; Appendices E and F are their model answers, with marks.

What these are for

The book has **no exam and nothing is marked**. What you get instead is a lab per chapter, checked on the target by `make test`, and eight to ten exercises per chapter ending in a Cross-check that makes you compute a number by hand and then measure it.

These papers do not replace that. **They are here to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. Nothing in the companion repository depends on them, and `make test` does not know they exist. They are useful where a written result is wanted anyway, for a certifying employer or a formal course credit, and useful on their own for finding out what you can reconstruct with nothing in front of you.

Take one after the last chapter. Both papers draw on all twelve chapters, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood. The intended point is after Chapter 12, once the driver probes off a device tree, services interrupts, blocks readers correctly, registers with two subsystems, and has had its latency characterised.

The two papers

Both cover the whole book, Chapters 1 to 12, and share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years. They do **not** weight the chapters identically, and that is deliberate.

Paper A

Question	Topic	Chapters	Marks
1	The image, and the licence on it	1	12
2	Asking a running kernel a question	2	8
3	Configuring a kernel, and what it costs	3	10
4	A module, written out	4	12
5	The character device contract	5	12
6	An address, translated and mapped	6, 10	12
7	A race, traced	7	10
8	An interrupt, acknowledged	8	10
9	A reader put to sleep	9	8
10	Frameworks, and the trade	11, 12	6
			100

Paper B

Question	Topic	Chapters	Marks
1	What leaves the building, and when it does not	1	10
2	A number that is not what it says	2, 9	12
3	An option that did not survive	3	10
4	A module that will not load	4	12
5	A read that returns zero	5, 9	12
6	A mapping that is wrong	6	10
7	A deadlock that has not happened	7	12
8	An interrupt that never stops	8	10
9	A driver that finds its own hardware	10, 11	6
10	Worse average, better maximum	12	6
			100

Paper A leans towards tracing a mechanism forwards: an address translated through a device tree, a module written from nothing, a `read()` traced through its four cases, an interrupt acknowledged in the right order.

Paper B leans towards the conditions under which the mechanism stops working: an option silently dropped by `olddefconfig`, a module refused at load, a read that returns end of file when it meant “not yet”, a mapping that reads plausible zeros, a deadlock reported by a tool without the deadlock occurring.

Code on a written paper

Both papers **do ask a candidate to write kernel C**. That is deliberate. This book teaches by building a driver: the specification of every lab is prose in its chapter and a test suite that

tells you whether what you wrote satisfies it, and a paper that avoided code entirely would be examining a different course.

The code questions come in two shapes, and neither is a typing exercise:

- **Write one function.** An `init` function, a `read()`, an interrupt handler. The answers are ten to twenty lines each. What carries the marks is the algorithm: the error path, the return value, the accessor used, the context the code is running in. Not the syntax.
- **Find what is wrong.** Both papers hand over a listing that compiles and does something other than what its author intended. The defects are the ones the chapters single out.

Both papers state in their rubric that answers may be written in kernel C **or in unambiguous pseudocode**, and that nothing is marked on syntax. A missing semicolon costs nothing. A `copy_to_user` replaced by `memcpy` costs everything.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **The kernel is 6.12 as pinned by the course.** Where an interface has changed within recent memory the paper says which version it means, because Chapters 3, 5 and 11 all make the point that the internal API is not stable.
- **HZ is 250** unless a question says otherwise, so a jiffy is 4 ms.
- **The device is `qa-dev`,** whose register map is the one in the course, and any register a question depends on is quoted in the question.
- **Only the values nobody could be expected to carry are supplied:** the SPI base of 32, the page size, a register offset where one is needed. Address translations, jiffy arithmetic and latency budgets are derived, because deriving them is the examinable skill.
- **“State the consequence”** means naming the defect earns half the marks and saying what it does to the running system earns the other half. “This is wrong” scores nothing.
- **Where a question asks you to find defects, the number is stated.** Listing more is not penalised, but only the stated number is marked, so put your strongest answers first.

Marking

Every model answer is written to be marked by somebody who has read the chapters and does not otherwise write kernel code, so each carries the reasoning rather than the answer alone. Marks are shown per part.

Three conventions worth agreeing before a paper is marked:

- **Method carries the marks.** A correct approach with an arithmetic slip in it is worth more than a correct number with no working, and both papers are built so that later parts consume earlier answers. Follow through an error rather than penalising it twice. A

candidate who translates a device tree address wrongly and then maps their own wrong address correctly has demonstrated the examinable skill.

- **The named traps are worth full marks on their own.** Several questions exist entirely to see whether a candidate avoids one specific mistake: `read` returning 0 when it meant “not yet”, a `memcpy` where `copy_to_user` was needed, dividing `/proc/stat` by `HZ` rather than by `USER_HZ`, taking `reg` at face value without translating through ranges, concluding from a clean unlocked run that a lock is unnecessary. Where a model answer flags one of these, a candidate who walks into it loses those marks and no others.
- **The discussion parts are not decoration.** “State the consequence” and “say what this is blind to” are where the book’s actual content is, and a paper marked only on the code and the arithmetic would pass a candidate who has understood none of it. Paper B in particular is mostly these.

The code questions are answered as checklists, not as listings. Where a paper asks a candidate to write a function, the model answer states what a correct answer must contain and what each element is worth, rather than printing a reference implementation. That is deliberate twice over: it is how these answers are actually marked, since two correct answers will not look alike; and a worked listing here would be a published solution to the lab that asks for the same thing, and the book publishes none for its labs or its Code exercises. Appendix A answers only the exercises that are not code.

Where a model answer says a candidate may reasonably disagree, they may. Two questions ask for a judgement rather than a fact: Paper A’s Question 10(c) and Paper B’s Question 8(d). Both answers state a position and say what an answer arguing the other way must contain to earn the marks.

Self-Assessment Paper A

Time: 4 hours. **Closed book.** No machine, no compiler, no reference material. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all twelve chapters, so **it is meant to be taken once the course is over**. The model answers are in [Appendix E](#).

Rubric

The kernel is Linux 6.12, configured as the course configures it: HZ=250, so one jiffy is 4 ms; CONFIG_PREEMPT=y unless a question says otherwise; two CPUs.

Code is marked on semantics, not on syntax. A missing semicolon, a forgotten header or a brace in the wrong column costs nothing. A memcpy where copy_to_user was needed costs everything. Answers may be written in kernel C or in unambiguous pseudocode.

Where a question says “state the consequence”, naming the defect earns half the marks and saying what it does to the running system earns the other half. “This is wrong” scores nothing.

Where a question asks you to find defects, the number is stated. Listing more is not penalised, but only the stated number is marked, so put your strongest answers first.

The device is qa-dev. Any register a question depends on is given in the question. You are expected to know that readl/writel are the only legal way to touch an __iomem pointer, and that its IRQ_STATUS is write-one-to-clear.

Supplied, because nobody could be expected to carry them: the GIC SPI base is 32; the page size is 4096 bytes; msecs_to_jiffies rounds up.

Question 1 The image, and the licence on it**12 MARKS**

A product ships an ARM64 board running Linux. Its flash contains a bootloader, a kernel, and a root filesystem holding BusyBox, an MQTT library under Apache-2.0, and a control daemon the company wrote. One driver was added to the kernel tree. One further driver is built out of tree and the company would prefer to keep it closed.

- (a) Name the four pieces of an embedded Linux system. One of them is not on the flash. Which, and what does it determine about the other three? (3 marks)
- (b) A customer requests the source. Go through the six components above and say for each what must be published. (4 marks)
- (c) BusyBox was not modified. Does that change your answer for it? State the rule. (2 marks)
- (d) The control daemon makes system calls into the kernel and is not published. The out-of-tree driver is linked into the kernel's address space. Explain why the first is settled and the second is not, naming the text that settles the first and the argument that applies to the second. (3 marks)

Question 2 Asking a running kernel a question**8 MARKS**

- (a) `cat /proc/uptime` prints two numbers. Where were those numbers a microsecond before the command ran? Give the mechanism, not a metaphor. (2 marks)
- (b) State two consequences of that mechanism for a program reading `/proc`, and for each say what a program that ignores it does wrong. (3 marks)
- (c) `/proc` and `/sys` are both generated on read. State the difference between them in terms of where their contents come from, and say which of the two is a documented ABI. (2 marks)
- (d) `ls -l /dev/null` shows two numbers where a size would be. Name them and say what each selects. (1 mark)

Question 3 Configuring a kernel, and what it costs

10 MARKS

From drivers/gpio/Kconfig:

```
config GPIO_SYSFS
    bool "/sys/class/gpio/... (sysfs interface)" if EXPERT
    depends on SYSFS
    select GPIO_CDEV
```

- (a) Your .config has CONFIG_SYSFS=y and # CONFIG_EXPERT is not set. You add CONFIG_GPIO_SYSFS=y to a fragment and run olddefconfig. What is in the final .config, and why? (3 marks)
- (b) Explain the difference between depends on and select in one sentence each. Which of the two can produce a .config that violates a depends on line elsewhere, and how? (3 marks)
- (c) A symbol has default y and its prompt is hidden. Is it set? Explain how a symbol nobody was asked about acquires a value. (2 marks)
- (d) A driver for the eMMC holding the root filesystem is configured =m, on a system with no initramfs. State what happens at boot and quote the shape of the message. (2 marks)

Question 4 A module, written out

12 MARKS

- (a) Write a complete kernel module. It must print a greeting on load and a farewell on unload, take a string parameter who defaulting to "world" that is readable but not writable through sysfs, and carry the macros a module needs. Use pr_fmt so every message is prefixed with the module name. (7 marks)
- (b) Your module_init function returns 0. What is running afterwards, and what is resident? (2 marks)
- (c) Name what __init does. It is not a comment. (1 mark)
- (d) modinfo reports vermagic: 6.12.30 SMP preempt mod_unload modversions aarch64. Name what two of those six fields assert, and say why a mismatch is an error rather than a warning. (2 marks)

Question 5 The character device contract

12 MARKS

A driver presents /dev/qa_fifo, backed by a byte ring buffer of capacity 256, currently empty.

- (a) Give the return value of every call in this sequence, and the level after each: (5 marks)

1. write of 200 bytes
2. write of 100 bytes
3. read of 512 bytes
4. read of 512 bytes
5. write of 300 bytes

(b) Step 2 is the one most people get wrong. State what it returns and which part of the contract decides it. (2 marks)

(c) Give what this driver should return in each of these situations, and say what userspace sees for each: a non-blocking read with nothing buffered; a write with the buffer full; a copy_to_user that fails; and a read asking for zero bytes. (3 marks)

(d) read is handed a char __user *. Name the three separate things that are wrong with dereferencing it directly. (2 marks)

Question 6 An address, translated and mapped

12 MARKS

A device tree contains:

```
/ {
    #address-cells = <2>;
    #size-cells = <2>;

    bus@c000000 {
        #address-cells = <1>;
        #size-cells = <1>;
        ranges = <0x00 0x00 0xc000000 0x2000000>;

        widget@0 {
            compatible = "acme,widget-1.0";
            reg = <0x00 0x1000>;
            interrupts = <0x00 0x70 0x04>;
        };
    };
};
```

(a) How many 32-bit cells does widget's reg occupy, and how do you know? (2 marks)

(b) What physical address do the registers start at, and how large is the window? Show the arithmetic. (3 marks)

(c) A colleague reads reg and maps 0x0. Name the two different things that could happen when the driver then reads its identity register, and say which of the two is worse to debug. (3 marks)

(d) Decode `interrupts = <0x00 0x70 0x04>`. Give the type, the number, the trigger, and the GIC hardware number. (2 marks)

(e) Rewrite the driver's acquisition of the register window as a platform driver would do it, in one line, and name what that line replaced. (2 marks)

Question 7 A race, traced

10 MARKS

Four kernel threads each increment a shared unsigned long twenty thousand times, with no lock, on a two-CPU machine.

(a) What is the expected final value? (1 mark)

(b) A run reports 63,562. How many increments were lost, and what percentage? (2 marks)

(c) `counter++` is one line of C. How many machine operations is it, and describe the interleaving that loses one increment. (3 marks)

(d) The same program with `increments=200` per thread returns the correct answer every time, over many runs. Is the bug absent at that size? Explain what is actually different. (2 marks)

(e) A colleague proposes making the counter `atomic_t` rather than adding a lock. For this program, does that work? Now suppose the threads must also maintain a running maximum alongside the counter. Does it still work? State the rule. (2 marks)

Question 8 An interrupt, acknowledged

10 MARKS

`/proc/interrupts` on a running system contains:

```
17:      1789          0  GIC-0 144 Level    qa_irq
```

and the device tree for that device says `interrupts = <0 112 4>`.

(a) Three different numbers identify this interrupt. Name all three, give their values, and say where each comes from. (3 marks)

(b) Write the body of a hard interrupt handler for a device whose `IRQ_STATUS` is write-one-to-clear and whose FIFO must be drained. Return the correct value in both the "mine" and "not mine" cases. (4 marks)

(c) Your handler acknowledges before draining rather than after. Describe the sequence of events that results, and say whether data is lost. (2 marks)

(d) Your handler does not acknowledge at all. State the consequence. (1 mark)

Question 9 A reader put to sleep

8 MARKS

(a) This code is intended to wait until data is available, and the reader has already joined the wait queue the interrupt handler wakes. Now and then it sleeps until the next interrupt instead, and if no further interrupt comes, forever. Describe the interleaving that causes it, being precise about where the interrupt lands. (3 marks)

```
if (fifo_empty()) {
    set_current_state(TASK_INTERRUPTIBLE);
    schedule();
}
```

(b) `wait_event_interruptible` takes a *condition* rather than a variable. Give two separate reasons: one about the failure in part (a), and one about what happens after a wake. (2 marks)

(c) State what `read` must return in each of these: data available; empty and blocking; empty with `O_NONBLOCK`; and a signal arriving during the wait. (2 marks)

(d) Name the two families of delay function, and give the single question that chooses between them. (1 mark)

Question 10 Frameworks, and the trade

6 MARKS

(a) A driver presents a character device with two `ioctl`s of its own. Name three things a subsystem would have given it that it does not have. (2 marks)

(b) Registering with IIO creates `in_voltage0_raw` in `sysfs`. Name what the driver had to implement for that to appear, and what it did **not** have to implement. (2 marks)

(c) The same measurement on two kernels gives a mean of 449 us and a maximum of 11,321 us on `PREEMPT`, and 551 us and 4,964 us on `PREEMPT_RT`. State which kernel is better and defend the answer in one sentence. (2 marks)

End of Paper A.

Self-Assessment Paper B

Time: 4 hours. **Closed book.** No machine, no compiler, no reference material. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all twelve chapters, so **it is meant to be taken once the course is over**. The model answers are in Appendix [F](#).

Rubric

The kernel is Linux 6.12, configured as the course configures it: HZ=250, so one jiffy is 4 ms; CONFIG_PREEMPT=y unless a question says otherwise; two CPUs.

Code is marked on semantics, not on syntax. A missing semicolon, a forgotten header or a brace in the wrong column costs nothing. A memcpy where copy_to_user was needed costs everything. Answers may be written in kernel C or in unambiguous pseudocode.

Where a question says “state the consequence”, naming the defect earns half the marks and saying what it does to the running system earns the other half. “This is wrong” scores nothing.

Where a question asks you to find defects, the number is stated. Listing more is not penalised, but only the stated number is marked, so put your strongest answers first.

The device is qa-dev. Any register a question depends on is given in the question. You are expected to know that readl/writel are the only legal way to touch an __iomem pointer, and that its IRQ_STATUS is write-one-to-clear.

This paper is about failure. Nearly every question hands you something that compiles, loads, or runs, and does something other than what its author intended. In most cases the system reports nothing at all, and saying **how you would have found it** carries marks in its own right.

Supplied: the GIC SPI base is 32; the page size is 4096 bytes; msecs_to_jiffies rounds up; USER_HZ is 100.

Question 1 What leaves the building, and when it does not 10 MARKS

- (a) State, in one sentence, the only question an open source licence answers. (1 mark)
- (b) A colleague says “we ship Linux, so all our code has to be open source”. Identify the part of that sentence that is wrong and the part that is right, and name the specific text that settles it. (3 marks)
- (c) Your product satisfies the GPL with a written offer rather than by shipping source alongside. State what you must be able to produce, and for how long. Name the engineering consequence, which is not a legal one. (3 marks)
- (d) An audit script greps for `SPDX-License-Identifier` and reports six distinct licences in a directory. Reading the files by hand finds three. Explain the discrepancy. (2 marks)
- (e) The same script finds no file marked `Proprietary` and the colleague concludes the tree contains nothing proprietary. Give two separate faults in that inference. (1 mark)

Question 2 A number that is not what it says 12 MARKS

- (a) `/proc/stat` reports CPU time in units of $1/100$ s regardless of `CONFIG_HZ`. A driver author divides those figures by HZ, which is 250. By what factor is the answer wrong, and in which direction? (2 marks)
- (b) Two consecutive samples of `/proc/stat` on a two-CPU machine, 2.50 s apart, show the idle field rising by 447 units. Convert that to seconds. It exceeds the wall-clock interval. Explain why that is not a mistake. (3 marks)
- (c) A script computes context switches per second by sampling `/proc/stat` twice and dividing by the `sleep 1` it asked for. Name the two separate errors in that method. (2 marks)
- (d) Ten runs of that script on one idle machine give ten different values, the largest several times the smallest. State what that spread tells you about the quantity being measured, and how many significant figures the result should be reported to. (2 marks)
- (e) HZ is 250. Give the measured duration of `msleep(1)`, `msleep(4)` and `msleep(10)`, and explain why two of the three are the same number. `msleep`’s own source contains no rounding-up term; name where the extra time comes from. (3 marks)

Question 3 An option that did not survive**10 MARKS**

You add `CONFIG_PREEMPT_RT=y` to a config fragment, merge it, run `olddefconfig` and build. The kernel builds cleanly, boots, and `/proc/version` says `PREEMPT`.

- (a) Explain what happened. `PREEMPT_RT` is declared depends on `EXPERT` && `ARCH_SUPPORTS_RT`, and the architecture selects the second. (3 marks)
- (b) State why this failure is worse than a build error, and describe the first symptom you would actually have noticed. (2 marks)
- (c) Describe a mechanical check that would have caught it at configure time, and say what makes it possible to write. (2 marks)
- (d) Removing `CONFIG_IKCONFIG` from a kernel shrinks `vmlinux`'s section total by 41,208 bytes and the Image by 65,536. The object it removes contributes 34,138 bytes. Account for both differences. (3 marks)

Question 4 A module that will not load**12 MARKS**

- (a) A module compiles and links, and `insmod` reports `Unknown symbol qa_answer (err -2)`. Give **three** distinct causes, and say how you would distinguish them. (4 marks)
- (b) One of those causes involves no error in your code at all and produces a message that does not mention the real problem. Name it and explain why the message is misleading. (3 marks)
- (c) A second module fails with `invalid module format`. What disagreed, and what would you do about it? (2 marks)
- (d) A build fails with `ERROR: modpost: GPL-incompatible module mine.ko uses GPL-only symbol 'bar'`. Contrast this failure with the one in part (b): which is better to receive, and why? (2 marks)
- (e) After loading a proprietary module, `/proc/sys/kernel/tainted` reads 4097 and one further line appears in `dmesg` that is not about licensing. State what it says and what has been lost. (1 mark)

Question 5 A read that returns zero**12 MARKS**

This is the read of a character device backed by a ring buffer. It compiles, it loads, and a program using it appears to work until it does not. `priv->lock` is a `spinlock_t` and `qa_fifo_get` returns the number of bytes it actually removed.

```

static ssize_t qa_read(struct file *file, char __user *buf, size_t count, loff_t *
↳ pos)
{
    unsigned long flags;
    u8          tmp[64];
    size_t      n;

    spin_lock_irqsave(&priv->lock, flags);

    if (priv->level == 0) {
        spin_unlock_irqrestore(&priv->lock, flags);
        return 0;
    }

    n = qa_fifo_get(priv->fifo, tmp, min(count, sizeof(tmp)));
    memcpy(buf, tmp, n);

    spin_unlock_irqrestore(&priv->lock, flags);
    return count;
}

```

- (a) This function has **four** defects. Name each one and state its consequence. (8 marks)
- (b) Give the correct return value in each of the four situations a blocking read faces: data available; empty and blocking; empty with `O_NONBLOCK`; and a signal arriving during the wait. (2 marks)
- (c) The same driver implements `read` but not `poll`. A program calls `select` on the descriptor. State what happens, and what the driver must implement. (2 marks)

Question 6 A mapping that is wrong**10 MARKS**

- (a) A driver maps a physical address one page past its device and reads the identity register. Describe what happens on ARM64, and what state the module and the machine are left in. (3 marks)
- (b) A different driver maps an address far below its device and reads the same register, obtaining 0x00000000 with no error. Which of the two failures is more dangerous, and why? (2 marks)
- (c) From parts (a) and (b), state the general rule about using “it did not crash” as evidence that a mapping is correct, and name the one defensive measure that would have caught both. (2 marks)
- (d) A driver stores the result of `ioremap` in a `void *` rather than a `void __iomem *`, and accesses it with `*ptr` rather than `readl`. It compiles and appears to work. Name **three** distinct things that can now go wrong. (3 marks)

Question 7 A deadlock that has not happened**12 MARKS**

A module’s init function, running on one thread, takes lock A then lock B and releases both, then takes lock B then lock A and releases both. It loads successfully. `dmesg` then contains `WARNING: possible circular locking dependency detected` and a diagram headed `*** DEADLOCK ***` with a CPU0 and a CPU1 column.

- (a) Nothing deadlocked and only one thread ran. Explain why the report was produced anyway. (3 marks)
- (b) The diagram has two CPU columns. How many CPUs executed this code? State what the columns actually represent. (2 marks)
- (c) Name the facility that produced the report, the config option that enables it, and what it costs. (2 marks)
- (d) A second module loses up to 38% of its increments to a data race, on the same kernel, and the same facility reports nothing at all. Explain precisely why, and state the general limitation. (3 marks)
- (e) A colleague argues an ABBA ordering that has run for two years without deadlocking is therefore correct. Rebut it in two sentences. (2 marks)

Question 8 An interrupt that never stops**10 MARKS**

A level-triggered device is given a handler that reads its status register, counts the interrupt, and returns `IRQ_HANDLED` without writing to `IRQ_STATUS`.

- (a) Describe what happens from the first interrupt onwards. State what `insmod` does. (3 marks)
- (b) Roughly half a minute later the console prints a message about RCU stalls and an NMI sent between CPUs. Explain the connection to the missing write, given that the message mentions neither interrupts nor the driver. (3 marks)
- (c) A test harness runs this. State what it should report, and why that is different from a failing test. (2 marks)
- (d) The same omission on an **edge**-triggered device does not wedge the machine. State what happens instead, and which of the two failures you would rather ship. (2 marks)

Question 9 A driver that finds its own hardware**6 MARKS**

- (a) A platform driver's probe is never called. Give **four** things you would check, in the order you would check them. (3 marks)
- (b) A driver is unbound by hand through `sysfs` while its module remains loaded. State what happens to the memory region, the interrupt and the `sysfs` attributes it acquired with `devm_`, and what that tells you about what `devm_` is attached to. (2 marks)
- (c) The same driver keeps its register pointer in a file-scope `static`. Describe the machine on which that stops working. (1 mark)

Question 10 Worse average, better maximum**6 MARKS**

The same latency measurement on two kernels, idle, over 2,000 samples:

PREEMPT	mean	449 us	max	11,321 us
PREEMPT_RT	mean	551 us	max	4,964 us

- (a) State which kernel a throughput-oriented reviewer would prefer, which a real-time requirement would demand, and why both are reading the table correctly. (2 marks)
- (b) Repeating the `PREEMPT` measurement gives a maximum of 2,502 us. State what that does to any conclusion drawn from the table, and what would be needed to draw one safely. (2 marks)
- (c) Both figures were measured under an emulator. Name **two** components other than the guest kernel that are inside those numbers, and state precisely what survives the comparison and what does not. (2 marks)

End of Paper B.

APPENDIX E

Model Answers to Paper A

Model answers for Appendix C, with marks shown per part.

Code questions are answered as checklists of what a correct answer must contain, not as listings. Two correct answers will not look alike, syntax is not marked, and a worked listing here would be a solution to the lab that asks for the same thing.

Every measured number quoted here was taken on the course's own target, and each comes from the chapter that publishes it: the FIFO sequence in Question 5 is the Cross-check of Exercise 5.9, the run in Question 7 is one of the ten in §7.1.2, the interrupt numbers in Question 8 are those of §8.1.1, and the latencies in Question 10 are those of §12.1.8.

Question 1 The image, and the licence on it

12 MARKS

(a)

3 MARKS

Bootloader, kernel, root filesystem, toolchain.

The **toolchain** is the one that does not ship. What it determines about the other three is the **ABI**: the C library and its version are baked into every binary built with it, so a glibc binary cannot run on a musl root filesystem, and a binary built against a newer glibc will not run on an older one. One mark for the four names, one for identifying the toolchain, one for naming the ABI consequence rather than saying “it compiles them”.

(b)**4 MARKS**

Two thirds of a mark each, rounded in the candidate's favour:

Component	Must publish
Bootloader	Complete corresponding source, including the board port, its config and build instructions
Kernel	The same, including the in-tree driver, the .config and the device tree sources
BusyBox	Source at the version shipped
MQTT library (Apache-2.0)	Nothing. Carry the licence text and the NOTICE file
Control daemon	Nothing
Out-of-tree driver	Nothing, if kept proprietary

The in-tree driver being *inside* the kernel tree is the point of that row: it is part of the work whose source must be published, and a candidate who lists it separately as "ours, so nothing" has missed it.

(c)**2 MARKS**

No, it changes nothing. **The obligation runs to whoever receives the binary and is not conditional on modification.** One mark for "no", one for stating the rule. "We did not change it" is the plausible wrong answer and scores zero; it does make compliance trivial, since the answer is the upstream tarball at the version shipped, and a candidate who says that earns the second mark.

(d)**3 MARKS**

The daemon is settled by the **syscall exception**, which the kernel's COPYING declares at its head and LICENSES/exceptions/Linux-syscall-note states: user programs using kernel services by normal system calls are not derived works. That is a clarification granted by the copyright holder, not something that falls out of the GPL, and a candidate who says so earns full credit for that half.

The driver is not settled because it is compiled against kernel headers and linked into the kernel's address space, so the derivative-work argument that the syscall exception excludes for applications applies to it directly. It is contested rather than decided; the kernel enforces a version of the argument mechanically through EXPORT_SYMBOL_GPL without the question ever being litigated.

One mark for naming the syscall note, by COPYING or by Linux-syscall-note, one for the address-space distinction, one for identifying the second as contested rather than asserting an answer.

Question 2 Asking a running kernel a question

8 MARKS

(a)

2 MARKS

They did not exist. There is no file and no storage: opening the path selects a kernel function, and reading it runs that function, which formats a string from live kernel variables at that instant. Closing the file discards it. One mark for “nowhere”, one for the mechanism.

(b)

3 MARKS

Any two of, one and a half marks each:

- **Every file appears to have size zero**, because the kernel does not know the length until it generates the answer. A program that trusts `stat` to size a buffer reads nothing.
- **Two reads give two different answers**, with no consistency guarantee between them. A program computing a rate from one read of a counter has computed nothing.
- **Most of it cannot be mapped or sought from the end**. A program that `mmaps` it, or seeks to the end to find its length, fails.
- **A read can be expensive**, walking kernel structures under a lock. Polling in a tight loop is measurably costly.

(c)

2 MARKS

`/proc` grew organically and holds whatever anyone put there, so its files share no format. `/sys` is generated from the driver model, with the rule of one value per file. **`/sys` is the documented ABI**, recorded in `Documentation/ABI/`. One mark each.

(d)

1 MARK

Major and minor. Major selects the **driver**; minor selects **which device of that driver's**. Both halves needed for the mark.

Question 3 Configuring a kernel, and what it costs

10 MARKS

(a)

3 MARKS

`CONFIG_GPIO_SYSFS` is **absent entirely** from the final `.config`. Not `n`, and not `# CONFIG_GPIO_SYSFS is not set`: the line the candidate added is discarded.

The prompt is `bool "... if EXPERT`, so with `EXPERT` off the symbol **has no prompt**, and a symbol with no prompt cannot be set by a user or by a fragment. `olddefconfig` gives it its default, which is absent.

One mark for “not there at all”, two for the prompt-behind-`EXPERT` reasoning. A candidate who says “it is set to `n`” has the common misconception and earns one mark at most.

(b)

3 MARKS

depends on B means the symbol is not offered unless B is on: a **precondition, which is checked**. select B means turning this symbol on **forces B on: an imperative, which is not checked**.

select is the one that can produce a violating .config, because it does not evaluate the selected symbol's own depends on. Selecting B when B depends on C and C is off leaves B=y with its own requirements unmet, and the failure surfaces as a compile or link error in unrelated code. One mark per definition, one for identifying select with the mechanism.

(c)

2 MARKS

Yes, it is set, provided its depends on is met. A hidden prompt means the symbol is **not asked about**, not that it is off; it then takes its default, here y. One mark for "yes", one for the distinction between "nobody was asked" and "the value is n".

(d)

2 MARKS

The kernel boots normally, prints its usual messages, and then **panics**, because the driver it needs to read the root filesystem is stored on the root filesystem:

```
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
```

One mark for the circularity, one for the shape of the message. Wording need not be exact; "unable to mount root fs" is enough.

Question 4 A module, written out

12 MARKS

(a)

7 MARKS

Marked against this checklist. Order and layout are free.

Element	Marks
An init function returning int, and an exit function returning void	1
Both registered, with module_init and module_exit	1
A message on load and a message on unload, using a pr_* wrapper	1
static char *who = "world"; with module_param(who, charp, ...)	1
A mode of 0444, or any read-only mode. 0644 is the trap and costs this mark	1
MODULE_LICENSE, plus author and description	1
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt before the includes	1

Deduct nothing for a missing header, a missing __init/__exit, or MODULE_PARM_DESC. Deduct the mode mark for a writable mode, since the question states the requirement explicitly.

(b)

2 MARKS

Nothing of the candidate's is running. The module is *resident and idle*: code and data occupying kernel memory, waiting to be called by something else. One mark for “nothing running”, one for “resident”. A candidate who says “the module keeps running” has the central misconception of the lecture and earns zero here.

(c)

1 MARK

It places the function in a section that is **discarded once initialisation is complete**, returning that memory to the system; the boot message `Freeing unused kernel memory` is this happening. Naming the section-and-free behaviour earns the mark; “it marks it as init code” does not.

(d)

2 MARKS

Any two of: the kernel version; SMP, so a kernel built without SMP has different locking primitives; the preemption model; `mod_unload`; `modversions`; the architecture. Half a mark each.

The mismatch is an error rather than a warning because **the kernel has no stable internal ABI**: a structure that gained a field between two releases would be read at the wrong offsets, and the result would be silent memory corruption rather than a refusal. One mark for turning corruption into an error message.

Question 5 The character device contract

12 MARKS

(a)

5 MARKS

One mark per line. Capacity 256, starting empty.

Step	Returns	Level after
write 200	200	200
write 100	56	256
read 512	256	0
read 512	-EAGAIN	0
write 300	256	256

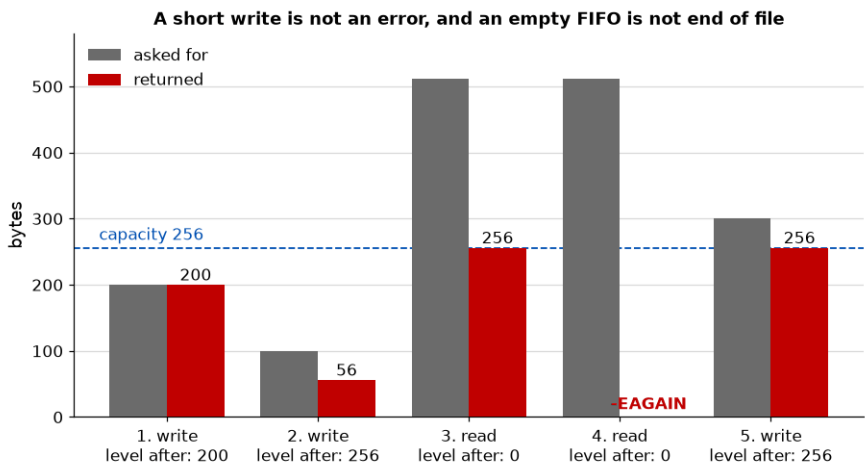


Figure E.1 A grouped bar chart of the five calls, showing what each asked for against what it returned. The second write asks for 100 and returns 56, a short write rather than an error; the second read returns no bar at all and is marked -EAGAIN, because an empty FIFO is not end of file.

(b)

2 MARKS

It returns **56**, a short write. The FIFO is not full when the call arrives, so -ENOSPC does not apply; the contract is that a short count is normal, not an error, and the caller is expected to read the return value. One mark for 56, one for naming the contract.

-ENOSPC and 100 are the two plausible wrong answers.

(c)

3 MARKS

Three quarters of a mark each:

Situation	Return	Userspace sees
Non-blocking, nothing buffered	-EAGAIN	read returns -1, errno is EAGAIN
Write with the buffer full	-ENOSPC	-1, errno is ENOSPC
copy_to_user fails	-EFAULT	-1, errno is EFAULT
A read of zero bytes	0	read returns 0, errno is unchanged

The last is not an error at all. The read that Chapter 5 specifies returns 0 for a count of 0 (§5.2.2): nothing was asked for, so nothing is wrong. -EINVAL is the plausible wrong answer and scores nothing for that line.

(d)

2 MARKS

Any three of these four, two thirds of a mark each, capped at 2:

- The page may not be present; userspace memory is demand-paged, and a direct dereference faults where no handler expects it.
- The address may not belong to the caller, and may point into kernel memory, so a driver that copies from it hands kernel memory to a program that asked for it.
- The mapping can change under you; another thread can munmap it between check and copy.
- Faulting it in may sleep, which is legal here and not in every context the code might be reused in.

Question 6 An address, translated and mapped

12 MARKS

(a)

2 MARKS

Two cells: one address and one size. You know because the **parent** declares `#address-cells = <1>` and `#size-cells = <1>`; the node itself does not say. One mark for “two”, one for “the parent declares it”. The second mark is the examinable half.

(b)

3 MARKS

reg’s first cell is the child address 0x0. The parent’s `ranges = <0x00 0x00 0xC000000 0x2000000>` maps child 0x0 to parent 0x0_0c000000. So:

$$0x0 + 0xc000000 = 0xc000000$$

The window is 0x1000, 4096 bytes. Two marks for the address with working shown, one for the size. A candidate who answers 0x0 has taken reg at face value and earns zero for this part; follow that error through into part (c) rather than penalising it twice.

(c)

3 MARKS

Either:

- the address is mapped but is not the device, and the identity register reads **plausible zeros** with no error at all; or
- the address is not backed by anything, and the read raises a **synchronous external abort**, killing insmod and leaving the module half-loaded.

One mark each. The third mark is for saying that **the zeros are worse to debug**: the abort is immediate, loud and points at the faulting instruction, whereas a driver reading zeros carries on and misbehaves somewhere else later.

(d)

2 MARKS

Type 0 is an **SPI**; number 0x70 is **112**; flag 4 is **level-triggered, active high**. The GIC number is $112 + 32 = 144$. One mark for the three fields, one for 144.

(e)

2 MARKS

`devm_platform_ioremap_resource(pdev, 0).`

One mark for the call, one for naming what it replaced: the address the driver used to compute by hand (the OF core translated reg through every ranges when it created the device, so the driver now only asks for resource 0), `request_mem_region`, `ioremap`, and the matching releases in the error path and in `remove`.

Question 7 A race, traced

10 MARKS

(a)

1 MARK

$4 \times 20,000 = 80,000$.

(b)

2 MARKS

$80,000 - 63,562 = 16,438$ lost, which is $16,438/80,000 = 20.5\%$. One mark each.

(c)

3 MARKS

Three: a load, an add, and a store.

The interleaving: CPU A loads the counter and gets n . CPU B loads the counter, also gets n . Both add one and both store $n+1$. Two increments happened and the counter advanced by one. One mark for “three”, two for an interleaving that actually loses an update; a candidate who says “they happen at the same time” without the load-load-store-store detail earns one.

(d)

2 MARKS

No, the bug is absent at no size. What differs is that the threads do not overlap for long enough to collide: at 200 iterations the work finishes inside the window it takes the other threads to get going, so the interleaving never occurs.

One mark for “not absent”, one for the overlap explanation. This is the question that distinguishes a candidate who has understood the lecture, and the plausible wrong answer, “the race needs more iterations to become likely”, earns one mark for being nearly right about the mechanism and missing that it is about *duration of overlap* rather than *number of attempts*.

(e)

2 MARKS

For the counter alone, **yes**: `atomic_inc` makes the load-add-store indivisible and fixes it completely.

With a running maximum alongside, **no**. The invariant now spans two variables and must hold across a read-compare-write sequence, which per-variable atomicity does not provide.

The rule: **atomics protect one variable; locks protect an invariant**. If the property you need mentions more than one variable, you need a lock. One mark for the pair of answers, one for the rule.

Question 8 An interrupt, acknowledged

10 MARKS

(a)

3 MARKS

One mark each:

- The **device tree cell**, 112, written by whoever described the board.
- The **GIC hardware number**, 144, which is 112 plus the 32-entry SPI base.
- The **Linux IRQ number**, 17, allocated at run time when the mapping is created and unrelated to either. It changes if the machine is booted with a different set of devices.

(b)

4 MARKS

Checklist:

Element	Marks
Reads its own status register first	1
Returns <code>IRQ_NONE</code> when nothing of its own is pending	1
Acknowledges by writing back the value it read	1
Drains the FIFO until the level reads zero, and returns <code>IRQ_HANDLED</code>	1

Deduct the acknowledgement mark for writing `0xFFFFFFFF` or an invented mask, since that clears bits set after the read and loses those events. The order of the acknowledgement and the drain is not marked here; part (c) is where it is examined.

(c)

2 MARKS

An event arriving between the acknowledge and the drain sets the status bit again *and* adds to the FIFO the handler is about to empty. The handler services it and returns with the status bit still set, producing **one spurious interrupt with nothing to do**.

No data is lost. One mark for the sequence, one for stating that this ordering is the safe one. A candidate who says data is lost has it backwards; that is the *other* ordering.

(d)

1 MARK

The line stays asserted, so the handler is re-entered immediately and forever. `insmod` never returns and the machine is unrecoverable. Either “interrupt storm” or “the machine wedges” earns the mark.

Question 9 A reader put to sleep

8 MARKS

(a)

3 MARKS

The reader evaluates `fifo_empty()` and finds it true. **The interrupt fires here:** the handler adds data and calls `wake_up`, which finds nothing asleep on the queue and does nothing at all. The reader then sets its state and calls `schedule()`, and sleeps waiting for a wakeup that has already happened: until the next one, and forever if there is none.

Two marks for the interleaving, one for placing the interrupt precisely between the test and the state change. A candidate who says “there is a race” without locating it earns one.

(b)

2 MARKS

One mark each:

- **Because of the failure in part (a):** the macro queues the task and sets its state *before* testing the condition, so a wake arriving during that window finds the task on the queue. Testing a condition rather than a flag is what allows the test to happen after queueing.
- **Because a wake does not mean the condition is true:** several readers may be woken and the first to run may take all the data, and spurious wakes happen. The macro loops and re-tests.

(c)

2 MARKS

Half a mark each: the bytes and their count; sleep until data arrives; `-EAGAIN`; `-ERESTARTSYS`.

Zero is never correct and a candidate who offers it for the second or third case loses that half mark and should have it pointed out.

(d)

1 MARK

The **delay** family (`udelay`, `mdelay`), which busy-waits and works in any context; and the **sleep** family (`msleep`, `usleep_range`), which gives up the CPU and needs process context.

The question: **can this code sleep?** Both halves needed for the mark.

Question 10 Frameworks, and the trade

6 MARKS

(a)

2 MARKS

Any three of, two thirds of a mark each: no way for a program that does not know about it to find it; no documented interface; no existing tools; no power management; no way to express what the device *is*; an `ioctl` ABI the author now maintains indefinitely, including its 32-on-64 compatibility.

(b)

2 MARKS

It had to implement a **channel specification** saying the value is a voltage, indexed, with `IIIO_CHAN_INFO_RAW`, and a **read_raw callback** returning the value.

It did **not** have to implement a `/dev` node, a `read()`, an `open()`, an `ioctl`, or any sysfs plumbing. One mark each way round.

(c)

2 MARKS

Neither is better in the abstract, and the question cannot be answered without the requirement. A throughput-oriented system should prefer `PREEMPT`, whose mean is 19% lower. A system with a deadline should prefer `PREEMPT_RT`, whose worst case is 2.3 times lower, because a deadline is a statement about the maximum and an average is not evidence about it.

Full marks for an answer that names the requirement as the deciding factor and reads both columns. One mark for an answer that picks `PREEMPT_RT` and defends it only by the maximum without noticing the mean got worse. **Zero for an answer that compares the means alone**, which is the trap.

A candidate may reasonably argue that the difference is within run-to-run noise and that neither conclusion is safe from one measurement; that earns full marks and is the better answer.

Model Answers to Paper B

Model answers for Appendix D, with marks shown per part.

Code questions are answered as checklists of what a correct answer must contain, not as listings. Two correct answers will not look alike, syntax is not marked, and a worked listing here would be a solution to the lab that asks for the same thing.

This paper is about failure, so most marks are in the second half of each answer: not what is wrong, but what it does to a running system that reports nothing. A script that returns a number and a script that returns the right number are indistinguishable from the outside, and several questions turn on exactly that.

Question 1 What leaves the building, and when it does not

10 MARKS

(a)

1 MARK

If I give somebody this binary, what am I obliged to give them along with it?

Any phrasing that puts the obligation on **distribution** earns the mark. Answers about “whether the code is free” or “whether we can use it” do not; using software you have is rarely the question, and it is distributing it that triggers everything.

(b)

3 MARKS

Wrong: “all our code.” Application code that talks to the kernel through system calls is not a derived work of the kernel, and its licence is your own business.

Right: the *kernel* is GPL-2.0-only, so the kernel you ship, including any modifications and any in-tree driver you added, must be published; and so must every other GPL component in the image.

The text that settles it is the syscall exception, which the kernel’s COPYING declares at its head and LICENSES/exceptions/Linux-syscall-note states: user programs using kernel services by normal system calls are not considered derived works. One mark each, and the third only for naming COPYING or the syscall exception specifically rather than gesturing at “the GPL”.

(c)

3 MARKS

Complete corresponding source for every GPL component in the shipped image: the source at the exact version shipped, the `.config`, the device tree sources, the scripts that control compilation and installation, and enough instructions to rebuild the binary you shipped. For **three years** from the last distribution of that product.

The engineering consequence is that your build must still work in three years' time. That means archiving the toolchain and not merely naming it, pinning every source and not tracking a branch, keeping the `.config` as a build artefact rather than as somebody's working file, and being able to identify which source corresponds to which shipped unit. It is a version control and artefact retention problem, and it lands on whoever owns the build.

One mark for the contents, one for three years, one for a consequence that is about reproducing an old build rather than about lawyers.

(d)

2 MARKS

The script counts **strings, not licences**. SPDX has current and deprecated spellings of the same licence, and a tree written over several years contains both: GPL-2.0 and GPL-2.0-only are one licence, GPL-2.0+ and GPL-2.0-or-later are another, and dual-licensed files carry an OR expression that a naive grep counts as a licence in its own right.

One mark for identifying the deprecated-alias problem, one for a concrete pair. A candidate who says only "the script has a bug" earns nothing; the question is which bug.

(e)

1 MARK

Half a mark each:

- **Nobody labels their own file Proprietary.** The absence of a tag is what proprietary looks like, so the interesting files are the untagged ones and the script does not report those at all.
- **The tree is not the product.** The shipped image contains binaries with no source in this tree: firmware blobs, prebuilt vendor libraries, the toolchain's own runtime.

Question 2 A number that is not what it says

12 MARKS

(a)

2 MARKS

Too small, by a factor of 2.5.

The figures are in units of `USER_HZ`, which is 100 and fixed by the ABI, so the correct conversion is to divide by 100. Dividing by 250 gives $100/250 = 0.4$ of the true value. One mark for the factor, one for the direction. A candidate who answers 2.5 without direction earns one.

(b)

3 MARKS

$447/100 = 4.47$ seconds.

It is not a mistake because **/proc/stat's first line sums over all CPUs**. With two CPUs, 2.50 s of wall clock offers 5.00 s of CPU time, and 4.47 s of it was idle: the machine was about 89% idle, which is the sensible reading. One mark for the conversion, two for the explanation.

A candidate who says "the machine was idle for 4.47 s" has read the number correctly and understood it wrongly.

(c)

2 MARKS

One mark each:

- **sleep 1 does not sleep for one second**, it sleeps for at least one second, and the interval actually measured also contains the script's own fork and exec on both sides. Dividing by the requested interval rather than by the elapsed one, which /proc/uptime will give you, builds in an error you never see.
- **The script perturbs what it measures**. Sampling /proc/stat means forking, reading and parsing, all of which cause context switches of their own, and on an otherwise idle machine the measurement is a substantial fraction of the activity.

(d)

2 MARKS

A spread that large means the quantity **has no single value**: context switch rate on an idle machine is a property of the particular moment, dominated by whatever happened to run, not a characteristic of the machine. Any single run is a sample, not a measurement.

It should be reported to **one significant figure at best**, and honestly to none: what you can report is a range. One mark each. A candidate who says "take the average of the ten" has missed that averaging a quantity with no stable value produces a number with no meaning, and earns one.

(e)

3 MARKS

Measured over 200 repetitions at HZ=250, where one jiffy is 4 ms:

Call	Measured
msleep(1)	8,000 us
msleep(4)	8,000 us
msleep(10)	15,998 us

msleep(1) and msleep(4) are the same because msec_to_jiffies rounds up to the same value: both 1 ms and 4 ms become **one jiffy**, and both then sleep the same two ticks. Asking for a quarter as long buys nothing at all.

The extra jiffy comes from schedule_timeout, not from msleep. Its guarantee is that *at least* the requested number of jiffies pass, and since the call arrives at an arbitrary point

inside the current jiffy, the remainder of that jiffy cannot be counted; so N requested jiffies cost $N+1$ ticks. The model $(\text{msecs_to_jiffies}(n) + 1)$ jiffies predicts all three exactly.

One mark for the figures, one for the rounding-up collision, one for locating the extra tick in `schedule_timeout`. Figures within a few percent are fine; the two equal ones must be equal.

Question 3 An option that did not survive

10 MARKS

(a)

3 MARKS

EXPERT is off in the `defconfig` this build starts from. With EXPERT off, `PREEMPT_RT`'s depends on is unsatisfied, so the symbol **has no prompt and cannot be set**; the line in the fragment survives the merge into `.config` but `olddefconfig` then discards it, because a symbol whose dependencies are unmet takes its default and the default is off.

Nothing warns, because a fragment line for an unsettable symbol is not an error in the Kconfig model; it is simply a request that could not be honoured.

Two marks for the dependency chain, one for saying the line is silently dropped rather than rejected.

(b)

2 MARKS

A build error stops you. This produces a kernel that builds cleanly, boots, runs, and is wrong, so it goes on to be measured, compared and reported as an RT kernel. Every number taken from it is a number about the wrong kernel.

The first symptom is that `/proc/version` or `uname -a` says PREEMPT where it should say PREEMPT_RT, closely followed by RT features being absent: no `[irq/N-...]` threads for interrupts that should be threaded, and latency figures indistinguishable from the non-RT kernel. One mark each. Accept "the latency measurement showed no improvement" as the symptom.

(c)

2 MARKS

Re-read the fragment after `olddefconfig` and check each of its assignments against the final `.config`, warning on any that did not survive; the course's own `ci/kernel.sh` does this.

What makes it possible is that the fragments are separate files containing explicit `CONFIG_X=y` assignments, so the *intent* exists in machine-readable form separately from the result. You cannot perform this check against a hand-edited `.config`, because there is nothing left to compare it to.

One mark for the check, one for identifying the separation of intent from result as the thing that makes it writable.

(d)

3 MARKS

Both gaps are **alignment**, and in neither case did any code disappear.

34,138 to 41,208. The object's bytes do not land in one section, and each section that loses bytes is itself aligned, so it shrinks by whole units rather than by what was removed:

Section	Delta	Content configs.o put there
.rodata	36,864	34,058, the compressed config and a string
.text	4,096	76
.init.text	112	
.rela.dyn	96	
.exit.text	40	
Elsewhere	0	4, the initcall entry
Total	41,208	34,138

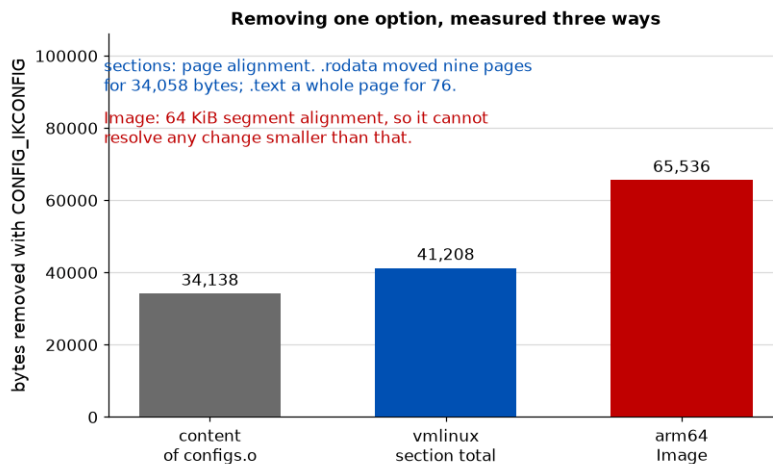


Figure F.1 A bar chart of the same one-line change measured three ways: 34,138 bytes of content in configs.o, 41,208 bytes of vmlinux sections, and 65,536 bytes of arm64 Image. The two gaps are annotated as page alignment and 64 KiB segment alignment.

Two sections carry almost all of it and both moved by an exact multiple of the 4,096-byte page: .rodata by nine pages for 34,058 bytes of content, and .text by a **whole page for 76 bytes**. The remaining 248 bytes are the sections that are not page-aligned. Removing a thing removes its padding with it, so the section total moves by more than the thing measured.

41,208 to 65,536. Image is a flat binary whose segments arm64 aligns to **64 KiB**; SEGMENT_ALIGN is SZ_64K in arch/arm64/include/asm/memory.h and vmlinux.lds.S aligns each segment to it. So the Image delta is quantised: it moves in whole 65,536-byte steps, and

here 41,208 bytes of sections happened to cost exactly one. A second option of the same size need not shrink it again by 65,536, and a small option may not shrink it at all; which it does depends on where each segment ends relative to the next 64 KiB boundary.

One mark for page alignment of the sections, one for the 64 KiB segment quantisation, one for concluding that the Image figure cannot resolve anything below 64 KiB and is therefore the wrong measurement to quote. **Link-time garbage collection is the plausible wrong answer** and earns nothing on its own; nothing was collected, and every byte is accounted for by padding.

Candidates are not expected to reproduce the per-section table from memory. Full marks for naming both alignments and the direction of each gap.

Question 4 A module that will not load

12 MARKS

(a)

4 MARKS

Three causes, one mark each, plus one for a method that actually separates them.

1. **The module that exports the symbol is not loaded.** Nothing provides it.
2. **The symbol exists but is not exported.** The provider is loaded, the function is there, but it carries no `EXPORT_SYMBOL`, so it is invisible to the module loader.
3. **The symbol is `EXPORT_SYMBOL_GPL` and your module's `MODULE_LICENSE` is not GPL-compatible.** The loader deliberately does not see it.

Distinguishing them: `grep qa_answer /proc/kallsyms`. Absent means cause 1, and `lsmod` confirms. Present means cause 2 or 3, and `modinfo -F license` on your own module settles it: a GPL-compatible licence leaves only cause 2, which the provider's source confirms by the missing `EXPORT_SYMBOL`.

(b)

3 MARKS

Cause 3. Your module is correct, the provider is loaded, the symbol is present in `/proc/kallsyms`, and the message says `Unknown symbol`.

It is misleading because the loader does not report a licence violation; it **hides GPL-only symbols from a GPL-incompatible module**, so the lookup genuinely fails and the loader reports, accurately, that it could not find the symbol. Nothing in the message mentions licensing, and the symbol is plainly there to anyone who looks.

Two marks for the mechanism of hiding rather than refusing, one for noticing that the message is technically true. This path is normally reached after a kernel upgrade turns an `EXPORT_SYMBOL` into an `EXPORT_SYMBOL_GPL`, since a fresh build would have been stopped at modpost; a candidate who says so earns full marks comfortably.

(c)

2 MARKS

vermagic disagreed. The string built into the .ko records the kernel version, SMP, the preemption model, module unloading and modversions, and it did not match the running kernel's.

The fix is to rebuild the module against the exact kernel it will load into. `modinfo -F vermagic mine.ko` against the target's own string identifies which field differs; the common cause in this course is switching between the PREEMPT and PREEMPT_RT builds and leaving stale objects behind. One mark each.

(d)

2 MARKS

The **modpost** failure is better.

It arrives **at build time on your own machine, names the licence, the module and the symbol, and produces no .ko at all**, so the defect cannot leave the build. The `insmod` failure arrives on the target, possibly at a customer, says `Unknown symbol` and does not mention the actual cause; diagnosing it requires already knowing this mechanism exists.

One mark for the choice with the build-time argument, one for contrasting the quality of the two messages.

(e)

1 MARK

Disabling lock debugging due to kernel taint.

Lockdep is now off for the rest of the boot. Every lock-ordering bug that the validator would have caught passes silently from this point, and the only way back is a reboot without that module.

Question 5 A read that returns zero

12 MARKS

(a)

8 MARKS

Two marks each: one for naming the defect, one for the consequence.

return 0 when the FIFO is empty. Zero means **end of file**. `cat` exits, a while `(read(...) > 0)` loop terminates, and the program concludes the device is finished when it has merely not produced anything yet. It should block, or return `-EAGAIN` under `O_NONBLOCK`.

memcpy to a char __user *. No verification that the address belongs to the caller, no fault handling if the page is not present, and no `__user` checking. An unmapped address oopses inside the driver; a kernel address supplied by the caller is written to happily. Must be `copy_to_user`, whose return is the number of bytes **not** copied.

The copy happens under spin_lock_irqsave. `copy_to_user` may sleep, because the destination page can be paged in on demand, and this is atomic context with interrupts disabled. With `CONFIG_DEBUG_ATOMIC_SLEEP` it is a `might_sleep` splat; without it, the machine can deadlock or corrupt. Copy into `tmp` under the lock, drop the lock, then copy out.

return count rather than n. Claims more bytes were delivered than were written. Only *n* bytes were copied, so the caller takes whatever its own buffer already held past *n* as data from the device: the byte stream is silently corrupted, and a short read, which is normal, is never seen as one.

Accept, for partial credit within a defect the candidate has otherwise missed, the observations that *priv* is never derived from *file->private_data* and that a 64-byte array on a small kernel stack is unwise. Neither is one of the four.

(b)

2 MARKS

Half a mark each: the byte count actually delivered, which may be short; sleep until data arrives, then that count; -EAGAIN; -ERESTARTSYS.

Zero is correct in none of the four, and offering it for the second or third is the defect in part (a) restated.

(c)

2 MARKS

With no *.poll* method, the VFS applies the default mask, which reports the file as **always readable and always writable**. *select* therefore returns immediately claiming the descriptor is ready, the program calls *read*, gets -EAGAIN on its non-blocking descriptor, calls *select* again, and **spins at 100% CPU** while appearing to work. On a blocking descriptor it sleeps in *read* instead, and every other descriptor it was watching waits with it.

The driver must implement *.poll*, calling *poll_wait* on the same wait queue *read* sleeps on and returning *EPOLLIN* | *EPOLLRDNORM* only when data is actually available. One mark for the busy loop, one for *poll_wait* on the same queue.

Question 6 A mapping that is wrong

10 MARKS

(a)

3 MARKS

Nothing is decoded at that address, so the read raises a **synchronous external abort**. The kernel prints an oops naming the faulting instruction and the driver's function.

The state left behind: *insmod* is killed, but **the module is half-loaded**, with whatever it allocated before the read still held and its *init* function never having returned. It cannot be removed, because *rmmod* requires a module that finished loading, and the resources are held until reboot. The machine survives; that module slot does not.

One mark for the abort, one for *insmod* dying, one for the half-loaded state.

(b)

2 MARKS

The **zeros**, decisively.

The abort is immediate, loud, and points at the exact instruction. Reading zeros produces no error of any kind: the driver believes it has a working mapping, sees an identity register that does not match and, if it does not check, carries on to configure a device that is not there. The failure then surfaces somewhere else entirely, at a different time, as a device that does not work, and nothing in the log connects it to the mapping.

One mark for the choice, one for “no error is worse than an error”.

(c)

2 MARKS

“It did not crash” is not evidence of anything. A wrong mapping has two outcomes and only one of them is loud, and which one you get depends on what happens to be at the wrong address, not on how wrong it is. Absence of a fault distinguishes nothing.

The defensive measure that catches both: **read a known identity register and refuse to probe unless it holds the expected value.** The abort case never reaches the comparison, and the zeros case fails it, so one check covers both. One mark each.

(d)

3 MARKS

Any three, one mark each:

- **The compiler treats it as ordinary memory.** It may cache the value in a register and never re-read it, so a loop polling a status register never observes it change; it may merge or discard writes it believes are redundant; it may reorder accesses against each other.
- **No ordering.** `readl` and `writel` carry the barriers that make MMIO ordered against other accesses and against DMA. A bare dereference carries none, so writes can reach the device in an order the driver never wrote.
- **No `__iomem` checking.** Sparse can no longer tell an I/O pointer from a normal one, so this class of mistake becomes invisible to the one tool that finds it, throughout the driver and not just here.
- **Access width and portability.** The compiler chooses the load width, and many device registers require an exact width; and on architectures where I/O needs distinct instructions, the code is simply wrong rather than merely fragile.

Question 7 A deadlock that has not happened

12 MARKS

(a)

3 MARKS

Lockdep does not watch for deadlocks; it **records the order in which locks are acquired and builds a graph of those orderings**, one edge per nesting it has ever seen, anywhere, at any time.

Taking A then B adds the edge A to B. Taking B then A later adds B to A, closing a **cycle**. A cycle in that graph means an ordering exists under which two threads can deadlock, and lockdep reports on the edge that closes it, immediately, whether or not anything was concurrent.

Two marks for the graph-of-orderings model, one for saying the report is about the *possibility* and fires on the first inversion. A candidate who says “lockdep detected a deadlock” has the wrong model and earns at most one.

(b)

2 MARKS

One CPU. The columns are **hypothetical**: lockdep constructs the interleaving that *would* deadlock, to show you what it is worried about. Nothing in the diagram is a record of anything that happened. One mark each, and the second is the examinable half.

(c)

2 MARKS

Lockdep, the lock validator, enabled by **CONFIG_PROVE_LOCKING**.

The cost is substantial: every acquisition and release is instrumented and checked against the graph, which is far too slow for production and uses memory that grows with the number of distinct lock classes. It is also **switched off entirely by the taint a proprietary module brings**, so loading one ends validation for the boot. One mark for the name and option, one for a real cost.

(d)

3 MARKS

Because **the racing module never took a lock**. Lockdep validates statements about locks that were acquired; where no lock was acquired there is no event to record, no edge to add, and nothing to contradict. Losing up to 38% of the increments involves no incorrect use of any lock, so from lockdep’s point of view nothing happened at all.

The general limitation: lockdep finds bugs in the locks you took, not the locks you did not take. It cannot know that a variable needed protection, and a clean lockdep report says nothing whatever about whether your data is protected.

Two marks for the mechanism, one for the limitation stated in general terms.

(e)

2 MARKS

Both sentences needed:

The ordering only deadlocks when the two paths interleave inside a window that may be a few instructions wide, and two years of not hitting a narrow window is evidence about the workload, not about the code; the machine that hits it will be a faster one, a busier one, or one with more CPUs.

And the argument proves too much: it is the same reasoning that says an unlocked counter is fine because 2,000 increments per thread never lost one, which this course measured going wrong at 20,000.

Question 8 An interrupt that never stops

10 MARKS

(a)

3 MARKS

The device's status bit stays set, so the line **stays asserted**. The GIC sees a level that is still high, so as soon as the handler returns the interrupt is delivered again, immediately and forever.

The CPU never leaves interrupt context on that core. **insmod never returns**, the console stops responding, and the machine is unrecoverable without a reset. Two marks for the level-stays-high mechanism, one for insmod.

(b)

3 MARKS

The connection is that **the CPU is never available to anything else**. RCU needs every CPU to pass through a quiescent state for a grace period to complete, and a CPU wedged in an interrupt handler never does. After the stall timeout, the RCU stall detector notices a grace period that has not ended and reports it.

The NMI is the detector asking the stuck CPU what it is doing, since it cannot be interrupted by anything less; the backtrace it returns is what actually names the driver. The message mentions neither interrupts nor your driver because **RCU is a victim, not the cause**: it is the first subsystem with a timeout long enough to notice.

Two marks for the quiescent-state argument, one for the NMI's purpose.

(c)

2 MARKS

It should report **TIMEOUT**, distinctly from a failure.

A failing test is one that ran, produced a result, and the result was wrong. This produced **no result at all**: the guest never reached the end of the test, so nothing is known about anything else the lecture tests either. Reporting it as a failure would imply the harness knows what went wrong, and treating it as a normal failure hides that every later test in that boot is unaccounted for. One mark each.

(d)

2 MARKS

On an edge-triggered line the controller latches a transition, not a level, so a status bit left set produces **no new edge and therefore no further interrupt**. You get exactly one interrupt and then silence: the FIFO fills, `OVERRUN` sets, and the device appears dead while the machine runs perfectly.

Which to ship: neither is shippable, and the honest answer is the trade. The wedge is undeniable, fires on the first interrupt, and **cannot escape testing**. The silent one passes a short test, passes review, and fails hours later in the field as an intermittent “device stopped responding” with nothing in the log.

The position expected: **the wedge**, on the ground that a failure which cannot reach a customer is preferable to one that reliably does. Full marks for an answer arguing the opposite on recoverability, that a wedged machine in the field is unrecoverable while a dead device may be restarted, provided it states that trade explicitly rather than merely preferring the quieter symptom.

Question 9 A driver that finds its own hardware

6 MARKS

(a)

3 MARKS

Three quarters of a mark each. Order matters: each step is cheaper than the next and rules out everything below it.

1. **Does the device exist?** Is the node in the device tree, is its status okay, and did a platform device appear under `/sys/bus/platform/devices/`? No device, no probe, and nothing about the driver is relevant.
2. **Did the driver register?** Is the module loaded and does `/sys/bus/platform/drivers/<name>/` exist? A driver that failed to register cannot match.
3. **Do the strings match exactly?** The compatible in the DT against the `of_match_table` entry, character for character, including the vendor prefix. This is the usual answer.
4. **Is the match table actually wired in?** `.of_match_table` assigned into the struct `platform_driver`’s `.driver` member, and the table terminated by an empty entry. A table that exists but is not attached matches nothing and warns about nothing.

(b)

2 MARKS

All three are released at unbind, while the module stays loaded: the memory region disappears from `/proc/iomem`, the interrupt is freed and its line disappears from `/proc/interrupts`, and the sysfs attributes are removed. Rebinding runs probe again and brings them all back.

What it tells you: **devm_ is attached to the device, not to the module**. The lifetime it manages is the bind, and unloading the module is merely the most common way for a bind to end, not the thing being tracked. One mark each.

(c)

1 MARK

A machine with two of these devices. The second probe overwrites the pointer the first stored, so both devices then operate on the second one's mapping, and the first device silently stops working while both appear to have probed successfully.

Question 10 Worse average, better maximum

6 MARKS

(a)

2 MARKS

A **throughput** reviewer prefers PREEMPT: its mean is 449 us against 551 us, so the average path through the system is 19% cheaper, and RT's overhead is real work being done on every interrupt.

A **real-time requirement** demands PREEMPT_RT: its maximum is 4,964 us against 11,321 us, a factor of 2.3. A deadline is a statement about the worst case, and no average is evidence about a maximum.

Both read correctly because **the table describes a trade rather than an improvement**: PREEMPT_RT buys a better bound by paying for it on every sample. One mark for the pair of readings, one for naming the trade rather than declaring a winner.

(b)

2 MARKS

The same kernel, measured again, moved its maximum from 11,321 us to 2,502 us: **a factor of four and a half, on one kernel, changing nothing**. That is larger than the kernel-to-kernel difference the table is being used to demonstrate, so **the table supports no conclusion at all**. The comparison is inside the noise of its own method.

To draw one safely: many runs of each configuration, reported as a distribution rather than a single maximum, with the run-to-run spread stated alongside; and for a bound, the interesting statistic is a high percentile over a long run, not the maximum of two thousand samples. One mark for invalidating the conclusion, one for what would replace it.

(c)

2 MARKS

Any two, half a mark each: **QEMU itself**, whose device emulation and timer delivery sit directly in the measured path; **the host kernel**, which schedules the QEMU process and can preempt it mid-interrupt; the host's own load; the host's power management and frequency scaling.

What survives: the **direction and the shape** of the comparison. PREEMPT_RT trading a worse mean for a better maximum is a real property of what it does, and it is visible here because the mechanism is real even when the timings are not.

What does not survive: **every absolute number, and the factor**. Not "approximately correct" but worthless, because the dominant term is the host and not the guest. A candidate who carries "RT is 2.3 times better" out of this course has taken the one thing the measurement cannot support. One mark for the components, one for separating shape from magnitude.